

INGENIERÍA INFORMÁTICA · SISTEMAS DE INTELIGENCIA
ARTIFICIAL

Compiladores para aprendizaje automático

Desde cero

Comprender una operación. Construir su compilador.

Entrenar un modelo y explicar cada resultado.

De una suma en C a un Transformer: representaciones intermedias, reescrituras, memoria, kernels CPU y GPU, autodiferenciación y entrenamiento.

Con el compilador **Lumbre**, laboratorios resueltos, proyectos de investigación, código completo y un mapa crítico de tinygrad, NVIDIA, AMD, Linux y Huawei Ascend.

Curso de un semestre · Itinerario preparatorio incluido

Edición ampliada con el syllabus: 5 de octubre de 2026

Corte de investigación original: 1 de octubre de 2026

Material didáctico independiente. Los ejemplos, diagramas y el compilador Lumbre son elaboración original. Incluye validación en CPU y CUDA sobre una RTX 5070 Ti Laptop, comparaciones reproducibles y entrenamiento de un decoder pequeño. Los resultados externos se atribuyen a sus fuentes. HIP queda pendiente de validación en hardware compatible.

Empieza aquí

Imagina que una persona te entrega cien temperaturas y te pide sumar dos grados a cada una. Puedes resolverlo con una calculadora, número a número. Puedes escribir un programa que repita esa operación. Y puedes escribir otro programa que construya automáticamente el programa anterior, escogiendo cómo recorrer los datos y cómo aprovechar la máquina. Ese último trabajo es el que aprenderás a hacer en este libro.

No empezaremos por una GPU, una letra griega ni una arquitectura de red neuronal. Primero habrá una lista de cuatro números, un bucle y una pregunta muy concreta: ¿qué debe producir? Después construiremos un lenguaje pequeño para describir el cálculo. Cuando ese lenguaje funcione, separaremos lo que se calcula de cómo se ejecuta. La separación permite conservar una misma intención y generar código distinto para CPU, CUDA o HIP.

El proyecto que conecta todo el curso se llama **Lumbre**. No es un nombre nuevo para tinygrad ni un envoltorio alrededor de PyTorch. Es un compilador didáctico original, escrito en Python, que genera C y dispone de rutas CUDA/HIP. Python organiza los grafos y llama al código compilado. Las operaciones de entrenamiento de la demostración se ejecutan en el C generado. NumPy proporciona almacenamiento, generación de datos y referencias de comprobación; no calcula los gradientes del modelo.

Una regla de lectura

Antes de memorizar una definición, debes poder predecir un ejemplo pequeño. Antes de optimizar un programa, debes poder explicar por qué produce el resultado correcto. Antes de publicar una mejora, debes poder decir exactamente qué has medido y qué has dejado fuera.

Las cajas azules dan nombre a una idea; las naranjas traducen fórmulas; las verdes desarrollan ejemplos; las violetas plantean tareas; las rojas señalan errores; las de color petróleo permiten comprobar comprensión. Un recuadro no sustituye el texto que lo rodea. Los símbolos aparecen después de la situación que intentan describir.

Cada bloque tiene tres niveles. La primera lectura construye intuición. La segunda convierte esa intuición en código o en un argumento preciso. La tercera analiza las condiciones bajo las cuales la técnica deja de funcionar. No necesitas completar los tres niveles en una sola sesión. Volver a una operación con más herramientas es parte del aprendizaje, no un fallo de la primera explicación.

0.1 Qué significa completar el curso

El primer hito consiste en generar C para programas elemento a elemento. El segundo incorpora bucles, índices, movimientos y reducciones. El tercero compara versiones lentas y rápidas con un contrato numérico común. El cuarto añade un runtime GPU y kernels que respetan la sincronización del dispositivo. El quinto construye y entrena un decoder Transformer. El sexto exige una extensión propia defendida con pruebas, medidas y limitaciones.

La frase del temario «cualquier modelo» se interpreta de forma técnicamente precisa: cualquier grafo expresable con las operaciones, tipos, formas y efectos que soporte el compilador, sujeto a memoria y recursos. No significa que unas pocas semanas produzcan compatibilidad automática con toda operación de cualquier framework, cualquier control de flujo y cualquier acelerador. Aprender a identificar una operación no soportada forma parte de construir un compilador correcto.

La referencia a código competitivo y a entrenar LLM de última generación se mantiene como **objetivo de ingeniería y de escalado**, no como un resultado garantizado por terminar una lista de ejercicios. El libro desarrolla GEMM, atención eficiente, precisión mixta, paralelismo, memoria de entrenamiento y la separación entre preentrenamiento y ajuste. Incluye una ruta de entrenamiento real de un modelo pequeño y un diseño de cómo llevar el mismo compilador hacia modelos mayores. No llama «SOTA» a una red de juguete ni atribuye rendimiento de H100 a un portátil.

Lo que se ha ejecutado y lo que no

La entrega conserva la validación original en CPU Linux y el entrenamiento de 75.584 parámetros. La revisión local del 1 de octubre añade ejecución CUDA bajo WSL2 en una RTX 5070 Ti Laptop: ejemplos numéricos, microbenchmarks, entrenamiento y WMMA. HIP sigue pendiente de un dispositivo compatible. El registro del apéndice y `code/reports/validation_20261001/` distinguen los resultados originales de los nuevos.

0.2 Un semestre con punto de partida honesto

El programa de semanas 1–10 presupone trabajo práctico continuado. Como aquí se empieza desde cero, se añade un bloque preparatorio sobre Python, C, terminal, matrices y números de coma flotante. Una planificación orientativa reserva entre veinte y treinta y cinco horas para esa preparación, según la experiencia real del estudiante. Es una estimación docente, no una medida experimental ni una promesa de aprendizaje idéntico para todas las personas.

Durante el semestre, una organización posible combina explicación, lectura de código, laboratorio y una sesión de depuración. El progreso se decide por competencias verificables, no por haber pasado de página. Si todavía no puedes calcular la dirección de un elemento de una matriz de dos filas, conviene repetir el capítulo de índices antes de estudiar una instrucción matricial GPU.

Etapa	Pregunta de control	Entregable
Preparación	¿Puedo ejecutar y comprobar un bucle?	Programa C y prueba manual
Semanas 1–2	¿Puedo transformar un grafo sin alterar su contrato?	UOps, simplificador, generador C
Semanas 3–4	¿Puedo convertir tensores en recorridos?	Bucles, movimientos, reducciones y rangeify
Semanas 5–6	¿Puedo diferenciar, entrenar y mejorar CPU?	Fisión, fusión, autodiff, MNIST y kernels rápidos
Semanas 7–8	¿Puedo ejecutar sin carreras en una GPU?	Runtime, mapeo de ejes y kernels

Etapa	Pregunta de control	Entregable
Semanas 9–10	¿Puedo construir y entrenar un decoder?	Formas simbólicas, modelo, optimizador y checkpoint
Semanas 11 en adelante	¿Puedo defender una extensión original?	Proyecto, informe y reproducción

0.3 Cómo utilizar el código entregado

El libro es autocontenido para leer: el listado completo del proyecto aparece al final. El paquete complementario evita transcribir miles de caracteres. Cada laboratorio indica una carpeta, una orden y una condición de éxito. Los fragmentos cortos del texto pueden ser ejemplos aislados; cuando no constituyen un programa completo se indica. El código del paquete es la referencia ejecutable de esta edición.

No copies todo el compilador el primer día intentando comprenderlo de golpe. Empieza con el primer bucle C. Después utiliza el proyecto para observar la salida del generador y relacionar cada fragmento con un capítulo. Un buen ejercicio consiste en predecir cuántos kernels tendrá una expresión antes de consultar el informe.

0.4 Fuentes, fechas y límites de esta investigación

El corte es el **1 de octubre de 2026**. Las páginas de proyectos pueden cambiar posteriormente. El recorrido central de tinygrad se ancla al commit `c3aec477b99d9bb87c54d91897cf60acd3f17441`, fechado el 30 de septiembre de 2026. Para otras páginas se identifica el recurso consultado y su estado visible, sin inventar un hash de código que no se haya recuperado. La bibliografía distingue artículos científicos, documentación y repositorios. [1]

Los artículos de arXiv se leen como trabajos con autores, métodos y un ámbito de evaluación; que estén en arXiv no demuestra revisión por pares. Una tabla de un proveedor es evidencia de lo que declara el proveedor. Una ejecución local es evidencia de una configuración concreta. Un razonamiento matemático tiene hipótesis que deben mantenerse al traducirlo a coma flotante.

La estética y el orden didáctico toman como referencia los apuntes de Teoría de Autómatas facilitados por el estudiante. Su contenido no se utiliza como fuente sobre GPU o aprendizaje automático. El temario de compiladores es el proporcionado en la solicitud; el desarrollo, los ejercicios y los proyectos son propios.

Programa del semestre: Compilers for Machine Learning

0.5 El encargo y la forma de trabajar

Esta ampliación, incorporada el **5 de octubre de 2026**, desarrolla el syllabus completo facilitado por el lector: un semestre práctico en el que cada estudiante construye su compilador desde programas elemento a elemento hasta el entrenamiento de modelos de lenguaje en GPU. La relación entre representación intermedia, hardware y estructura del modelo guía las decisiones. Se estudian reescritura de términos, generación de código, movimientos, fusión de kernels, jerarquías de memoria, arquitectura GPU, autodiferenciación y FlashAttention.

El proyecto se realiza individualmente o en equipos de dos. El lenguaje de implementación es libre. El syllabus establece la especificación y propone un formato sencillo de intercambio de UOps; no entrega código inicial ni impone cómo implementar el compilador. Lumbre y los tutoriales de este repositorio son material de consulta y contraste. Para seguir la modalidad de construcción desde cero, crea tu propio proyecto y resuelve cada entrega antes de consultar su solución en Lumbre.

Aunque no hay prerequisites formales, el curso original presupone dominio de sistemas equivalente a 15-213, soltura en C, cierta experiencia GPU, dominio del lenguaje elegido y capacidad para mantener un proyecto durante meses. El itinerario preparatorio del libro sirve para detectar y cubrir lagunas; no sustituye automáticamente esa experiencia. La modalidad presencial del syllabus incluye cuestionarios frecuentes y clases que no se publican en línea. Estas condiciones describen el curso de referencia; el repositorio no anuncia sesiones presenciales propias.

La entrega semanal se realiza **el miércoles antes de las 10:00, antes de clase**. El orden de las sesiones es miércoles, viernes y lunes. No se proporciona zona horaria ni fechas del calendario académico: deben fijarse al organizar una edición concreta. Cada entrega conserva las capacidades de las anteriores. Una regla de simplificación incorrecta en la primera semana puede invalidar gradientes y modelos meses después.

Gestionar el trabajo que todavía no se entiende

La advertencia del syllabus sobre el «slop» se convierte aquí en una práctica observable: toda transformación debe tener un contrato, un contraejemplo que podría hacerla fallar y una prueba de regresión. Poder generar mucho código no sustituye poder explicar qué hace. Un integrante debe poder defender cambios escritos por el otro y cualquier fragmento producido con ayuda automática.

0.6 El semestre de un vistazo

Semanas	Trabajo acumulado	Hito del syllabus
1–2	UOps, operaciones elementales, simbólico, reescrituras, renderer y runtime	Compilar y simplificar programas elemento a elemento a C
3–4	Bucles, movimientos, broadcasting, reducción y rangeify	Compilar grafos de modelos expresables a C correcto, inicialmente lento
5–6	Fisión, fusión, autodiff, memoria, hilos, upcasting, GEMM y convolución	Entrenar MNIST y aspirar a código CPU competitivo con el estado del arte
7–8	Kernels, GPU, mapeo de ejes, aceleradores y tensor cores	Aspirar a código CUDA/HIP competitivo con PyTorch
9–10	Modelos reales, formas simbólicas y entrenamiento	Implementar y entrenar un LLM
11 en adelante	Proyecto propio sobre el compilador	Implementar un artículo, portar a otro hardware o desarrollar una extensión

La tabla final del syllabus agrupa autodiff con modelos reales; el desarrollo semanal lo introduce en la **semana 5**. Lo aprenderemos entonces y lo utilizaremos de nuevo en las semanas 9–10. Igualmente, **CALL** aparece en la semana 1 aunque las llamadas a kernels de dispositivo se estudien después. Las jerarquías de memoria y la aceleración CPU culminan en la semana 6; no se pospone autodiff para acomodarlo al resumen.

0.7 Semana 1. UOps y reescrituras

Construye nodos inmutables con interfaz conceptual `UOp(op, src:tuple[UOp, ...], arg)` y hash-consing. Introduce la propiedad recursiva `dtype` con valores `bool`, `i32` y `f32`. El conjunto inicial es **PARAM**, **CALL**, **CONST**, **CAST**, **NEG**, **RECIP**, **EXP2**, **LOG2**, **ADD**, **MUL**, **CMPLT**, **CMPE**, **MAX**, **FLOORDIV**, **FLOORMOD** y **WHERE**. Para booleanos, **MUL**, **MAX** y **CMPE** significan **AND**, **OR** y **XOR**.

El miércoles se presentan curso, completitud de Turing, **UOp**, formato y primera tarea. El viernes se trabajan **ALU**, reescritura voraz de abajo arriba y plegado de constantes. El lunes se estudian llamadas, parámetros, simplificación simbólica y sistemas de álgebra computacional. El plegador debe poder evaluar programas constantes del subconjunto soportado; mencionar completitud de Turing en clase no convierte por sí solo ese subconjunto en un lenguaje universal.

Entrega un constructor, impresor y lector del subconjunto acordado, un verificador, un evaluador de referencia y un motor de reescrituras. Demuestra que una subexpresión pura repetida se comparte, que los parámetros mantienen su ámbito y que cada regla termina. Lectura: «UOps: construir un grafo de intenciones», simplificación simbólica y laboratorios 1–2. El contrato ampliado al final del libro concreta casos límite.

0.8 Semana 2. Renderer, runtime y memoria

Añade `void`, la propiedad `shape:tuple[int, ...]` y `addrspace` con `MEM` y `ALU`. Introduce `BUFFER`, `ALLOC`, `STACK`, `INDEX`, `LOAD`, `STORE`, `AFTER` y `LINEAR`. `BUFFER` representa memoria persistente global y no se define dentro de una llamada; `ALLOC` reserva almacenamiento local a la llamada. Un buffer externo puede pasarse como argumento de la función sin volver a declararlo como global dentro de su cuerpo.

El miércoles se estudian orden topológico, renderer y repaso de C. El viernes, runtime y gestión de memoria. El lunes, sistemas de tipos para forma, dtype y espacio de direcciones. La entrega genera, compila y ejecuta funciones C **sin bucles** que combinan inmediatos y memoria. No es necesario anticipar `RANGE` para demostrar una carga, una suma y una escritura.

La prueba decisiva llama dos veces a la función con datos distintos y comprueba memoria y valores. Incluye una dependencia de lectura después de escritura y rechazo de índices inválidos. Lectura: «Del grafo al C», planificación de memoria y laboratorio 3. No basta con imprimir C: la firma, los tamaños y la vida de los buffers forman parte de la entrega.

0.9 Semana 3. Bucles, GEMM y convolución

`RANGE` recibe como fuente el límite superior y como argumento el nombre del rango. Un límite de tres recorre 0, 1 y 2. `END` recibe (`passthrough`, `range`) y cierra el rango correspondiente. El renderer debe preservar anidamiento, acumuladores y valores disponibles al salir.

Escribe una reducción, una multiplicación de matrices rectangular y una convolución pequeña usando índices explícitos. Prueba rangos de longitud cero y uno donde el contrato los permita. Una suma vacía conserva su identidad; una reducción sin identidad definida requiere un diagnóstico. Lectura: bucles, reducción, GEMM y laboratorio de convolución. La semántica de bucles aparece aquí; la transformación automática de formas a bucles llegará en la semana siguiente.

0.10 Semana 4. Movimientos y rangeify

Incorpora `RESHAPE`, `EXPAND`, `PERMUTE`, `FLIP`, `PAD`, `SHRINK` y `REDUCE`, parametrizada por número de ejes y operación. Obtén GEMM y convolución componiendo movimientos y reducciones. El alumno debe poder seguir una coordenada desde la salida hasta cada entrada sin materializar todas las vistas.

Rangeify elimina las formas de las operaciones del grafo de cálculo bajado: el trabajo queda expresado con rangos, índices y operaciones escalares. El runtime sigue necesitando tamaños y metadatos de almacenamiento. La entrega conserva el grafo de alto nivel para contrastar valores y comprueba que el backend recibe su representación escalar esperada. Lectura: movimientos, broadcasting, layouts y laboratorio 3.

0.11 Semana 5. Fisión, fusión y autodiferenciación

`STAGE` representa un buffer temporal indexable. Úsalo para decidir cuándo materializar un valor compartido y cuándo fusionar su cálculo en el consumidor. Contar kernels sin contar

cómputo repetido puede premiar el plan equivocado.

Autodiff actúa sobre **grafos puros con formas**, antes de introducir efectos. No se aplica a **STORE**, **CALL**, **AFTER**, **RANGE** ni **END**. Genera el grafo de gradientes y bájalo después con los mismos mecanismos que el forward. El hito es entrenar el primer clasificador de MNIST con tu compilador. Los datos, la pérdida y el protocolo experimental deben documentarse; este hito es una entrega propuesta, no una ejecución MNIST ya incluida entre las evidencias de Lumbre.

Lectura: planificación de kernels, autodiff y laboratorio 8. La guía ampliada desarrolla el backward del broadcasting, un clasificador y las pruebas necesarias para evitar una falsa sensación de éxito basada solo en la pérdida.

0.12 Semana 6. Memoria, multihilo y kernels rápidos en CPU

Clasifica los rangos como **LOOP**, **UPCAST** o **THREAD**. Upcasting transforma trabajo de un eje en varias operaciones explícitas que el backend puede combinar o vectorizar; no significa convertir FP32 a FP64. Añade **GROUP**, que agrupa salidas `void` sin ordenar sus fuentes y tiene forma escalar `()`.

Trabaja localidad, tiling, acumuladores, hilos, conteo de FLOPs, GEMM, reducciones, convoluciones y atención con memoria acotada. La CPU sigue siendo el destino. La meta del syllabus es código C competitivo con el estado del arte: una afirmación así exige suite, referencia fuerte, precisión, hilos, hardware y medidas reproducibles. Lectura: CPU rápida, benchmarks, atención eficiente y proyecto C.

0.13 Semanas 7–8. GPU y aceleradores

Introduce kernels de dispositivo, runtime GPU, mapeo de ejes a bloques e hilos, jerarquía de memoria y tensor cores. Mantén un camino correcto para formas no compatibles con el kernel especializado. CUDA e HIP son destinos del programa; validar uno no valida automáticamente el otro.

La meta es competir con PyTorch en cargas acordadas. Compara operaciones equivalentes y declara si incluyes compilación, transferencias, conversiones y sincronización. Lectura: GPU básica, aceleradores, laboratorio 7 y el nuevo puente del tutorial CUDA. Las prácticas existentes de vectores, reducción, GEMM y streams ofrecen referencias para revisar el código que genere el compilador propio.

0.14 Semanas 9–10. Modelos reales y dimensiones simbólicas

Integra un modelo de lenguaje con embeddings, atención causal, normalización, bloque feed-forward, pérdida, gradientes, optimizador y checkpoints. Lleva las dimensiones simbólicas desde las restricciones del frontend hasta las comprobaciones de ejecución y la clave de caché. Un tamaño variable no puede invalidar silenciosamente un tile o una reserva.

Entrena primero una configuración pequeña que puedas depurar y después escala dentro del presupuesto disponible. Registra tokens, arquitectura, memoria, tiempo por paso y evaluación. La aspiración a LLM de última generación del syllabus requiere además datos, recursos y comparación externa; los resultados existentes del repositorio acreditan un decoder pequeño. Lectura: modelo de lenguaje, entrenamiento, escalado y laboratorios 9–11.

0.15 Semanas 11 en adelante. Proyecto y defensa

Escoge una contribución sobre el compilador: reproducir una idea de un artículo, portar a hardware distinto, implementar backward de atención por tiles, mejorar selección de kernels o ampliar el entrenamiento. Los proyectos A–E del libro dan ejemplos. El syllabus deja abierto el número de semanas finales; cualquier calendario de catorce semanas de este material es una propuesta organizativa.

Entrega código propio, pruebas, instrucciones, resultados, límites y una comparación que permita activar y desactivar la contribución. Para equipos de dos, ambos deben poder explicar un caso nuevo desde el tensor hasta la ejecución. Una defensa útil modifica una forma o una dependencia y pide predecir el efecto antes de ejecutar.

0.16 Qué se conserva entre entregas

Cada miércoles etiqueta una versión reproducible con especificación vigente, gramática admitida, ejemplos de IR, pruebas acumuladas y un informe breve. Al añadir una operación, registra sus propiedades, aridad, precondiciones y fase legal. Al añadir una optimización, conserva un programa mínimo que detecte su fallo más probable. Al añadir un backend, mantén los mismos valores de entrada y el mismo contrato numérico.

Los capítulos «Contrato de UOps» y «De rangeify al entrenamiento» amplían el syllabus con decisiones didácticas explícitas. El formato del mensaje original es un ejemplo y no una gramática completa. No se atribuyen al curso reglas de ABI, literales o evaluación que su texto no fija.

CAPÍTULO 1

Un programa que fabrica otro programa

La pregunta de este capítulo

¿Cómo puede una lista de números convertirse en una tarea para un compilador? Empezaremos ejecutando a mano. Después escribiremos instrucciones, y solo al final construiremos un programa que escriba esas instrucciones por nosotros.

1.1 Cuatro temperaturas y una regla

Tenemos los números 10, 12, 15 y 9. Queremos sumar dos a cada uno. La salida debe ser 12, 14, 17 y 11. No estamos ordenando los números ni sumándolos entre sí. Tampoco estamos pidiendo el promedio. Precisar la tarea es el primer paso de cualquier optimización: una versión que resuelve otro problema no es una versión más rápida.

Imagina cuatro casillas numeradas desde cero. La casilla 0 contiene 10; la 1 contiene 12; la 2 contiene 15; la 3 contiene 9. La numeración permite referirse a una posición sin escribir su valor. Esa posición se llama **índice**. El valor puede cambiar sin que cambie la posición.

Ejecución sin ordenador

En el paso 0 leemos 10 y escribimos 12 en la casilla de salida 0. En el paso 1 leemos 12 y escribimos 14 en la salida 1. Después hacemos lo mismo con 15 y con 9. Cada paso depende de una única entrada. Por tanto, el orden de esos cuatro pasos no afecta a la salida, siempre que cada uno escriba en su propia casilla y las entradas no se destruyan antes de leerlas.

Ya hemos descubierto una propiedad de paralelismo sin usar la palabra GPU. Las tareas son independientes porque los datos que necesitan no son resultados de otras tareas. Más adelante encontraremos operaciones donde esa independencia no existe, como acumular una suma en una única casilla.

1.2 Variables: etiquetas para valores

En Python podemos guardar un número con una etiqueta:

```
x = 10
y = x + 2
print(y)
```

La primera línea asocia `x` al número 10. La segunda calcula una suma y asocia su resultado a `y`. La tercera muestra 12. El signo `=` en un programa es una instrucción de asignación, no una

afirmación matemática eterna. Si después escribimos $x = 50$, no cambia retrospectivamente el cálculo anterior de y .

Una lista agrupa varias posiciones. La expresión `temperaturas[2]` accede a la tercera, porque empezamos a contar por cero. Esa convención puede resultar incómoda al principio, pero simplifica direcciones: la primera posición se encuentra a desplazamiento cero respecto al comienzo.

```
temperaturas = [10, 12, 15, 9]
salida = []
for x in temperaturas:
    salida.append(x + 2)
assert salida == [12, 14, 17, 11]
```

`for` repite el bloque sangrado para cada valor. `append` añade una casilla al final. `assert` pide detenerse si una condición no se cumple. En este ejemplo la aserción convierte una intención verbal en una comprobación automática. No prueba todos los casos posibles; prueba este caso concreto.

1.3 De la lista al array de C

C permite describir un array de valores del mismo tipo. Aquí utilizaremos `float`, un formato de coma flotante que estudiaremos después. El siguiente programa es completo.

```
#include <stdio.h>

int main(void) {
    float x[4] = {10, 12, 15, 9};
    float y[4];
    for (int i = 0; i < 4; i++) {
        y[i] = x[i] + 2.0f;
    }
    for (int i = 0; i < 4; i++) {
        printf("%.1f\n", y[i]);
    }
    return 0;
}
```

El primer `for` contiene tres decisiones: empezar en cero, seguir mientras el índice sea menor que cuatro y aumentar uno al terminar cada vuelta. La condición es $i < 4$, no $i \leq 4$. La casilla 4 no pertenece a un array de cuatro posiciones. Leerla o escribirla constituye un acceso fuera de límites.

Las llaves delimitan bloques. El punto y coma termina muchas instrucciones. La letra `f` en `2.0f` indica una constante de tipo `float`. `return 0` devuelve al sistema operativo un código de finalización satisfactoria. Estos detalles son sintaxis: permiten que el compilador interprete inequívocamente nuestro texto.

1.4 Compilar y ejecutar no son la misma acción

Guarda el programa como `temperaturas.c`. Compilar significa convertir ese texto en otro artefacto ejecutable. En un entorno Linux con un compilador C instalado:

```
cc -O0 temperaturas.c -o temperaturas
./temperaturas
```

La primera orden produce el archivo ejecutable. La segunda lo ejecuta. Si modificas el archivo C pero no vuelves a compilar, el ejecutable anterior no se actualiza por intuición. Si una compilación falla y después ejecutas un archivo antiguo que todavía existe, puedes estar observando el programa equivocado. Por eso los laboratorios conservan la orden, el código de salida y el hash del artefacto.

`-O0` desactiva gran parte de las optimizaciones del compilador C y facilita comenzar. Más adelante compararemos con `-O3`. Aumentar el nivel de optimización no convierte en correcto un acceso inválido ni vuelve segura una carrera entre hilos.

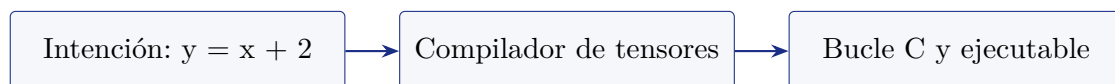
El primer error útil

Cambia temporalmente `i < 4` por `i <= 4`. El programa puede parecer funcionar, mostrar un valor extraño o fallar. Esa variabilidad no demuestra que el error sea inofensivo: has pedido una escritura que no pertenece al array. La ausencia de un fallo visible no equivale a seguridad de memoria.

1.5 Qué hace un compilador de aprendizaje automático

Un compilador C recibe instrucciones como bucles, llamadas y operaciones escalares. Nuestro compilador de ML recibirá intenciones más altas: sumar tensores, reorganizar sus ejes, multiplicar matrices o calcular una reducción. Su trabajo consiste en decidir qué bucles y accesos implementan esas intenciones.

Para las temperaturas, la intención es «cada salida es su entrada más dos». Esa descripción no obliga a recorrer de izquierda a derecha, usar una única CPU ni materializar resultados intermedios. El compilador puede escoger una ejecución siempre que preserve el contrato de entrada y salida.



No llamaremos compilador a cualquier función que invoque una biblioteca. Las llamadas a bibliotecas pueden formar parte legítima de un compilador, pero el trabajo de representación, análisis, transformación y generación debe quedar visible. En Lumbre construiremos esas etapas en lugar de ocultarlas detrás de `torch.compile`.

1.6 Un generador de texto ya es un primer paso

Este programa Python escribe un fragmento de C. Todavía no optimiza ni valida nada; sirve para entender que los programas también son datos.

```
def generar_suma(n: int, incremento: float) -> str:
    if n < 0:
        raise ValueError("n no puede ser negativo")
    return (
        f"for (int i = 0; i < {n}; i++) {{\n"
        f"    y[i] = x[i] + {incremento:.9g}f;\n"
        f"}}\n"
    )

print(generar_suma(4, 2.5))
```

El resultado es texto, no el array de salida. Ese texto puede colocarse dentro de una función C y compilarse. Hay ahora dos momentos distintos: el generador se ejecuta para construir código; el código construido se ejecuta después para procesar datos. Mezclarlos es una de las confusiones más frecuentes del curso.

La pequeña función tampoco es todavía un renderer robusto de constantes: para un valor como 2 podría imprimir `2f`, que no es la forma C que buscamos. Esta limitación es deliberada y local al ejemplo introductorio. El compilador completo utiliza literales hexadecimales de coma flotante y conserva un contrato de tipos. Aprenderemos por qué no basta con convertir cualquier objeto a texto.

1.7 Un problema, tres tiempos

El tiempo de construir el grafo, el tiempo de compilar el código y el tiempo de ejecutarlo responden a preguntas distintas. Supón que generar y compilar cuesta 0,3 segundos, mientras ejecutar tarda 0,001 segundos. Una sola llamada cuesta aproximadamente 0,301 segundos. Mil llamadas que reutilizan el ejecutable cuestan aproximadamente 1,3 segundos, sin contar otros gastos. No es correcto anunciar 0,001 segundos como tiempo de la primera llamada.

Estas cifras son un ejemplo aritmético inventado, no un benchmark. Su utilidad es mostrar la **amortización**: un coste inicial se reparte entre varias ejecuciones. La caché de compilación intenta evitar pagar ese coste otra vez para programas equivalentes bajo la misma configuración.

1.8 Ejercicios con respuesta explicada

Predecir antes de ejecutar

Cambia la regla por «multiplica por tres y resta uno». Calcula la salida para 2, 0 y -1. Después escribe la expresión Python y el cuerpo del bucle C. ¿Necesitas que la entrada tenga exactamente tres elementos?

Solución. Las salidas son 5, -1 y -4. La expresión es $3 \cdot x - 1$. El cuerpo C puede ser `y[i] = 3.0f*x[i] - 1.0f;`. El número de posiciones es independiente de la regla aritmética: el bucle utiliza una longitud `n`. El resultado de una posición no depende de las otras.

Reconocer el artefacto

Un programa Python devuelve la cadena `z[0] = x[0] + 2;`". ¿Ha calculado ya la salida? Un compilador produce un ejecutable sin errores. ¿Ha demostrado que la fórmula corresponde a la tarea pedida?

Solución. En el primer caso ha construido una instrucción, no la ha aplicado a datos. En el segundo ha aceptado el programa bajo las reglas del lenguaje y sus propias comprobaciones; no conoce necesariamente la intención humana. Una fórmula equivocada puede ser un programa perfectamente compilable.

Antes de continuar

Debes poder distinguir índice y valor, texto C y ejecutable, compilación y ejecución, una prueba concreta y una garantía general. Explica también por qué cuatro tareas independientes pueden repartirse sin cambiar su significado.

CAPÍTULO 2

Preparar el taller y aprender a leer un fallo

2.1 Una carpeta es parte del experimento

Antes de aprender instrucciones complejas, necesitamos un taller reproducible. Un resultado sin código, configuración y datos identificables es difícil de estudiar. La organización mínima separa fuente, pruebas, ejemplos e informes. No necesitas una plataforma empresarial para conseguirlo.

```
lumbre-curso/  
  lumbre/      # Compilador y representación intermedia  
  examples/    # Programas que usan el compilador  
  tests/       # Comprobaciones automáticas  
  kernels/     # Laboratorios de bajo nivel  
  reports/     # Resultados de ejecuciones
```

El archivo fuente es la receta; una prueba es una pregunta que debe responder; un informe registra lo que realmente ocurrió. Los nombres tienen una función: permiten que otra persona ejecute la misma tarea sin adivinar cuál de diez archivos llamados `final_nuevo.py` era el importante.

2.2 Terminal sin conocimientos previos

Una terminal recibe órdenes de texto. La carpeta de trabajo determina cómo se interpretan rutas relativas. `pwd` muestra dónde estás en Linux; `ls` enumera archivos; `cd` cambia de carpeta. Una ruta absoluta empieza desde la raíz del sistema; una relativa se interpreta desde la carpeta actual.

```
pwd  
ls  
cd lumbre-curso  
python3 --version  
cc --version
```

No copies los símbolos de un prompt como si fueran parte de la orden. En el libro los bloques contienen únicamente lo que debes escribir, salvo que se indique explícitamente una salida. Una línea con `#` en un script de shell suele ser un comentario. Una ruta con espacios debe ir entre comillas.

Para Windows se recomienda realizar los laboratorios POSIX en una distribución Linux dentro de WSL2 o en una máquina Linux separada. Esta recomendación responde a que el runtime

de esta edición genera bibliotecas `.so` y utiliza opciones de compilación POSIX. El código Python no transforma automáticamente ese runtime en uno nativo de Windows. Un puerto nativo tendría que gestionar DLL, compilador, ABI y carga dinámica.

La documentación de NVIDIA diferencia el controlador Windows que ofrece CUDA a WSL y los paquetes de desarrollo dentro de la distribución. No debes instalar por inercia un controlador de pantalla Linux dentro de WSL siguiendo instrucciones destinadas a un Linux instalado directamente. Confirma siempre las instrucciones oficiales para la combinación usada. [7]

2.3 Entorno Python aislado

Un entorno virtual separa dependencias de este proyecto de las de otros. Desde la carpeta del paquete:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
PYTHONPATH=. python -m pytest -q
```

`python -m pip` asegura que se utiliza el instalador asociado al intérprete seleccionado. `PYTHONPATH=.` hace visible el paquete de la carpeta actual en estos ejemplos. La versión de NumPy registrada en las pruebas de esta edición es 2.3.5 y el intérprete es Python 3.13.5. No se deduce que versiones distintas fallen; significa que esa fue la combinación comprobada.

Para un entorno compartido o de producción conviene fijar versiones y hashes mediante un sistema de bloqueo de dependencias. El libro no convierte «instala siempre la versión más reciente» en un procedimiento reproducible. El archivo de requisitos del paquete da una base; el registro de ejecución conserva las versiones observadas.

2.4 Funciones: una tarea con entradas y salida

Una función agrupa instrucciones y recibe datos. Este ejemplo devuelve una lista nueva sin modificar la original:

```
def desplazar(valores: list[float], delta: float) -> list[float]:
    return [x + delta for x in valores]

assert desplazar([1.0, 4.0], 2.0) == [3.0, 6.0]
```

Las anotaciones `list[float]` describen el tipo esperado. Por sí solas no hacen que Python compruebe todos los valores en ejecución. En la API del compilador escribiremos validaciones explícitas para formas, tipos, dimensiones y parámetros obligatorios.

Una función que cambia un objeto existente tiene un **efecto**. Una que produce un valor sin alterar estado visible puede tratarse como una función pura bajo su contrato. Esta distinción reaparecerá cuando intentemos reordenar operaciones. Reordenar dos sumas puras es diferente de reordenar una lectura y una escritura sobre la misma memoria.

2.5 Excepciones y códigos de salida

Un error útil debe decir qué contrato se ha violado. «No funciona» no identifica si falta un compilador, si las dimensiones son incompatibles o si una comparación numérica excede la tolerancia. En Python, una excepción interrumpe el flujo normal y conserva una traza de llamadas.

```
def longitud_valida(n: int) -> int:
    if not isinstance(n, int):
        raise TypeError("la longitud debe ser entera")
    if n < 0:
        raise ValueError("la longitud no puede ser negativa")
    return n
```

Una traza se lee desde el error final hacia la llamada que lo provocó. No es necesario comprender toda una biblioteca para reparar una forma incompatible: primero identifica el objeto y sus dimensiones. El compilador no debe ocultar la orden fallida del compilador C ni reemplazar todos los errores por una salida cero que parece correcta.

2.6 La primera prueba automatizada

Un test contiene preparación, ejecución y comparación. En `pytest` una función cuyo nombre comienza por `test_` puede ser descubierta automáticamente. No necesita imprimir «correcto» para pasar; termina sin que falle una aserción.

```
def desplazar(xs, d):
    return [x + d for x in xs]

def test_desplazamiento():
    entrada = [10, 12, 15, 9]
    salida = desplazar(entrada, 2)
    assert salida == [12, 14, 17, 11]
    assert entrada == [10, 12, 15, 9]
```

La segunda aserción comprueba que la entrada no se ha modificado. Es una propiedad diferente del resultado. Un algoritmo podría devolver la salida correcta y destruir por accidente unos datos que otra parte del programa necesita.

Para coma flotante usaremos comparaciones con tolerancia. No todas las diferencias son errores, pero tampoco toda diferencia pequeña es aceptable. Una tolerancia debe relacionarse con el tipo, el número de operaciones y la escala de los datos. El capítulo numérico prepara esa discusión.

2.7 Tres clases de prueba

Una prueba unitaria examina una pieza pequeña, por ejemplo el cálculo de strides. Una prueba de integración conecta piezas, como generar C, compilarlo, cargarlo y ejecutar una matriz. Una prueba de extremo a extremo verifica una tarea completa, como entrenar un modelo y reanudar su checkpoint.

Ninguna sustituye a las otras. Si únicamente probamos el entrenamiento completo, localizar una transposición errónea puede ser muy costoso. Si únicamente comprobamos sumas aisladas, podemos no detectar que el plan de memoria reutiliza un buffer antes de su último uso.

Una prueba que no basta

Para comprobar una multiplicación de matrices, llenar todo con unos es cómodo: el resultado de cada casilla suele ser el tamaño de la reducción. Sin embargo, ciertas transposiciones incorrectas siguen dando lo mismo. Añade matrices no cuadradas y valores distintos por fila y columna. El objetivo no es acumular ejemplos fáciles, sino hacer visibles decisiones diferentes.

2.8 Leer el código generado

Lumbre guarda el código fuente generado en su caché. El informe del programa incluye la ruta. Es importante abrirlo: un compilador educativo no debería convertirse en una caja negra justo cuando empieza a funcionar.

```
from lumbre import param, Program

x = param("x", (4,))
y = x + 2.0
with Program([y]) as programa:
    print(programa.report())
```

Esta construcción compila un programa pero todavía no ha inicializado **x**. Intentar ejecutarlo sin datos produce un error explícito. Inicializar un parámetro, compilar un grafo y ejecutar sus kernels son acciones distintas.

2.9 Seguridad básica del JIT

Un compilador JIT convierte datos estructurados en código ejecutable. Si acepta nombres, opciones o texto arbitrario de usuarios no confiables, aparece una superficie de ataque. En los ejercicios locales no ejecutamos fuentes descargadas sin inspección ni utilizamos `shell=True` para concatenar órdenes.

La práctica segura es pasar la orden al sistema como una lista de argumentos, limitar rutas, comprobar retornos, fijar un tiempo máximo y mantener una caché privada al usuario. Estas medidas no convierten el JIT en una sandbox. Ejecutar código nativo no confiable requiere aislamiento del sistema operativo y una política específica.

2.10 Git como cuaderno de decisiones

Un commit conserva una versión identificable. Antes de optimizar, guarda una versión correcta. Después de cambiar un pase, registra qué propiedad intentas mejorar. Si un resultado deja de coincidir, podrás comparar dos versiones pequeñas en lugar de reconstruir de memoria una semana de cambios.

```
git init
```

```
git add lumbre tests examples  
git commit -m "Compilador de referencia y primeras pruebas"
```

No incluyas claves, credenciales, datasets privados ni enormes cachés binarios por defecto. El hash de un commit identifica código, pero no identifica automáticamente el dataset, el driver ni las opciones de compilación. El pasaporte experimental del final del libro une esas piezas.

Antes de continuar

Localiza la carpeta actual, activa el entorno, ejecuta una prueba y abre un archivo generado. Explica la diferencia entre un fallo de instalación y un fallo numérico. Ninguna de esas tareas requiere todavía saber qué es un tensor core.

CAPÍTULO 3

Tensores: números con una organización

La idea antes del nombre

Un tensor del curso es una colección de números organizada por ejes. Una temperatura es un número; las temperaturas de una semana forman una fila; las de varias ciudades forman una tabla. Añadir ejes no añade magia: añade preguntas para localizar cada valor.

3.1 Escalar, vector y matriz

Un **escalar** es un único valor. Un **vector** es una secuencia con un eje. Una **matriz** tiene dos ejes, que imaginaremos como filas y columnas. Un tensor puede tener más ejes: lote, altura, anchura y canal, por ejemplo. En este libro «tensor» designa la estructura computacional de un array multidimensional; no necesitamos empezar por la definición geométrica de tensores de la física.

Una tabla de dos ciudades y tres días tiene forma $(2, 3)$. El primer número indica cuántas ciudades; el segundo, cuántos días. Contiene seis elementos, no cinco. La forma cuenta combinaciones: para cada una de las dos ciudades existen tres posiciones de día.

Ciudad / día	Día 0	Día 1	Día 2
Ciudad 0	10	12	15
Ciudad 1	8	9	11

El elemento de índices $(1, 2)$ vale 11. El rango, en el sentido de número de ejes usado por las APIs de arrays, es dos. No confundas este uso de «rango» con el rango algebraico de una matriz, que es otra propiedad.

3.2 Forma y número de elementos

Una forma se escribe como una tupla. Para $(2, 3, 4)$ hay dos grupos, cada uno con tres filas de cuatro números: veinticuatro elementos. Una forma escalar se representa por la tupla vacía $()$, y contiene un elemento. Una forma $(0, 3)$ contiene cero elementos. Es importante distinguir el escalar de un array vacío.

La fórmula después de contar

Si la forma es $(d_0, d_1, \dots, d_{r-1})$, el número de elementos es el producto de sus dimensiones: $d_0 d_1 \cdots d_{r-1}$. Para la forma sin ejes, el producto vacío se toma como uno. Si alguna dimensión vale cero, el producto vale cero.

Los casos vacíos revelan errores en compiladores: una suma sobre ningún elemento puede tener una identidad bien definida, mientras un máximo vacío necesita una convención adicional o un error. No añadiremos un acceso a la primera posición para «simplificar» el código cuando esa posición no existe.

3.3 Operaciones elemento a elemento

Sumar dos matrices de igual forma significa sumar posiciones correspondientes. Para $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ y $B = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$, la suma es $\begin{bmatrix} 11 & 22 \\ 33 & 44 \end{bmatrix}$. Ningún resultado utiliza toda una fila ni toda una columna; utiliza una pareja de valores.

Multiplicar elemento a elemento también empareja posiciones: el resultado sería $\begin{bmatrix} 10 & 40 \\ 90 & 160 \end{bmatrix}$. En muchas APIs se escribe $A * B$. La multiplicación matricial, que veremos después, hace otra cosa y se suele escribir $A @ B$. La diferencia no es un detalle sintáctico: cambia las dependencias y el coste.

```
import numpy as np
A = np.array([[1, 2], [3, 4]], dtype=np.float32)
B = np.array([[10, 20], [30, 40]], dtype=np.float32)
assert np.array_equal(A + B, [[11, 22], [33, 44]])
assert np.array_equal(A * B, [[10, 40], [90, 160]])
```

Aquí NumPy es un instrumento para comprobar una definición. Más adelante generaremos nuestros propios bucles para producir esas mismas salidas.

3.4 Broadcasting: reutilizar sin inventar dimensiones

Queremos sumar a cada ciudad una corrección distinta por día: $[1, 0, -1]$. La fila tiene forma $(3,)$ y la tabla $(2,3)$. Podemos reutilizar esa fila en ambas ciudades. No necesitamos almacenar físicamente dos copias para expresar la operación.

La regla general alinea las formas por la derecha. Dos dimensiones son compatibles si son iguales o si una de ellas vale uno. Una dimensión ausente se considera uno a efectos de esta comparación. Por tanto, $(2,3)$ y $(3,)$ se alinean como $(2,3)$ y $(1,3)$ y producen $(2,3)$.

Tres comparaciones completas

$(4,1,7)$ con $(3,7)$ se alinea con $(1,3,7)$ y produce $(4,3,7)$. En cambio, $(4,2)$ con $(3,)$ falla porque 2 y 3 no coinciden y ninguna vale uno. Finalmente, $(0,3)$ con $(1,3)$ produce $(0,3)$: no aparecen datos de la nada. Usar simplemente el máximo de cada pareja daría uno para la primera dimensión y sería incorrecto.

Broadcasting no significa «todas las formas se pueden combinar». Tampoco significa concatenar arrays. Es una regla concreta para reutilizar valores a lo largo de ejes de tamaño uno. En el backward, esa reutilización se transforma en una suma de contribuciones.

3.5 Reducciones: muchas posiciones producen una

Sumar los tres días de cada ciudad transforma una tabla (2,3) en una fila (2,): 37 y 28. Hemos reducido el eje de los días. Sumar todas las ciudades y todos los días produce el escalar 65. Mantener un eje de longitud uno, con `keepdim`, permite recordar qué posición dimensional se ha reducido.

La operación no es elemento a elemento porque un resultado depende de varios datos. En un programa secuencial utilizaremos un acumulador. En uno paralelo necesitaremos combinar resultados parciales. Cambiar el orden de esa combinación puede afectar a la coma flotante, aunque la suma matemática sea la misma.

```
A = np.array([[10, 12, 15], [8, 9, 11]], dtype=np.float32)
assert np.array_equal(A.sum(axis=1), [37, 28])
assert A.sum() == 65
assert A.sum(axis=1, keepdims=True).shape == (2, 1)
```

`axis=1` identifica el segundo eje, no la segunda fila. Esta diferencia merece una pausa: un eje es una dirección de organización; una fila concreta es una selección dentro de ella.

3.6 Producto escalar: comparar dos listas de características

Supón que un objeto tiene características [2,3] y damos importancia [4,5] a esas características. Multiplicamos parejas y sumamos: $2*4 + 3*5 = 23$. El resultado es un único número. A esta operación se la llama **producto escalar**.

La longitud de ambas listas debe coincidir. Cada posición tiene una pareja y todas las parejas contribuyen. La expresión compacta es $\sum_k a_k b_k$. El signo de suma significa repetir la multiplicación para cada índice permitido y acumularla; no introduce una operación distinta de la que acabamos de realizar.

3.7 Multiplicación matricial desde una sola casilla

Sea $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ y $B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$. A tiene dos filas y tres columnas; B tiene tres filas y dos columnas. El resultado tendrá dos filas y dos columnas. Para su casilla (0,0), multiplicamos la fila 0 de A por la columna 0 de B: $1*7 + 2*9 + 3*11 = 58$.

La casilla (0,1) vale $1*8 + 2*10 + 3*12 = 64$. La (1,0) vale $4*7 + 5*9 + 6*11 = 139$. La (1,1) vale $4*8 + 5*10 + 6*12 = 154$. Hemos construido todo el resultado sin esconder ninguna suma.

Las tres letras de GEMM

Usaremos M para las filas de A, K para sus columnas y N para las columnas de B. A tiene forma (M, K) , B tiene forma (K, N) y C tiene forma (M, N) . La dimensión compartida K es la que se reduce:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

Cada salida necesita K multiplicaciones y, según cómo contemos la inicialización, K o $K-1$ sumas. En informes de rendimiento es habitual contar aproximadamente $2MNK$ operaciones de coma flotante. Debes declarar esa convención al convertir tiempo en FLOP/s.

3.8 Transponer no es invertir

Transponer una matriz intercambia filas y columnas. La matriz $[[1,2,3],[4,5,6]]$ se convierte en $[[1,4],[2,5],[3,6]]$. La forma cambia de $(2,3)$ a $(3,2)$. No estamos calculando la inversa algebraica ni cambiando los valores.

Un compilador puede representar la transposición como un cambio de cómo calcula direcciones, sin copiar datos inmediatamente. Sin embargo, que una vista no copie no significa que todos sus consumidores accedan con la misma eficiencia. Una lectura que antes recorría posiciones contiguas puede pasar a saltar distancias mayores.

3.9 Lotes de matrices

Una red suele procesar varios ejemplos a la vez. Si A tiene forma (B,M,K) y $Bmat$ tiene forma (B,K,N) , hay B multiplicaciones independientes. El resultado es (B,M,N) . La letra B de lote no debe confundirse con el nombre de la segunda matriz; por eso en explicaciones largas escribiremos «lote» o `batch`.

Lumbre admite multiplicaciones con dos o más ejes y broadcasting de los ejes de lote. No admite por esa misma función todas las convenciones especiales que una API externa pueda tener para vectores de rango uno. La firma de una operación forma parte del contrato, incluso cuando el nombre coincide entre bibliotecas.

3.10 Ejercicios resueltos

Forma, operación y resultado

Tienes X de forma $(5,4)$ y un sesgo de forma $(4,)$. ¿Qué forma tiene X más el sesgo? Después multiplicas por W de forma $(4,3)$. ¿Qué forma sale? ¿Cuántas multiplicaciones escalares requiere esta multiplicación matricial?

Solución. El sesgo se reutiliza en las cinco filas, por lo que la suma conserva $(5,4)$. La multiplicación por W produce $(5,3)$. Cada una de sus quince salidas acumula cuatro productos: sesenta multiplicaciones. El sesgo no altera la dimensión que debe coincidir con las filas de W .

Una igualdad aparente

Con una matriz identidad, ciertas multiplicaciones dejan la otra matriz igual. ¿Es una buena única prueba para un compilador de GEMM? ¿Y con matrices llenas de cero?

Solución. Son pruebas útiles de casos particulares, pero insuficientes. Un error que omita productos o invierta índices puede quedar oculto por ceros, simetrías o patrones especiales. Añade datos no simétricos, dimensiones distintas y un ejemplo calculado a mano.

Antes de continuar

Calcula una casilla de GEMM, explica una reducción y decide compatibilidad de broadcasting. Debes poder hacer estas tres tareas sin recurrir a «lo hace NumPy». Las siguientes transformaciones del compilador se apoyan en esa comprensión.

CAPÍTULO 4

Los números de la máquina no son los números de la pizarra

4.1 Una regla con pocas marcas

Imagina una regla que solo tiene marcas de centímetro. Una longitud de 2,4 centímetros debe aproximarse al utilizarla. La longitud real no cambia, pero la representación tiene precisión limitada. Los formatos de coma flotante funcionan con otra distribución de marcas: cerca de números pequeños hay más detalle absoluto; a medida que crece la magnitud, la separación entre valores representables también crece.

La metáfora no cubre todos los detalles, pero aclara la idea central: guardar y operar con un número puede introducir redondeo. Por eso una transformación correcta sobre números reales puede no conservar exactamente el programa que ejecuta una CPU o una GPU.

4.2 Bits, bytes y tipos

Un bit distingue dos estados. Ocho bits forman un byte. Un `float32` ocupa cuatro bytes y un `float64` ocho. La memoria disponible se mide en bytes, mientras la forma de un tensor cuenta elementos. Confundir ambas unidades produce presupuestos de memoria erróneos.

Una matriz de 1024 por 1024 elementos `float32` contiene 1.048.576 valores y ocupa 4.194.304 bytes, es decir, 4 MiB. El prefijo MiB indica potencias de 1024; MB suele indicar un millón de bytes. Si comparas cifras de herramientas distintas, comprueba qué unidad utiliza cada una.

Un tipo numérico especifica más que el tamaño. También define cómo interpretar los bits: entero, coma flotante, máscara o puntero. Los mismos cuatro bytes pueden representar cosas radicalmente diferentes. Cambiar la etiqueta sin transformar el contenido es un `bitcast`, no una conversión numérica ordinaria.

4.3 Signo, exponente y fracción

Un formato de coma flotante representa aproximadamente un signo, una parte significativa y una escala de potencia de dos. El exponente controla la escala; la fracción almacena detalle dentro de esa escala. FP32 dispone de ocho bits de exponente y veintitrés bits de fracción almacenada, con una unidad implícita para números normales. BF16 y FP16 reparten de forma diferente sus dieciséis bits. [8]

No necesitas memorizar todas las codificaciones para escribir tu primer kernel. Sí debes entender que **rango** y **precisión** son propiedades diferentes. Un formato puede representar

magnitudes grandes y ofrecer pocos dígitos de detalle; otro puede distinguir mejor números cercanos pero desbordarse antes.

Formato	Bits totales	Exponente	Fracción almacenada
FP32	32	8	23
FP16	16	5	10
BF16	16	8	7

Esta tabla describe los formatos habituales usados en el curso. No convierte FP8 o FP4 en nombres de una única codificación universal. Sus variantes requieren especificar exponente, fracción, valores especiales, escala y reglas de conversión.

4.4 El orden de una suma puede cambiar el resultado

En números reales, $(a+b)+c$ y $a+(b+c)$ son iguales. En coma flotante las sumas intermedias se redondean. Para una magnitud muy grande y otra pequeña, la pequeña puede desaparecer al sumar porque no hay una marca representable cercana que la conserve.

```
import numpy as np
a = np.float32(1e20)
b = np.float32(-1e20)
c = np.float32(3.0)
izquierda = np.float32(np.float32(a + b) + c)
derecha = np.float32(a + np.float32(b + c))
print(izquierda, derecha)
```

En el comportamiento FP32 ordinario de este ejemplo, la primera agrupación produce 3 y la segunda 0. El punto importante no es memorizar esas magnitudes, sino localizar el redondeo que pierde información. Un compilador que reordene sumas debe tener permiso para cambiar esa semántica o aceptar un criterio aproximado declarado.

Una reescritura no es una identidad porque resulte familiar

Factorizar $a*b + a*c$ como $a*(b+c)$ cambia operaciones y redondeos. Reemplazar una división por una multiplicación aproximada por el recíproco también puede cambiar el error. Estas transformaciones requieren un contrato numérico, no una confianza general en el álgebra escolar.

4.5 Infinito, NaN y ceros con signo

La coma flotante incluye valores especiales. Un desbordamiento puede producir infinito. Operaciones indefinidas pueden producir NaN, «no es un número». Existen además cero positivo y cero negativo, que pueden distinguirse en ciertas operaciones y observaciones.

La reescritura $x * 0 \rightarrow 0$ parece inocente. Sin embargo, con x infinito el producto puede ser NaN; con NaN también puede mantenerse NaN; el signo del cero puede importar. Por eso

Lumbre separa las simplificaciones exactas de índices enteros de las posibles optimizaciones numéricas de tensores.

Comparar NaN con una tolerancia habitual tampoco funciona como comparar dos números finitos. Un test que decide «todos los resultados son aproximadamente iguales» debe especificar qué hacer con NaN e infinitos. Aceptar NaN en ambas salidas puede esconder un error que vuelve inválido todo el entrenamiento.

4.6 FMA: una multiplicación y una suma con un único redondeo

Una instrucción fused multiply-add calcula una expresión de la forma $\mathbf{a}*\mathbf{b}+\mathbf{c}$ redondeando una sola vez el resultado combinado. Una multiplicación seguida de una suma ordinarias puede redondear dos veces. FMA suele ser valiosa para rendimiento y precisión, pero no siempre reproduce bit a bit la secuencia no fusionada. [8]

El backend CPU de referencia de Lumbre utiliza `-ffp-contract=off`. Esto ayuda a mantener una política explícita al comparar recorridos. No afirma que FMA sea mala. En un proyecto de rendimiento podrás habilitarla, medir su efecto y cambiar la definición de equivalencia de manera documentada.

El compilador también debe distinguir la contracción de una multiplicación-suma de la fusión de kernels. Podemos evitar escribir un tensor intermedio en memoria sin necesariamente introducir FMA; son transformaciones relacionadas, pero no idénticas.

4.7 Error absoluto y relativo

El error absoluto compara la distancia: $|\text{resultado} - \text{referencia}|$. El relativo compara esa distancia con una escala, normalmente ligada al valor de referencia. Si la referencia está cerca de cero, dividir por ella puede producir una medida enorme o indefinida aunque la diferencia absoluta sea pequeña.

Una regla práctica frecuente acepta una casilla si su error absoluto es menor que una tolerancia absoluta más una tolerancia relativa multiplicada por la magnitud de la referencia. Las tolerancias no deben escogerse después de observar el peor error para forzar un aprobado. Primero fija el contrato y luego ejecuta el test.

$$|y - \hat{y}| \leq \text{atol} + \text{rtol} |\hat{y}|.$$

Leer una tolerancia con números

Si la referencia es 100, `atol=0.001` y `rtol=0.0001`, el margen es 0,011. Si la referencia es cero, el margen es 0,001. La primera componente protege los valores pequeños; la segunda adapta la comparación a la escala. No se deduce que estos valores sean adecuados para cualquier modelo.

4.8 Reducciones largas y acumulación

Sumar mil productos acumula más oportunidades de redondeo que sumar dos. Una reducción por árbol cambia el orden respecto al recorrido secuencial. Un tensor core puede multiplicar entradas de menor precisión y acumular en FP32. Hay que describir todos esos pasos al comparar con una referencia de FP64.

Una referencia de mayor precisión es útil para aproximar la respuesta real, pero no equivale necesariamente a la semántica exacta de la API que intentamos reproducir. Conviene tener dos preguntas separadas: ¿reproduce el contrato del framework? ¿Qué error tiene respecto a una referencia más precisa?

Técnicas como acumulación compensada, sumas por bloques o acumuladores más anchos pueden reducir error con costes distintos. El compilador necesita conocer cuándo la aplicación valora reproducibilidad bit a bit, estabilidad numérica o velocidad dentro de una tolerancia.

4.9 Softmax y el truco que evita un desbordamiento

Softmax convierte puntuaciones en pesos positivos que suman uno. Primero exponencia cada puntuación y después divide entre la suma de exponenciales. Si las puntuaciones son 1000 y 1001, calcular directamente esas exponenciales puede desbordar. Restar el máximo convierte el problema en exponenciar -1 y 0.

No cambia el resultado matemático porque todos los términos se multiplican por el mismo factor, que se cancela entre numerador y denominador. Sí cambia favorablemente el comportamiento de la máquina al evitar magnitudes enormes. En el capítulo de atención construiremos una versión que actualiza ese máximo mientras procesa bloques.

$$\frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i-m}}{\sum_j e^{x_j-m}}, \quad m = \max_j x_j.$$

Aquí cada símbolo ya tiene un trabajo: i selecciona una salida; j recorre todas las puntuaciones; m es el mayor valor. La fórmula no es un adorno que se añade a la implementación: explica por qué la transformación conserva la intención.

4.10 Qué comprobar antes de optimizar

Una batería numérica útil incluye valores pequeños, magnitudes desiguales, signos opuestos, ceros, dimensiones no múltiplos del tile y casos que estresen las reducciones. Los valores especiales se prueban si pertenecen al dominio de la operación; en otro caso se valida y documenta su exclusión.

Para un entrenamiento, controla además pérdida finita, norma de gradiente, magnitud de parámetros y estados del optimizador. Una pérdida decreciente no demuestra que todos los gradientes sean correctos: algunas direcciones equivocadas pueden mejorar temporalmente una tarea diminuta.

Clasificar una transformación

¿Es siempre válida la regla $n \neq 0 \rightarrow 0$ cuando n es un índice entero matemático sin desbordamiento? ¿Y cuando n es un tensor FP32 que puede contener infinito? ¿Es el mismo problema que cambiar el orden de dos bucles independientes?

Solución. En el dominio entero indicado sí es una identidad exacta. En FP32 con valores especiales no lo es en general. Cambiar el orden de bucles independientes puede conservar cada operación individual, mientras reordenar una reducción cambia la secuencia de redondeos. Hay que identificar el dominio y las dependencias, no aplicar una regla por parecido visual.

Antes de continuar

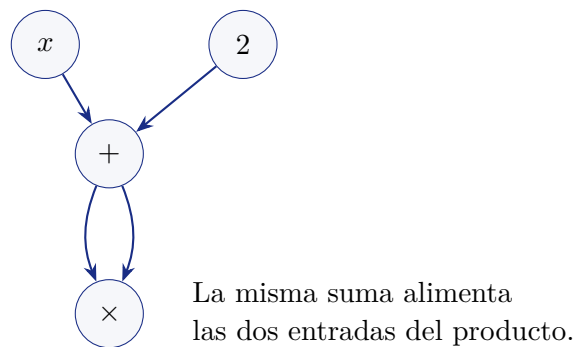
Distingue almacenamiento y valor, error absoluto y relativo, precisión y rango, fusión de kernels y FMA. Explica por qué el compilador tiene que conocer el contrato de coma flotante antes de simplificar una expresión.

CAPÍTULO 5

UOps: construir un grafo de intenciones

5.1 De una fórmula escrita a piezas conectadas

Queremos calcular $y=(x+2)*(x+2)$. Un texto repite dos veces $x+2$. Un dibujo puede mostrar una sola suma cuyo resultado se utiliza dos veces. Esa representación conserva explícitamente la dependencia: la multiplicación necesita la salida de la suma, y la suma necesita x y la constante dos.



Cada pieza es una operación con sus entradas. En la tradición de tinygrad se utiliza el nombre **UOp**. Lumbre toma ese nombre didáctico, pero su estructura y su conjunto de operaciones son propios. No prometemos compatibilidad de código con la API interna de tinygrad. [2]

5.2 Qué información necesita un nodo

Para interpretar una suma necesitamos saber que es una suma, cuáles son sus operandos, qué forma tiene su salida y qué tipo numérico utiliza. Una constante necesita además su valor. Una permutación necesita el orden de los ejes. Un parámetro necesita un nombre que conecte el grafo con los datos de ejecución.

```
from dataclasses import dataclass

@dataclass(frozen=True, eq=False)
class Nodo:
    op: str
    shape: tuple[int, ...]
    src: tuple["Nodo", ...] = ()
    arg: object = None
    dtype: str = "float32"
```

Este es un modelo explicativo reducido. El listado completo añade utilidades y validaciones. `frozen=True` expresa que no modificaremos los campos después de crear el nodo. La inmutabilidad facilita reutilizar nodos y razonar sobre transformaciones: una suma antigua no cambia por accidente cuando otro pase la inspección.

`src` contiene referencias a otros nodos, no copias de sus valores numéricos. El grafo describe una computación pendiente. Crear el nodo de una suma no exige tener todavía el array que sumaremos.

5.3 Un grafo dirigido sin ciclos

Las aristas van de las entradas hacia sus usos. Si ninguna operación termina dependiendo de sí misma a través de un camino, tenemos un grafo dirigido acíclico, o DAG. El nombre parece abstracto, pero aquí significa que podemos ordenar las operaciones para calcular antes sus entradas.

El grafo de una ejecución tensorial finita no tiene por qué representar todos los bucles del programa original como ciclos. Puede contener una operación de reducción que más adelante se convierta en un bucle. También puede construirse un grafo distinto para cada iteración de entrenamiento, reutilizando su estructura y cambiando parámetros.

Los bucles y el control de flujo general requieren representaciones adicionales si queremos compilarlos sin desplegar cada paso. Lumbre trabaja con grafos estáticos de operaciones tensoriales; esta decisión reduce la superficie inicial y hace visibles las extensiones necesarias.

5.4 Orden topológico explicado como lista de tareas

Imagina una receta: primero necesitas ingredientes, después una mezcla, después el horno. Un orden topológico es una lista donde toda tarea aparece después de sus requisitos. Para $(x+2)^2$, un orden válido es `x`, constante, suma, multiplicación.

Un recorrido en profundidad puede visitar entradas antes de añadir un nodo a la lista. Hay que recordar qué nodos ya se han incluido, porque una misma suma puede utilizarse dos veces. De lo contrario repetiríamos trabajo y podríamos contar mal los usos.

```
def ordenar(salidas):
    vistos, resultado = set(), []
    def visitar(nodo):
        if nodo in vistos:
            return
        vistos.add(nodo)
        for entrada in nodo.src:
            visitar(entrada)
        resultado.append(nodo)
    for salida in salidas:
        visitar(salida)
    return resultado
```

Este fragmento supone que el grafo ya es acíclico y es pequeño. Un frontend abierto debe detectar ciclos o impedirlos por construcción. La versión ejecutable usa un recorrido iterativo para no depender de la profundidad máxima de recursión de Python.

5.5 Identidad frente a igualdad estructural

Dos objetos pueden describir la misma suma sin ser el mismo objeto. Compartirlos permite representar una expresión común una sola vez. A esta técnica se la suele llamar **hash-consing**: construir una clave inmutable y consultar si ya existe un nodo con esa descripción.

En Lumbre la clave incluye operación, forma, entradas, argumento y tipo. Las entradas se comparan por identidad de nodo dentro del grafo. Si construimos repetidamente una misma operación sobre las mismas entradas, el constructor con caché puede devolver el mismo nodo.

No podemos compartir indiscriminadamente operaciones con efectos. Dos lecturas de un buffer separadas por una escritura no tienen necesariamente el mismo valor. La deduplicación de operaciones puras es una cosa; el control de dependencias de memoria es otra.

Contar usuarios correctamente

En $z=a*a$, el nodo a aparece dos veces como entrada de la misma multiplicación. Si contamos usos para decidir materialización, hay dos aristas aunque solo haya una operación consumidora. Un algoritmo debe decidir si necesita contar aristas o consumidores distintos y utilizar esa definición de manera consistente. Lumbre cuenta referencias de entrada para su heurística de fusión.

5.6 El frontend verifica el significado

El constructor de suma calcula la forma resultante mediante broadcasting. El de `matmul` comprueba la dimensión K . El de permutación valida que cada eje aparezca una vez. Cuanto antes detectemos un contrato inválido, más comprensible será el error.

Un `renderer` no debería descubrir por primera vez que las matrices no encajan cuando ya está escribiendo una llamada de bajo nivel. La separación deseable es: el frontend define y valida una operación; los pases conservan sus invariantes; el backend la implementa para un dispositivo concreto.

```
from lumbre import param
A = param("A", (2, 3))
B = param("B", (3, 4))
C = A @ B
assert C.shape == (2, 4)
```

Esto construye un nodo con forma $(2,4)$. No multiplica arrays en Python. Para ejecutar, tendremos que crear un programa, inicializar A y B , y llamar a su runtime.

5.7 Por qué la forma es información del compilador

Conocer dimensiones permite calcular cuánto espacio reservar, qué índices son válidos y qué bucles generar. Una dimensión simbólica introduce otra pregunta: ¿qué sabemos sobre su valor y qué debemos comprobar en ejecución? El primer compilador utiliza formas estáticas para separar ambas dificultades.

Estático no significa constante: A y B pueden cambiar en cada llamada manteniendo sus formas. Así se actualizan pesos sin reconstruir dimensiones.

5.8 Parámetros y constantes

Un parámetro recibe datos al ejecutar; una constante forma parte del programa. El incremento dos puede incorporarse al código o leerse como parámetro. La primera opción especializa más; la segunda reutiliza el ejecutable para distintos valores.

Especializar cada longitud, escala y valor multiplica variantes, caché y compilaciones. La API debe distinguir qué valores identifican el programa y cuáles son datos de ejecución.

5.9 Representación impresa y representación interna

Imprimir el grafo ayuda a depurar, pero sus identificadores pueden depender del orden de creación. Una clave de caché estable utiliza contenido canónico y configuración, no direcciones de objetos Python ni un texto de depuración accidental.

Lumbre utiliza el código generado y la identidad del toolchain para su caché de módulos. Esa estrategia es sencilla y visible. No es la única posible: compiladores grandes también almacenan IR serializada, resultados de autotuning y artefactos intermedios con versiones propias.

5.10 Ejercicios de construcción

Dibujar antes de programar

Dibuja $r = (a+b) * (a+b) + c$. ¿Qué nodos pueden compartirse? Si a, b y c tienen forma (8,), ¿qué forma tiene cada nodo? ¿En qué orden podrías evaluarlos?

Solución. Los parámetros a, b y c son hojas. La suma $a+b$ puede representarse una vez y alimentar las dos entradas del producto. El último nodo suma ese producto y c. Todos tienen forma (8,). Un orden válido es a, b, suma, producto, c, suma final; c también puede aparecer antes porque no depende de los otros cálculos.

Encontrar el contrato ausente

Alguien construye un nodo `matmul` sin comprobar formas y afirma que el backend decidirá qué significa. ¿Qué problema crea? ¿Basta con que el producto total de elementos coincida?

Solución. El mismo nodo podría representar cálculos diferentes o accesos inválidos. La igualdad del número total de elementos no determina las dimensiones de contracción. El frontend debe fijar una semántica inequívoca antes de que el backend escoja una implementación.

Antes de continuar

Explica UOp, DAG, orden topológico, parámetro, constante y operación pura con el ejemplo de una sola fórmula. Después localiza esas ideas en `lumbre/ir.py` sin intentar memorizar todas sus líneas.

CAPÍTULO 6

Del grafo al C: renderer, ABI y primera ejecución

6.1 Traducir sin cambiar de tarea

Ya sabemos representar $y=(x+2)*3$. Ahora necesitamos instrucciones que calculen sus valores. Un **renderer** recorre una representación y escribe código en un lenguaje objetivo. En el primer backend el objetivo es C: bucles, accesos a memoria y operaciones escalares.

Para cuatro elementos podemos generar un bucle que calcule toda la fórmula por posición. No hace falta crear arrays intermedios para $x+2$ si ninguna otra parte necesita observarlos. La representación tensorial se transforma en una expresión escalar dentro de un recorrido.

```
void desplazar_escalar(const float* x, float* y, long n) {
    for (long i = 0; i < n; i++) {
        y[i] = (x[i] + 2.0f) * 3.0f;
    }
}
```

El código tiene un contrato: x apunta a n valores legibles; y apunta a n posiciones escribibles; las reglas de aliasing que adoptemos deben respetarse. No basta con imprimir una fórmula correcta si los punteros no apuntan al espacio adecuado.

6.2 Qué es un puntero sin metáforas misteriosas

Un puntero contiene una dirección de memoria. Podemos imaginar la memoria como una calle de bytes y el puntero como el número del primer portal de un array. Para acceder al elemento i de `float*`, C calcula un desplazamiento de i elementos del tamaño de `float`, no de i bytes.

Si un array FP32 comienza en una dirección A , su elemento 3 comienza doce bytes después. Es importante mantener separadas dos unidades: índices en elementos y desplazamientos de memoria en bytes. El planificador de memoria de Lumbre calcula offsets en bytes; las expresiones de acceso dentro de un tensor utilizan índices de elementos.

6.3 Firmas y ABI

Una firma describe argumentos y resultado de una función. La **ABI**, interfaz binaria de aplicación, define cómo se transmiten esos argumentos y resultados en el código máquina de una plataforma. Python y C tienen que ponerse de acuerdo para que una llamada no interprete una dirección como un entero pequeño o un número de coma flotante como un puntero.

En `ctypes` declaramos tipos explícitamente. El siguiente fragmento ilustra una función que recibe dos punteros y una longitud; no incluye la compilación de la biblioteca.

```
import ctypes as C
biblioteca = C.CDLL("./operaciones.so")
funcion = biblioteca.desplazar_escalar
funcion.argtypes = [C.POINTER(C.c_float), C.POINTER(C.c_float), C.c_long]
funcion.restype = None
```

No conviene suponer que `long` tiene el mismo tamaño en todos los sistemas. En el código del proyecto se prefieren tipos de anchura explícita para índices y se define la firma del runtime como un puntero base. La portabilidad necesita comprobar ABI, alineación y plataforma, no solo cambiar la extensión del archivo.

6.4 Compilar una biblioteca compartida

Una biblioteca compartida contiene funciones cargables desde otro programa. Para un ejemplo Linux:

```
cc -O3 -std=c11 -shared -fPIC operaciones.c -o operaciones.so
```

`-shared` pide una biblioteca compartida y `-fPIC` genera código adecuado para cargarse en distintas direcciones. Lumbre añade las opciones numéricas y la biblioteca matemática que necesita. La orden real se construye como una lista de argumentos; se captura su salida y se comprueba el código de retorno.

El hecho de que la compilación termine solo garantiza que el compilador C ha producido un artefacto. Todavía faltan carga, inicialización y ejecución. Algunas incompatibilidades aparecen al cargar una biblioteca; otras, al ejecutar una instrucción que el procesador no soporta.

6.5 Un renderer recursivo reducido

Para operaciones puras elemento a elemento, podemos traducir cada nodo a una expresión. Este fragmento es un ejemplo aislado de estructura, no el renderer completo de Lumbre.

```
def expresion(u, i):
    if u.op == "param":
        return f"{u.arg}[{i}]"
    if u.op == "const":
        return literal_float(u.arg)
    if u.op == "add":
        return f"({expresion(u.src[0], i)} + {expresion(u.src[1], i)})"
    if u.op == "mul":
        return f"({expresion(u.src[0], i)} * {expresion(u.src[1], i)})"
    raise NotImplementedError(u.op)
```

`literal_float` debe imprimir una constante válida del tipo requerido. El renderer real también traduce índices de broadcasting, movimientos, reducciones y operaciones materializadas. La

recursión es una forma cómoda de mostrar dependencias, pero un grafo grande necesita controlar profundidad, duplicación y coste de generación.

6.6 Paréntesis y precedencia

La expresión $a+b*c$ no significa $(a+b)*c$. Un generador de código debe conservar la estructura del grafo y no confiar en que la forma impresa se «parezca» a la fórmula. Añadir paréntesis explícitos en un renderer inicial simplifica la corrección, aunque produzca texto más largo.

También hay que evitar inyectar nombres arbitrarios en el código. Lumbre utiliza identificadores internos para punteros y valida los parámetros mediante el grafo. Un usuario no debería poder introducir una instrucción C escondida en el nombre de un tensor.

6.7 Fusión: un recorrido en lugar de varios

Considera $a=x+2$, $b=a*3$, $y=b-1$. Una implementación separada realiza tres recorridos y escribe a y b en memoria. Una fusionada calcula $(x[i]+2)*3-1$ y escribe únicamente y . El número de operaciones aritméticas puede ser el mismo, pero disminuye el tráfico de memoria intermedia.

La fusión no siempre gana. Una expresión compartida por varios consumidores puede repetirse si se inserta en cada uno. Una expresión excesivamente grande puede aumentar el tiempo de compilación o la presión de registros. Por eso Lumbre utiliza una heurística limitada de profundidad y usos, en lugar de fusionar indiscriminadamente todo el grafo.

Contar bytes sin medir todavía

Para N elementos FP32, cada recorrido que lee un array y escribe otro mueve aproximadamente $8N$ bytes a nivel de accesos solicitados. Tres recorridos elementales requieren aproximadamente $24N$ bytes; uno fusionado, $8N$. La cifra es un modelo de tráfico solicitado, no una medición exacta de DRAM: cachés, escrituras y otros detalles pueden cambiar el tráfico físico.

6.8 Primera ejecución completa con Lumbre

```
import numpy as np
from lumbre import param, Program

x = param("x", (4,))
y = (x + 2.0) * 3.0 - 1.0
with Program([y]) as programa:
    resultado = programa.run({"x": np.array([1, 2, 3, 4], dtype="float32")})[0]
    np.testing.assert_array_equal(resultado, [8, 11, 14, 17])
    print(programa.report())
```

`Program([y])` declara qué salidas necesitamos conservar. `run` inicializa x con los datos recibidos, ejecuta el módulo y devuelve una copia de cada salida solicitada. La lista externa permite programas con varias salidas, como pérdida, norma de gradiente y nuevos parámetros.

La copia de salida es una decisión del runtime didáctico: evita devolver una vista que cambie

inesperadamente en la siguiente ejecución. Cuando medimos únicamente kernels residentes podemos pedir `read=[]` para no incluir esa copia. El nombre de la medición debe reflejarlo.

6.9 Caché de compilación

Si el código generado, el compilador y las opciones coinciden, podemos reutilizar una biblioteca existente. Una clave incompleta es peligrosa. Dos módulos con el mismo texto pero compilados para arquitecturas distintas no son necesariamente intercambiables. Tampoco lo son variantes con contratos numéricos incompatibles.

La clave de Lumbre incluye fuente, ruta y versión del compilador, opciones, máquina y sistema. Se escribe primero un artefacto temporal y después se publica mediante renombrado. Esto reduce la posibilidad de observar un archivo a medio escribir. No convierte la caché en un almacén distribuido seguro ni resuelve por sí solo todos los casos de concurrencia.

6.10 Errores que deben resultar visibles

Un compilador correcto rechaza una operación no soportada en lugar de devolver ceros. Un runtime correcto detecta parámetros sin inicializar. Un test correcto verifica salida y forma. Una herramienta reproducible conserva la fuente generada cuando el compilador C falla.

Hay un riesgo particular en los sistemas educativos: hacer que el ejemplo «funcione» delegando silenciosamente a NumPy cuando una operación no se puede compilar. Eso puede ser una estrategia válida si se declara como fallback, pero falsea un benchmark de compilador propio si se oculta. Lumbre no utiliza ese fallback para los kernels de entrenamiento.

Diseñar un test de integración

Construye $(x+1)*(x-1)$ con cinco elementos, incluyendo cero y valores negativos. Compara el resultado generado con el cálculo Python. Después repite la ejecución con otros valores pero la misma forma. ¿Debería recompilarse por haber cambiado x ?

Solución. La identidad matemática es $x*x-1$, pero el test debe evaluar la expresión que realmente se ha pedido bajo el contrato elegido. Para $[-2,-1,0,1,2]$ se obtienen $[3,0,-1,0,3]$. Cambiar los datos de x no modifica el grafo ni sus formas, de modo que el mismo programa compilado puede reutilizarse.

Antes de continuar

Explica qué recibe una función C, qué conserva el grafo y qué identifica la caché. Localiza un bucle en la fuente generada y relaciónalo con una salida del programa. Después distingue el tiempo de compilar del de volver a ejecutar.

CAPÍTULO 7

Índices simbólicos: calcular con lo que todavía no sabemos

7.1 Una casilla cuyo número depende de otra

Supón que una fila tiene ocho elementos y quieres acceder al elemento situado dos posiciones después de i . Si i vale tres, la dirección lógica es cinco. Pero el compilador puede estar construyendo un programa que servirá para todos los valores de i entre cero y cinco. Necesita representar $i+2$ sin conocer todavía i .

Una expresión simbólica es una receta para calcular un valor cuando se conozcan sus variables. No es una predicción estadística ni una cadena de texto opaca. Podemos analizar su estructura, simplificar partes y deducir límites.

Los índices de Lumbre se estudian con un pequeño lenguaje entero separado del de valores FP32. Esa separación evita aplicar por accidente reglas enteras a números de coma flotante. En el paquete, `symbolic.py` funciona como laboratorio autocontenido de expresiones e intervalos; el renderer tensorial principal utiliza sus propias fórmulas de índices estáticos. No se presenta el laboratorio como un solver general ya integrado en todos los pases.

7.2 Variables, constantes y operaciones

Una constante como 8 tiene valor conocido. Una variable como i tendrá valor en ejecución. Una suma une dos expresiones. La estructura de $i*8+j$ contiene una multiplicación y una suma, igual que el grafo tensorial, pero ahora cada nodo describe un índice entero.

```
from lumbre.symbolic import var, const, simplify

i = var("i")
j = var("j")
expresion = i * 8 + j
print(expresion.evaluate({"i": 2, "j": 3}))
```

La intención del ejemplo es obtener 19: dos filas completas de ocho posiciones más tres posiciones dentro de la siguiente. El método evalúa recursivamente la receta con los valores del diccionario.

7.3 Por qué los intervalos son útiles

Sabemos que i está entre cero y tres, y j entre cero y siete. Entonces $i*8+j$ está entre cero y treinta y uno. Hemos demostrado que el índice cabe dentro de un array de treinta y dos

elementos sin probar todas las combinaciones una por una.

El intervalo es una aproximación del conjunto de valores posibles. Para ciertas expresiones la cota será exacta; para otras perderá información. Esa pérdida puede impedir una optimización segura, pero no debe permitir una optimización insegura.

Propagar límites

Si $a \in [a_{\min}, a_{\max}]$ y $b \in [b_{\min}, b_{\max}]$, entonces la suma pertenece a $[a_{\min} + b_{\min}, a_{\max} + b_{\max}]$. Para el producto general se comparan los cuatro productos de extremos. Si sabemos que una constante multiplicadora es positiva, el orden de extremos se conserva.

La notación $a \in [\text{mínimo}, \text{máximo}]$ dice que el valor de a se encuentra dentro de ese intervalo. No significa que a sea un array ni que todos los valores del intervalo deban alcanzarse realmente.

7.4 División y resto: encontrar fila y columna

Si una tabla tiene ocho columnas y un índice lineal q vale diecinueve, la fila es $q//8 = 2$ y la columna $q\%8 = 3$. Hemos separado cuántas filas completas caben y cuánto sobra. Esta pareja de operaciones permite convertir un único bucle lineal en coordenadas multidimensionales.

Para índices no negativos y divisor positivo, la identidad es $q = (q//8)*8 + q\%8$, con resto entre cero y siete. La restricción de signos importa porque los lenguajes pueden tener convenciones diferentes para división y resto de enteros negativos.

```
for q in range(32):
    fila, columna = q // 8, q % 8
    assert q == fila * 8 + columna
    assert 0 <= fila < 4
    assert 0 <= columna < 8
```

Este test cubre un dominio finito pequeño. El argumento general procede de la definición de cociente y resto bajo el contrato indicado. Ambos son útiles: el argumento explica la regla y el test detecta errores en nuestra implementación.

7.5 Simplificaciones que necesitan hipótesis

$(i*8+j)//8 \rightarrow i$ es válida si j está entre cero y siete y trabajamos con enteros sin desbordamiento. Sin esa hipótesis, j podría valer nueve y añadir una fila. Una reescritura debe registrar las condiciones bajo las que se aplica.

Análogamente, $(i*8+j)\%8 \rightarrow j$ necesita ese mismo intervalo de j . Si solo sabemos que j es no negativo, podríamos simplificar a $j\%8$, pero no a j . Aprender a conservar una condición es más importante que memorizar una colección de patrones.

Un contraejemplo mínimo

Para $i=2$ y $j=9$, $(i*8+j)//8$ vale 3, mientras i vale 2. La regla sin guardia cambia la dirección. Un test con $j=0,1,2$ no detectaría el fallo porque permanece dentro del intervalo donde sí es válida.

7.6 Máscaras como proposiciones

Un kernel vectorizado puede generar índices que exceden el tamaño real. Para $N=10$ y grupos de cuatro, el último grupo considera posiciones 8,9,10,11. Solo las dos primeras son válidas. Una máscara representa la condición $q < N$.

El compilador puede simplificar una máscara a verdadera si demuestra que todos los índices del grupo están dentro de rango. Puede simplificarla a falsa si demuestra que ninguno lo está. Si no puede demostrar ninguna de esas dos cosas, debe conservarla.

No confundir «no he encontrado un índice inválido» con «he demostrado que no existe». El análisis conservador prefiere mantener una comprobación cuando falta información. Perder una optimización es un coste de rendimiento; eliminar una comprobación necesaria es un error de corrección y potencialmente de seguridad.

7.7 Aritmética matemática y anchura de máquina

El lenguaje simbólico puede razonar con enteros Python, que crecen según sea necesario. El código generado utiliza enteros de anchura fija. Una prueba de límites solo se traslada si las expresiones intermedias no desbordan esa anchura, o si la semántica modela explícitamente el desbordamiento.

Aunque el índice final quepa, un producto intermedio puede no caber. Por ejemplo, multiplicar dimensiones antes de comprobarlas exige cuidado. Un frontend robusto verifica tamaños y límites de dirección antes de reservar o generar memoria. No puede confiar en que una reserva fallida repare por sí sola una expresión desbordada.

7.8 Rangos relacionados: lo que un intervalo pierde

Si j es exactamente $7-i$ y i está entre cero y siete, la suma $i+j$ siempre vale siete. Un análisis que solo recuerde los intervalos independientes $[0,7]$ y $[0,7]$ concluirá que la suma está entre cero y catorce. La cota es segura, pero imprecisa.

Para conservar relaciones más ricas se utilizan dominios abstractos y análisis afines. No los necesitamos para el primer compilador, pero debemos reconocer el límite de nuestro análisis. Un motor de intervalos no se convierte automáticamente en un demostrador de todas las identidades aritméticas.

7.9 Expresiones afines y transformaciones de bucles

Una expresión afín combina variables multiplicadas por constantes y una constante adicional, como $3*i+2*j+5$. Los accesos afines permiten analizar regiones y dependencias de manera estructurada. Divisiones y módulos por constantes introducen extensiones útiles para tiling y layouts.

Los compiladores de ML aprovechan estas estructuras para generar bucles, transformar índices y vincular ejes lógicos con ejes físicos. MLIR ofrece representaciones y dialectos para expresar

distintos niveles de abstracción; su tutorial Toy es una ruta de comparación con nuestro compilador pequeño. [34]

7.10 Cómo probar un simplificador simbólico

Primero genera expresiones pequeñas sobre variables con dominios acotados. Después evalúa la original y la simplificada en todas las combinaciones de esos dominios. Añade pruebas específicas de fronteras: cero, último índice válido, primer índice inválido y divisores que cambian un cociente.

No permitas divisores cero en los generadores cuando el lenguaje los prohíbe. Pero sí prueba que el frontend rechaza esas expresiones. Un test no debe descartar silenciosamente todas las entradas que incomodan al algoritmo; debe representar el contrato real.

Demostrar o conservar

Sabes que $0 \leq t < 16$. Simplifica $(32*b+t)//16$ y $(32*b+t)\%16$, suponiendo b no negativo. Después explica qué cambia si t puede valer 16.

Solución. La primera expresión vale $2*b$ y la segunda t , porque $32*b$ es múltiplo de 16 y t es un resto válido. Si t puede valer 16, la primera puede ser $2*b+1$ y la segunda cero. La frontera superior estricta es una parte esencial de la regla.

Antes de continuar

Debes poder transformar un índice lineal en coordenadas, propagar una cota de suma y encontrar una hipótesis que falte en una reescritura. El objetivo es justificar direcciones, no practicar álgebra simbólica sin dominio.

CAPÍTULO 8

Reescrituras: mejorar el programa conservando su contrato

8.1 Cambiar el plan, no el resultado pedido

Un compilador transforma representaciones. Algunas transformaciones hacen el grafo más pequeño; otras lo hacen más grande para exponer paralelismo o generar instrucciones legales. La palabra **optimización** no significa siempre reducir el número de nodos.

Para una suma entera, $x+0$ puede reemplazarse por x bajo la semántica correspondiente. Para una transposición seguida de su inversa, podemos recuperar el tensor original si no hay efectos que debamos conservar. Para una multiplicación matricial, expandirla en bucles produce muchos elementos de IR pero acerca el programa al hardware.

Una transformación necesita dos descripciones: qué patrón reconoce y qué condición garantiza que su reemplazo sea válido. La condición puede depender de tipos, formas, intervalos, aliasing, valores especiales y política de coma flotante.

8.2 Pattern matching explicado con piezas

Imagina una caja de piezas donde buscas «una suma cuyo segundo operando es cero». No te interesa el nombre concreto del primer operando. Un patrón permite poner una variable, por ejemplo x , en esa posición y capturar la pieza encontrada.

Patrón:	ADD(x , CONST(0))
Condición:	dominio entero exacto compatible
Reemplazo:	x

La variable del patrón no es necesariamente una variable de entrada del programa. Puede capturar un subgrafo completo. Esa distinción permite escribir reglas que no dependan de la complejidad interna del operando.

Tinygrad utiliza `UPat`, `PatternMatcher` y recorridos de reescritura dentro de un pipeline de UOps. Lumbre enseña la idea mediante componentes más pequeños; no copia una lista universal de reglas suponiendo que todas son seguras para su propia semántica. [4]

8.3 Reescribir de abajo hacia arriba

Si una expresión es $(3+4)*x$, simplificar primero la suma produce $7*x$. Un recorrido de abajo hacia arriba transforma las entradas antes de examinar el nodo que las utiliza. Esto permite que una regla descubra constantes que antes estaban escondidas.

El recorrido debe conservar el compartido del grafo. Si un nodo común se transforma, todos sus usos compatibles deberían referirse a la nueva versión, no a copias incoherentes. Una tabla que asocie nodo antiguo y nodo nuevo ayuda a garantizar esa consistencia.

```
def reescribir(orden, regla):
    nuevos = {}
    for viejo in orden:
        entradas = tuple(nuevos[x] for x in viejo.src)
        candidato = reconstruir(viejo, entradas)
        nuevos[viejo] = regla(candidato) or candidato
    return nuevos
```

Este es un esquema de algoritmo. `reconstruir` debe utilizar el constructor canónico y conservar los metadatos que no cambian. Devolver `None` indica «la regla no se aplica», no «el resultado es un tensor vacío».

8.4 Punto fijo y riesgo de bucles

Una pasada puede hacer aplicable otra regla. Por eso algunos motores repiten hasta que el grafo deja de cambiar, un **punto fijo**. Pero si incluimos reglas opuestas, como expandir y factorizar la misma expresión sin control, podemos alternar indefinidamente.

Una solución es ordenar fases con propósitos diferentes. Otra consiste en orientar reglas según una medida que disminuye. También puede imponerse un límite de iteraciones con diagnóstico explícito. El límite evita un bloqueo, pero no sustituye una argumentación sobre terminación o estabilidad del pipeline.

Dos reglas que se pelean

Regla A: $x*(y+z) \rightarrow x*y+x*z$. Regla B: $x*y+x*z \rightarrow x*(y+z)$. Aplicarlas repetidamente sin estrategia no elige una versión óptima; produce un ciclo. Además, en FP32 ni siquiera se garantiza equivalencia exacta sin permisos numéricos adicionales.

8.5 Constant folding

Calcular una subexpresión formada por constantes durante la compilación se llama constant folding. Para enteros matemáticos es directo mientras respetemos el dominio. Para FP32, evaluar con un `float` de Python puede usar otra precisión y otro orden de redondeo que el programa objetivo.

Un compilador cuidadoso define cómo redondea constantes y operaciones plegadas. En Lumbre las constantes escalares se convierten al tipo FP32 al construirlas y se almacenan mediante una representación hexadecimal. Esto preserva su identidad con más fidelidad que un texto decimal abreviado arbitrariamente.

Plegar funciones trascendentes introduce una dificultad adicional: la biblioteca matemática de la máquina anfitriona puede no coincidir con la del dispositivo objetivo. No basta con que ambas funciones se llamen `exp` para garantizar el mismo último bit.

8.6 Eliminar trabajo muerto

Si una operación no contribuye a ninguna salida y no tiene efectos observables, puede eliminarse. En un grafo puro basta recorrer desde las salidas hacia las entradas para encontrar lo necesario. Una suma construida y abandonada no debe generar un kernel solo porque existe un objeto Python que la describe.

Si una operación escribe un archivo, modifica un parámetro o comunica con otro dispositivo, la ausencia de una salida numérica no significa que sea irrelevante. Por eso la representación de efectos y dependencias de control es necesaria en compiladores más generales.

Lumbre separa la evaluación funcional de nuevos valores y el compromiso de actualizaciones mediante `assign`. Durante la evaluación, el grafo lee el estado viejo. Después se copian las salidas declaradas hacia los parámetros. Esta disciplina hace más fácil razonar sobre qué trabajo es observable.

8.7 Equivalencia: qué debemos conservar

La equivalencia puede significar identidad bit a bit, igualdad matemática bajo hipótesis, proximidad dentro de tolerancia o equivalencia de comportamiento con efectos. No debemos cambiar de criterio a mitad de una comparación.

Para movimientos de memoria, además de valores pueden importar aliasing y escrituras futuras. Para entrenamiento, además de forward importa backward. Una transformación que conserva la salida pero destruye el gradiente no es una optimización válida del programa de entrenamiento.

Un contrato de transformación

Una regla se documenta con entradas admitidas, precondiciones, salida esperada, efectos, semántica numérica y propiedades conservadas. «Probada con cien tensores» describe evidencia experimental; «válida para toda forma estática no negativa bajo estas hipótesis» requiere un argumento general.

8.8 E-graphs y saturación de igualdades

Un motor de reescrituras dirigido elige una representación y puede perder alternativas útiles. Un e-graph agrupa expresiones equivalentes y conserva varias formas. La saturación de igualdades aplica reglas para ampliar ese conjunto y después extrae una opción según un modelo de coste.

El trabajo de `egg` proporciona una implementación y técnicas de mantenimiento de esas clases de equivalencia; su publicación original es de 2021, aunque existe una presentación en Communications of the ACM de 2026. No se debe presentar esa reedición como si el algoritmo hubiera nacido en 2026. [38]

La idea no elimina las precondiciones numéricas. Introducir una igualdad falsa en un e-graph contamina las alternativas. Tampoco garantiza automáticamente el mejor rendimiento real: la extracción depende del coste que se haya modelado y de los límites de búsqueda.

8.9 Costes que no se suman de manera local

Supón que dos expresiones comparten una suboperación costosa. Elegir por separado la mejor forma de cada una puede duplicarla. El coste global depende de reutilización, materialización, tamaños y presión de memoria. Una regla «menos nodos siempre es más rápido» no captura estas interacciones.

También hay costes de lanzamiento, compilación y transferencia. Un kernel algo más lento puede ser preferible si evita una transferencia grande. Un grafo muy fusionado puede perder frente a dos kernels porque consume demasiados registros. Las decisiones necesitan modelos y medición.

8.10 Pruebas diferenciales y metamórficas

Una prueba diferencial compara dos implementaciones del mismo contrato: intérprete y C generado, versión sin optimizaciones y versión optimizada, o referencia FP64 y candidato FP32 bajo tolerancia. Una prueba metamórfica utiliza una relación que debe mantenerse, como transponer dos veces o cambiar una dimensión de lote independiente.

Las relaciones metamórficas también tienen hipótesis. Permutar los términos de una reducción no garantiza igualdad bit a bit en FP32. Escalar todas las entradas puede cambiar el régimen de desbordamiento. El test debe declarar qué observa y qué propiedades espera.

8.11 Un expediente de regla

Para cada nueva reescritura, crea una ficha pequeña. La primera línea da el patrón; la segunda, las precondiciones; la tercera, un ejemplo válido; la cuarta, un contraejemplo cuando se elimina una precondición. Añade un test que falle antes de corregir una implementación defectuosa.

Esta disciplina evita colecciones de trucos sin explicación. También facilita revisar código generado por un asistente: una propuesta puede parecer convincente y, aun así, olvidar el signo del cero o una dependencia de memoria.

Diseñar una regla completa

Quieres eliminar dos permutaciones consecutivas. La primera cambia ejes $(0,1,2)$ a $(2,0,1)$ y la segunda aplica el orden $(1,2,0)$ al resultado. ¿Qué permutación conjunta obtienes? ¿Puedes eliminarla?

Solución. La segunda selecciona, en orden, los ejes originales que quedaron en posiciones 1,2,0: son 0,1,2. La composición es la identidad. En una representación pura de vistas puede recuperarse la entrada. Si entre ambas hay una materialización o efecto exigido por el contrato, hay que analizarlo por separado; no se borra una escritura observable solo porque los valores finales coincidan.

Antes de continuar

Explica cuándo se aplica una regla, qué hipótesis exige y cómo se evitan ciclos. Una optimización debe hacer visibles esas condiciones.

CAPÍTULO 9

Movement ops: cambiar el mapa sin mover las cajas

9.1 Una estantería y dos maneras de leerla

Coloca seis cajas en una estantería lineal con los valores 1,2,3,4,5,6. Puedes dibujar alrededor una tabla de dos filas y tres columnas, o una de tres filas y dos columnas. Las cajas no se han movido; ha cambiado la forma de interpretar sus posiciones.

Las **movement ops** reorganizan ejes, seleccionan regiones o reutilizan valores. A menudo pueden representarse como cambios de índices sin copiar inmediatamente el contenido. Tinygrad destaca esta separación entre operaciones aritméticas, reducciones y movimientos. La ausencia de copia es una posibilidad de representación, no una garantía universal de coste cero para cualquier cadena de consumidores. [3]

9.2 Strides: cuánto avanzar al cambiar una coordenada

En una matriz contigua de dos filas y tres columnas, aumentar una columna avanza un elemento; aumentar una fila avanza tres. Los strides son (3,1). La dirección lógica del elemento (i,j) es $3*i+j$, más un desplazamiento inicial si la vista comienza dentro de otro buffer.

Para una forma (2,3,4) contigua por filas, los strides son (12,4,1). El primer eje salta un bloque de doce elementos; el segundo, una fila de cuatro; el tercero, una posición. La última dimensión varía más deprisa.

Dirección de una vista afín

Con offset o , coordenadas $i_0, \dots, i_{r-1}$ y strides $s_0, \dots, s_{r-1}$, el índice de almacenamiento es:

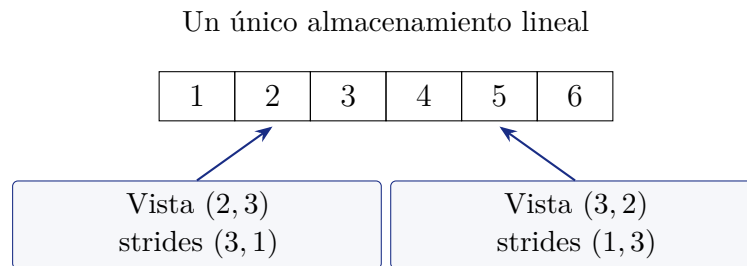
$$\text{addr}(i_0, \dots, i_{r-1}) = o + \sum_{a=0}^{r-1} i_a s_a.$$

La suma recorre ejes, no los valores del tensor. Multiplicar después por bytes por elemento convierte ese índice en un desplazamiento de bytes.

9.3 Transposición como intercambio de strides

La matriz original (2,3) tiene strides (3,1). Su transpuesta tiene forma (3,2) y strides (1,3). El elemento (2,1) de la transpuesta apunta a $2*1+1*3=5$, donde está el valor 6. No hemos cambiado el buffer.

El acceso secuencial por filas de la vista transpuesta salta ahora distancias de tres elementos. Para algunos consumidores esto es adecuado; para otros conviene materializar una copia contigua. La decisión depende de cuántas veces se utilizará y de la eficiencia de los accesos que evita.



9.4 Reshape no siempre es una vista simple

Cambiar una forma contigua (2,3) a (3,2) conserva el orden lineal. Pero intentar reinterpretar una transpuesta no contigua puede requerir más que asignar nuevos strides. La pregunta no es únicamente si el número de elementos coincide: también importa el orden lógico de esos elementos.

El laboratorio `layout.py` permite un reshape contiguo y rechaza otro que no pueda justificar. El IR tensorial de Lumbre, en cambio, representa reshape como una transformación de su orden lógico y puede insertar materialización en las fronteras elegidas por el scheduler. No debemos confundir estas dos superficies de la entrega: una es un modelo de vistas afines; la otra, una representación de operaciones tensoriales.

Un reshape que revela el orden

Partimos de `[[1,2,3],[4,5,6]]`. Su transpuesta se lee lógicamente como `[[1,4],[2,5],[3,6]]`. Aplanarla en ese orden produce `[1,4,2,5,3,6]`, no `[1,2,3,4,5,6]`. Cambiar solo la etiqueta de forma sobre el buffer original daría la segunda secuencia y sería incorrecto para ese contrato.

9.5 Expand y stride cero

Un valor que se reutiliza a lo largo de un eje puede representarse con stride cero en ese eje. Una fila de tres números expandida a cuatro filas tiene forma (4,3) y strides (0,1): cambiar de fila no cambia la dirección.

Leer esa vista es sencillo. Escribir en ella es peligroso si varias posiciones lógicas apuntan al mismo lugar. Dos hilos que escriben valores distintos en filas diferentes podrían estar escribiendo realmente la misma dirección. El compilador necesita restricciones de escritura o una semántica explícita de combinación.

Por eso una vista de broadcasting no debe tratarse como un array independiente solo porque tiene una forma grande. El número de posiciones lógicas no es necesariamente el número de ubicaciones físicas distintas.

9.6 Slicing: cambiar el inicio y la extensión

Seleccionar columnas 1 y 2 de una matriz de cuatro columnas conserva el stride de fila cuatro, pero cambia el offset inicial en uno y la forma de la selección. Una región puede ser rectangular en coordenadas lógicas y no contigua en memoria.

Para un slice con paso uno, el mapeo de una coordenada de salida j a la entrada es $j + \text{inicio}$. Para pasos diferentes, sería $\text{inicio} + j * \text{paso}$, con más condiciones de límites. El IR ejecutable de esta edición admite slices de paso uno con extremos no negativos; el capítulo enseña cómo ampliar el contrato sin afirmar que esa ampliación ya esté implementada.

9.7 Invertir un eje y strides negativos

Una fila $[1, 2, 3, 4]$ puede leerse al revés empezando en la última posición y usando stride -1 . El offset debe cambiar a tres. Si solo cambias el signo del stride y mantienes el inicio en cero, intentarás leer antes del buffer.

La regla general para invertir un eje de longitud n añade $(n-1) * \text{stride}$ al offset y cambia el signo del stride. Para ejes vacíos hay que definir el caso sin crear un acceso. El laboratorio de vistas cubre inversión como ejercicio de direcciones; el frontend tensorial no ofrece toda esa API de strides al runtime C.

9.8 Padding: una lectura condicional

Añadir un borde de ceros crea posiciones de salida que no corresponden a ninguna entrada. Podemos evitar materializar el borde si el consumidor utiliza una lectura condicional: dentro de la región válida lee la entrada; fuera, devuelve cero.

El orden de la condición importa. No es seguro leer primero una dirección inválida y después multiplicar el valor por una máscara cero. La lectura ya ha ocurrido. En C se puede utilizar una expresión condicional que solo evalúe la rama seleccionada bajo la semántica del lenguaje.

```
float valor = (j >= inicio && j < final)
    ? entrada[j - inicio]
    : 0.0f;
```

Un backend GPU debe confirmar que su instrucción de carga enmascarada respeta el acceso, no solo el valor final. Las máscaras de carga, las de cálculo y las de escritura son conceptos relacionados pero diferentes.

9.9 Componer movimientos

Una permutación, un slice y un expand pueden componerse en una única función de índice. Esto evita crear buffers intermedios para cada movimiento. Pero una expresión de índice demasiado complicada puede añadir divisiones, módulos y ramas a cada acceso.

La política óptima depende del uso. Una transposición compartida por veinte multiplicaciones puede merecer una copia contigua reutilizable. Una selección usada una sola vez puede

integrarse en la carga. El compilador debe comparar coste de materializar, coste de recomputar índices y beneficio de acceso.

9.10 El backward de un mapa

Una vista no cambia necesariamente los valores, pero sí cómo se acumulan sus gradientes. Una permutación se deshace con la permutación inversa. Un reshape recupera la forma anterior. Un expand suma las contribuciones de todas las posiciones que reutilizaron un mismo dato. Un slice coloca sus gradientes en las posiciones seleccionadas y deja cero en las demás.

Esta relación anticipa una idea fundamental: **reutilizar en forward implica acumular en backward**. Si tres salidas leen el mismo escalar, las tres pueden influir en su gradiente. No basta con escoger la contribución de la primera.

Calcular una dirección completa

Una vista tiene forma $(3, 2)$, strides $(1, 4)$ y offset 2. Calcula la dirección lógica de $(2, 1)$. Después invierte el primer eje: ¿qué offset y strides necesitas para conservar las mismas posiciones en orden inverso?

Solución. La dirección es $2+2*1+1*4=8$. Al invertir el eje de longitud tres, el offset pasa a $2+(3-1)*1=4$; los strides pasan a $(-1, 4)$. El elemento nuevo $(0, 1)$ apunta a ocho y corresponde al antiguo $(2, 1)$.

Antes de continuar

Debes distinguir forma, strides, offset y almacenamiento. Explica por qué expand puede ser barato al leer y problemático al escribir. Después demuestra con seis números por qué no todo reshape no contiguo puede resolverse cambiando etiquetas.

CAPÍTULO 10

Loops y rangeify: convertir un tensor en recorridos

10.1 La receta tensorial necesita índices

Una operación de suma dice qué relación tienen entradas y salida, pero no enumera posiciones. Para ejecutarla necesitamos rangos: i recorre filas, j columnas, k términos de una reducción. Introducir explícitamente esos rangos es el puente entre álgebra tensorial y código de bucles.

En el vocabulario de tinygrad, **rangeify** forma parte de ese descenso hacia rangos y accesos. El pipeline real cambia con el proyecto; el recorrido de esta edición se vincula al commit fijado. Lumbre implementa un descenso más sencillo a expresiones de índices y bucles C. La similitud es conceptual, no identidad de implementación. [1, 4]

10.2 Un bucle lineal puede recorrer varios ejes

Para una salida $(2,3)$, q recorre de cero a cinco. La fila es $q//3$ y la columna $q\%3$. La misma técnica se amplía a más ejes: dividimos por el producto de dimensiones posteriores y tomamos resto por el tamaño del eje.

Para $(2,3,4)$ y $q=17$, el primer eje vale $17//12=1$; el segundo $(17//4)\%3=1$; el tercero $17\%4=1$. La coordenada es $(1,1,1)$. Reconstruir el índice da $1*12+1*4+1=17$.

Un recorrido verificable

Enumera las coordenadas de $q=0,1,3,4,11,12,23$ para $(2,3,4)$. Son $(0,0,0)$, $(0,0,1)$, $(0,0,3)$, $(0,1,0)$, $(0,2,3)$, $(1,0,0)$ y $(1,2,3)$. Las fronteras muestran cuándo se reinicia un eje y avanza el anterior.

10.3 Bucles anidados y equivalencia con el índice lineal

La versión anidada recorre explícitamente cada eje. La versión lineal calcula coordenadas a partir de q . Ambas pueden visitar las mismas posiciones en el mismo orden para un layout contiguo. La elección afecta al código que ve el compilador C y a las oportunidades de optimización.

```
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        y[i*3+j] = x[i*3+j] + 2.0f;
    }
}
```

Si las dimensiones son constantes, un compilador C puede simplificar divisiones y módulos. Si son variables, algunos índices resultan más caros. Esto no obliga a evitar siempre la linealización: permite mapear fácilmente un hilo GPU a una posición. El diseño compara ventajas y costes.

10.4 Ejes de salida y ejes de reducción

En GEMM, i y j identifican una salida; k recorre valores que contribuyen a ella. Los dos tipos de eje no son intercambiables. Repartir salidas entre trabajadores no requiere combinarlas. Repartir k obliga a sumar resultados parciales de una misma salida.

```
for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) {
        float acc = 0.0f;
        for (int k = 0; k < K; k++) {
            acc += A[i*K+k] * B[k*N+j];
        }
        C[i*N+j] = acc;
    }
}
```

El acumulador se reinicia para cada pareja (i, j) . Situarlo fuera del bucle de j sumaría resultados de columnas diferentes. Esta es una de las primeras pruebas de scope que debe cubrir un compilador de reducciones.

10.5 Invariantes de bucle

Un invariante es una afirmación que se mantiene en un punto del bucle. Después de procesar los primeros k términos de una salida GEMM, el acumulador representa la suma de esos términos bajo el orden y redondeo del programa. Al empezar no se ha procesado ninguno y el acumulador vale cero. Cada vuelta añade exactamente el siguiente producto.

Al terminar $k=K$, se han incluido todos los términos requeridos. Ese argumento explica la corrección para cualquier K permitido. Probar $K=3$ es útil para detectar errores, pero no sustituye el invariante.

El invariante en notación compacta

Antes de procesar el término k , el acumulador contiene la reducción ordenada de $A_{it}B_{tj}$ para $0 \leq t < k$. En aritmética real se escribe $\sum_{t=0}^{k-1} A_{it}B_{tj}$. En FP32 debemos añadir que se ha ejecutado la secuencia concreta de redondeos elegida.

10.6 División de rangos en bloques

Para recorrer $N=10$ posiciones en bloques de cuatro, un índice exterior selecciona el bloque y uno interior la posición dentro de él. La coordenada es $q=4*b+t$. Los bloques $b=0,1,2$ producen q de 0 a 11, por lo que necesitamos la guardia $q<10$ en el último.

Otra posibilidad separa una región completa de ocho elementos y una cola de dos. Ambas

implementaciones pueden ser correctas. La elección afecta a ramas, vectorización y tamaño de código. El compilador puede generar una variante rápida para tamaños múltiplos del bloque y otra general para el resto.

10.7 Intercambiar bucles

Si dos ejes de salida son independientes, intercambiar sus bucles puede conservar el cálculo de cada casilla y cambiar el orden de acceso a memoria. En una matriz por filas, recorrer columnas en el bucle interior suele dar accesos contiguos. Recorrer filas en el interior salta por el stride de fila.

Intercambiar un eje de reducción con otro bucle requiere analizar qué acumuladores siguen vivos y dónde se inicializan. En GEMM, el orden i, k, j permite reutilizar $A[i, k]$ a lo largo de varias columnas, pero exige que las salidas C se hayan inicializado adecuadamente antes de acumular.

```
for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) C[i*N+j] = 0.0f;
    for (int k = 0; k < K; k++) {
        float a = A[i*K+k];
        for (int j = 0; j < N; j++) C[i*N+j] += a * B[k*N+j];
    }
}
```

El orden de k dentro de cada salida puede conservarse, pero el tráfico de C y la vectorización cambian. No afirmamos que esta versión sea siempre mejor: es una alternativa que debe medirse.

10.8 Dependencias de memoria

Considera $x[i] = x[i-1] + 1$ para i creciente. La iteración i utiliza el resultado de la anterior. Reordenarlas libremente cambia la tarea. El hecho de que un bucle tenga muchas iteraciones no implica que sean paralelas.

Un análisis de dependencias identifica lecturas y escrituras que pueden referirse a la misma dirección. Para un compilador tensorial funcional, muchos efectos quedan fuera del grafo de valores. Cuando añadimos actualizaciones in-place, buffers compartidos o comunicación, esas dependencias deben representarse explícitamente.

10.9 Lowering por etapas

No intentamos transformar una red completa directamente en ensamblador con una sola función. Una secuencia útil conserva niveles: operaciones tensoriales, índices y rangos, asignación de buffers, kernels, instrucciones del backend y ejecución. Cada nivel resuelve una clase de decisiones.

Esta separación permite localizar errores. Si la forma del tensor es incorrecta, el problema está antes del mapeo de GPU. Si el C es correcto pero el runtime copia el tamaño equivocado,

el problema está después del renderer. Un buen compilador permite inspeccionar las fronteras entre etapas.

10.10 Verificar que no faltan ni sobran posiciones

Una transformación de rangos debe demostrar cobertura y ausencia de duplicación de escrituras, salvo que exista una operación de combinación definida. Para $q=4*b+t$, con b en el rango apropiado y t entre cero y tres, la guardia asegura que solo se escriben posiciones válidas. La división y el resto recuperan b y t para cualquier q válido, lo que demuestra que no se pierde ninguna posición.

Esta forma de argumento reaparecerá al mapear bloques e hilos GPU. La GPU no elimina la necesidad de razonar sobre rangos: la hace más importante, porque un error puede convertirse en una carrera difícil de reproducir.

Diseñar un tile con cola

Transforma un rango de 23 elementos en bloques de ocho. Escribe los índices generados por el último bloque y una condición que impida accesos inválidos. ¿Cuántas posiciones válidas tiene ese bloque?

Solución. Hay tres bloques. El último genera 16 a 23. Son válidos 16 a 22, siete elementos. La condición es $q < 23$. Una comprobación $q \leq 23$ incluye un elemento inexistente. Una condición aplicada solo a la escritura no protege una lectura inválida realizada antes.

Antes de continuar

Distingue ejes de salida y reducción, formula un invariante de acumulador y justifica un bloque con cola. Después identifica qué decisión de loop order puede mejorar accesos sin alterar la fórmula de cada salida.

CAPÍTULO 11

Reducciones: identidad, orden y cooperación

11.1 La suma de una caja vacía

Una reducción combina muchos valores en uno. Para comenzar necesita un valor inicial. En una suma, cero permite que añadir el primer elemento produzca ese elemento. En un producto, la identidad es uno. En un máximo, la identidad conceptual para valores extendidos puede ser menos infinito, pero un API puede preferir rechazar una reducción vacía.

Lumbre admite suma vacía y rechaza máximo sobre un eje vacío. Esa elección es explícita. No se deduce de que ambos algoritmos utilicen un acumulador. Las operaciones tienen dominios y contratos diferentes.

11.2 Una reducción por fila

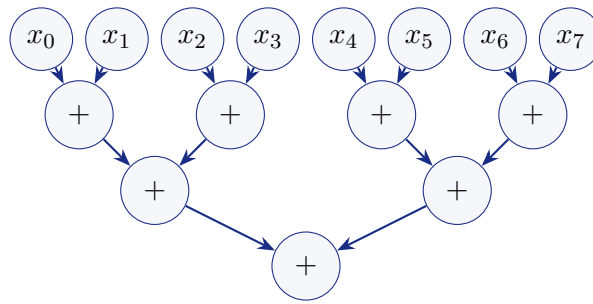
Para una matriz (M,N) , cada salida de suma por fila procesa N elementos. Podemos asignar una fila a cada trabajador. Dentro de una fila, el trabajador acumula secuencialmente. Es una implementación sencilla y correcta, pero puede desperdiciar paralelismo cuando hay pocas filas muy largas.

```
for (int row = 0; row < M; row++) {
    float acc = 0.0f;
    for (int col = 0; col < N; col++) acc += x[row*N+col];
    y[row] = acc;
}
```

La fórmula de dirección mezcla dos decisiones: qué fila produce cada salida y qué eje se reduce. Reducir columnas en vez de filas cambia la secuencia de accesos y el número de resultados.

11.3 Árbol de reducción

Para ocho números, podemos sumar parejas, luego parejas de resultados y finalmente los dos valores restantes. La profundidad de dependencias baja frente a una cadena secuencial. Hay más operaciones que pueden ejecutarse simultáneamente.



En enteros sin desbordamiento, la suma conserva el resultado. En FP32 cambia el orden de redondeos. Un árbol puede incluso reducir ciertos errores, pero no garantiza identidad bit a bit con la cadena. El contrato de comparación debe reconocer esa diferencia.

11.4 Particionar y combinar

Si una fila es demasiado grande para un bloque, varios bloques pueden producir sumas parciales. Un segundo kernel las combina. La frontera entre kernels proporciona un punto de orden que no existe automáticamente entre bloques de un mismo kernel ordinario.

Intentar implementar una barrera global haciendo que todos los bloques esperen dentro de un kernel puede bloquear el dispositivo si no todos están residentes simultáneamente. El plan seguro inicial utiliza varios lanzamientos o mecanismos de cooperación explícitamente soportados y correctamente configurados.

11.5 Atomics no significan determinismo numérico

Una operación atómica permite combinar actualizaciones de una ubicación sin perder escrituras bajo su semántica. No garantiza un orden fijo entre todos los productores. En una suma FP32, distintos órdenes pueden dar últimos bits diferentes.

Además, muchas actualizaciones a una sola dirección pueden serializarse. Un diseño jerárquico reduce primero localmente y limita el número de operaciones atómicas globales. Es otra muestra de que corrección, determinismo y rendimiento son tres preguntas separadas.

11.6 Máximo y empates

El máximo de una lista puede aparecer en varias posiciones. Para forward eso no presenta ambigüedad de valor. Para el gradiente hay que elegir una convención de subgradiente en los empates. Lumbre reparte la contribución entre las posiciones iguales al máximo dentro de la reducción.

Ese comportamiento debe probarse explícitamente. Si una optimización cambia una comparación o el tratamiento de NaN, puede alterar tanto la salida como las posiciones a las que se asigna gradiente. Una capa de softmax estable utiliza el máximo como desplazamiento y lo desconecta del gradiente en la implementación para simplificar el camino numérico sin alterar la derivada matemática del softmax finito.

11.7 Promedio y ejes de tamaño uno

El promedio puede implementarse como suma dividida por el número de términos. Si ese número es cero, no debemos dividir por cero y fingir un promedio válido. Si es uno, la reducción puede desaparecer en el valor, aunque quizá deba conservarse o reconstruirse la forma según `keepdim`.

Las dimensiones unitarias son excelentes casos de test porque revelan confusiones entre eliminar un eje y reducirlo manteniéndolo de longitud uno. Las dimensiones cero revelan accesos indebidos e identidades mal elegidas.

11.8 Log-sum-exp

La expresión $\log(\text{sum}(\exp(\mathbf{x})))$ aparece en pérdidas de clasificación. Se estabiliza restando un máximo y añadiéndolo al final: $\mathbf{m} + \log(\text{sum}(\exp(\mathbf{x}-\mathbf{m})))$. Esta transformación evita muchas exponenciales enormes y es la base de una entropía cruzada estable.

No conviene calcular primero softmax, convertir probabilidades diminutas a cero por redondeo y después aplicar log. El resultado puede ser menos estable que calcular directamente log-softmax. El orden algebraico de una fórmula puede tener un impacto práctico grande aunque describa el mismo objeto matemático.

11.9 RMSNorm como combinación de reducciones y operaciones locales

RMSNorm calcula la media de los cuadrados a lo largo de un eje, añade una pequeña constante, toma la raíz y divide cada elemento por ella, con una escala aprendida. La reducción produce una estadística compartida por la fila; la normalización posterior es elemento a elemento.

Un compilador puede fusionar parte del trabajo, pero tiene que conservar la dependencia: no puede normalizar una fila con una estadística que todavía no está completa. En GPU, la combinación de reducción local y escrituras de la fila necesita sincronización entre los participantes.

11.10 Pruebas que separan mecanismos

Para una suma por filas, prueba una sola fila larga, muchas filas de un elemento, dimensiones no potencias de dos y una fila que combine magnitudes muy diferentes. Para un máximo, añade negativos y empates. Para log-sum-exp, añade un desplazamiento grande común a todas las entradas.

Una buena propiedad para softmax es que sumar una constante finita común a toda la fila no cambia matemáticamente las probabilidades. La prueba numérica debe utilizar valores que no provoquen por sí mismos pérdida extrema de información al representar ese desplazamiento.

Dos fases o una atómica

Debes sumar un millón de valores con muchos bloques GPU. Propón una implementación con sumas parciales y otra con atomics. ¿Qué compararías además del tiempo?

Solución. En la primera, cada bloque reduce su región y un segundo kernel combina las parciales. En la segunda, cada bloque puede reducir localmente y añadir una parcial a una salida inicializada. Hay que comparar precisión, repetibilidad entre ejecuciones, coste de inicialización, número de lanzamientos, memoria auxiliar y restricciones de la operación atómica. No basta medir una sola llamada caliente.

Antes de continuar

Explica identidad, árbol, suma parcial y efecto del orden. Distingue una barrera de bloque de una sincronización global y explica por qué una operación atómica no fija necesariamente el resultado bit a bit de una reducción FP32.

CAPÍTULO 12

Un modelo se convierte en varios kernels

12.1 La diferencia entre fórmula y plan de ejecución

Un grafo describe dependencias de valores. Un plan de ejecución decide qué operaciones se agrupan, dónde se almacenan resultados y en qué orden se lanzan tareas. A una unidad de trabajo compilada para ejecutarse en un dispositivo la llamaremos **kernel de cálculo**. No es el kernel del sistema operativo Linux.

Para $y=(x+1)*2$, un único kernel suele ser suficiente. Para una normalización que necesita una reducción completa y después combina su resultado con muchas posiciones, el compilador puede generar varios kernels o uno cooperativo especializado. La respuesta depende del backend y de la estrategia de memoria.

La pregunta del scheduler

¿Qué valores deben existir en memoria, cuáles pueden calcularse dentro de sus consumidores y qué dependencias deben respetar los lanzamientos? El scheduler no inventa una nueva función matemática: escoge una realización de la que ya existe.

12.2 Materializar: dar una ubicación a un resultado

Un resultado **materializado** se escribe en un buffer para que otros lo lean. Uno no materializado puede quedar como expresión dentro de otro kernel. La materialización cuesta memoria y tráfico, pero puede evitar repetir trabajo o facilitar una implementación eficiente.

Supón $a=\exp(x)$, $b=a+1$, $c=a*2$. Si a se inserta en ambos consumidores, se calcula la exponencial dos veces. Si se materializa una vez, ambos leen el mismo resultado. La mejor opción depende del coste de $\exp$, del tamaño de x , de la fusión posible y del tráfico.

No todas las operaciones compartidas merecen materializarse siempre. Una suma escalar barata puede repetirse sin coste relevante. Lumbre utiliza una heurística sencilla basada en número de usos, clase de operación y profundidad; el libro enseña a sustituirla por un modelo más rico cuando existan pruebas que lo justifiquen.

12.3 Fronteras naturales y fronteras elegidas

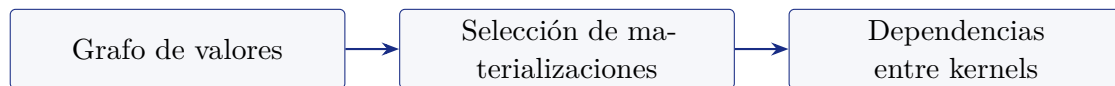
Las salidas declaradas necesitan conservarse para el usuario. Los parámetros son entradas persistentes. Reducciones, GEMM, gather y llamadas especializadas suelen formar fronteras útiles porque requieren recorridos distintos o implementaciones propias.

Pero «suele» no es «siempre». Una biblioteca GEMM puede admitir un epílogo que añade sesgo y activación sin escribir una salida intermedia. Un kernel de atención puede integrar dos productos matriciales y un softmax por bloques. El scheduler general puede reconocer esos patrones y delegar en una implementación especializada con contrato explícito.

12.4 Dependencias de un kernel fusionado

Si un kernel contiene una expresión de varios nodos, sus entradas reales son los buffers materializados que aparecen al recorrer esa expresión. No necesitamos pasar como argumento cada suma interna, porque no tiene ubicación independiente.

Un algoritmo de dependencias recorre desde la raíz del kernel. Cuando encuentra otro nodo materializado, lo registra y se detiene en esa rama. Cuando encuentra una operación interna fusionable, continúa hacia sus entradas. Esta distinción evita tanto argumentos redundantes como lecturas de buffers que nunca se escribieron.



12.5 Orden topológico del plan

Todo kernel que produce una entrada de otro debe ejecutarse antes, salvo que el runtime utilice una sincronización explícita equivalente. Un orden topológico secuencial es una primera solución correcta. Más adelante podremos superponer kernels independientes o transferencias, pero solo después de representar sus dependencias.

El runtime GPU didáctico de Lumbre lanza en una secuencia y sincroniza al finalizar `run`. Esta política es conservadora y fácil de observar. No pretende reproducir la planificación asíncrona avanzada de un sistema distribuido.

12.6 Por qué una fusión grande puede ser peor

Fusionar muchas operaciones mantiene más valores intermedios vivos en registros. Si el hardware no dispone de suficientes registros por hilo o grupo, parte del estado puede derramarse a memoria, un fenómeno llamado **spill**. También puede disminuir el número de grupos residentes y limitar la capacidad de ocultar latencia.

El código generado puede crecer hasta hacer lenta la compilación o impedir simplificaciones del compilador de bajo nivel. Por eso hay que medir coste de primera compilación, tamaño de módulo y rendimiento caliente. Una optimización que solo considera número de lanzamientos puede empeorar la experiencia total.

12.7 Epílogos de GEMM

Después de $C=A@B$, muchas redes calculan $\text{relu}(C+b)$ o una función similar. Un epílogo aplica esas operaciones antes de escribir la salida final. Así evita volver a leer y escribir todo C en un kernel separado.

La implementación debe fijar dónde se realiza cada conversión de tipo. Aplicar activación en FP32 antes de redondear a FP16 no es necesariamente igual a redondear primero y aplicar después. La fusión tiene que preservar el contrato numérico elegido, no solo la forma de las expresiones.

CUTLASS y hipBLASLt ofrecen mecanismos y configuraciones de GEMM y epílogos en sus respectivos ecosistemas. Estudiarlos ayuda a separar la operación matemática de la variedad de realizaciones de hardware. No significa que una llamada a esas bibliotecas sea automáticamente compatible con cualquier layout o tipo. [11, 14]

12.8 Scheduling con efectos

Una actualización de parámetros no puede adelantarse a un kernel que todavía necesita los valores viejos. Una copia entre dispositivos no puede consumirse antes de que termine. Un buffer no puede liberarse mientras una operación asíncrona lo utiliza.

Estas dependencias no siempre aparecen como aristas de valores puras. Los compiladores pueden introducir tokens de efecto, eventos, regiones o relaciones explícitas de orden. El nombre concreto cambia entre sistemas; la obligación de representar el efecto no cambia.

Lumbre utiliza una barrera conceptual simple: evaluar todo el grafo de nuevos valores y después llamar a `assign`. Rechaza usar directamente otro parámetro como salida de actualización para evitar ciclos de copia que pisarían un valor antes de leerlo. Una implementación más general podría utilizar copias temporales o análisis de aliasing.

12.9 Programas con varias salidas

Un programa de entrenamiento produce más que la pérdida. También puede producir gradientes, normas, nuevos momentos del optimizador y parámetros actualizados. Si esas salidas se declaran, el plan de memoria debe conservarlas hasta que el runtime termine de leerlas o comprometerlas.

Eliminar una salida del conjunto puede permitir reutilizar su buffer antes. Por tanto, cambiar qué observamos puede cambiar el plan y el rendimiento. Un benchmark que solicita todos los intermedios para depuración no mide exactamente el mismo programa que una ejecución de producción que solo conserva la salida final.

12.10 Un experimento de fusión

Construye una cadena de cinco operaciones elemento a elemento y compárala con `fuse=False`. Guarda cantidad de kernels, memoria de arena, tiempo de compilación y tiempo de ejecución residente. Después introduce un subgrafo compartido por dos salidas y repite.

La hipótesis razonable es que la fusión reduzca tráfico en la cadena simple, pero el experimento debe permitir que el resultado la contradiga. Para tensores diminutos puede dominar el coste de llamada; para expresiones complejas puede aumentar el tiempo de compilación. La conclusión debe explicar el régimen observado.

12.11 Preguntas de revisión de un scheduler

Al revisar un plan, sigue un valor desde su producción hasta su último uso. Comprueba que el productor se ejecuta antes, que existe un buffer o expresión válido, que la forma es consistente y que ninguna escritura lo destruye prematuramente. Después examina las salidas y los efectos que no se ven en una sola fórmula.

Una visualización de kernels es útil si muestra dependencias, tamaños y buffers, no solo cajas con nombres. La originalidad de un proyecto puede estar en hacer observable este recorrido y detectar violaciones automáticamente.

Escoger materializaciones

Tienes $a=x*2$, $b=\exp(a)$, $c=b+1$, $d=b*b$, y necesitas c y d . Propón un plan sencillo de dos o tres kernels. Explica dónde evitarías repetir $\exp$ y qué cambiaría si solo necesitas c .

Solución. Un plan materializa b calculando $\exp(x*2)$ en el primer kernel y produce c y d en uno o dos kernels posteriores según la capacidad de múltiples salidas. Así no repite $\exp$. Si solo se necesita c , toda la cadena puede fusionarse en una única salida. La propuesta no demuestra que sea el plan más rápido; define una alternativa correcta y fácil de medir.

Antes de continuar

Distingue grafo de valores y grafo de kernels. Explica materialización, dependencia, epílogo y efecto. Debes poder identificar por qué un nodo aparece como argumento de un kernel o queda insertado en su cuerpo.

CAPÍTULO 13

Memoria del programa: reservar, reutilizar y no pisar el pasado

13.1 Un almacén compartido por tareas sucesivas

Si cada resultado intermedio conserva un buffer para siempre, un modelo puede consumir mucha más memoria de la necesaria. Una vez que un valor ya no se utilizará, su espacio puede servir para otro. El problema consiste en saber exactamente cuándo termina su vida útil.

Imagina tres bandejas y una secuencia de tareas. La primera prepara una mezcla; la segunda la consume por completo; la tercera prepara otra mezcla del mismo tamaño. Podemos reutilizar la bandeja después de la segunda tarea, pero no mientras esa tarea todavía lee de ella.

13.2 Vida útil y último uso

Para cada buffer registramos el kernel que lo produce y el último kernel que lo consume. Las salidas declaradas se mantienen vivas hasta el final observable del programa. Los parámetros persistentes se conservan entre ejecuciones y no se mezclan sin más con temporales.

Valor	Se produce	Último uso	Puede reutilizarse después de
a	Kernel 0	Kernel 2	Kernel 2 completo
b	Kernel 1	Kernel 3	Kernel 3 completo
salida	Kernel 3	Lectura del usuario	Lectura o siguiente contrato

Si un kernel lee a y escribe un resultado nuevo, reutilizar exactamente el espacio de a dentro de ese kernel exige analizar si los accesos pueden solaparse de forma segura. El planificador simple evita esa situación: libera solo valores cuyo último uso es estrictamente anterior al kernel actual.

13.3 Arena y offsets

Una **arena** es una reserva grande de memoria dividida en regiones. Cada tensor materializado recibe un offset y un tamaño. El runtime suma el offset al puntero base para obtener la dirección del tensor. Esto reduce muchas reservas pequeñas y hace visible el presupuesto total.

Lumbre alinea regiones a múltiplos de 64 bytes. La alineación facilita ciertos accesos y evita direcciones arbitrarias, pero desperdicia algo de espacio en tensores pequeños. La cifra 64 es

una decisión de esta implementación, no una ley universal de todos los dispositivos.

13.4 Best fit y fragmentación

Cuando una región queda libre, el planificador puede guardarla en una lista. Para una nueva petición, una estrategia best fit escoge la región libre más pequeña que todavía sea suficiente. Si sobra espacio, conserva el resto como otro fragmento.

La estrategia puede fragmentar la memoria: existen suficientes bytes libres en total, pero repartidos en trozos demasiado pequeños. Un planificador más avanzado combina regiones vecinas, reordena decisiones o calcula una asignación global. La versión didáctica no pretende resolver óptimamente todos los casos.

Reutilización con tamaños

Queda libre una región de 1024 bytes y necesitamos 256. Podemos asignar los primeros 256 y devolver los 768 restantes a la lista libre. Si más tarde se liberan dos regiones contiguas de 256, una política que no las combine no podrá atender con ellas una petición única de 512, aunque físicamente estén juntas. Ese es un ejemplo de fragmentación administrativa.

13.5 Alias: dos nombres, una dirección

Dos vistas pueden apuntar al mismo almacenamiento. Dos parámetros también podrían recibir arrays externos que se solapan. El runtime de Lumbre copia los datos de entrada a regiones propias, lo que simplifica el contrato de aliasing externo, a costa de una copia explícita.

En un sistema zero-copy, el compilador debe conocer mejor los solapamientos. Marcar punteros como `restrict` en C promete ciertas ausencias de aliasing. Si el programa viola esa promesa, el compilador C puede generar resultados inesperados bajo las reglas del lenguaje. No se utiliza `restrict` como un talismán de rendimiento sin comprobar su precondition.

13.6 Actualizaciones de entrenamiento

Supón que calculas nuevos pesos a partir de pesos viejos y gradientes. Si sobrescribes un peso mientras otro gradiente todavía lo necesita, mezclas estados de dos iteraciones. El error puede producir una pérdida que aparentemente disminuye y, sin embargo, implementar otro algoritmo.

Lumbre construye un grafo funcional para nuevos parámetros y momentos. Después de ejecutarlo, `assign` copia las salidas hacia las regiones persistentes. Esa frontera es una simplificación pedagógica poderosa: permite razonar sobre una iteración con un estado inicial y otro final bien separados.

13.7 Memoria asíncrona

En CPU secuencial, al volver de una función normalmente han terminado sus accesos ordinarios. En GPU, lanzar un kernel puede devolver el control antes de que termine. La vida útil física

de un buffer depende entonces de eventos y streams, no solo de la posición de una llamada en el programa anfitrión.

Un allocator asíncrono debe respetar qué streams utilizan cada región. Liberar desde el host no significa que los consumidores del dispositivo hayan acabado. La implementación del curso sincroniza al finalizar cada `run`; una versión avanzada puede eliminar esa sincronización global, pero debe reemplazarla por dependencias correctas.

13.8 Pico de memoria y memoria acumulada

La suma de tamaños de todos los intermedios creados durante una ejecución no es el pico simultáneo. Un plan puede producir un terabyte acumulado de temporales a lo largo del tiempo y necesitar solo unos pocos gigabytes a la vez. Para decidir si un modelo cabe importa el pico, junto con reservas persistentes y workspace.

Tampoco basta sumar pesos y activaciones visibles. El optimizador, gradientes, buffers de comunicación, cachés de compilación, áreas de bibliotecas y memoria reservada por allocators pueden contribuir. En el capítulo de escalado construiremos un presupuesto completo por categoría.

13.9 Recomputation frente a almacenamiento

Guardar una activación permite utilizarla en backward sin repetir forward. Recalcularla evita mantenerla viva durante mucho tiempo. Este intercambio se llama checkpointing o rematerialización de activaciones, distinto de guardar un checkpoint de entrenamiento en disco.

La decisión depende de coste de recomputar, tamaño del valor y momento de uso. Una operación barata que produce un tensor grande es una buena candidata conceptual. Una operación muy costosa con salida pequeña puede merecer almacenamiento. Un compilador puede automatizar parte de esa elección mediante análisis de vida útil y costes.

13.10 Verificar un plan de memoria

Una prueba útil ejecuta el mismo grafo con reutilización activada y desactivada. Deben coincidir sus salidas bajo el contrato numérico. Añade grafos con ramas, valores compartidos y varias salidas. Los grafos lineales simples no estresan todos los errores de vida útil.

Para depuración, rellena regiones liberadas con patrones especiales o utiliza herramientas de detección de accesos inválidos cuando el backend lo permita. Ese modo no debe mezclarse con el benchmark final sin declararlo: el relleno añade trabajo y puede cambiar cachés.

13.11 Un certificado sencillo de no solapamiento

Dos buffers pueden compartir un intervalo de bytes si sus vidas útiles no se solapan. Si sus vidas sí se solapan, sus regiones de bytes deben ser disjuntas, salvo un alias explícito permitido

por el contrato. Esa condición puede comprobarse automáticamente para cada pareja de buffers materializados.

Es una forma de convertir una propiedad difícil de observar mediante salidas en una verificación estructural. No demuestra por sí sola que los índices internos de cada kernel sean válidos, pero cubre una clase diferente de errores.

Encontrar un uso tardío

Un tensor *a* se produce en el kernel 0 y se utiliza en los kernels 1 y 4. Un planificador lo libera después del kernel 1 porque «ya se ha consumido». ¿Qué dato olvidó? Diseña un grafo pequeño que haga visible el error.

Solución. Olvidó que el consumo no es único: debe registrar el último uso, aquí el kernel 4. Un ejemplo es una rama corta que utiliza *a* inmediatamente y otra rama larga que vuelve a combinarlo al final. Reutilizar su región durante la rama larga cambia el valor observado por la combinación final.

Antes de continuar

Debes poder dibujar intervalos de vida, distinguir memoria persistente y temporal, y explicar por qué último uso igual al kernel actual no permite una reutilización arbitraria. También debes diferenciar checkpoint de activaciones y checkpoint en disco.

CAPÍTULO 14

Descomponer modelos: operaciones generales y límites explícitos

14.1 Un catálogo pequeño puede expresar muchas redes

Las redes no necesitan una instrucción de hardware para cada nombre de capa. Una capa lineal se expresa mediante GEMM y sesgo. Una normalización combina reducciones y operaciones locales. Una convolución puede expresarse mediante accesos a ventanas, productos y sumas. La atención combina productos matriciales, máscara, softmax y otro producto.

Esta composicionalidad explica por qué un núcleo relativamente pequeño puede ejecutar modelos variados. No demuestra que cualquier operación imaginable ya esté soportada, ni que la descomposición general sea rápida. Corrección por descomposición y rendimiento por especialización son etapas diferentes.

14.2 Capa lineal

Para una entrada de forma `(lote, características)` y una matriz de pesos `(características, salidas)`, GEMM produce `(lote, salidas)`. Un sesgo de forma `(salidas,)` se añade por broadcasting. Si cada fila representa un ejemplo independiente, cambiar el orden de filas solo cambia el orden de resultados.

Una red de varias capas añade funciones no lineales entre esas transformaciones. Sin ellas, ciertas cadenas de matrices podrían combinarse en otra transformación lineal, aunque el entrenamiento y la parametrización tengan propiedades distintas.

14.3 Activaciones

ReLU devuelve el máximo entre x y cero. Sigmoid comprime valores a un intervalo entre cero y uno. Tanh devuelve valores entre menos uno y uno. SiLU multiplica x por $\text{sigmoid}(x)$, y SwiGLU utiliza una rama de compuerta junto con otra proyección.

Cada activación necesita un contrato para valores extremos y una regla de gradiente. Una aproximación rápida puede ser aceptable si su error y su efecto en entrenamiento se evalúan. No debe sustituirse una función por otra únicamente porque ambas tienen una curva parecida.

El núcleo ejecutable dispone de `exp`, `log`, `sqrt` y `tanh`, a partir de las que se construyen varias de esas funciones. Para ReLU puede utilizarse una condición y `where`. No hay una biblioteca oculta que resuelva automáticamente cualquier activación no declarada.

14.4 Convolución unidimensional desde una ventana

Supón una señal $[1, 2, 3, 4]$ y un filtro $[2, 1]$, sin padding y con paso uno. La primera ventana $[1, 2]$ produce $1*2+2*1=4$; la segunda $[2, 3]$ produce 7; la tercera $[3, 4]$ produce 10. Hemos aplicado el mismo conjunto de pesos a posiciones distintas.

En aprendizaje automático muchas APIs llaman convolución a una correlación cruzada, sin invertir el filtro. El contrato debe aclarar esa convención. Cambiar la orientación del filtro puede pasar desapercibido con filtros simétricos, por lo que conviene utilizar uno asimétrico en el test.

14.5 Dimensiones de salida de una convolución

Con longitud de entrada L , filtro de longitud R , padding p por lado, dilatación d y stride s , el tamaño efectivo del filtro es $d*(R-1)+1$. La última ventana debe caber dentro de la región extendida. De ahí se obtiene el número de posiciones válidas.

$$L_{\text{salida}} = \left\lfloor \frac{L + 2p - d(R - 1) - 1}{s} \right\rfloor + 1.$$

La fórmula presupone parámetros válidos y una convención simétrica de padding. Para casos donde no cabe ninguna ventana hay que definir si la API produce una dimensión cero o rechaza la configuración. No se debe permitir que un tamaño negativo llegue a una reserva de memoria.

14.6 Convolución bidimensional

En una imagen, cada salida recorre filas y columnas del filtro, además de canales de entrada. Una disposición posible es NCHW: lote, canal, altura, anchura. Otra es NHWC. El nombre de la capa no fija por sí solo el layout físico.

Para cada lote n , canal de salida o y posición (y, x) , se suman productos de pesos y entradas de la ventana correspondiente. El acceso incorpora stride, dilatación y padding. Es una reducción multidimensional con un patrón de movimiento; los conceptos anteriores ya bastan para describirla correctamente.

14.7 Im2col: hacer explícitas las ventanas

Podemos convertir cada ventana en una fila de una matriz y aplicar GEMM. Esa transformación se conoce como im2col en una de sus formas habituales. Simplifica la reutilización de un kernel GEMM eficiente, pero puede crear un intermedio grande que repite muchos píxeles.

Una implementación implícita calcula los índices de ventana durante la carga y evita materializar toda esa matriz. El intercambio es claro: menos memoria intermedia, más complejidad de índices y de scheduling. No existe una elección universalmente mejor para todas las formas.

Estimar el crecimiento

Una imagen de un canal con H por W posiciones y filtro de 3 por 3 puede contribuir aproximadamente nueve valores por posición de salida a una matriz de ventanas, ignorando bordes para una estimación. El intermedio puede acercarse a nueve veces el número de posiciones originales. La cifra exacta depende de padding, stride, canales y tamaño de salida.

14.8 Gather y embedding

Una tabla de embeddings tiene una fila de características por identificador de token. Gather selecciona las filas indicadas por una lista de enteros. A diferencia de una suma elemento a elemento, las direcciones dependen de datos de entrada, no solo de los índices de salida.

Lumbre implementa gather para una tabla FP32 bidimensional y un tensor de índices int32. El runtime de entrenamiento genera identificadores válidos a partir de un vocabulario cerrado. El kernel evita una lectura fuera de rango, pero la validez semántica de etiquetas e identificadores también debe comprobarse en la canalización de datos.

En backward, dos tokens iguales contribuyen a la misma fila de gradiente. Se necesita una suma de contribuciones, un scatter-add conceptual. La versión de referencia utiliza un recorrido determinista sencillo; una versión GPU rápida necesitará una estrategia de atomics, agrupación o reducción segmentada.

14.9 Formas dinámicas y guardias

Si el número de tokens cambia, un compilador estático puede generar una variante por longitud. Un compilador con formas simbólicas puede aceptar varias longitudes bajo guardias de rango y relaciones entre dimensiones. Otra estrategia agrupa longitudes en buckets y rellena hasta un tamaño conocido.

Cada estrategia tiene costes: compilación, memoria desperdiciada, complejidad de índices o diversidad de variantes. Una API que acepta una longitud distinta no es necesariamente dinámica en todos sus niveles; puede estar recompilando silenciosamente.

14.10 Control de flujo

Una condición que depende de una configuración conocida puede resolverse al construir el grafo. Una condición que depende de datos exige representar selección, ramas o control de flujo en ejecución. **where** calcula una selección elemento a elemento, pero no sustituye todos los bucles y ramas de un lenguaje general.

Un bucle cuyo número de iteraciones depende de una búsqueda, una estructura de datos irregular o un criterio de convergencia necesita soporte adicional. La universalidad de Python como lenguaje anfitrión no transfiere automáticamente esa capacidad al grafo compilado.

14.11 Un manifiesto de operaciones soportadas

Un proyecto serio mantiene un catálogo con operación, tipos, formas, backward, backends y tests. «Soporta atención» es demasiado impreciso si solo funciona para una longitud, un layout, una máscara y forward sin gradiente.

En Lumbre se distinguen las operaciones generales del IR, el laboratorio de vistas, el kernel de atención online y los backends de dispositivo. C y CUDA cuentan con pruebas ejecutadas; HIP sigue pendiente de hardware compatible. El catálogo de la entrega permite saber qué constituye una implementación ejecutada y qué constituye una práctica avanzada.

Descomponer una capa sin inventar una instrucción

Describe una capa que normaliza cada fila por la raíz de la media de sus cuadrados y luego aplica una proyección lineal. Enumera qué operaciones primitivas necesita y dónde aparecen reducciones.

Solución. Se multiplica la fila por sí misma, se calcula la media sobre características, se añade epsilon, se toma la raíz, se divide la fila por esa estadística y se aplica GEMM. Puede añadirse una escala aprendida por broadcasting. La media es una reducción; la proyección contiene otra reducción sobre su dimensión K. Las demás operaciones son locales o movimientos de forma.

Antes de continuar

Debes poder descomponer una capa conocida y declarar una limitación concreta del compilador. Un mensaje «no soportado» bien definido es preferible a una promesa ambigua de compilar cualquier modelo.

CAPÍTULO 15

La jerarquía de memoria: por qué multiplicar no es todo el trabajo

15.1 La cocina y el almacén

Imagina que cada vez que cortas una verdura debes caminar hasta un almacén situado a cien metros. Un cuchillo diez veces más rápido apenas resolvería el problema. Primero necesitas organizar qué ingredientes mantienes cerca y cuántas veces los reutilizas.

Una CPU tiene una jerarquía de almacenamiento: registros muy próximos a las unidades de ejecución, varios niveles de caché y memoria principal. La analogía de distancias no representa literalmente sus tiempos, pero ayuda a separar dos recursos: capacidad de calcular y capacidad de suministrar datos.

15.2 Latencia y ancho de banda

La **latencia** es cuánto tarda en completarse una petición. El **ancho de banda** es cuánto volumen puede trasladarse por unidad de tiempo cuando existe suficiente trabajo en curso. Un sistema puede tener alta latencia y alto ancho de banda: una petición aislada tarda, pero muchas peticiones simultáneas entregan gran cantidad de datos.

Un kernel de aprendizaje automático puede estar limitado por cualquiera de ellos, por dependencias, por ocupación o por costes de lanzamiento. «La memoria es lenta» es una explicación insuficiente si no identificamos qué comportamiento concreto domina.

15.3 Caché y localidad

Una caché conserva datos que probablemente volveremos a usar. La **localidad temporal** aparece cuando reutilizamos pronto un valor. La **localidad espacial** aparece cuando accedemos a posiciones cercanas. Los arrays contiguos ayudan a explotar esta segunda propiedad.

En una suma de vectores, cada entrada suele utilizarse una vez. En GEMM, un elemento de A puede contribuir a varias columnas y uno de B a varias filas. Esa reutilización potencial explica por qué GEMM puede beneficiarse tanto del blocking.

No es suficiente que los datos sean reutilizables matemáticamente. El orden del programa debe acercar esos usos en el tiempo y mantener un conjunto de trabajo que quepa en el nivel de memoria relevante.

15.4 Líneas de caché y accesos dispersos

Las cachés mueven datos en unidades que contienen varios bytes consecutivos. Leer un único valor de cada región muy distante puede desperdiciar gran parte de los datos trasladados. Leer posiciones contiguas aprovecha mejor cada transferencia.

El tamaño exacto de líneas y cachés depende del procesador. No debemos usar cifras de un modelo como si describieran todos los portátiles. En un informe se registra la CPU y, cuando sea necesario, sus propiedades consultadas mediante herramientas del sistema.

15.5 Intensidad aritmética

La intensidad aritmética compara operaciones de cálculo con bytes movidos. Una suma de vectores FP32 realiza aproximadamente una operación por dos lecturas y una escritura, unos doce bytes solicitados por elemento. Su intensidad es baja.

Una GEMM de tamaño grande puede reutilizar datos muchas veces, aumentando operaciones por byte trasladado desde memoria lejana. La intensidad depende de qué nivel de la jerarquía estamos midiendo y de la implementación, no solo de la fórmula del operador.

Un cociente con unidades

$$I = \frac{\text{operaciones de coma flotante}}{\text{bytes transferidos}}.$$

Si I vale 0,1 FLOP/byte y el ancho de banda sostenible es 100 GB/s, un techo idealizado asociado a esa transferencia es 10 GFLOP/s. Son cifras inventadas para practicar unidades, no especificaciones de una máquina de esta entrega.

15.6 El modelo roofline

Roofline compara un techo de capacidad de cómputo con otro impuesto por ancho de banda e intensidad. La cota idealizada de rendimiento es el mínimo entre ambos. El modelo ayuda a decidir si tiene sentido reducir operaciones o, antes, reducir tráfico. [39]

$$P \leq \min(P_{\text{cómputo}}, B_{\text{memoria}} I).$$

Una cota no es una predicción exacta. Ignora o simplifica latencias, dependencias, instrucciones, cachés, paralelismo disponible y comportamiento del compilador. Si un kernel queda muy por debajo de ambos techos, el modelo no queda refutado: falta explicar qué otros límites aparecen.

15.7 Contar tráfico con cuidado

Leer A y B y escribir C una vez da una estimación mínima para GEMM, pero una implementación ingenua puede leer los mismos valores muchas veces desde distintos niveles. Algunas escrituras provocan tráfico adicional. El resultado de un contador de hardware tampoco coincide necesariamente con el número de accesos del código fuente.

Un informe debe distinguir bytes lógicos solicitados, bytes estimados por un modelo y bytes observados por contadores. Mezclarlos puede producir intensidades imposibles o comparaciones engañosas.

15.8 Blocking: trabajar por porciones que se reutilizan

En lugar de calcular una salida completa y pasar a la siguiente, podemos calcular un bloque de salidas. Un pequeño conjunto de A y B alimenta varias multiplicaciones antes de abandonar la caché o los registros.

Para un bloque de cuatro filas y ocho columnas, un valor de A puede multiplicarse por ocho valores de B; una fila parcial de B puede utilizarse para cuatro filas de salida. El trabajo se organiza para conservar esos datos cerca de las unidades de cálculo.

La elección de tamaños depende de registros, caché, SIMD, número de hilos y dimensiones reales. Un tile mayor no siempre es mejor: puede expulsar datos que se querían reutilizar o producir demasiados acumuladores.

15.9 Prefetch y predicción de accesos

Algunos procesadores intentan anticipar patrones de acceso. Un recorrido regular puede facilitar esa anticipación. Los programas también pueden incluir instrucciones de prefetch, pero usarlas sin medir puede empeorar el tráfico o traer datos demasiado pronto.

Antes de añadir prefetch manual, consigue un recorrido claro, un conjunto de trabajo razonable y un benchmark estable. La complejidad de bajo nivel no es un indicador de calidad. Un kernel sencillo que vectoriza bien puede superar uno lleno de instrucciones explícitas mal organizadas.

15.10 NUMA y afinidad

En sistemas con varios dominios de memoria, el lugar donde se reserva o se toca primero un buffer puede influir en qué procesador accede a él con menor coste. Esto se conoce como NUMA. También importa dónde se ejecutan los hilos y si migran entre núcleos.

No todos los equipos del curso tienen varios nodos NUMA, pero el concepto explica por qué un benchmark de servidor puede cambiar al modificar afinidad. Registra número de hilos, política de colocación y configuración; no atribuyas automáticamente toda variación al compilador.

15.11 Una investigación guiada

Ejecuta una suma de vectores con tamaños que aumenten por potencias de dos. Para cada tamaño mide varias repeticiones calientes y una política separada de datos fríos. Convierte tiempo a bytes lógicos por segundo y anota dónde cambia el comportamiento.

No etiquetes cada cambio como «L1», «L2» o «RAM» únicamente por su forma visual. Utiliza información del procesador y, si está disponible, contadores. Una curva sugiere hipótesis; no identifica por sí sola un componente físico.

Decidir qué optimizar

Un kernel lee dos arrays FP32, suma sus elementos y escribe uno. Has eliminado una operación entera de cálculo de índice, pero el tiempo apenas cambia. Propón una explicación coherente con roofline y otra que no dependa de ancho de banda.

Solución. Puede estar limitado por tráfico de memoria, de modo que ahorrar una operación pequeña no cambia el techo dominante. También puede ser tan pequeño que domine la llamada o el lanzamiento. Hay que medir varios tamaños y separar costes antes de concluir cuál explicación corresponde.

Antes de continuar

Debes manejar unidades de FLOP, byte y segundo; distinguir localidad temporal y espacial; y explicar por qué la reutilización matemática necesita un orden de ejecución adecuado para convertirse en reutilización física.

CAPÍTULO 16

Upcasting, unrolling y SIMD: varias casillas por instrucción

16.1 Ocho lápices que avanzan juntos

Imagina que puedes escribir ocho resultados de la misma clase a la vez, utilizando ocho parejas de entradas. SIMD significa «una instrucción, múltiples datos». La operación es común, pero cada carril trabaja con valores distintos.

No confundas SIMD con varios núcleos independientes. Un núcleo puede ejecutar instrucciones vectoriales, y varios núcleos pueden hacerlo a la vez. Tampoco confundas SIMD con una GPU completa: comparten ideas de paralelismo de datos, pero sus modelos de ejecución y memoria tienen diferencias importantes.

16.2 Del bucle escalar al vector

Un bucle escalar produce una casilla por vuelta. Un bucle vectorizado puede procesar grupos de cuatro u ocho, según tipo e ISA. Para N que no sea múltiplo del ancho, hace falta tratar la cola mediante máscaras o un bucle escalar.

```
int i = 0;
for (; i + 3 < N; i += 4) {
    y[i]    = x[i]    + 2.0f;
    y[i+1]  = x[i+1]  + 2.0f;
    y[i+2]  = x[i+2]  + 2.0f;
    y[i+3]  = x[i+3]  + 2.0f;
}
for (; i < N; i++) y[i] = x[i] + 2.0f;
```

El código muestra un desenrollado de cuatro posiciones. No garantiza por sí solo que el compilador produzca una instrucción SIMD concreta. La generación real depende del compilador, las opciones, el target y los obstáculos de aliasing o dependencias.

16.3 Unrolling no es necesariamente vectorización

Desenrollar copia el cuerpo del bucle para varias iteraciones. Puede reducir control de bucle y exponer independencia de instrucciones. Vectorizar agrupa operaciones en instrucciones de múltiples carriles. Podemos desenrollar sin vectorizar, vectorizar sin un desenrollado visible en el fuente o combinar ambas técnicas.

Un exceso de desenrollado aumenta tamaño de código y valores vivos. El compilador puede necesitar más registros o perder eficacia de la caché de instrucciones. Por eso la cantidad de

copias es un parámetro de scheduling, no una mejora automática.

16.4 Qué significa upcasting en este contexto

En el vocabulario de ciertos compiladores de tensores, upcasting puede referirse a convertir parte de un eje iterativo en un conjunto de valores procesados dentro de una misma instancia, a menudo representados como carriles o vectores. No debe confundirse con promover FP16 a FP32, que es otro uso habitual de la palabra en programación.

En el commit estudiado de tinygrad aparecen ejes UPCAST y UNROLL y transformaciones que expanden rangos y vectores. El sentido concreto se entiende leyendo el pase y sus invariantes, no trasladando una definición de «casting de tipos» sin mirar el código. [4]

16.5 Acumuladores múltiples

Una reducción secuencial tiene una dependencia: cada suma necesita el acumulador anterior. Utilizar varios acumuladores independientes puede exponer más paralelismo. Por ejemplo, sumar posiciones de cuatro clases de resto por separado y combinar al final.

```
float s0=0, s1=0, s2=0, s3=0;
int i=0;
for (; i+3<N; i+=4) {
    s0 += x[i];    s1 += x[i+1];
    s2 += x[i+2]; s3 += x[i+3];
}
float s = (s0+s1)+(s2+s3);
for (; i<N; i++) s += x[i];
```

El orden numérico ya no coincide con la cadena original. No ocultes ese cambio bajo la palabra «vectorización». Debe estar permitido por el contrato y probado con tolerancias apropiadas.

16.6 Alineación

Una dirección alineada cumple un múltiplo requerido o conveniente para una instrucción. Algunas instrucciones admiten cargas no alineadas; otras exigen condiciones más estrictas. Incluso cuando una carga no alineada es legal, cruzar ciertas fronteras puede cambiar el coste.

No añadas una promesa de alineación al compilador sin garantizar que el allocator y los offsets la cumplen. En Lumbre la arena y sus regiones tienen una política explícita; las vistas arbitrarias pueden romper alineaciones de subregiones, por lo que necesitarían análisis adicional.

16.7 Aliasing como obstáculo y como contrato

Si el compilador C no sabe si x e y se solapan, puede generar comprobaciones o evitar ciertos reordenamientos. `restrict` ofrece una promesa que puede ayudar, pero su uso incorrecto viola el contrato del lenguaje.

Una API de kernels puede declarar que entradas y salida no se solapan. Entonces el runtime debe hacer cumplir esa condición o elegir un camino seguro. La solución no consiste en añadir palabras clave hasta que el benchmark mejore, sino en conectar garantías de alto nivel con supuestos de bajo nivel.

16.8 Auto-vectorización y ensamblador

La primera estrategia razonable es escribir bucles simples, compilar para el target y consultar informes de vectorización. Después inspecciona ensamblador de un caso pequeño. Busca cargas vectoriales, operaciones aritméticas y tratamiento de colas, pero no deduzcas rendimiento únicamente contando instrucciones.

```
cc -O3 -S -fverbose-asm kernel.c -o kernel.s
```

Las opciones exactas de diagnóstico dependen del compilador. Guarda su versión. Una recomendación de GCC no se convierte automáticamente en una opción válida de Clang ni de un compilador de acelerador.

16.9 Intrinsics y despacho por ISA

Un intrinsic permite expresar una operación ligada a una ISA desde C o C++. Puede ofrecer control útil, pero reduce portabilidad y exige gestionar capacidades. Ejecutar instrucciones no soportadas puede fallar aunque el programa compile en otra máquina.

Una biblioteca portable puede compilar varias versiones y escoger en ejecución según la CPU. El fallback no debe desaparecer. El compilador educativo puede empezar con C escalar y auto-vectorización; el proyecto avanzado añade versiones específicas con guardias y tests sobre el mismo contrato.

16.10 Reducir presión de registros

Cada acumulador ocupa recursos. Un microkernel de cuatro por ocho salidas mantiene treinta y dos acumuladores escalares o su equivalente vectorial. Aumentar a ocho por dieciséis puede multiplicar necesidades y causar spills.

El tamaño óptimo depende de cuántos registros existen, qué otras variables están vivas y cómo el compilador asigna instrucciones. Por eso una búsqueda de tiles debe incluir medición y descartar configuraciones que superen recursos, en lugar de escoger siempre el bloque de mayor superficie.

16.11 Un ejercicio de predicción de colas

Con $N=19$ y ancho ocho, hay dos grupos completos de ocho y una cola de tres. Una versión enmascarada puede lanzar un tercer grupo con cinco carriles inactivos. Una versión escalar puede procesar tres iteraciones finales. Ambas deben evitar leer más allá de la entrada.

El relleno de la cola depende de la operación. Para suma parcial, cero es identidad. Para máximo, cero sería incorrecto si todos los datos válidos son negativos; hay que usar una identidad compatible, como menos infinito bajo el contrato adoptado.

Qué cambió realmente

Una optimización transforma una reducción de un acumulador en ocho acumuladores y luego reduce esos ocho. El tiempo mejora y el último bit cambia. ¿Es necesariamente un bug? ¿Qué información necesitas para responder?

Solución. Necesitas conocer la equivalencia exigida. Si se requiere reproducibilidad bit a bit con el recorrido original, el cambio incumple el contrato. Si se permite una reducción reordenada dentro de una tolerancia y las pruebas la cumplen, puede ser válida. La mejora de tiempo no decide por sí sola la cuestión.

Antes de continuar

Distingue upcasting de ejes y conversión de tipos, unrolling y SIMD, legalidad de una carga y su eficiencia. Explica por qué cada promesa de alineación o ausencia de alias debe respaldarse desde el runtime.

CAPÍTULO 17

GEMM rápida: del triple bucle al microkernel

17.1 Empezar por una salida correcta

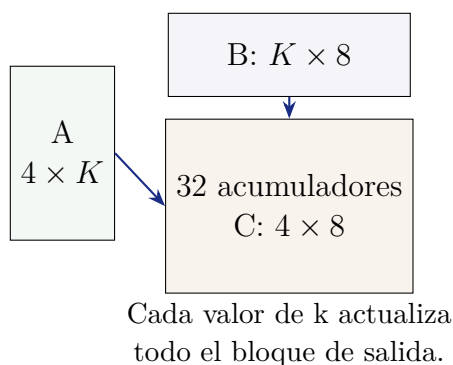
La multiplicación matricial es un caso central porque aparece en capas lineales, atención y otros operadores. La primera implementación debe ser transparente: para cada salida, sumar K productos en orden. Esa versión sirve como referencia y como instrumento para localizar errores de índices.

Optimizar antes de disponer de esa referencia complica el diagnóstico. Cuando un microkernel tiene tiles, packing, colas y vectorización, un resultado incorrecto puede tener muchas causas. Una implementación lenta no es un fracaso: es una pieza del sistema de verificación.

17.2 Reutilizar una entrada de A

En el orden i, j, k , un valor $A[i, k]$ vuelve a cargarse lógicamente para varias columnas j . Cambiar a un bloque de columnas permite usarlo con varios valores contiguos de B . El microkernel mantiene varias salidas en acumuladores.

Para cuatro filas y ocho columnas, cada paso k carga cuatro valores de A y ocho de B , y actualiza treinta y dos productos. Esta cuenta idealizada ilustra reutilización; el número real de instrucciones y transferencias depende de cómo se implemente y compile.



17.3 Tres niveles de blocking

Un microkernel organiza registros. Un bloque mayor organiza caché. Una partición entre hilos organiza paralelismo de CPU. Son escalas diferentes. Un tamaño adecuado para registros no tiene por qué llenar una caché, y un bloque de caché no tiene por qué ser la unidad de reparto entre núcleos.

Las bibliotecas de álgebra lineal de alto rendimiento combinan estos niveles con packing, kernels específicos y selección por forma. Nuestro objetivo docente es entender sus razones y construir una versión medible, no afirmar que unas decenas de líneas reemplazan todos sus años de ingeniería.

17.4 Packing

Empaquetar copia un panel de datos a un formato que facilita el microkernel: contigüidad, alineación y un orden de acceso regular. La copia tiene un coste inicial. Compensa cuando el panel se reutiliza lo suficiente o cuando evita accesos muy ineficientes.

El packing debe incluir o tratar las colas de forma segura. Si rellenamos un panel con ceros, el microkernel puede ejecutar un tamaño fijo sin leer fuera de rango, siempre que las posiciones de salida y la semántica numérica se controlen. No debemos escribir fuera de C aunque las entradas estén correctamente rellenadas.

Una matriz pequeña puede perder rendimiento por el coste de packing. Por eso los sistemas maduros suelen tener caminos distintos para formas pequeñas, matrices estrechas y grandes GEMM cuadradas.

17.5 El microkernel incluido en Lumbre

La opción `gemm="blocked"` genera una implementación CPU con bloques de cuatro filas, ocho columnas y tramos de K de hasta sesenta y cuatro. Mantiene un array local de acumuladores, recorre K en orden creciente y trata explícitamente las colas de M y N.

No utiliza BLAS ni delega el producto a NumPy. Tampoco implementa packing completo, despacho por ISA ni paralelismo multicore. Es una mejora original y comprobada frente al backend de referencia del propio compilador, no una afirmación de liderazgo frente a BLIS, oneDNN, MKL u otra biblioteca.

```
A = param("A", (128, 128))
B = param("B", (128, 128))
C = A @ B
programa = Program([C], gemm="blocked")
```

El scheduler materializa los operandos de la llamada para garantizar buffers contiguos con un contrato sencillo. Esa materialización puede costar cuando una entrada es una vista o expresión. Una versión avanzada puede integrar movimientos o packing en la carga, pero debe medir el coste de extremo a extremo.

17.6 Colas y dimensiones cero

Para M=9 y N=17 aparecen bloques incompletos en ambas direcciones. Cada acceso y cada escritura debe comprobar su posición. K=0 produce una suma vacía: las salidas deben valer cero bajo este contrato.

Un test que solo utiliza 128 por 128 no examina estas fronteras. La batería de la entrega

incluye formas no múltiplos del microkernel y un caso $K=0$. Las pruebas de broadcasting de lote comprueban además que una misma matriz B puede reutilizarse en varios lotes.

17.7 Orden de acumulación

El microkernel conserva el orden creciente de k dentro de cada salida. La política CPU desactiva contracción FMA. Esto facilita comparar con la referencia, aunque un compilador de bajo nivel puede seguir elegir instrucciones y optimizaciones legales dentro de sus reglas.

Para un proyecto SIMD con FMA, el contrato puede permitir diferencias pequeñas. Guarda métricas de error máximo y distribución, no solo una bandera de aprobado. Si un cambio reduce error respecto a FP64 pero rompe una exigencia bit a bit, sigue siendo una modificación de contrato que debe declararse.

17.8 Matrices por lotes y transpuestas

Los ejes de lote se convierten en offsets independientes. Si B tiene lote de tamaño uno y A varios, B puede reutilizarse. Los ejes de matrices siguen teniendo su contrato (M,K) y (K,N) . No conviene mezclar broadcasting de lote con una transposición implícita de los dos últimos ejes.

Un backend de biblioteca puede aceptar flags de transposición. Un microkernel propio puede preferir materializar o empaquetar. El compilador decide qué convención entrega a cada implementación y conserva una referencia que compruebe el resultado.

17.9 Medidas locales de esta edición

El script `examples/bench_kernels.py` compara los dos caminos CPU con datos residentes, después de calentamiento y sin incluir compilación ni copias de salida. En la ejecución registrada, la mediana para $128 \times 128 \times 128$ fue aproximadamente 1,566 ms en la referencia y 0,157 ms en la versión bloqueada. Para $256 \times 256 \times 256$ fue aproximadamente 15,247 ms y 1,371 ms, respectivamente.

Estas cifras pertenecen a la CPU virtual del entorno de elaboración y a esa configuración concreta. No predicen tiempos de un portátil ni demuestran superioridad frente a bibliotecas de producción. El interés didáctico es que una transformación visible de organización produjo una mejora medible sin cambiar la operación solicitada.

17.10 Cuándo decir «competitivo»

Para justificar ese término, define competidor, versión, hardware, forma, tipo, layout, número de hilos, calentamiento, modo numérico y alcance temporal. Compara varias formas representativas, no solo la que favorece al microkernel. Incluye costes de packing o conversiones que el usuario real deba pagar.

Una curva de speedup por forma es más informativa que un único máximo. También lo es informar de casos donde se pierde. La meta del temario es aprender a producir y evaluar

código competitivo; la palabra no debe sustituir al experimento que la demostraría.

17.11 Progresión de ingeniería

Después de la versión bloqueada, una extensión razonable incorpora packing y un microkernel vectorial. La siguiente añade selección por forma y varios hilos. Más tarde se pueden estudiar prefetch, división de K y layouts específicos. Cada paso debe conservar tests y aislar qué cambio explica el rendimiento.

No cambies a la vez algoritmo, tipo, tolerancia y número de hilos para luego atribuir todo el speedup a «mi compilador». La ablación, es decir, activar y desactivar componentes de forma controlada, permite separar sus efectos.

Contar la reutilización

Un microkernel produce un bloque de 2×4 salidas. Para cada k , ¿cuántos valores distintos de A y B necesita idealmente y cuántos productos calcula? ¿Por qué esa cuenta no basta para conocer su tiempo?

Solución. Necesita dos valores de A y cuatro de B y calcula ocho productos. El tiempo depende además de cargas reales, registros, instrucciones, dependencias, alineación, caché, colas y overhead. La cuenta muestra una oportunidad de reutilización, no un benchmark.

Antes de continuar

Explica registros, caché y reparto de hilos como tres niveles distintos. Identifica dónde aparecen packing, epílogo y colas. Después lee el helper `lm_gemm` generado por Lumbre y relaciona cada bucle con una de esas decisiones.

Convoluciones y kernels de reducción rápidos

18.1 Reutilización que se solapa entre ventanas

En una convolución de filtro 3×3 y paso uno, dos ventanas vecinas comparten gran parte de sus entradas. Una implementación que vuelve a traer toda la ventana desde memoria lejana desaprovecha esa estructura. El problema se parece a GEMM, pero el patrón de acceso incorpora geometría, bordes y canales.

El primer objetivo sigue siendo una referencia correcta. El segundo es escoger una organización que reutilice entradas y pesos. El tercero es decidir si una especialización compensa frente a una descomposición en GEMM. No hay una única «convolución rápida» independiente de forma, layout y hardware.

18.2 Contrato completo de conv2d

Una firma debe especificar layout de entrada, layout de pesos, stride, padding, dilatación, grupos, tipo y forma de salida. Una convolución con grupos divide canales en subconjuntos; una depthwise utiliza una estructura aún más particular. No basta con declarar que el filtro mide tres.

En la referencia del laboratorio usaremos NCHW y pesos OIHW, sin grupos. Las coordenadas de entrada para una posición de salida (oy,ox) y filtro (ry,rx) son $iy = oy * stride + ry * dilation - padding$ e $ix = ox * stride + rx * dilation - padding$. Si esas coordenadas quedan fuera de la imagen, aportan cero bajo el padding elegido.

```
float acc = 0.0f;
for (int c = 0; c < Cin; c++)
    for (int ry = 0; ry < R; ry++)
        for (int rx = 0; rx < S; rx++) {
            int iy = oy*stride + ry*dilation - padding;
            int ix = ox*stride + rx*dilation - padding;
            if (0 <= iy && iy < H && 0 <= ix && ix < W)
                acc += X[((n*Cin+c)*H+iy)*W+ix]
                    * F[((o*Cin+c)*R+ry)*S+rx];
        }
```

El fragmento es el cuerpo de una salida; los bucles sobre n, o, oy y ox lo rodean. El kernel completo del paquete valida tamaños y calcula todas las salidas.

18.3 Separar interior y bordes

Las posiciones interiores tienen ventanas completamente válidas y no necesitan comprobar cada acceso. Los bordes sí. Un compilador puede generar un kernel o región rápida para el interior y otro camino general para el borde.

El beneficio depende del tamaño de la imagen y del filtro. En imágenes diminutas, gran parte de las posiciones son borde y la separación puede añadir complejidad sin ganar. En imágenes grandes, eliminar ramas del interior puede facilitar vectorización y optimización de direcciones.

No elimines la máscara del borde basándote en que «normalmente el padding es pequeño». La prueba debe derivar el rango exacto de coordenadas de salida cuyo filtro queda dentro de límites.

18.4 Transformar a GEMM paso a paso

Para cada posición de salida, enumera los valores de su ventana a través de canales y coordenadas de filtro. Esa lista tiene longitud $C_{in} * R * S$. Colócala como una fila de una matriz P . Aplana cada filtro de salida como una columna de una matriz W . Entonces $P @ W$ produce los canales de salida para todas las posiciones.

La transformación conserva el orden de los productos si se define una enumeración consistente. El resultado debe reorganizarse al layout de salida. El coste de construir P , su tamaño y sus copias forman parte del operador, no son un gasto que se pueda ocultar al comparar con una convolución directa.

Un caso de una dimensión

La señal $[1, 2, 3, 4]$ y filtro $[2, 1]$ producen P con filas $[1, 2]$, $[2, 3]$, $[3, 4]$. Multiplicar P por la columna $[2, 1]$ da $[4, 7, 10]$. El ejemplo muestra qué se duplica: los valores 2 y 3 aparecen en varias filas de P .

18.5 Convolución implícita

Una versión implícita no escribe P completa. El microkernel solicita un elemento de P y una función de índices lo traduce a X o a cero de padding. Esto reduce almacenamiento intermedio, pero introduce cálculos de dirección y puede complicar cargas vectoriales.

El compilador puede precomputar partes de esos índices, desenrollar tamaños de filtro pequeños o empaquetar paneles en lugar de toda la matriz. Es un espacio de decisiones más rico que «directa o im2col», y permite proyectos de autotuning bien delimitados.

18.6 Layouts de canales

En NCHW, posiciones espaciales vecinas de un mismo canal son contiguas. En NHWC, los canales de un píxel son contiguos. Una implementación puede preferir una u otra según cómo vectoriza y cómo alimenta instrucciones matriciales.

Cambiar todo el modelo a otro layout tiene un coste si algunas capas necesitan conversiones. Un kernel aislado más rápido puede empeorar la red completa al introducir transposiciones adicionales. El benchmark de una capa debe complementarse con una medida de la cadena donde se utiliza.

18.7 Reducciones especializadas

Una suma por fila y una suma por columna tienen la misma clase algebraica, pero diferentes accesos en un layout dado. Un kernel rápido puede asignar varios elementos contiguos a cada carril, acumular localmente y reducir entre carriles. Para columnas puede convenir una transposición por bloques o una distribución distinta.

El tamaño de la reducción también cambia la estrategia. Una reducción de ocho valores no necesita el mismo diseño que una de un millón. El coste de lanzamiento y la cantidad de filas disponibles para paralelismo orientan la selección.

18.8 Fusionar estadísticas y epílogos

En normalización, el cálculo de estadísticas y la transformación final están relacionados. Mantener una fila o un bloque cerca permite evitar lecturas repetidas. Sin embargo, si la fila no cabe en memoria local, pueden ser necesarias varias pasadas.

Para varianza, fórmulas algebraicamente equivalentes pueden tener estabilidad diferente. Calcular media de cuadrados menos cuadrado de la media puede perder precisión cuando los valores tienen una media grande y una dispersión pequeña. Una optimización de reducción debe analizar también ese régimen numérico.

18.9 Convolución y backward

El gradiente respecto a pesos suma contribuciones de todas las ventanas. El gradiente respecto a entrada acumula contribuciones de las salidas cuyas ventanas incluían cada posición. Esa segunda operación tiene solapamientos: varias salidas influyen en una misma entrada.

Una estrategia de referencia puede asignar un elemento de gradiente de entrada a cada trabajador y recorrer sus contribuciones. Otra puede repartir salidas y acumular mediante atomics. Ambas pueden ser correctas bajo sus contratos, pero difieren en trabajo, regularidad y determinismo numérico.

18.10 Pruebas indispensables

Utiliza filtros asimétricos, imágenes no cuadradas, stride mayor que uno, dilatación y padding que creen bordes. Comprueba un caso pequeño a mano. Después compara una referencia directa con la descomposición `im2col` en FP64 o FP32 según el contrato.

Para backward, realiza diferencias finitas en unas pocas posiciones de entrada y pesos. No necesitas perturbar millones de parámetros para detectar una inversión del filtro o una condición de borde incorrecta.

18.11 Objetivo de rendimiento

La meta de semanas 5–6 no es declarar que toda convolución generada gana a una biblioteca. Es construir una ruta desde una referencia general hasta especializaciones evaluables. Para una afirmación competitiva, la matriz de casos debe incluir tamaños reales, layouts y costes de conversión.

En el libro, la convolución directa se aporta como laboratorio de referencia y base de extensión; el núcleo de entrenamiento del decoder no utiliza conv2d. Esta distinción evita confundir cobertura del temario con dependencias de un modelo concreto.

Elegir una estrategia

Tienes un filtro 1×1 sobre muchos píxeles y numerosos canales. ¿Qué simplifica respecto a un filtro 5×5 ? ¿Por qué puede parecerse especialmente a GEMM?

Solución. No hay una ventana espacial de varios píxeles: cada salida combina canales del mismo píxel, salvo el efecto de stride y otros parámetros. Reorganizando píxeles como filas y canales como características, la operación se acerca a una proyección matricial. El layout y las conversiones siguen importando.

Antes de continuar

Debes poder escribir la coordenada de una ventana, explicar qué repite `im2col` y detectar por qué un gradiente de entrada necesita acumulación. Después propone una prueba que distinga correlación cruzada de convolución con filtro invertido.

CAPÍTULO 19

Medir, perfilar y autotunear sin engañarse

19.1 El cronómetro también tiene un contrato

Antes de medir, escribe qué intervalo temporal te interesa. ¿Primera llamada con compilación? ¿Kernel caliente con datos residentes? ¿Operador completo con packing? ¿Entrenamiento por token incluyendo carga de datos? Son preguntas diferentes y todas pueden ser útiles.

Un benchmark no es bueno por tener muchos decimales. Es bueno cuando la cifra corresponde a una pregunta clara, el procedimiento es reproducible y la incertidumbre se conserva. No redondees una variación grande hasta convertirla en una mejora aparente pequeña.

19.2 Calentamiento

Las primeras llamadas pueden incluir carga de módulos, asignaciones, compilación diferida, cachés frías y cambios de frecuencia. Un benchmark caliente suele realizar varias llamadas antes de medir. Un benchmark de arranque debe conservar esos costes en lugar de eliminarlos.

No existe un número universal de warmups. Comprueba que el comportamiento se estabiliza para el caso estudiado y registra la política. En el script de la entrega se utilizan tres llamadas previas como procedimiento sencillo, no como garantía de estacionariedad de cualquier dispositivo.

19.3 Sincronizar lo que se mide

En una GPU, medir el tiempo del lanzamiento con un reloj CPU puede contar únicamente cuánto tarda el host en encolar trabajo. Para medir ejecución necesitas eventos del dispositivo o una sincronización bien situada. Sin esa precaución, puedes obtener un kernel «más rápido» que todavía no ha terminado.

Tampoco debes sincronizar más de lo que requiere el escenario real y llamar a ese resultado «rendimiento asíncrono». Una medida de kernels aislados y otra de una cadena con solapamiento deben describir sus puntos de sincronización.

19.4 Muestras y estadísticas

Recoge varias muestras. La mediana reduce la influencia de algunos valores extremos, pero no explica por sí sola la distribución. Conserva mínimo, máximo, cuantiles y, cuando sea útil, intervalos de confianza calculados sobre un procedimiento adecuado.

Si seleccionas la mejor de mil ejecuciones de una versión y la media de diez de otra, la comparación favorece artificialmente a la primera. Aplica la misma política. Intercalar versiones o aleatorizar el orden puede reducir efectos de deriva térmica y de carga externa.

19.5 Corrección antes de tiempo

Un candidato que no produce la salida correcta queda fuera de la comparación de rendimiento del mismo operador. No puede ganar por omitir una reducción, usar etiquetas equivocadas o escribir solo una parte de la salida.

Las pruebas de corrección se realizan fuera del intervalo medido, pero con los mismos tipos, formas y parámetros del candidato. No pruebes un kernel escalar seguro y midas otro vectorizado sin validarlo. El hash del código ayuda a unir evidencia de corrección y evidencia de tiempo.

19.6 Throughput y latencia

La latencia mide cuánto tarda una tarea. El throughput mide cuántas tareas o tokens se procesan por unidad de tiempo. Aumentar batch puede mejorar throughput y empeorar latencia por petición. En entrenamiento suele interesar tokens por segundo a una configuración de batch y secuencia; en generación interactiva también importa el tiempo por token y la primera respuesta.

La cantidad de FLOP/s es una métrica de utilización bajo una convención de conteo. No es una medida directa de calidad del modelo ni de eficiencia económica. Un algoritmo que realiza menos operaciones puede ser más rápido y mostrar menos FLOP/s.

19.7 El comparador PyTorch

Para comparar con PyTorch, identifica si utiliza ejecución eager, compilación, una biblioteca fusionada o un kernel especial. Fija dtype, TF32 cuando corresponda, layout, dispositivo y forma. Una llamada llamada `matmul` puede escoger implementaciones distintas según esos detalles.

No atribuyas a PyTorch el coste de transferir datos desde CPU mientras tu candidato recibe datos residentes, salvo que esa sea precisamente la comparación de escenarios que deseas estudiar y la expliques. El usuario necesita saber qué costes pagará en su aplicación.

19.8 Autotuning

Autotuning explora configuraciones de scheduling: tamaños de tile, número de hilos, unrolling, stages, layout y elección de algoritmo. Cada candidato pasa validación y medición. Después se conserva la configuración elegida para una clave de problema.

El espacio debe limitarse mediante restricciones de recursos y conocimiento del hardware. Probar combinaciones ilegales consume tiempo y puede bloquear o fallar. Una estrategia útil

comienza con candidatos seguros y utiliza modelos para priorizar, sin confundir predicción de coste con tiempo observado.

```
Para cada configuración candidata:
  comprobar restricciones estáticas
  generar y compilar
  verificar resultados
  medir con la política acordada
  registrar artefacto, error y tiempos
Elegir según el objetivo y guardar la clave completa
```

19.9 Coste de buscar

Si autotunear tarda diez segundos y ahorra un milisegundo por ejecución, hacen falta aproximadamente diez mil ejecuciones para recuperar ese coste, ignorando otros gastos. La cuenta es un ejemplo de amortización, no una razón para prohibir búsqueda.

Un servicio que ejecuta la misma forma millones de veces puede beneficiarse mucho. Una herramienta interactiva con formas casi siempre nuevas puede preferir una buena heurística inicial. El compilador puede separar un modo de arranque rápido y otro de optimización persistente.

19.10 Overfitting del autotuner

Escoger el tile que gana en un único tamaño puede producir una política frágil. Un proyecto de selección por forma debe separar casos de ajuste y casos de evaluación. También debe distinguir entre memorizar cada forma exacta y aprender una regla que generalice a formas nuevas.

El mismo principio se aplica a agentes que generan kernels. Si el agente ve los tests finales y puede especializarse a sus datos concretos, el resultado no demuestra una implementación general del operador. Deben variar valores y formas dentro del contrato.

19.11 Perfilado

Un perfilador ayuda a identificar dónde se gasta tiempo, cuánta memoria se mueve y qué recursos limitan. Empieza por una vista de alto nivel: tiempo por kernel, número de lanzamientos, transferencias y sincronizaciones. Después profundiza en el kernel dominante.

No optimices una operación que representa el uno por ciento del tiempo esperando multiplicar por dos el programa completo. La ley de Amdahl formaliza esta intuición: incluso eliminar por completo una parte pequeña deja casi todo el tiempo intacto.

$$S = \frac{1}{(1 - f) + f/s}.$$

Aquí f es la fracción original mejorada y s su factor de aceleración. Si $f=0,1$ y $s=10$, el speedup total es aproximadamente 1,10, no diez. La fórmula presupone que el resto no cambia; en sistemas reales puede haber efectos secundarios que también deban medirse.

19.12 Energía y coste

Un kernel más rápido puede consumir más potencia instantánea y aun así menos energía total, o no. La energía integra potencia durante el tiempo. Un presupuesto de entrenamiento debe considerar horas, precio de recursos, almacenamiento y transferencias, no solo el pico de FLOP/s anunciado.

El libro no fija precios de nube a octubre de 2026 sin una consulta específica y una región: cambian y dependen de reservas, disponibilidad y proveedor. Para planificar un proyecto, utiliza un presupuesto parametrizado y actualiza sus entradas al contratar recursos.

19.13 Una tabla de resultados honesta

Incluye forma, tipo, backend, estrategia, error, mediana, dispersión, memoria y coste de compilación. Conserva además configuración del sistema y fuente generada. Una tabla sin estas columnas puede servir como borrador personal, pero no sostiene una afirmación comparativa fuerte.

Los informes JSON del paquete registran muestras y artefactos del compilador. No incluyen contadores de hardware ni una comparación con bibliotecas externas; esas ausencias se declaran en lugar de rellenarse con estimaciones presentadas como mediciones.

Interpretar una mejora

Una versión reduce el kernel de 1 ms a 0,5 ms, pero añade un packing de 0,8 ms en cada llamada. La original no necesitaba packing. ¿Cuál gana si el panel se usa una vez? ¿Y si se usa cien veces sin volver a empaquetar?

Solución. Una vez, la nueva cuesta 1,3 ms y pierde frente a 1 ms. Cien veces, la original cuesta 100 ms y la nueva $0,8 + 50 = 50,8$ ms, suponiendo que el panel puede reutilizarse sin otros cambios. La reutilización es una precondition del segundo resultado.

Antes de continuar

Debes poder escribir el intervalo medido, separar latencia y throughput, justificar calentamiento y explicar el coste de buscar una configuración. Un benchmark debe permitir que tu hipótesis pierda.

CAPÍTULO 20

GPU: repartir posiciones entre miles de tareas

20.1 Una GPU no adivina el paralelismo

El compilador debe decidir qué trabajo corresponde a cada hilo, qué datos lee y dónde escribe. La GPU ejecuta esa organización bajo su modelo de hardware. Una fórmula con muchas operaciones no se vuelve paralela automáticamente por enviarla a un dispositivo.

Empezamos con una suma elemento a elemento porque sus salidas son independientes. Un hilo calcula una posición. Más adelante varios hilos cooperarán para una salida o un bloque de salidas. La dificultad crece cuando comparten memoria o dependen de resultados parciales.

20.2 Hilo, bloque y grid

Un **hilo** es una instancia del programa de kernel con índices propios. Un **bloque** agrupa hilos que pueden cooperar con recursos y sincronizaciones de bloque. Una **grid** reúne los bloques de un lanzamiento. CUDA proporciona identificadores para cada nivel. [9]

Para $N=10$ y bloques de cuatro hilos, lanzamos tres bloques. En el bloque b , el hilo t calcula $q=b*4+t$. Los primeros diez q son válidos; los dos últimos deben abstenerse de acceder a la salida. Esta es exactamente la división de rangos que ya hicimos en CPU.

```
__global__ void sumar(const float* x, float* y, long long n) {
    long long q = (long long)blockIdx.x * blockDim.x + threadIdx.x;
    if (q < n) y[q] = x[q] + 2.0f;
}
```

El keyword `__global__` marca una función lanzable como kernel. No indica que todas sus variables vivan en memoria global. Los nombres de CUDA requieren estudiar el contexto en que aparecen.

20.3 La guardia de la cola

Si un hilo tiene q fuera de rango, no debe leer $x[q]$ ni escribir $y[q]$. Poner la condición alrededor de ambas operaciones es la solución inicial. La expresión del índice utiliza un tipo suficientemente ancho antes de multiplicar índices, evitando que el producto se haga primero en una anchura menor.

El número de bloques se calcula con división redondeada hacia arriba: $(N+B-1)//B$ para B positivo. Para $N=0$, el runtime puede no lanzar nada. Lanzar una grid vacía o permitir

tamaños negativos debe manejarse mediante un contrato explícito del runtime.

20.4 Mapeo bidimensional

Para una matriz, podemos asignar una coordenada de fila y otra de columna a partir de bloque e hilo. Un bloque de 16×16 contiene 256 hilos lógicos y cubre una región de 16×16 salidas. Las colas de ambas dimensiones requieren máscaras.

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
if (row < M && col < N) C[row*N+col] = A[row*N+col] + B[row*N+col];
```

El producto de dimensiones del bloque debe respetar los límites del dispositivo. No copies un bloque de 1024×1024 porque la matriz tenga ese tamaño: bloque de ejecución y tamaño total del problema no son lo mismo.

20.5 SIMT y grupos de ejecución

En NVIDIA, los hilos se ejecutan en grupos llamados warps. Comparten una organización de ejecución que hace relevante si siguen el mismo camino de instrucciones. AMD utiliza wavefronts con características que dependen de arquitectura y modo. No se debe imponer un tamaño único de grupo a todos los backends. [9, 19]

Para un principiante, la intuición útil es que instrucciones similares sobre posiciones vecinas suelen aprovechar mejor el hardware que decisiones completamente divergentes. Pero la semántica no autoriza a omitir sincronizaciones por suponer que todos los hilos avanzan exactamente juntos.

20.6 Divergencia

Si unos hilos toman una rama y otros otra, el hardware debe gestionar los caminos. Una condición de borde suele ser inevitable y afecta solo a ciertos grupos. Una condición irregular basada en datos puede producir más divergencia.

Divergencia no significa automáticamente incorrección. El problema de corrección aparece cuando una operación colectiva o una barrera exige participación y algunos hilos no llegan. El rendimiento y la validez de la sincronización son cuestiones distintas.

20.7 Registros y memoria local

Las variables privadas de un hilo pueden residir en registros si el compilador y los recursos lo permiten. Si no, pueden almacenarse en memoria asociada a ese hilo. En terminología CUDA, «local memory» no significa necesariamente una memoria física cercana y rápida como shared memory.

Esta terminología confunde porque distintos niveles del stack usan «local» con sentidos diferentes. En un informe conviene escribir si hablamos de almacenamiento privado, memoria

compartida del bloque, caché o memoria global del dispositivo.

20.8 Ocupación

La ocupación se relaciona con cuántos grupos de ejecución pueden estar activos respecto a una capacidad del dispositivo. Registros por hilo, memoria compartida por bloque y tamaño del bloque pueden limitar esa residencia.

Una ocupación mayor no garantiza un kernel más rápido. Un microkernel con más registros puede reutilizar mejor datos y ganar aunque reduzca el número de grupos residentes. La ocupación es una señal para analizar latencia y recursos, no un objetivo que deba maximizarse aislado.

20.9 Memoria del host y del dispositivo

Una CPU y una GPU pueden tener espacios de memoria y mecanismos de transferencia distintos. En el runtime inicial reservamos buffers de dispositivo y copiamos explícitamente entradas y salidas. No suponemos que un puntero de NumPy sea directamente utilizable por cualquier kernel GPU.

La memoria unificada y los mecanismos de acceso compartido ofrecen otras posibilidades, pero también tienen costes y restricciones. Empezar con transferencias explícitas permite observar el movimiento de datos y no confundirlo con cálculo.

20.10 Primer hito GPU

El objetivo inicial no es alcanzar el techo del hardware. Es ejecutar una operación simple con tipos, tamaños, máscaras, errores y sincronización correctos. Después se incorporan reducciones y GEMM de referencia. Solo entonces tiene sentido optimizar accesos y cooperación.

Lumbre proporciona una ruta CUDA y una HIP que generan kernels a partir del mismo IR. La revisión del 1 de octubre ejecuta CUDA en una RTX 5070 Ti Laptop bajo WSL2, con toolkit 12.9 y destino `sm_120`; sus evidencias se guardan en `code/reports/validation_20261001/`. HIP sigue pendiente de hardware compatible. La validación en el dispositivo del estudiante sigue siendo una tarea concreta del laboratorio.

20.11 CUDA frente a HIP

HIP ofrece un modelo y APIs cercanos a CUDA para facilitar portabilidad, especialmente hacia AMD. La proximidad sintáctica permite compartir parte del renderer y del runtime, pero no garantiza rendimiento portable ni disponibilidad idéntica de instrucciones. [19]

Una suma independiente puede portarse con cambios pequeños. Una GEMM que depende de instrucciones matriciales, layouts de fragmentos, barreras y tamaños de grupo necesita un análisis específico. La portabilidad tiene varios niveles: fuente, compilación, corrección y rendimiento.

Cobertura de una grid

Una matriz tiene 33 filas y 17 columnas. Usas bloques de 16×16 . ¿Cuántos bloques lanzas en cada eje? ¿Cuántos hilos del bloque situado al final de ambos ejes escriben una salida válida?

Solución. Hacen falta tres bloques en filas y dos en columnas. El último bloque empieza en fila 32 y columna 16. Solo existe una fila y una columna válidas en esa región: un hilo escribe una salida. Los demás deben respetar las máscaras. Esto no significa que deban saltarse barreras colectivas si el kernel las utiliza.

Antes de continuar

Calcula el índice de un hilo, el tamaño de grid y una máscara de borde. Explica por qué una GPU necesita un plan de trabajo y por qué portabilidad de sintaxis no equivale a portabilidad de rendimiento.

CAPÍTULO 21

Call y runtime: hacer que el código llegue al dispositivo

21.1 Un kernel compilado todavía no se ha ejecutado

Entre la fórmula y un resultado GPU hay varias etapas: generar fuente o IR, compilar para un target, cargar un módulo, reservar memoria, transferir datos, resolver una función, preparar argumentos, lanzar y sincronizar. Un fallo en cualquiera puede impedir el resultado aunque la aritmética del kernel sea correcta.

Una operación **call** representa una invocación con un contrato. Puede llamar a un kernel generado, a una biblioteca especializada o a una función del runtime. El compilador necesita conocer argumentos, salidas, workspace, efectos y condiciones de validez.

21.2 Un contrato de llamada

Para GEMM, la llamada debe especificar dimensiones, tipos, layouts, alineación, dispositivo y si entradas y salida pueden solaparse. Para una operación asíncrona, también necesita definir qué evento indica finalización. Para una biblioteca, puede exigir un workspace y una configuración que dependan de la forma.

La firma «tres punteros» no contiene por sí sola toda esa información. El compilador puede mantener metadatos separados y validar antes del lanzamiento. Una llamada especializada no es una excepción a la corrección: es una frontera donde el contrato debe resultar especialmente claro.

21.3 Cargar un módulo y resolver una función

En el backend CPU, una biblioteca compartida se carga mediante **ctypes**. En GPU, una ruta puede utilizar APIs de módulo y funciones del driver, o compilar una biblioteca anfitriona que incluya lanzamientos. Lumbre utiliza esta segunda vía como solución didáctica.

El código generado contiene wrappers de reserva, copia, sincronización y liberación. El runtime Python declara sus firmas y comprueba códigos de retorno. La arquitectura concreta se indica para CUDA mediante **LUMBRE_CUDA_ARCH**; no se elige una capacidad arbitraria por el nombre comercial del ordenador.

21.4 Comprobar el target

El nombre de una familia de GPU no identifica todas sus instrucciones. La tabla de NVIDIA distingue, por ejemplo, las capacidades de H100/H200, B200/B300 y GeForce RTX 50. Una tarjeta Blackwell de consumo no debe tratarse como si fuera idéntica a una GPU Blackwell de centro de datos. [55]

La práctica correcta consulta el dispositivo real y el soporte del compilador instalado. Las variantes de target con sufijos y características específicas tienen reglas de compatibilidad propias. Compilar para un target avanzado no añade al silicio instrucciones que no posee.

21.5 Streams

Un stream organiza una secuencia de trabajo en un dispositivo. Operaciones dentro de una secuencia pueden respetar un orden, mientras varias secuencias permiten potencialmente solapamiento. Para coordinar streams se utilizan eventos y dependencias explícitas.

El solapamiento no es gratuito: requiere recursos disponibles y transferencias o kernels que realmente puedan ejecutarse a la vez. Dos tareas que compiten por el mismo ancho de banda pueden no mejorar al superponerse. La ausencia de sincronización global puede revelar carreras si los buffers se comparten sin dependencias.

21.6 Eventos

Un evento marca un punto de progreso. Otro stream o el host puede esperar a que se alcance. Los eventos también pueden servir para medir intervalos en el dispositivo, bajo las reglas de la API.

Un evento debe representar exactamente el trabajo del que depende el consumidor. Esperar a un evento grabado antes de una copia no garantiza que la copia haya terminado. Los diagramas de runtime deben indicar orden temporal, no solo conexiones visuales entre nombres.



Esta secuencia es el punto de partida. Una versión avanzada puede superponer la copia del siguiente batch con el cálculo del actual, manteniendo buffers separados y dependencias correctas.

21.7 Errores asíncronos

Un error de lanzamiento puede detectarse al encolar. Un acceso inválido dentro del kernel puede aparecer al sincronizar más tarde. Por eso un runtime no debe comprobar únicamente la llamada de lanzamiento y asumir que la ejecución fue correcta.

En Lumbre se comprueba el estado de lanzamiento y se sincroniza al terminar `run`. Para depurar, conviene reducir el caso a una forma pequeña y conservar la fuente generada. Después se utiliza la herramienta de diagnóstico disponible para el dispositivo, sin afirmar una validación que no se ha ejecutado.

21.8 Vida de buffers y módulos

Un buffer debe permanecer vivo hasta que todos sus consumidores terminen. Un módulo no debe descargarse mientras existan llamadas que dependan de él. Un objeto Python destruido por el recolector no debería liberar memoria que todavía utiliza un stream asíncrono.

La gestión mediante context manager ayuda a cerrar recursos de forma explícita, pero no elimina la necesidad de sincronización. La ruta del curso prioriza un ciclo de vida sencillo y visible. Un runtime de producción puede utilizar pools y referencias asociadas a eventos.

21.9 Caché del JIT y capacidades

Una caché GPU debe incluir arquitectura, opciones, versión del compilador y otras dependencias relevantes. Algunas bibliotecas generan variantes para formas y tipos concretos. Reutilizar un módulo con una clave incompleta puede producir un fallo o un resultado sutilmente incorrecto.

DeepJIT, publicado por DeepSeek, aborda compilación, caché, carga y lanzamiento para CUDA y Ascend desde una interfaz C++20. En el estado consultado, ciertas APIs Python y funciones de precalentamiento figuran como trabajo pendiente; no se utilizan en el libro como si fueran interfaces ya disponibles. [27]

21.10 Bibliotecas externas y fallback

Un compilador puede escoger una implementación de biblioteca cuando resulte adecuada y un kernel propio en otros casos. Debe registrar esa decisión. Una medida de un backend que llama a cuBLAS o hipBLASLt no se presenta como si todo su código aritmético hubiera sido generado desde cero por el estudiante.

El fallback es valioso para cobertura, pero debe ser observable en logs y métricas. Un programa que mezcla kernels propios y bibliotecas puede ser una solución excelente; ocultar esa mezcla impide interpretar sus resultados.

21.11 Un pasaporte de llamada

Registra nombre del operador, hash del código o biblioteca, dimensiones, tipo, layout, argumentos de lanzamiento, memoria compartida, workspace, stream y política numérica. Ese pasaporte permite reproducir un fallo y comparar implementaciones sin depender de nombres ambiguos.

El proyecto final de «pasaportes de kernels» extiende esta idea a verificación de precondiciones y selección automática. La originalidad no exige inventar una nueva instrucción de GPU: puede estar en unir contratos, evidencia y observabilidad de forma útil.

Una carrera entre copia y cálculo

Copias nuevos pesos en un stream y lanzas un kernel que los usa en otro, sin evento. A veces funciona. ¿Qué falta? ¿Basta con que ambas llamadas aparezcan en ese orden en

Python?

Solución. Falta una dependencia de finalización entre la copia y el consumidor. El orden de llamadas del host no establece por sí solo todas las dependencias entre streams distintos. Debe usarse una espera o una organización de stream que garantice el orden requerido. Que algunas ejecuciones funcionen no demuestra ausencia de carrera.

Antes de continuar

Enumera las etapas entre fuente y resultado. Distingue error de lanzamiento y error de ejecución, y explica qué información pertenece a la clave de caché y qué información pertenece a los datos de una llamada.

CAPÍTULO 22

Coalescing, shared memory y sincronización

22.1 Una fila de hilos y una fila de datos

Cuando hilos vecinos acceden a posiciones vecinas, el hardware puede agrupar solicitudes de memoria de forma eficiente. A esta propiedad se la llama coalescing en el contexto GPU. El patrón exacto de transacciones depende de arquitectura, alineación y tamaño de acceso. [9]

Si cada hilo salta una gran distancia, puede requerirse mover muchas regiones para utilizar pocos valores de cada una. La fórmula matemática no cambia, pero sí el coste de suministrar datos. La relación entre layout y mapeo de hilos es por tanto una decisión central del compilador.

22.2 Elegir qué eje sigue al hilo x

En un array por filas, las columnas contiguas suelen asignarse a hilos vecinos del eje x. Si asignamos filas a esos hilos, cada uno puede saltar un stride grande. En otros layouts o patrones de reutilización puede convenir otra organización.

El criterio no es «x siempre representa columnas» por una ley del lenguaje. Es «qué mapeo produce accesos útiles para la operación y el hardware». Las APIs ofrecen índices; el compilador les da significado.

22.3 Shared memory como mesa de trabajo del bloque

La memoria compartida de bloque permite que varios hilos carguen datos, los reutilicen y cooperen. No se rellena automáticamente: el programa debe decidir quién carga qué y cuándo los demás pueden leerlo.

En GEMM, un bloque puede cargar un tile de A y otro de B, sincronizar, calcular productos y repetir con el siguiente tramo de K. La reutilización dentro del bloque reduce accesos repetidos a memoria global.

22.4 Dos barreras por iteración

Después de cargar un tile, todos los participantes deben ver datos completos antes de calcular. Después de calcular, nadie debe sobrescribir ese almacenamiento con el siguiente tile mientras otro hilo todavía lee el anterior. Por eso un diseño sencillo utiliza una barrera tras la carga y otra antes de reutilizar el buffer.

```
cargar_tile_A_y_B();  
__syncthreads();  
acumular_productos_del_tile();  
__syncthreads();
```

El fragmento muestra el orden, no una función ejecutable por sí solo. El kernel completo generado por Lumbre contiene ambas barreras y cargas enmascaradas que escriben cero en posiciones fuera de rango.

22.5 La trampa del retorno temprano

En una suma sin cooperación, un hilo fuera de rango puede salir. En un kernel con barreras de bloque, hacer que solo algunos hilos retornen antes de una barrera puede violar sus requisitos. Los hilos de borde pueden no producir una salida válida y aun así tener que participar cargando ceros y alcanzando las barreras.

Una máscara de salida no es una máscara de participación

No encierres todo el cuerpo de una GEMM cooperativa en `if (row<M && col<N)` si dentro hay barreras que deben alcanzar todos los hilos del bloque. Enmascara accesos y escritura final; conserva la participación requerida en las operaciones colectivas.

22.6 GEMM compartida de 16×16

El backend bloqueado GPU de Lumbre utiliza dos arrays compartidos de 16×16 . Cada hilo carga una posición de A y una de B. Tras sincronizar, acumula dieciséis productos del tile en su salida. Después sincroniza de nuevo y avanza K.

La implementación es pedagógica: cada hilo mantiene un acumulador escalar FP32. No utiliza tensor cores ni una canalización asíncrona avanzada. Permite estudiar coalescing, reutilización y colas con un código que se corresponde directamente con los capítulos anteriores.

22.7 Bancos de shared memory

La memoria compartida se organiza en bancos. Ciertos patrones de acceso pueden hacer que varios hilos compitan por un mismo banco, salvo casos soportados como difusión de un mismo dato. Padding y swizzling pueden cambiar el mapeo de direcciones para reducir conflictos.

El patrón óptimo depende del tamaño de elemento y de las instrucciones que consumen los datos. Una disposición que parece ideal para cargas escalares puede no ser la que necesita una instrucción matricial. Por eso los layouts de bibliotecas como CuTe se tratan como funciones de coordenadas a ubicaciones, no como simples nombres «row-major» y «column-major».

22.8 Swizzling

Un swizzle reorganiza direcciones mediante una transformación estructurada, a menudo utilizando combinaciones de bits de índices. La intención es distribuir accesos entre bancos

o adaptar los datos a un consumidor. No cambia qué valor corresponde a cada coordenada lógica si el mapa y su uso son consistentes.

Para comenzar, dibuja una tabla pequeña de coordenadas y direcciones. Comprueba que no hay colisiones indebidas y que la transformación es invertible sobre el dominio requerido. Después evalúa el patrón de bancos. No copies una fórmula XOR de otro kernel sin entender sus dimensiones y alineaciones.

22.9 Registros compartidos por una operación colectiva

Algunas instrucciones de grupo distribuyen fragmentos entre carriles. Un fragmento lógico de matriz no reside necesariamente como una submatriz intuitiva en cada hilo. El contrato de carga, cómputo y almacenamiento define cómo cooperan.

No uses el contenido de un fragmento de otro target suponiendo que el reparto interno es idéntico. Cuando una API documenta un layout opaco, debe tratarse como opaco. Un backend de bajo nivel que maneja registros explícitos necesita especificaciones más detalladas y pruebas por arquitectura.

22.10 Double buffering

Mientras se calcula con un tile, se puede preparar el siguiente en otra región. Esto intenta solapar movimiento y cálculo. Se necesitan dos buffers y un protocolo que indique cuál está listo y cuál puede reutilizarse.

Un error típico confunde «la copia ha sido emitida» con «la copia ha terminado». Otro sobrescribe un buffer que todavía consume una instrucción asíncrona. El número de stages no es una mejora aislada: cambia recursos y dependencias.

22.11 Profundidad de pipeline

Más stages pueden ocultar latencia, pero consumen más memoria compartida y estado de control. Eso puede reducir residencia de bloques. Un pipeline profundo puede perder frente a uno corto si el kernel ya está limitado por otro recurso.

El estudio de programas tile de 2026 incluye fallos relacionados con sincronización y vida útil. Su utilidad para el curso es recordar que un DSL no elimina automáticamente todas las obligaciones de cooperación; la evaluación debe cubrir el protocolo, no solo una salida fácil. [54]

22.12 Pruebas de concurrencia

Repite kernels con formas de borde, pocos y muchos bloques, diferentes tamaños de tile y datos cambiantes. Utiliza herramientas de detección de carreras y accesos cuando estén disponibles. Conserva un caso reducido para cada fallo.

Una prueba que pasa una vez no demuestra ausencia de carrera. Aun así, pruebas de estrés y comprobaciones estructurales son evidencia útil. Deben complementarse con el razonamiento

de productores, consumidores y barreras.

Encontrar la barrera que falta

Un bloque carga un tile, sincroniza, calcula y comienza inmediatamente a cargar el siguiente en el mismo buffer. ¿Qué carrera puede existir aunque todos hayan superado la primera barrera?

Solución. Un hilo rápido puede sobrescribir datos del tile anterior mientras uno lento todavía los lee durante el cálculo. La primera barrera protege la disponibilidad inicial, no la reutilización posterior. Hace falta una segunda sincronización o un protocolo equivalente que garantice finalización de todos los consumidores.

Antes de continuar

Explica quién produce y quién consume cada región compartida. Distingue máscara de acceso, máscara de salida y participación colectiva. Después justifica las dos barreras del kernel GEMM de referencia.

CAPÍTULO 23

Tensor cores: una operación colectiva, no una multiplicación mágica

La pregunta de este capítulo

Ya sabemos repartir un producto de matrices entre hilos. Ahora imaginaremos una calculadora que no acepta un número cada vez, sino dos pequeños mosaicos completos. Aprender a alimentarla y recoger su resultado es tan importante como invocar la instrucción.

23.1 De una celda a un mosaico

En el producto de matrices del capítulo de CPU, una celda de salida se obtenía recorriendo una fila de A y una columna de B . Podíamos mantener varias celdas en acumuladores para reutilizar los datos. Un acelerador matricial lleva esa idea al hardware: ejecuta una operación de la forma $D = AB + C$ sobre pequeños bloques, con formatos y disposiciones permitidos por su arquitectura.

Imagina una imprenta que imprime dieciséis tarjetas simultáneamente. No basta con entregarle una tarjeta y pedir que sea dieciséis veces más rápida. Hay que colocar las tarjetas en las posiciones correctas, cargar la tinta y esperar a que el mecanismo termine. Si preparar la bandeja cuesta más que imprimirla, una tirada diminuta no aprovecha la máquina.

El paralelismo matricial tampoco elimina el tráfico de memoria, las conversiones de tipos, las máscaras ni los epílogos. Un GEMM rápido organiza todo ese trabajo alrededor de los bloques que puede consumir la unidad especializada.

Tres niveles que conviene distinguir

Una operación matemática define qué resultado queremos. Una instrucción matricial define un bloque y una cooperación concreta del hardware. Un kernel define cómo muchas instrucciones, cargas y escrituras producen una operación completa de cualquier tamaño admitido.

23.2 Fragmentos: cada participante guarda solo parte

La interfaz WMMA de CUDA permite describir fragmentos de matrices y ejecutar una multiplicación cooperativa. No hay que interpretar un fragmento como un array ordinario cuya posición cero corresponde siempre a la primera celda matemática. La distribución interna pertenece al contrato de la interfaz y puede depender de la arquitectura. La programación correcta utiliza sus operaciones de carga, multiplicación y almacenamiento. [9, 10]

Nuestro archivo `kernels/wmma_demo.cu` usa exactamente un warp y un bloque $16 \times 16 \times 16$:

entradas FP16, acumulación FP32 y salida FP32. No es todavía un GEMM general. Es una lupa que permite verificar el primer uso de la operación colectiva.

El núcleo relevante es el siguiente; el archivo completo añade asignación, transferencia, comprobación de errores y referencia numérica.

```
using namespace nvcuda;
wmma::fragment<wmma::matrix_a,16,16,16,half,wmma::row_major> a;
wmma::fragment<wmma::matrix_b,16,16,16,half,wmma::row_major> b;
wmma::fragment<wmma::accumulator,16,16,16,float> acc;
wmma::fill_fragment(acc,0.0f);
wmma::load_matrix_sync(a,A,16);
wmma::load_matrix_sync(b,B,16);
wmma::mma_sync(acc,a,b,acc);
wmma::store_matrix_sync(C,acc,16,wmma::mem_row_major);
```

Lee 16 en la carga como dimensión principal de esta disposición, no como número de hilos. Todas las operaciones del fragmento se ejecutan cooperativamente. No se coloca la multiplicación dentro de un `if` que solo toma una parte del warp.

La inicialización de los datos utiliza valores distintos y representables de manera sencilla. Una matriz de unos es útil como primera prueba, pero puede esconder una transposición: muchas permutaciones de unos siguen siendo unos. Los patrones por fila y columna revelan qué orientación se ha interpretado.

23.3 Las cuatro preguntas antes de compilar

Primera: ¿qué GPU tenemos realmente? Segunda: ¿qué instrucciones y tipos soporta ese destino? Tercera: ¿el compilador instalado conoce el destino? Cuarta: ¿el controlador puede cargar el código producido?

Una marca comercial no resuelve esas preguntas. La tabla oficial distingue, por ejemplo, capacidades de cómputo diferentes entre Blackwell de centro de datos y GeForce RTX 50. Un kernel que depende de características específicas de `sm_100a` no se convierte en compatible con una GeForce por compartir el nombre Blackwell. [55, 11]

Para compilar la demostración se debe sustituir `sm_120` por el destino real si tu dispositivo es diferente y comprobar que el toolkit lo conoce. El comando siguiente usa el destino de la GPU validada, sin detectarlo automáticamente:

```
nvcc -O2 -std=c++17 -arch=sm_120 kernels/wmma_demo.cu -o wmma_demo
./wmma_demo
```

Estado de esta práctica

La revisión del 1 de octubre compiló y ejecutó este archivo en una RTX 5070 Ti Laptop con CUDA 12.9 y destino `sm_120`. El error absoluto máximo frente a su referencia fue cero; el registro está en `code/reports/validation_20261001/wmma_run.log`. Ajusta el destino al dispositivo real del laboratorio. Este resultado comprueba la operación, sin demostrar rendimiento competitivo.

23.4 De la demostración a un GEMM completo

Para multiplicar matrices mayores, divide la salida en tiles. Cada bloque de salida recorre K por fragmentos. Acumula contribuciones y escribe el resultado al terminar. Esta descripción se parece al GEMM de memoria compartida porque resuelve la misma necesidad de reutilización, pero cambia la unidad de cómputo y su contrato de layout.

Considera $M=20$, $N=18$, $K=17$. Un tamaño de tile de 16 deja bordes en las tres dimensiones. Si el hardware exige un fragmento completo, no podemos leer fuera de los arrays para fingir que esos elementos existen. Una solución didáctica consiste en preparar bloques con ceros para las posiciones no válidas y almacenar únicamente las celdas de salida válidas.

El relleno en K debe aportar cero a la suma. El relleno en M y N no debe producir escrituras fuera de rango. Son dos obligaciones diferentes: una protege el valor, la otra protege la memoria.

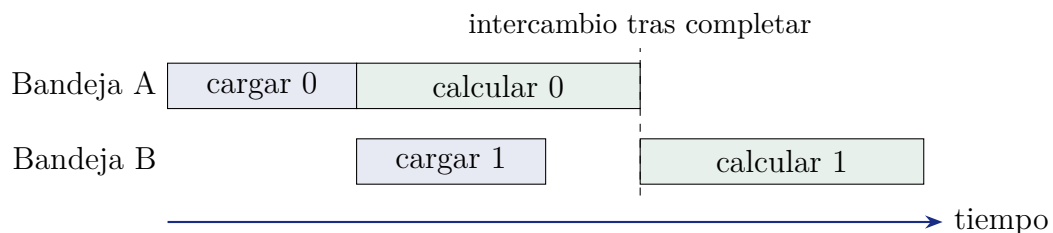
Una ruta más sofisticada puede separar los bloques interiores de los bordes y elegir otro kernel para tamaños pequeños. El compilador conserva una alternativa genérica para los casos que no cumplen alineamiento, tipos o tamaños de la ruta rápida. Eso es una especialización con guardas, no una excusa para rechazar silenciosamente entradas.

23.5 Carga asíncrona: iniciar no significa terminar

Imagina dos bandejas. Mientras se calcula con la primera, se prepara la segunda. En la siguiente iteración intercambiamos sus papeles. El beneficio potencial es solapar traslado y cómputo; el peligro es reutilizar una bandeja que todavía está siendo leída.

En una implementación real hacen falta estados para cada fase: espacio disponible, copia iniciada, copia completada, consumidores terminados. Una barrera genérica puesta al azar no sustituye el protocolo específico de la operación asíncrona. Las instrucciones modernas tienen ámbitos y mecanismos de finalización diferentes; el renderer debe elegirlos según el destino. [10]

El siguiente dibujo es un modelo temporal propio. Cada fila es una bandeja, no una garantía de que una GPU concreta ejecute exactamente estos ciclos.



Antes de añadir doble buffering, mide un kernel de una sola etapa. Después compara registros, memoria compartida y tiempo. Doblar el número de buffers puede reducir la ocupación o complicar el epílogo. Solapar más trabajo no implica automáticamente terminar antes.

23.6 El epílogo también tiene arquitectura

La salida del acelerador está distribuida entre participantes y puede necesitar una reorganización antes de escribir. Además, queremos quizá sumar un sesgo, escalar, aplicar una activación o cuantizar. Ese tramo es el epílogo.

Fusionarlo evita escribir una matriz intermedia y volver a leerla. Sin embargo, una reducción por filas en el epílogo exige comunicación entre quienes poseen diferentes columnas. Una función escalar por elemento y una normalización de toda la fila no requieren el mismo mecanismo.

Un buen diseño de IR conserva la información suficiente para decidir si la fusión es legal y rentable. No basta con reconocer el patrón textual `matmul + bias`; también hay que conocer sus formas, el eje del sesgo, el tipo de acumulación y los consumidores posteriores.

23.7 CuTe, CUTLASS y cuTile no son tres nombres de lo mismo

CUTLASS ofrece componentes para construir operaciones de álgebra lineal. CuTe representa layouts y particiones de datos e hilos; su DSL permite expresar esas ideas desde Python. cuTile es otro proyecto y otra interfaz. Que dos herramientas hablen de tiles no hace que compartan sintaxis, modelo de memoria o nivel de control. [11, 12]

La lección para Lumbre es arquitectónica: separar la decisión matemática, la disposición de datos y el mecanismo de cómputo. Una transformación de layout debería poder justificarse sin mezclarla con la lógica del optimizador de entrenamiento.

El README de CUTLASS consultado presenta la edición 4.8.0 de septiembre de 2026 y soporte preliminar de Rubin. Advierte además que ejecutar esos kernels exige un controlador futuro respecto de la entrega preliminar citada. En este libro eso es una observación de código y documentación, no una práctica disponible universalmente el día del corte. [11]

23.8 Una prueba oral que detecta comprensión

¿Por qué una sola instrucción puede necesitar treinta y dos hilos?

Explica por qué no podemos lanzar un hilo, ejecutar `mma_sync` y esperar una matriz completa. Después explica por qué treinta y dos hilos no bastan para garantizar que todo sea correcto.

La primera respuesta es que la interfaz representa una operación cooperativa: los fragmentos están repartidos y las llamadas tienen requisitos colectivos. La segunda es que siguen importando la participación uniforme, el tipo y layout de cada fragmento, las direcciones, el alineamiento, las dimensiones principales, el destino compilado y el almacenamiento de la salida. Contar participantes solo comprueba una parte del contrato.

CAPÍTULO 24

AMD: portar el significado antes de perseguir instrucciones

24.1 Una traducción que parece demasiado fácil

Muchos ejemplos de suma de vectores cambian `cudaMalloc` por `hipMalloc` y conservan casi todo el kernel. Eso enseña una correspondencia útil entre interfaces. Pero un kernel que dependía de una disposición concreta de registros, un tamaño de warp o una instrucción matricial no se vuelve portátil mediante una sustitución de nombres.

Pensemos en una clase que organiza a sus estudiantes en grupos. Cambiar el nombre de la escuela no nos dice cuántas personas hay en cada grupo ni cómo se reparten los ejercicios. Si el algoritmo suponía que el estudiante número 31 era siempre el último, necesitamos revisar esa suposición.

HIP documenta propiedades y extensiones para el modelo de ejecución. La ruta correcta consulta y especializa las características del destino; no convierte una suposición de NVIDIA en una constante universal. [19]

24.2 Tres capas del port

La capa semántica define `sum`, `matmul`, `where`, los tipos y los ejes. No debería cambiar porque cambiemos de fabricante. La capa de planificación decide tiles, grupos, memoria local y sincronización. Puede necesitar decisiones diferentes. La capa de emisión produce las instrucciones e interfaces del destino.

En Lumbre, el backend HIP básico sigue la misma matemática y genera kernels escalares por elemento o un GEMM bloqueado de memoria compartida. No contiene una implementación optimizada de MFMA. Tener backend HIP, por tanto, significa que existe una ruta de emisión y runtime; no que hayamos igualado las bibliotecas especializadas de AMD.

Portabilidad semántica frente a portabilidad de rendimiento

La primera pregunta es «¿calcula lo mismo dentro del contrato numérico?». La segunda es «¿usa bien esta máquina?». Resolver la primera es obligatorio. La segunda suele exigir especialización y mediciones nuevas.

24.3 Un ejemplo de reducción dependiente del grupo

Supón que una reducción parcial usa intercambios entre lanes con distancias 16, 8, 4, 2 y 1. Ese patrón presupone una agrupación concreta. En otro tamaño de grupo, parte de los valores podría no participar o podríamos comunicar posiciones no válidas.

Una implementación portable empieza por una reducción de memoria compartida cuya corrección puede explicarse con el número de elementos activos. Después añade una versión especializada con operaciones de subgrupo para los destinos que cumplen sus guardas.

No basta con reemplazar 32 por una variable si el resto del código aún codifica máscaras de 32 bits, fragmentos fijos o layouts de almacenamiento. Las dependencias de una suposición pueden estar repartidas por varios módulos. Por eso las pruebas incluyen tamaños menores, mayores y no múltiplos del subgrupo.

24.4 LDS y memoria compartida: parecido no significa idéntico coste

En el razonamiento del compilador podemos llamar memoria local del bloque al espacio cooperativo cercano a los hilos. En la documentación de AMD aparece LDS. La abstracción ayuda a expresar un tile reutilizable, pero el rendimiento depende de bancos, acceso, capacidad y recursos del destino.

Si cada lane accede a una columna de una matriz local con un stride desfavorable, puede aparecer contención. Una reorganización o un padding puede mejorar el patrón. Hay que revisar tanto quién escribe como quién lee: arreglar la carga inicial no garantiza que el consumo matricial quede bien distribuido.

Un experimento educativo cambia únicamente el layout local manteniendo las mismas operaciones aritméticas. Se registra error, tiempo, memoria reservada y recursos. Si cambian también el tamaño del tile, el número de hilos y la precisión, ya no podemos atribuir la mejora a una única causa.

24.5 Dónde mirar en el ecosistema

El repositorio histórico Composable Kernel y el de hipBLASLt remiten al monorepo ROCm/rocm-libraries. AITER reúne implementaciones orientadas a operaciones de IA, mientras que el fork LLVM de ROCm pertenece a la infraestructura de compilación. Son piezas de niveles diferentes. [15, 14, 16, 17, 18]

La lectura recomendada no es abrir todos los archivos. Escoge un GEMM y reconstruye su contrato de entrada, selección de implementación, parámetros de tile, epílogo y pruebas. Después sigue una configuración concreta hasta el código que ejecutaría. El producto de la lectura es un diagrama y una tabla de decisiones, no una colección de nombres.

Pieza	Pregunta que ayuda a responder	Lo que no debemos inferir
HIP	¿Cómo lanzo y sincronizo trabajo?	Que todos los kernels son óptimos.
hipBLASLt	¿Cómo selecciono un GEMM especializado?	Que es un compilador general de modelos.
Composable Kernel	¿Cómo se componen operaciones y layouts?	Que copiar una configuración sirve en cualquier GPU.

Pieza	Pregunta que ayuda a responder	Lo que no debemos inferir
AITER	¿Qué kernels de IA puedo estudiar o contrastar?	Que cada ruta soporta todos los modelos y tipos.
LLVM de ROCm	¿Cómo se baja código hacia el destino?	Que un frontend de tensores viene incluido en nuestra aplicación.

24.6 Laboratorio de port con presupuesto acotado

Empieza por el ejemplo de elementwise de Lumbre. Conserva los mismos datos guardados en un archivo y la misma referencia CPU. Ejecuta la ruta HIP solo después de verificar instalación, dispositivo y compilador. Anota expresamente la arquitectura detectada y la versión de la cadena.

Después prueba el GEMM compartido con tamaños 31, 47 y 65. Esos números fuerzan bordes. Si falla solo el tamaño irregular, sospecha primero de máscaras o índices; si falla también el bloque exacto, revisa layout, dimensiones principales y barreras.

Para rendimiento, compara kernels con operandos residentes. No atribuyas a la multiplicación el coste de construir un contexto en la primera llamada. Tampoco escondas ese coste: preséntalo por separado porque puede importar en una aplicación de vida corta.

El siguiente paso es incorporar una instrucción matricial o una biblioteca como ruta externa declarada. Eso cambia la afirmación que podemos hacer: «Lumbre despacha a esta implementación» no equivale a «Lumbre genera sus instrucciones desde cero». Ambas opciones son útiles si se documentan con precisión.

Antes de pasar a otro acelerador

Debes poder señalar qué parte del compilador conserva la matemática, qué parte cambia el reparto del trabajo y qué parte depende de la API del fabricante. También debes poder explicar por qué un test numérico idéntico no garantiza un perfil de rendimiento idéntico.

CAPÍTULO 25

Huawei Ascend y DeepSeek: separar cálculo, comunicación y compilación

25.1 La palabra kernel estaba escondiendo dos cosas

Un kernel de cálculo es un programa que realiza una operación, como un producto matricial, en un acelerador. El kernel de Linux es el núcleo del sistema operativo. Los controladores pueden vivir parcialmente dentro de este último y permitir que el primero se ejecute, pero no son el mismo programa.

Por eso «el kernel de Huawei» resulta insuficiente como descripción técnica. En este curso distinguimos la familia de aceleradores Ascend, su software de programación, las bibliotecas de kernels de DeepSeek para Ascend y las capas de sistema que gestionan dispositivos.

La precisión en los nombres evita un error de diseño: intentar encontrar un algoritmo de GEMM en un controlador de memoria, o esperar que una biblioteca de comunicación implemente la multiplicación del modelo.

25.2 Una fábrica con estaciones especializadas

Imagina una fábrica con una estación para transportar cajas, otra para operaciones vectoriales y otra para multiplicar bloques de matrices. El desafío no es mantener ocupada una sola estación, sino organizar el flujo completo sin que una consuma una caja antes de que la anterior la haya preparado.

La documentación y ejemplos de Ascend permiten estudiar esa división de responsabilidades y sus espacios de memoria. Aquí usamos la idea de cálculo matricial, cálculo vectorial y transferencia como modelo pedagógico. Las capacidades, nombres de instrucciones y límites concretos deben obtenerse del chip y de la versión de CANN utilizados. [28]

No trasladamos automáticamente un bloque CUDA a una unidad Ascend. Reutilizamos la matemática y volvemos a diseñar el reparto del trabajo, el layout y el protocolo entre etapas.

25.3 Un GEMM con epílogo como primer port

La operación objetivo es sencilla de escribir:

$$Y_{ij} = \max \left(0, \sum_{k=0}^{K-1} A_{ik} B_{kj} + b_j \right).$$

La multiplicación puede aprovechar una estación matricial. La suma de sesgo y la activación requieren trabajo adicional. Las entradas y resultados parciales deben estar en un formato que ambas partes interpreten correctamente.

Un primer prototipo separa las etapas y comprueba cada frontera. Se guarda el resultado del GEMM antes de aplicar el epílogo. Después se fusiona la ruta y se vuelve a comparar. Si la salida fusionada falla, esa descomposición permite localizar si el error viene de la multiplicación, de la interpretación del tile o del epílogo.

La validación incluye una matriz que distingue filas y columnas, K no múltiplo del bloque y valores positivos y negativos para que ReLU no oculte todos los errores. Si todos los resultados se recortan a cero, una referencia aparentemente perfecta puede estar comprobando muy poco.

25.4 Qué publica cada repositorio de DeepSeek

DeepGEMM se centra en cómputo matricial. DeepEP se centra en comunicación útil para repartir y reunir trabajo, especialmente en mezclas de expertos. DeepJIT aporta infraestructura de compilación, caché, carga y lanzamiento. Las variantes Ascend adaptan partes de esa pila a otro acelerador. [23, 25, 27, 24, 26]

Esta separación es una lección de ingeniería. Si la red está ociosa porque el empaquetado de tokens tarda demasiado, acelerar solo el GEMM quizá no ayude. Si un kernel tarda milisegundos pero recompilarlo tarda segundos en cada llamada, el problema está en la vida del programa compilado.

Nivel	Entrada conceptual	Salida conceptual
Kernel matricial	Tiles y escalas numéricas	Acumuladores o resultados.
Comunicación de expertos	Tokens, destinos y metadatos	Tokens redistribuidos o combinados.
JIT	Fuente, configuración y destino	Programa cargable y reutilizable.
Runtime	Programa y buffers	Trabajo ejecutado y finalización observable.

El compilador Lumbre conserva también esta separación, aunque sus implementaciones sean mucho más pequeñas: construir un grafo, elegir kernels, generar una unidad compilable y ejecutarla son pasos identificables.

25.5 La lectura del README también es parte del laboratorio

Al corte de esta edición, DeepEP-Ascend describe validación sobre hardware PoC Ascend 950DT y una combinación concreta de CANN y dependencias. El mismo repositorio sitúa la disponibilidad de un HDK comercial alrededor del 15 de octubre, una fecha posterior al 1 de octubre. También enumera funciones pendientes o restringidas. No sería correcto transformar esa publicación en «cualquiera puede reproducir todos los resultados hoy». [26]

DeepJIT se presenta como infraestructura C++20 y documenta rutas CUDA y Ascend.

Algunas funciones de su interfaz Python y calentamiento de caché figuran como trabajo en curso. La existencia del repositorio no autoriza a escribir comandos para una API todavía no disponible. [27]

Disponibilidad documental, binaria y física

Podemos leer código público sin disponer de un paquete instalable compatible. Podemos instalar un paquete sin tener el acelerador. Podemos tener el acelerador y un controlador que no soporte esa ruta. Cada nivel necesita evidencia propia.

25.6 Cuantización por bloques sin perder la escala

Imagina que guardamos cada número con pocas cifras y, junto a un grupo, guardamos una escala. El número aproximado se reconstruye multiplicando la cifra almacenada por esa escala. El formato del grupo y el eje de la escala forman parte del dato.

Para un ejemplo propio, supón A aproximada como $s_A \hat{A}$ y B como $s_B \hat{B}$, con escalas constantes en el producto considerado. Entonces AB se aproxima por $s_A s_B (\hat{A} \hat{B})$. Si la escala cambia dentro de K , ya no podemos sacar un único producto de escalas fuera de toda la suma. Hay que agrupar contribuciones compatibles o aplicar las escalas donde corresponde.

Ese detalle explica por qué copiar únicamente el array de valores comprimidos no porta una operación cuantizada. También hay que portar las escalas, su layout, el redondeo, las saturaciones y el tipo del acumulador.

Un test útil construye dos bloques con magnitudes muy diferentes. Si ambos usan accidentalmente la primera escala, el bloque pequeño o el grande revelará el fallo. Un tensor aleatorio de rango estrecho podría disimularlo.

25.7 Planificar una tubería con estados explícitos

Representaremos cada buffer con tres estados: libre, preparado y en uso. La estación de copia solo escribe en uno libre; al terminar lo marca preparado. La estación de cómputo solo consume uno preparado y, al finalizar, lo devuelve a libre.

La implementación real utilizará eventos, colas o primitivas del dispositivo, no necesariamente estas cadenas de texto. Pero el modelo permite formular invariantes: ningún buffer tiene dos escritores simultáneos; ningún consumidor usa datos incompletos; ningún productor sobrescribe datos aún necesarios.

```
libre --iniciar copia--> copia pendiente
copia pendiente --finalizacion--> preparado
preparado --iniciar computo--> en uso
en uso --ultimo consumidor termina--> libre
```

Esta máquina de estados puede comprobarse en una simulación CPU del scheduler antes de disponer del acelerador. No valida la instrucción concreta ni el rendimiento, pero sí descubre errores lógicos de reutilización.

El proyecto de port del final del libro exige precisamente dos evidencias distintas: validación

del protocolo abstracto y validación del backend real. Cuando falta acceso al hardware, la segunda queda pendiente; no se convierte en un resultado mediante una captura de una simulación.

25.8 Qué conservar de esta comparación

Las máquinas cambian; las preguntas fundamentales se repiten. ¿Quién posee cada dato? ¿En qué formato está? ¿Qué etapa puede leerlo? ¿Quién sabe que la operación ha terminado? ¿Cuánto trabajo útil se hace por byte trasladado?

La originalidad de un port serio no está en renombrar llamadas. Está en reconocer qué invariantes sobreviven y qué decisiones eran específicas del hardware anterior. Ese es el conocimiento transferible que buscamos al estudiar repositorios de fabricantes distintos.

CAPÍTULO 26

Linux y los controladores: el suelo sobre el que corre el compilador

26.1 Una aplicación no posee físicamente toda la GPU

Cuando un programa reserva memoria, recibe una forma de referirse a recursos que el sistema administra. El controlador participa en la creación de contextos, gestión de memoria y envío de trabajo. El sistema debe coordinar aplicaciones, proteger recursos y observar cuándo terminan operaciones.

Es útil imaginar un edificio de talleres. El compilador escribe las instrucciones de fabricación; el runtime entrega el pedido; el controlador y el sistema administran el acceso al taller. Confundirlos dificulta depurar: un cálculo erróneo, un fallo al cargar un módulo y un dispositivo ausente no tienen por qué compartir causa.

La documentación DRM y AMDGPU describe interfaces y mecanismos de gestión de memoria y sincronización. Los módulos abiertos de NVIDIA son otra fuente para estudiar una parte de la pila. Que un módulo del kernel sea abierto no implica que todo el compilador o toda la biblioteca matemática del fabricante lo sean. [21, 20, 13]

26.2 Dirección virtual, residencia y transferencia

Una dirección es un identificador dentro de un espacio de direcciones, no una explicación completa de dónde están los bytes. La memoria puede requerir mapeos, migraciones o transferencias. El tiempo de una operación aparentemente pequeña puede estar dominado por preparar sus datos.

Nuestro runtime didáctico adopta una decisión sencilla: mantiene una arena del dispositivo y realiza copias explícitas. Eso hace visible el coste y reduce la cantidad de políticas que el estudiante debe comprender a la vez. No pretende modelar todos los mecanismos de memoria unificada o virtualización.

Cuando una prueba tarda mucho solo la primera vez, una hipótesis posible es compilación o carga. Otra es creación de contexto. Otra es asignación o movimiento de páginas. Separamos fases antes de culpar al kernel de cálculo.

26.3 Fences: una promesa de finalización

Una fence representa una condición de finalización que otros participantes pueden esperar. La idea es más general que un `sleep`: esperar diez milisegundos no demuestra que el trabajo

haya terminado; observar la señal de finalización correcta sí establece una dependencia según su contrato.

Supón que un productor genera un buffer y un consumidor lo reutiliza. La dependencia debe seguir al buffer y al trabajo que lo produce, no a una suposición sobre la velocidad habitual de la máquina. Una GPU menos cargada puede ocultar durante semanas un error que aparece al introducir otra aplicación.

En una arena con reutilización, el último consumidor del grafo no siempre coincide con el último consumidor físicamente terminado. El planificador estático conoce un orden lógico; el runtime asíncrono debe convertirlo en dependencias efectivas. Si varias colas se solapan, los intervalos de vida necesitan esa información adicional.

26.4 Fallo de compilación, fallo de ejecución y reinicio del dispositivo

Un error de sintaxis pertenece a la generación o compilación. Una arquitectura no admitida pertenece a compatibilidad de destino. Una lectura fuera de rango pertenece al programa o sus contratos de memoria. Un error de dispositivo puede tener causas de software, controlador, alimentación o hardware.

No debemos diagnosticar un fallo físico a partir de una excepción genérica de la biblioteca. Tampoco debemos atribuir todo al controlador para evitar revisar un índice. La investigación empieza por un caso mínimo y por conservar el mensaje completo.

Una ficha de incidencia útil incluye la operación, forma, tipo, configuración, fuente generada, versiones, momento de aparición y si se reproduce tras reiniciar el proceso. Se separa lo observado de la causa propuesta.

Una incidencia bien formulada

«El primer GEMM $31 \times 47 \times 65$ falla al sincronizar; la compilación termina y la suma vectorial funciona. Con $32 \times 48 \times 64$ no falla. La primera hipótesis es un borde mal protegido». Eso permite diseñar una prueba. «HIP está roto» no delimita ninguna.

26.5 Observación segura para principiantes

Leer información del sistema, consultar versiones y revisar logs autorizados es suficiente para el laboratorio inicial. No se pide descargar firmware experimental, desactivar protecciones de memoria, cambiar parámetros de IOMMU ni cargar un módulo propio como administrador.

Los repositorios de firmware o control de dispositivos son objetos de lectura avanzada. Su existencia en una organización que también desarrolla un compilador no convierte su instalación en requisito del curso. Una modificación de bajo nivel puede dejar el equipo inestable y no ayuda a aprender la primera reescritura de UOps.

El ejercicio consiste en dibujar el camino de una suma: programa Python, IR, C o código GPU, compilador del destino, biblioteca cargada, runtime, controlador y dispositivo. Añade al lado de cada frontera una evidencia que podrías recoger sin modificar el sistema.

26.6 El límite útil del semestre

Un estudiante puede entender la separación de capas sin implementar un controlador. Nuestro proyecto construye un compilador y un runtime pequeño que usan interfaces existentes. Escribir un controlador sería otro proyecto, con un modelo de seguridad, pruebas y acceso al hardware mucho más exigentes.

La ambición del curso no se mide por cuántas capas reescribimos. Se mide por si sabemos explicar cuáles controlamos, cuáles delegamos y qué evidencia respalda cada resultado.

CAPÍTULO 27

Aprender significa cambiar números: la derivada desde cero

Empezamos con una perilla

Una máquina predice el precio de una entrada multiplicando el número de personas por una tarifa. Si la predicción se equivoca, queremos saber en qué dirección mover la tarifa. La derivada será una forma precisa de expresar esa sensibilidad local.

27.1 Un error que podemos calcular a mano

Supongamos que tres personas deberían pagar doce unidades. Nuestro modelo tiene una tarifa ajustable w y predice $p = 3w$. Si w vale dos, predice seis. Para convertir la discrepancia en un número no negativo, elevamos al cuadrado el error: $L = (p - 12)^2$. L se llama pérdida.

Con $w=2$, $L=36$. Con $w=2.1$, $p=6.3$ y $L=32.49$. Aumentar un poco w ha reducido la pérdida. Con $w=1.9$, $p=5.7$ y $L=39.69$: disminuir w la empeora. Todavía no hemos necesitado una fórmula de derivación para identificar la dirección correcta.

La pregunta cuantitativa es cuánto cambia L por un cambio pequeño de w . El cociente entre ambos cambios aproxima una pendiente. Cerca de $w=2$ esa pendiente es negativa: al avanzar hacia la derecha, la pérdida baja.

Qué dice el signo

Pendiente positiva: aumentar la variable aumenta localmente la función. Pendiente negativa: aumentarla disminuye localmente la función. Pendiente cero: la primera variación local no indica una dirección; no garantiza por sí sola un mínimo.

27.2 Derivar no es dividir dos números escritos

La notación dL/dw representa una sensibilidad. En este ejemplo puede obtenerse con álgebra: $L = 9w^2 - 72w + 144$, así que la pendiente es $18w - 72$. En $w=2$ vale -36 .

El número -36 no es la pérdida ni la tarifa nueva. Dice que un incremento muy pequeño de w produce aproximadamente -36 veces ese incremento en L . Para un incremento de 0.1 , la aproximación predice una reducción de 3.6 ; la reducción real es 3.51 porque la función es curva.

Si escogemos un paso de aprendizaje de 0.01 y nos movemos contra la pendiente, obtenemos w nueva igual a $2 - 0.01(-36) = 2.36$. No saltamos directamente a la solución $w=4$. Damos un paso cuya magnitud depende de una decisión del optimizador.

27.3 La regla de la cadena como transmisión de cambios

Nuestro cálculo tiene varias estaciones: w produce p ; p produce un error e ; e produce L . Cada estación transforma una pequeña variación que recibe. La variación final resulta de multiplicar esas sensibilidades locales a lo largo del camino.



En $w=2$, $p=6$ y $e=-6$. La sensibilidad de p respecto a w es 3. La de e respecto a p es 1. La de L respecto a e es $2e = -12$. Multiplicamos: $(-12) \cdot 1 \cdot 3 = -36$.

Ahora podemos escribir la versión compacta sin que sea una colección de símbolos nuevos:

$$\frac{dL}{dw} = \frac{dL}{de} \frac{de}{dp} \frac{dp}{dw}.$$

Cada factor responde una pregunta local. El compilador conoce qué operación creó cada nodo y puede construir automáticamente esas preguntas en orden inverso.

27.4 Dos caminos obligan a sumar

Considera $y = x \cdot x$. x aparece en los dos operandos de la multiplicación. Si x cambia, cambia tanto el primer factor como el segundo. Cada camino contribuye x a la pendiente, de modo que el total es $2x$.

Un motor de autodiferenciación que guardara solo la última contribución devolvería x en lugar de $2x$. Este fallo es especialmente fácil de cometer en un grafo con subexpresiones compartidas: el grafo no es siempre un árbol.

Con $y = x \cdot z + x$, hay dos caminos desde x hasta y . El producto aporta z y la suma directa aporta 1. La sensibilidad total es $z+1$. Para $z=3$ vale 4, aunque ninguna operación local individual tuviera por derivada 4.

Asignar y acumular no son lo mismo

En la propagación inversa, cuando un nodo recibe otra contribución usamos suma. Reemplazar la contribución anterior destruye información de los demás caminos.

27.5 Qué necesitamos guardar del avance

Para derivar una multiplicación, necesitamos los valores de sus operandos. Para derivar una exponencial, podemos reutilizar su resultado. Para una suma, no necesitamos los valores de entrada: su sensibilidad local es constante.

Esa diferencia conecta autodiff con memoria. Guardar todos los valores facilita el retroceso, pero consume espacio. Volver a calcular algunos reduce memoria a cambio de cómputo. La decisión no debería hacerse antes de saber qué datos necesita cada regla de gradiente.

Nuestro compilador construye un grafo de gradientes y después lo planifica como cualquier otro. No ejecuta una fórmula en Python por cada elemento del tensor durante el entrenamiento.

Python construye la receta; el código C generado hace los cálculos numéricos.

27.6 Una derivada numérica para comprobar, no para entrenar todo

La diferencia central aproxima la pendiente usando dos evaluaciones:

$$\frac{\partial L}{\partial w_i} \approx \frac{L(w + he_i) - L(w - he_i)}{2h}.$$

Aquí e_i significa cambiar únicamente la componente i . Para entenderlo, imagina un panel de cien perillas: movemos solo una hacia arriba y hacia abajo, dejando las otras noventa y nueve en el mismo sitio.

La aproximación tiene dos problemas. Si h es muy grande, deja de medir la pendiente local. Si es demasiado pequeño, el redondeo puede borrar la diferencia entre las dos evaluaciones. No existe una elección universal que arregle todos los casos.

Para un tensor pequeño y funciones suaves, la diferencia central es una prueba excelente. Para millones de parámetros sería muy costosa: requiere evaluaciones adicionales por cada componente. El retroceso reutiliza la estructura del cálculo para obtener muchas sensibilidades juntas.

27.7 Primera práctica con Lumbre

```
import numpy as np
from lumbre import param, Program, gradients

w = param("tarifa", ())
pred = 3.0 * w
loss = (pred - 12.0) * (pred - 12.0)
grad = gradients(loss, [w])[0]
with Program([pred, loss, grad]) as program:
    values = program.run({"tarifa": np.array(2, dtype="float32")})
    print([float(x) for x in values])
```

La salida esperada es 6, 36 y -36. Primero comprueba esos tres números, no solo el último. Si la predicción ya es incorrecta, investigar la regla del gradiente sería empezar por el lugar equivocado.

Después cambia w a cuatro. La predicción es doce, la pérdida cero y el gradiente cero. Cambia w a cinco: predicción quince, pérdida nueve y gradiente dieciocho. El signo ahora señala que conviene reducir la tarifa.

27.8 Dónde acaba esta intuición

En varias dimensiones, cada componente del gradiente mide el cambio respecto a una variable manteniendo las demás fijas. El conjunto de esas sensibilidades orienta un paso local. No garantiza encontrar el mejor mínimo global de una red neuronal.

Además, algunas operaciones no son diferenciables en ciertos puntos. ReLU cambia de pendiente en cero; máximo puede tener empates. Un framework elige una convención o una subderivada para esos casos. El compilador debe documentarla y probarla, no fingir que existe una única derivada clásica donde no la hay.

CAPÍTULO 28

Autodiferenciación inversa sobre un grafo de UOps

28.1 El retroceso empieza con una pregunta

Queremos saber cómo cambia una pérdida escalar al cambiar cada parámetro. La sensibilidad de la pérdida respecto a sí misma es uno: si L aumenta una unidad, L aumenta una unidad. Ese uno es la semilla del retroceso.

Recorremos los nodos en orden topológico inverso. Cuando visitamos una operación, ya hemos reunido las contribuciones procedentes de sus consumidores. Aplicamos su regla local y añadimos las contribuciones a sus operandos.

```
adjunto[perdida] = 1
para nodo en orden_topologico_inverso:
    g = adjunto[nodo]
    para operando, contribucion in regla_local(nodo, g):
        adjunto[operando] += contribucion
```

Este es pseudocódigo explicativo. La implementación completa de `gradients` está en el apéndice y devuelve nuevos UOps, no arrays NumPy ya calculados.

28.2 Qué significa un adjunto

Llamaremos adjunto de x a la sensibilidad de la pérdida respecto a x . El nombre evita repetir una frase larga. Si x es un tensor, su adjunto tiene la forma necesaria para representar la sensibilidad respecto a cada elemento de x .

Una operación vectorial podría describirse con una gran matriz de derivadas llamada jacobiano. No necesitamos construir esa matriz completa. La regla inversa calcula directamente cómo una sensibilidad de salida se transforma en sensibilidades de entrada. Esa operación es un producto vector-jacobiano, o VJP.

Una suma de mil elementos

Para $s = x_0 + \dots + x_{999}$, cada entrada tiene sensibilidad uno respecto a s . Si el adjunto de s es 0.2, cada entrada recibe 0.2. No construimos una matriz de mil columnas para descubrirlo: expandimos un escalar.

28.3 Reglas locales que debemos poder justificar

Para $z=x+y$, ambas entradas reciben g . Para $z=x-y$, x recibe g e y recibe $-g$. Para $z=xy$, x recibe gy e y recibe gx . Para $z=x/y$, x recibe g/y e y recibe $-gx/y^2$.

Para $z=\exp(x)$, el adjunto de x es gz . Para $z=\log(x)$, es g/x , dentro del dominio donde la función está definida. Para $z=\sqrt{x}$, es $g/(2z)$ en los puntos donde la derivada ordinaria existe. Para $z=\tanh(x)$, es $g(1-z^2)$.

No memorices la tabla sin verificar una operación. Con $z=xy$ y y fijo, aumentar x en una cantidad h aumenta z en yh . Esa es la razón de la contribución gy . La misma explicación intercambiando los factores da gx .

Operación	Contribución hacia x	Dato que hace falta
$x+y$	g	No necesita valores de x ni y .
$x \cdot y$	$g \cdot y$	El otro operando.
$\exp(x)$	$g \cdot \exp(x)$	Entrada o resultado.
$\log(x)$	g/x	La entrada.
$\tanh(x)$	$g \cdot (1-z^2)$	El resultado z .

El dominio numérico sigue siendo parte del contrato. Una regla algebraica no convierte $\log(-1)$ en una operación válida ni evita que aparezcan infinitos al dividir por cero.

28.4 Broadcasting hacia delante, suma hacia atrás

Supón que sumamos un sesgo de tres elementos a dos filas:

$$Y_{ij} = X_{ij} + b_j, \quad i = 0, 1; \quad j = 0, 1, 2.$$

El sesgo b_0 se utilizó en dos celdas: Y_{00} y Y_{10} . Por tanto, su gradiente recibe las dos contribuciones. Si los adjuntos de salida son las filas $[1,2,3]$ y $[4,5,6]$, el gradiente del sesgo es $[5,7,9]$.

Devolver el tensor de dos filas como gradiente de un sesgo de una fila sería un error de forma y de significado. La operación inversa del broadcasting suma sobre los ejes que se replicaron y conserva o restaura las dimensiones de tamaño uno.

Lumbre implementa esa reconciliación mediante una reducción hacia la forma original. Los ejes añadidos por delante también cuentan. La regla debe contemplar la diferencia entre $(3,)$, $(1,3)$ y $(2,3)$, aunque algunas operaciones produzcan los mismos valores visibles.

28.5 Reducir hacia delante, expandir hacia atrás

Para una suma por filas, cada elemento de una fila contribuyó a su resultado. El adjunto del resultado se expande a todas esas posiciones. Para una media, además se divide por el número de elementos reducidos.

Si $x=[2,4,6]$ y $y=\text{media}(x)$, el resultado es cuatro. Aumentar solo x_0 en 0.3 aumenta la media en 0.1. La sensibilidad respecto a cada entrada es un tercio. Una implementación que olvidara esa división entrenaría con gradientes multiplicados por el tamaño del eje.

Para máximo, únicamente las posiciones que alcanzan el valor máximo participan según la convención elegida. Nuestra implementación distribuye el adjunto entre las posiciones empatadas. Es una elección explícita; otros sistemas pueden seleccionar una posición. Las pruebas deben compararse con una referencia que use la misma convención.

28.6 El producto de matrices explicado con una celda

Si $Y = AB$, la celda Y_{ij} es la suma de $A_{ik}B_{kj}$. Una entrada A_{ik} influye en todas las columnas j de la fila i . Cada contribución se multiplica por B_{kj} . Al sumar las sensibilidades de salida aparece:

$$\overline{A} = \overline{Y}B^T, \quad \overline{B} = A^T\overline{Y}.$$

La barra significa adjunto. Comprueba las formas antes de calcular: si A es $M \times K$, B es $K \times N$ y el adjunto de Y es $M \times N$, el primer producto devuelve $M \times K$ y el segundo $K \times N$.

En un `matmul` por lotes con `broadcasting`, esas fórmulas producen contribuciones por cada lote expandido. Después hay que sumarlas hacia las formas originales. Omitir ese paso puede dar gradientes correctos cuando los lotes coinciden y fallar cuando un operando se comparte.

28.7 Movimientos: deshacer direcciones, no derivar etiquetas

Una permutación se deshace con la permutación inversa. Un `reshape` recupera la forma de entrada. Un `slice` coloca su gradiente en las posiciones seleccionadas y deja cero en las demás. Un `padding` elimina del gradiente las posiciones añadidas.

Un `gather` de embeddings exige más cuidado. Si dos tokens seleccionan la misma fila de una tabla, sus contribuciones se suman en esa fila. No se sobrescriben. La operación inversa es una acumulación dispersa, habitualmente descrita como `scatter-add`.

Un embedding con un token repetido

Una tabla tiene cuatro filas y dos columnas. Los índices son $[2,0,2]$. El adjunto de las tres selecciones es $[[1,2],[3,4],[5,6]]$. ¿Qué recibe cada fila de la tabla?

La fila cero recibe $[3,4]$. La fila uno recibe $[0,0]$. La fila dos recibe $[6,8]$, suma de la primera y tercera selección. La fila tres recibe $[0,0]$. Este caso pequeño detecta una implementación que simplemente escribe el último gradiente recibido.

28.8 Comparaciones, selección y stop-gradient

Una comparación produce una decisión discreta. En nuestro motor no propagamos una derivada hacia el cambio de esa decisión. En `where(cond,a,b)`, el adjunto se dirige a `a` donde

cond es verdadera y a b donde es falsa.

Eso no implica que todas las funciones con decisiones sean suaves. Describe la regla que usamos para una ejecución y una convención concreta. En una frontera donde la condición cambia, la diferencia numérica puede no coincidir con esa regla.

`detach` conserva el valor hacia delante y corta la propagación hacia atrás. Es útil para constantes auxiliares o para escribir algoritmos con una separación explícita de dependencias. No debe añadirse solo para hacer desaparecer un gradiente que revela un error.

28.9 Autodiff también se compila

La salida de `gradients` es otro grafo. Podemos simplificarlo, identificar subexpresiones compartidas, fusionar elementwise, planificar reducciones y generar C. Esa decisión hace que el desarrollo de optimizaciones beneficie tanto al avance como al retroceso.

Sin embargo, un patrón rápido del avance no define automáticamente un retroceso correcto. Si añadimos una operación nueva de atención online, necesitamos su VJP o una descomposición diferenciable equivalente. El compilador debe rechazar una operación sin regla cuando se piden gradientes, en lugar de devolver ceros por defecto.

28.10 Pruebas por capas

Primero se prueba cada regla local con tensores diminutos y valores lejos de singularidades. Después se prueban composiciones, broadcasting y nodos compartidos. Finalmente se comprueba una actualización de parámetros y una pérdida que descienda en un problema controlado.

Una pérdida decreciente no demuestra por sí sola que todos los gradientes sean correctos: un modelo podría aprender lentamente con un gradiente parcialmente incorrecto. A la inversa, una pérdida que no desciende no demuestra necesariamente un fallo de autodiff; puede haber una tasa de aprendizaje inadecuada o datos sin señal.

El diagnóstico combina pruebas numéricas locales y comportamiento global. Esta separación evita culpar al optimizador antes de comprobar el signo de una derivada.

CAPÍTULO 29

Optimizar parámetros sin corromper el estado

29.1 Descenso de gradiente con un ejemplo completo

Para $L = (w - 3)^2$, el gradiente es $2(w - 3)$. Con $w=0$ y tasa 0.1, el primer paso da 0.6. La pérdida pasa de 9 a 5.76. El segundo gradiente es -4.8 y el nuevo valor 1.08. La pérdida vuelve a disminuir.

El siguiente código compila tanto la pérdida como la propuesta de actualización. La llamada `assign` copia los nuevos valores a los parámetros después de que el programa termine.

```
import numpy as np
from lumbre import param, gradients, Program

w = param("w", ())
loss = (w-3.0)*(w-3.0)
g = gradients(loss, [w])[0]
new_w = w-0.1*g
with Program([loss, new_w]) as p:
    p.set("w", np.array(0.0, dtype="float32"))
    for step in range(5):
        before, proposal = p.run()
        print(step, float(before), float(proposal))
    p.assign({"w": new_w})
```

La pérdida impresa corresponde a los parámetros antes de aplicar la propuesta de esa iteración. Confundir ese momento puede hacer que un gráfico parezca desplazado un paso o que se compare con el checkpoint equivocado.

29.2 Por qué la actualización no debe pisar sus entradas

Supón que dos parámetros se actualizan usando valores antiguos de ambos. Si escribimos el primero antes de calcular el segundo, el resultado puede dejar de representar un paso simultáneo. El orden de copia habrá cambiado el algoritmo.

Nuestro diseño calcula las salidas de actualización y después las asigna. Además, rechaza intercambios directos de parámetros como `a<-b`, `b<-a` a través de la API de asignación cuando las salidas son los propios parámetros: esa operación requiere un manejo explícito del aliasing. Un rechazo claro es preferible a una corrupción silenciosa.

En una implementación de producción se pueden demostrar condiciones de actualización in-place o usar buffers alternos. El optimizador y el planificador de memoria deben compartir

información sobre qué valores antiguos siguen vivos.

29.3 Momentum: recordar una dirección reciente

Una pendiente puede cambiar mucho entre minibatches. Momentum mantiene una media de gradientes anteriores para suavizar la dirección. Imagina empujar un carrito: un pequeño cambio de pendiente no invierte instantáneamente su movimiento.

La analogía tiene un límite: el algoritmo no es una simulación física exacta, sino una regla numérica. Hay que especificar cómo se pondera la memoria anterior y cómo se usa para actualizar el parámetro. Dos convenciones de momentum pueden parecer distintas y estar relacionadas por una escala de la tasa.

29.4 AdamW en cuatro piezas

Adam mantiene una media del gradiente y una media de su cuadrado. La primera orienta la actualización. La segunda adapta su escala por componente. Como ambas medias empiezan en cero, se corrige el sesgo de las primeras iteraciones. AdamW aplica además un decaimiento desacoplado de los pesos.

La separación entre Adam y decaimiento desacoplado se documenta en sus trabajos originales. [56, 57] La implementación didáctica usa estas ecuaciones:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \widehat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \widehat{v}_t &= \frac{v_t}{1 - \beta_2^t}, & w_{t+1} &= w_t(1 - \eta\lambda) - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}. \end{aligned}$$

Lee las ecuaciones en ese orden. t es el número de actualización, no el número de tokens. η es la tasa de aprendizaje, λ el decaimiento y ϵ una protección numérica. Los cuadrados y divisiones se aplican por elemento.

En el primer paso, con m y v iniciales cero, las correcciones de sesgo recuperan aproximadamente g y g^2 . Eso permite una comprobación manual sencilla para un parámetro escalar.

La política de qué parámetros reciben decaimiento es una decisión adicional. En redes reales se suelen separar grupos según su función; el ejemplo del paquete mantiene una política simple y explícita. No presentamos esa simplificación como la receta óptima para entrenar todos los LLM.

29.5 Recortar el gradiente sin cambiar cada componente a ojo

El clipping por norma global calcula una longitud conjunta de todos los gradientes. Si supera un umbral, multiplica todos por el mismo factor. Así limita la magnitud conservando la dirección global del vector de gradiente.

Para gradientes [3,4], la norma es 5. Con umbral 1, se aplica factor 0.2 y quedan[0.6,0.8]. Recortar cada componente por separado a 1 produciría[1,1], una dirección diferente. Son algoritmos distintos.

El cálculo de la norma también forma parte del grafo compilado. Tiene un coste y requiere una reducción; no conviene omitirlo del tiempo de entrenamiento si se usa en cada actualización.

29.6 Gradientes acumulados y lotes efectivos

Cuando no cabe un lote grande, podemos sumar contribuciones de varios microbatches antes de actualizar. Para equivaler a una media sobre todos los ejemplos, hay que ponderar correctamente sus tamaños y pérdidas.

Si dos microbatches tienen cuatro y dos tokens válidos, promediar sus dos pérdidas con peso un medio no equivale a promediar los seis tokens. El primero debería pesar cuatro sextos y el segundo dos sextos. Las máscaras y longitudes variables hacen que este detalle sea frecuente en modelos de lenguaje.

Además, el contador t de Adam debe avanzar por actualización del optimizador, no por cada microbatch si todavía no se actualiza. El clipping global se aplica normalmente al gradiente agregado cuando esa es la receta que se pretende reproducir.

29.7 Checkpoint: mucho más que pesos

Para continuar el mismo proceso necesitamos pesos, estados del optimizador, contador de actualización, configuración del modelo y estado del generador aleatorio. Si cambió el vocabulario, también debe detectarse. Un archivo que contiene solo pesos sirve para iniciar otro proceso, no para prometer una reanudación equivalente.

El ejemplo guarda esos componentes en un archivo NPZ y valida configuración y vocabulario al reanudar. La equivalencia bit a bit entre distintos dispositivos o toolchains no se promete: el orden numérico puede cambiar. Dentro del mismo entorno, una prueba de interrupción y reanudación permite comprobar la lógica del checkpoint.

Qué debe poder explicar el estudiante

Una actualización usa gradientes calculados con los parámetros adecuados, conserva los estados que necesita el paso siguiente y tiene un momento de aplicación definido. El entrenamiento es una secuencia de estados, no solo una función que devuelve una pérdida pequeña.

De letras a predicciones: construir un modelo de lenguaje

Un juego de completar una secuencia

Escribimos «el gato» y pedimos al modelo una distribución sobre lo que podría venir después. No le entregamos una regla gramatical escrita a mano. Ajustamos sus parámetros usando ejemplos donde sí conocemos la continuación.

30.1 Texto, token e identificador

Un ordenador necesita una representación numérica del texto. En el ejemplo didáctico, cada carácter del corpus tiene un identificador entero. La letra a podría ser el número dos y el espacio el número cero. Esos identificadores son etiquetas: no significa que a sea el doble de un espacio.

Un token puede ser un carácter, un byte, una palabra o una pieza de palabra según el tokenizador. El vocabulario es la colección de tokens disponibles. Su tamaño, V , determina cuántas alternativas puede predecir el modelo en cada posición.

El entrenamiento del paquete usa caracteres para mantener visible todo el proceso. Un tokenizador por subpalabras es más habitual en grandes modelos, pero añade decisiones sobre normalización, bytes, tokens especiales y segmentación. Cambiar el tokenizador cambia tanto los datos como las dimensiones de la entrada y salida.

No ordenar palabras por su identificador

Si «gato» tiene identificador 17 y «perro» 18, esa cercanía numérica no representa por sí sola parecido semántico. El embedding es la tabla de vectores que el entrenamiento aprende a asociar a esas etiquetas.

30.2 La tabla de embeddings

Cada fila de una tabla de tamaño $V \times D$ contiene D números. D es el ancho de la representación. Para convertir una secuencia de identificadores en vectores, seleccionamos sus filas. Esa selección es exactamente el gather que ya implementamos.

Si los tokens son $[2, 0, 2]$, seleccionamos dos veces la fila dos y una vez la fila cero. Durante el retroceso, las contribuciones de ambas apariciones de la fila dos se suman. El pequeño ejercicio de scatter-add del capítulo anterior ya estaba preparando esta operación real de un modelo de lenguaje.

Un lote de B secuencias de longitud T tiene identificadores de forma $B \times T$. Después del embedding, la forma es $B \times T \times D$. B identifica ejemplos, T posiciones y D características. Aunque todas sean dimensiones de un tensor, tienen funciones diferentes y no deben intercambiarse accidentalmente.

30.3 La etiqueta está desplazada una posición

Para una secuencia de identificadores $[4,1,7,3,2]$, podemos usar como entrada $[4,1,7,3]$ y como etiquetas $[1,7,3,2]$. En cada posición se intenta predecir el token siguiente.

Posición	Token de entrada	Etiqueta que debe predecir
0	4	1
1	1	7
2	7	3
3	3	2

Si usamos la misma secuencia como entrada y etiqueta sin desplazamiento, el modelo podría aprender a copiar el token actual. La pérdida bajaría, pero no estaríamos entrenando la tarea pretendida. Por eso la construcción del batch es una parte verificable del sistema, no un detalle administrativo.

30.4 Logits y probabilidades

La salida de una posición son V puntuaciones reales llamadas logits. No son todavía probabilidades: pueden ser negativas y no tienen por qué sumar uno. Softmax las convierte en cantidades positivas normalizadas.

Para logits $[0,0,0]$, cada alternativa recibe un tercio. Para $[0,1,0]$, la segunda recibe más peso, pero no certeza. Añadir la misma constante a todas las puntuaciones no cambia las probabilidades: importa su diferencia relativa.

La pérdida de entropía cruzada de una etiqueta correcta t es el negativo del logaritmo de su probabilidad. Si se le asigna 0.5, la pérdida es aproximadamente 0.693; si se le asigna 0.1, aproximadamente 2.303. Castigamos más una respuesta que da poca probabilidad a lo observado.

En el código evitamos calcular una probabilidad diminuta y después su logaritmo si podemos usar una forma estable con log-sum-exp. El capítulo de números ya justificó restar el máximo. La estadística y la estabilidad numérica se encuentran aquí en una operación concreta.

30.5 Una derivada especialmente útil

Para softmax seguido de entropía cruzada de una etiqueta, la derivada respecto a cada logit es la probabilidad predicha menos la etiqueta one-hot. One-hot significa un vector con uno en la alternativa correcta y cero en las demás.

Con probabilidades $[0.2, 0.5, 0.3]$ y etiqueta primera, el gradiente es $[-0.8, 0.5, 0.3]$. El signo negativo en el primer logit indica que aumentarlo reduce la pérdida localmente. Los otros signos indican que conviene disminuirlos. Si promediamos varias posiciones, se incorpora el factor de esa media.

Esta fórmula permite comprobar el motor de autodiff sin confiar en otro framework. En las pruebas del paquete se usan logits grandes positivos y negativos para comprobar también que la estabilización funciona.

30.6 Causalidad: no mirar la solución

Durante entrenamiento disponemos de toda la secuencia, incluidas las etiquetas futuras. Eso permite calcular muchas posiciones en paralelo, pero no autoriza a que una posición use información que no tendría al generar texto.

Para predecir después de la posición i , solo puede utilizar posiciones hasta i . Una máscara causal elimina las contribuciones posteriores. El triángulo permitido incluye la diagonal porque el token actual sí forma parte de la entrada conocida.

```
Posicion que pregunta -> columnas que puede consultar
0 : 1 0 0 0
1 : 1 1 0 0
2 : 1 1 1 0
3 : 1 1 1 1
```

El test de causalidad cambia el último token y exige que las salidas anteriores permanezcan iguales dentro de la tolerancia. Es una prueba mucho más directa que observar una pérdida sospechosamente baja. También puede detectar una permutación de ejes que aplica la máscara al lugar equivocado.

30.7 Qué significa «entrenar un LLM» en este curso

Construimos una arquitectura decoder-only con componentes usados en modelos de lenguaje modernos y hacemos que su avance, retroceso y actualización se ejecuten mediante nuestro compilador. El ejemplo pequeño sirve para comprobar toda esa cadena.

Pero «modelo de lenguaje» y «gran modelo de lenguaje de calidad competitiva» no son sinónimos. Un modelo de decenas de miles de parámetros y un corpus de unas frases no tiene la capacidad ni la evidencia de un sistema de frontera. Las siguientes partes desarrollan la arquitectura y los mecanismos necesarios para escalar; los resultados locales se etiquetan por su tamaño real.

Una definición operacional del hito

El estudiante alcanza el hito cuando puede explicar cómo se construyen las etiquetas, verificar causalidad y gradientes, ejecutar actualizaciones compiladas, guardar y reanudar el estado, y evaluar sobre datos definidos. Una muestra de texto llamativa por sí sola no satisface esos requisitos.

CAPÍTULO 31

Atención: consultar recuerdos con pesos aprendidos

31.1 Tres papeles para una misma secuencia

Imagina que cada palabra entrega tres fichas. La primera dice qué información busca; la segunda, qué tipo de información ofrece; la tercera contiene la información que puede aportar. Las llamaremos consulta Q , clave K y valor V .

Las fichas son vectores calculados mediante proyecciones aprendidas. No son frases que el modelo escriba en lenguaje natural. La analogía nos ayuda a separar funciones: Q y K deciden pesos; V aporta el contenido que se mezcla.

La atención del Transformer utiliza productos entre consultas y claves para construir puntuaciones, normaliza esas puntuaciones y combina valores. El trabajo original del Transformer es la referencia histórica de esta organización; el ejemplo numérico siguiente es una construcción didáctica propia. [40]

31.2 Una consulta y dos recuerdos

Tomemos una consulta de dos componentes $q=[1,0]$. Dos claves son $k_0=[1,0]$ y $k_1=[0,1]$. Los productos escalares valen 1 y 0. Si dividimos por la raíz de la dimensión, las puntuaciones son aproximadamente 0.707 y 0.

Softmax asigna aproximadamente 0.670 a la primera y 0.330 a la segunda. Con valores $v_0=[2,0]$ y $v_1=[0,4]$, la mezcla es aproximadamente $[1.340, 1.321]$. No hemos elegido un único recuerdo: hemos construido una combinación ponderada.

Cambiar los valores sin cambiar consultas y claves conserva los pesos, pero cambia el contenido mezclado. Cambiar una clave puede cambiar los pesos aunque su valor permanezca igual. Esta distinción ayuda a depurar una implementación que accidentalmente intercambia K y V .

31.3 Todas las consultas juntas

Para T posiciones y dimensión de cabeza d , Q y K tienen forma $T \times d$. El producto QK^T tiene forma $T \times T$: cada fila corresponde a una consulta y cada columna a una clave.

Aplicamos softmax a cada fila, no a toda la matriz a la vez. Cada consulta debe tener su propia distribución sobre recuerdos. Después multiplicamos por V , de forma $T \times d$, y obtenemos otra matriz $T \times d$.

$$S = \frac{QK^T}{\sqrt{d}} + M, \quad P = \text{softmax}_{\text{filas}}(S), \quad O = PV.$$

M representa la máscara: cero donde se permite consultar y un valor que excluye la posición donde no. En una formulación matemática exacta se usa menos infinito. En una implementación con un número finito muy negativo hay que verificar el rango de puntuaciones y el comportamiento numérico; no es una equivalencia sin condiciones.

31.4 Por qué aparece la raíz de d

El producto escalar suma d productos. Bajo un modelo simplificado de componentes independientes, centradas y de varianza comparable, su dispersión crece con la dimensión. Escalar ayuda a mantener las puntuaciones en un rango manejable para softmax. No es una normalización que garantice una distribución fija para cualquier dato real.

Si todas las puntuaciones se separan demasiado, softmax puede concentrar casi todo el peso en una alternativa y producir sensibilidades muy pequeñas para otras. El factor forma parte de la arquitectura que queremos reproducir; omitirlo cambia el modelo, aunque las formas de los tensores sigan siendo válidas.

31.5 Múltiples cabezas: varias consultas en paralelo

Podemos dividir la representación en H cabezas, cada una con dimensión $d=D/H$. Cada cabeza aprende proyecciones y mezclas distintas. Después concatenamos sus resultados y aplicamos una proyección de salida.

En la implementación usamos forma $B \times H \times T \times d$ para las operaciones de atención. Antes veníamos de $B \times T \times D$. El reshape separa D en H y d; la permutación coloca H antes de T. Al volver, se invierte ese movimiento antes de reunir H y d otra vez.

Un error común consiste en hacer un reshape directo entre $B \times T \times H \times d$ y $B \times H \times T \times d$ sin permutar. El número total de elementos coincide, pero el significado de sus posiciones no. Una prueba con valores que codifican sus coordenadas permite verlo sin entrenar nada.

Codificar coordenadas para detectar una permutación

Asigna a cada elemento el valor $1000 \cdot b + 100 \cdot t + 10 \cdot h + j$. Después de reorganizarlo, puedes leer el número y reconstruir a qué batch, posición, cabeza y componente pertenecía. Un array de ceros no revela ese error.

31.6 GQA: compartir claves y valores

Grouped-query attention permite más cabezas de consulta que de claves y valores. Varias consultas comparten el mismo grupo de K y V. La familia incluye configuraciones intermedias entre una cabeza KV compartida y una por cada consulta. [58]

Nuestro ejemplo exige que H sea divisible por el número de cabezas KV, HK. Con $H=4$ y $HK=2$, cada grupo KV sirve a dos cabezas de consulta. La asociación entre grupos debe mantenerse al expandir o indexar.

La implementación didáctica expresa la repetición mediante movimientos de tensor. Es correcta para el contrato del modelo, pero una implementación de alto rendimiento puede evitar materializar copias. Esa diferencia vuelve a separar una arquitectura matemática de su realización eficiente.

31.7 Qué memoria crece más deprisa

Q, K, V y O crecen linealmente con T para B,H,d fijos. La matriz de puntuaciones crece como T^2 . Duplicar T multiplica por cuatro sus elementos.

Con B=1,H=8,T=4096 y FP32, una sola matriz $T \times T$ por cabeza ocupa $1 \cdot 8 \cdot 4096^2 \cdot 4$ bytes, unos 512 MiB. Si además guardamos probabilidades y gradientes semejantes, la presión aumenta. Esta es una cuenta de tamaño, no una medición de un framework concreto.

La atención eficiente que estudiaremos después no elimina necesariamente los productos entre todas las parejas. Evita materializar algunas matrices grandes y reorganiza el acceso a memoria. Confundir reducción de almacenamiento con reducción de complejidad aritmética lleva a expectativas equivocadas.

31.8 El retroceso de atención por etapas

Si $O=PV$, el adjunto de V es $P^T \bar{O}$ y el de P es $\bar{O} V^T$. Para una fila de softmax, el adjunto de S puede calcularse sin una matriz jacobiana completa:

$$\bar{S} = P \odot \left(\bar{P} - \text{sum}(P \odot \bar{P}, \text{eje final}) \right).$$

La suma se conserva como dimensión de tamaño uno para que pueda expandirse sobre la fila. El símbolo $\odot$ representa multiplicación por elemento.

Finalmente, para $S=QKt$ dividido por raíz de d, obtenemos los gradientes de Q y K mediante dos productos matriciales y el mismo factor. Las posiciones enmascaradas deben contribuir cero.

En Lumbre, la operación de atención online usa precisamente un VJP que reconstruye una atención densa para el retroceso. Eso proporciona gradientes comprobables, pero no un backward de memoria lineal. El proyecto final desarrolla cómo mejorar esa limitación sin esconderla.

CAPÍTULO 32

RMSNorm, RoPE y SwiGLU: entender cada pieza del decoder

32.1 Normalizar sin borrar la identidad de la fila

Las activaciones pueden cambiar de escala durante el entrenamiento. RMSNorm divide cada fila por una medida de su magnitud y aplica un peso aprendido por componente. A diferencia de una normalización que resta la media, esta operación usa la raíz de la media de cuadrados. [60]

Para $x=[3,4]$, la media de cuadrados es 12.5 y su raíz aproximadamente 3.536. Sin el pequeño epsilon y con pesos uno, la salida es aproximadamente [0.849,1.131]. No obligamos a que la media sea cero; controlamos otra propiedad de escala.

$$r = \left(\frac{1}{D} \sum_j x_j^2 + \epsilon \right)^{-1/2}, \quad y_j = x_j r \gamma_j.$$

El peso γ tiene D componentes y se comparte entre todas las filas. Su gradiente suma contribuciones de batch y posiciones. Este es otro uso concreto de la regla de unbroadcast.

32.2 Derivar RMSNorm para comprobar el grafo

Sea g el adjunto de y y definamos $u_j = g_j \gamma_j$. El gradiente de x resulta:

$$\bar{x}_j = r u_j - \frac{r^3 x_j}{D} \sum_k u_k x_k.$$

El primer término es la dependencia directa de $x_j r$. El segundo refleja que cambiar cualquier componente modifica la magnitud r que se comparte con toda la fila. Si tratáramos r como una constante, perderíamos ese segundo término.

La implementación de Lumbre no escribe esta fórmula como kernel especial: construye RMSNorm a partir de multiplicación, media, suma, raíz, división y peso. El autodiff produce la derivada compuesta. La fórmula cerrada sirve como referencia independiente para un futuro kernel fusionado.

32.3 RoPE: rotar parejas según la posición

Sin información posicional, una colección de vectores no expresa por sí sola el orden que queremos distinguir. RoPE aplica rotaciones dependientes de la posición a consultas y claves. [59]

Empieza por una pareja $[a, b]$ y un ángulo θ . Su rotación es:

$$[a', b'] = [a \cos \theta - b \sin \theta, a \sin \theta + b \cos \theta].$$

Con ángulo cero no cambia nada. Con un cuarto de vuelta, $[1, 0]$ se convierte en $[0, 1]$. Las normas se conservan en la aritmética ideal. En el modelo, diferentes parejas usan frecuencias diferentes y θ depende de la posición.

La implementación del paquete empareja la primera mitad de componentes con la segunda mitad. Otras implementaciones emparejan componentes consecutivas. Ambas convenciones pueden expresar rotaciones, pero sus pesos y layouts no son intercambiables sin una conversión coherente.

Una forma correcta no demuestra una convención correcta

Dos implementaciones de RoPE pueden devolver exactamente el mismo shape y utilizar los mismos senos y cosenos, pero rotar parejas distintas. Importar un checkpoint exige comprobar esa convención, no solo el tamaño de las matrices.

32.4 Posición absoluta y relación relativa

Si rotamos q con un ángulo asociado a la posición i y k con uno asociado a j , su producto depende de la diferencia de esas rotaciones. Esa propiedad motiva el uso de RoPE para introducir relaciones posicionales en la atención.

No se deduce que un modelo entrenado con contexto corto funcione perfectamente con un contexto arbitrariamente largo. El comportamiento fuera de la distribución y las técnicas de extensión de contexto requieren evaluación. Una fórmula que acepta un índice grande no constituye una garantía de comprensión a esa longitud.

En nuestro ejemplo, senos y cosenos se preparan para la longitud estática del programa y se pasan como parámetros fijos. El trabajo inicial de construcción de esas tablas no es una operación diferenciable de entrenamiento.

32.5 SwiGLU: una puerta que modula contenido

Después de la atención, el bloque aplica una red por posición. SwiGLU construye dos proyecciones: una produce contenido y otra una puerta suave. La puerta se transforma con SiLU y multiplica el contenido, antes de una proyección de vuelta a D . [61]

$$u = xW_u, \quad a = xW_g, \quad \text{SiLU}(a) = \frac{a}{1 + e^{-a}}, \quad z = (\text{SiLU}(a) \odot u)W_d.$$

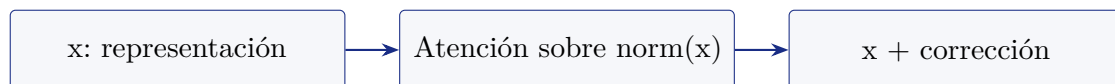
La dimensión intermedia F puede ser distinta de D . W_u y W_g tienen forma $D \times F$; W_d , $F \times D$. La operación se aplica independientemente a cada posición después de que la atención haya mezclado información entre posiciones.

No es equivalente a aplicar ReLU a una sola proyección. Tampoco es una operación matricial única: tiene dos GEMMs iniciales, trabajo elementwise y otro GEMM. Un compilador puede fusionar algunos epílogos, pero debe respetar esa dependencia.

32.6 Conexiones residuales: sumar una corrección

Un bloque residual produce x más una transformación de x . Podemos imaginar que la transformación propone una corrección en lugar de reemplazar por completo la representación. La forma de ambos sumandos debe coincidir.

En el decoder del paquete hay una normalización antes de la atención, una suma residual, otra normalización antes de SwiGLU y otra suma residual. El orden define una arquitectura pre-norm. Mover la normalización al otro lado de la suma no es una optimización algebraica gratuita: cambia el modelo.



El dibujo se repite para la subcapa feed-forward. En el código completo se conservan los nombres de cada peso, lo que permite relacionar una celda del checkpoint con la operación que utiliza.

32.7 Embedding compartido con la salida

La tabla de embeddings transforma identificadores en vectores D . Al final, su transpuesta puede transformar una representación D en V logits. Compartir esos pesos reduce el número de parámetros y une dos usos de la misma tabla.

Para el autodiff significa que el mismo parámetro recibe contribuciones desde la entrada y desde la salida. El motor debe sumarlas. Si se crean dos parámetros diferentes con valores inicialmente iguales, no se ha implementado weight tying: durante entrenamiento podrían divergir.

En Lumbre, ambos usos apuntan al mismo UOp de parámetro. Esta decisión muestra por qué la identidad y el compartido de nodos eran importantes desde las primeras semanas.

CAPÍTULO 33

Ensamblar el Transformer y comprobarlo antes de entrenar

33.1 El contrato del constructor

`Config` define vocabulario, batch, contexto, ancho, cabezas, cabezas KV, capas y dimensión intermedia. El constructor rechaza dimensiones no positivas, un ancho no divisible por las cabezas, cabezas no divisibles por HK y una dimensión de cabeza impar para la convención RoPE utilizada.

Esas comprobaciones forman parte de la interfaz. Un mensaje claro durante construcción es mejor que una lectura fuera de rango en el kernel número cuatrocientos.

```
from lumbre.nn import Config, Decoder

config = Config(vocab=24, batch=2, context=32, dim=64,
                heads=2, kv_heads=1, layers=2, hidden=128)
model = Decoder(config, seed=7)
print(model.parameter_count())
print(model.logits.shape, model.loss.shape)
```

En esta configuración el recuento es 75,584 parámetros. Los logits tienen forma(2,32,24) y la pérdida es escalar. Son valores verificables con el constructor incluido.

33.2 Contar parámetros para detectar duplicaciones

El embedding compartido aporta $V \cdot D$. Cada bloque aporta matrices de consulta y salida de $D \times D$, dos matrices KV de $D \times (H_K \cdot d)$, tres matrices del feed-forward con un total $3 \cdot D \cdot F$ y dos vectores de normalización de tamaño D . Al final hay otro vector de normalización.

$$N_{\text{param}} = VD + L \left(2D^2 + 2D(H_K d) + 3DF + 2D \right) + D.$$

Para $V=24$, $D=64$, $H=2$, $H_K=1$, $d=32$, $F=128$, $L=2$, el embedding aporta 1536; cada bloque 36992; la normalización final 64. Sumamos $1536+73984+64=75584$.

Las tablas fijas de RoPE y la máscara no son parámetros aprendidos y no se incluyen en ese recuento. Si tu suma duplica el embedding de salida, probablemente has perdido la compartición de pesos.

33.3 Seguir una capa con shapes

Etapa	Forma de salida	Operación principal
Identificadores	$B \times T$	Entrada int32.
Embedding	$B \times T \times D$	Gather.
Consultas	$B \times H \times T \times d$	Proyección y movimientos.
Claves y valores	$B \times HK \times T \times d$	Proyección y movimientos.
Atención por cabeza	$B \times H \times T \times d$	Productos y softmax causal.
Reunión de cabezas	$B \times T \times D$	Permutación y reshape.
Feed-forward	$B \times T \times D$	SwiGLU.
Logits	$B \times T \times V$	Producto con embedding transpuesto.

Escribe esta tabla para cualquier nueva configuración antes de ejecutar. Una dimensión que parece pequeña puede esconder una expansión costosa, como repetir K y V hasta H o materializar una matriz $T \times T$.

33.4 Prueba uno: el avance tiene números finitos

Inicializa los pesos con la semilla declarada. Carga todos los parámetros fijos que el programa utiliza y un batch de tokens válidos. Exige logits y pérdida finitos.

Un valor NaN inicial puede venir de una operación inválida, una fila de softmax completamente enmascarada, un parámetro sin inicializar o un índice fuera del vocabulario. No se corrige aumentando el número de pasos de entrenamiento.

La API de Lumbre detecta parámetros no cargados antes de ejecutar. En una aplicación más grande conviene diferenciar parámetros aprendidos, constantes y entradas de cada iteración para que el diagnóstico sea todavía más específico.

33.5 Prueba dos: cambiar el futuro no cambia el pasado

Construye dos secuencias iguales salvo su último token. Compara logits anteriores. Deben coincidir dentro de la tolerancia. La posición final sí puede cambiar, porque ahora recibe un token distinto dentro de su contexto permitido.

Después cambia un token inicial. Es razonable que cambien posiciones posteriores: la prueba de causalidad no exige independencia del pasado. Un test que exige igualdad de toda la salida ante cualquier cambio de entrada validaría precisamente un modelo que ignora sus datos.

33.6 Prueba tres: reducir a una instancia diminuta

Para gradientes numéricos usa una capa, ancho pequeño, pocas posiciones y vocabulario diminuto. Selecciona algunas coordenadas de matrices diferentes y del embedding compartido. Compara diferencias centrales con el gradiente compilado.

No es necesario perturbar todos los parámetros de un modelo grande para empezar. La prioridad es cubrir tipos de operación y caminos compartidos. Después se amplía la muestra

o se usa una referencia independiente para una comprobación más extensa.

33.7 Prueba cuatro: sobreajustar un batch deliberadamente

Fija un batch pequeño y repite actualizaciones sobre él. La pérdida debería poder descender de forma clara con una configuración adecuada. Este experimento no mide generalización: elimina variabilidad para comprobar que existe una señal de aprendizaje y que la actualización la aprovecha.

Si falla, vuelve a problemas menores: un parámetro escalar, una capa lineal, una clasificación de pocas clases. Un modelo grande combina demasiadas hipótesis para ser el primer depurador.

33.8 Prueba cinco: el checkpoint conserva el significado

Guarda una ejecución corta, cierra el proceso y reanuda con la misma configuración. Verifica que no se reinicien los momentos de Adam ni el contador. Cambia deliberadamente el vocabulario o la longitud de contexto y comprueba que se rechaza una reanudación incompatible cuando ese es el contrato del formato.

El resultado final de estas pruebas es más valioso que un pantallazo de texto generado: demuestra que el compilador ha unido correctamente todas las etapas que empezaron con una suma de vectores.

CAPÍTULO 34

Tokenización por bytes y BPE: una extensión completa

34.1 El carácter desconocido revela una decisión pendiente

Nuestro vocabulario de caracteres se obtiene del corpus didáctico. ¿Qué ocurre si después aparece un símbolo nuevo? No basta con cambiar su identificador a cero sin explicarlo: eso sustituye información por otro token. Para una demostración cerrada es una simplificación visible; para una aplicación general necesitamos un contrato mejor.

Una opción es empezar por los 256 valores posibles de un byte. Todo texto UTF-8 se representa como una secuencia de esos bytes. El vocabulario básico puede codificar cualquier texto válido sin inventar un token desconocido para cada carácter nuevo.

El precio es que un carácter puede ocupar varios tokens. La longitud en caracteres, bytes y tokens deja de ser la misma. Comparar pérdidas por token entre tokenizadores distintos requiere cuidado: cada uno está resolviendo unidades diferentes.

34.2 Una caja de piezas que aprende a pegar

BPE añade piezas que representan parejas frecuentes de piezas anteriores. Empezamos con bytes. Contamos parejas adyacentes en los datos de entrenamiento, elegimos una y la sustituimos por un identificador nuevo. Repetimos hasta alcanzar el límite o no encontrar una pareja suficientemente repetida.

Para entender la mecánica usamos símbolos visibles. En `ababab`, la pareja `ab` aparece tres veces. Creamos `X=ab` y la secuencia queda `XXX`. Después podríamos crear `Y=XX`, que representa `abab`, y quedaría `YX`. El texto no ha cambiado; ha cambiado su segmentación.

La implementación `lumbre/tokenizer.py` es original, pequeña y deliberadamente simple: no aplica normalización, expresiones regulares de pretokenización ni tokens especiales. Su entrenamiento recuenta parejas en cada iteración, por lo que no es una implementación eficiente para corpus masivos.

34.3 Parejas solapadas no pueden sustituirse dos veces

En `aaa`, la pareja `aa` aparece en posiciones 0–1 y 1–2, pero ambas ocurrencias comparten el carácter central. Si reemplazamos de izquierda a derecha sin solapamiento, obtenemos `Xa`, no `XX`.

El código `replace_pair` avanza dos posiciones cuando sustituye y una cuando no. Esa pequeña regla evita contar un mismo byte dos veces en la representación resultante.

```
from lumbre.tokenizer import replace_pair
assert replace_pair([1,1,1], (1,1), 256) == [256,1]
```

Contar frecuencias de parejas y ejecutar sus sustituciones son pasos diferentes. El contador puede observar solapamientos; la sustitución debe producir una secuencia válida sin reutilizar posiciones.

34.4 Un orden reproducible para los empates

Si dos parejas tienen la misma frecuencia, necesitamos una regla de desempate. El ejemplo elige de forma determinista según la pareja de identificadores. Sin esa decisión, dos entrenamientos del tokenizador podrían producir vocabularios distintos aunque los datos fueran iguales.

Cada nueva pieza solo puede referirse a piezas ya definidas. El constructor comprueba esa propiedad y rechaza referencias hacia el futuro o parejas duplicadas. Así, reconstruir los bytes de una pieza termina siempre siguiendo una lista ordenada.

El vocabulario también es un programa

Los primeros 256 tokens representan bytes. Cada token posterior contiene una regla que concatena dos tokens anteriores. Guardar únicamente el número de tokens no basta: necesitamos las reglas y su orden para interpretar los identificadores.

34.5 Codificar y decodificar

Para codificar, se transforma el texto a bytes y se aplican las fusiones aprendidas en orden. Para decodificar, se concatenan los bytes de cada token y se interpreta el resultado como UTF-8.

Una secuencia generada arbitrariamente por un modelo de bytes puede no ser UTF-8 válido. El ejemplo de muestreo usa sustitución de secuencias inválidas para poder mostrar una cadena. Esa sustitución pertenece a la presentación; no demuestra que el modelo haya generado caracteres correctos.

```
from lumbre.tokenizer import ByteBPE

text = "la memoria guarda datos. la memoria guarda ideas."
codec = ByteBPE.fit(text, max_merges=20)
ids = codec.encode(text)
assert codec.decode(ids) == text
assert codec.decode(codec.encode("un texto nuevo")) == "un texto nuevo"
copy = ByteBPE.from_state(codec.state())
assert copy.encode(text) == ids
```

El segundo `assert` comprueba que no dependemos de haber visto todos los caracteres durante el entrenamiento del tokenizador. La base de bytes hace posible representar textos nuevos, aunque su compresión en tokens sea peor.

34.6 Ajustar el tokenizador solo con entrenamiento

La ruta BPE separa primero el texto en entrenamiento y validación y aprende las fusiones con la primera parte. Después codifica ambas con el mismo vocabulario. Así no se utilizan frecuencias de validación para elegir piezas.

Esto no resuelve automáticamente la contaminación del corpus. Si el texto repite las mismas frases en ambos lados, sigue habiendo solapamiento de contenido. Separar correctamente las etapas de preparación es necesario, pero también hay que diseñar una división que responda a la pregunta experimental.

Para documentos, una división por documento suele ser más informativa que cortar a mitad de una repetición. Para autores, periodos o dominios distintos, la separación debe reflejar qué generalización queremos medir.

34.7 Usarlo con el compilador

El programa de entrenamiento acepta bytes y fusiones BPE mediante estas opciones:

```
python examples/train_decoder.py --byte-tokens --bpe-merges 30 --steps 20 --gemm blocked  
  ↪ --output reports/mi_bpe
```

Con cero fusiones y `--byte-tokens` se usa el vocabulario de 256 bytes. Con un número positivo se añaden tantas fusiones como permita la regla de entrenamiento, hasta ese máximo. El número final no tiene por qué ser exactamente 256 más el límite solicitado: el algoritmo puede detenerse antes.

La dimensión de salida del modelo cambia con el vocabulario. El constructor, la pérdida, el muestreo y el checkpoint deben usar la misma tabla. El estado del tokenizador y el hash del corpus forman parte de la configuración guardada.

34.8 Qué demuestra la prueba incluida

Las pruebas comprueban ida y vuelta con cadenas vacías, repeticiones, texto nuevo y caracteres multibyte; validan el orden de las reglas y el caso de solapamiento. Además, se ejecutó un entrenamiento corto con BPE que produjo pérdidas finitas.

Ese entrenamiento no produjo texto útil. Es una comprobación de integración: el pipeline puede cambiar de representación sin romper el compilador. La calidad lingüística exige una escala y evaluación distintas.

Diseñar una prueba que falle de verdad

¿Por qué `decode(encode(text)) == text` no basta para demostrar que dos tokenizadores asignan los mismos identificadores? Construye una explicación antes de seguir.

Dos vocabularios pueden codificar el mismo texto de formas diferentes y decodificarlo correctamente. La ida y vuelta comprueba conservación del contenido, no identidad de segmentación. Para cargar pesos entrenados con otro tokenizador necesitamos igualdad del mapeo de identificadores, reglas, normalización y tokens especiales, no solo que ambos sepan leer español.

CAPÍTULO 35

Ejecutar el entrenamiento completo y leer sus resultados

35.1 El primer comando tiene un objetivo pequeño

Desde la carpeta del paquete, después de preparar el entorno descrito al principio, ejecuta:

```
python examples/train_decoder.py --steps 60 --gemm blocked --output reports/  
↪ primer_decoder
```

El programa construye el modelo, crea el grafo de pérdida y AdamW, compila, inicializa parámetros y ejecuta actualizaciones. Después guarda un checkpoint, calcula una pérdida de validación sobre cuatro batches y genera una muestra.

Si no has exportado PYTHONPATH ni instalado el paquete, ejecuta el comando con PYTHONPATH=. en una terminal POSIX. El README incluye la instalación editable para evitar depender de esa variable. No cambies varias opciones a la vez durante la primera ejecución.

35.2 Lo que ocurre en una iteración

Se eligen posiciones de inicio en los datos de entrenamiento. Se construyen entradas y etiquetas desplazadas. Se cargan las entradas y escalares de la actualización. El programa compilado calcula pérdida, gradientes, norma global, momentos nuevos y propuestas de pesos. Se comprueba finitud y se aplican las propuestas.

La preparación de índices y la selección del batch ocurren en Python. El cálculo tensorial del entrenamiento ocurre en el C generado. NumPy sirve para almacenar datos, inicializar pesos, comparar resultados y presentar muestras; no ejecuta en secreto el avance ni el retroceso del Transformer.



35.3 Una configuración ejecutada durante la preparación

El siguiente comando reproduce la configuración principal de la edición. Los tiempos exactos dependerán del entorno y de si existe caché de compilación.

```
python examples/train_decoder.py --dim 64 --layers 2 --context 32 --steps 600 --gemm  
↪ blocked --online --output reports/decoder_final
```

Se usaron 24 caracteres, batch 2, dos cabezas de consulta, una cabeza KV, dimensión intermedia 128 y 75,584 parámetros. La atención online se usa en el avance; su retroceso reconstruye la versión densa. Todo el experimento se ejecutó en CPU.

Medida	Resultado observado	Interpretación
Actualizaciones	600	Pasos de AdamW, no épocas.
Primera pérdida de entrenamiento	3.21643686	Batch del primer paso.
Última pérdida de entrenamiento	0.27465653	Batch del paso 600.
Pérdida de validación	0.33215784	Media de cuatro batches del split descrito.
Tiempo de bucle	3.43718 segundos	Excluye compilación, evaluación y muestreo.
Kernels del grafo de entrenamiento	447	Fronteras del plan generado.
Arena	2,100,160 bytes	No es memoria total del proceso.

La compilación reportó cero segundos porque se reutilizó un binario en caché. Eso no significa que el compilador sea instantáneo. El reporte conserva el hash de la unidad compilada y distingue esa fase del bucle.

35.4 La muestra no autoriza una afirmación de calidad

La muestra observada comienza así:

```
el sol rga yla rila laa la l miria lardiorgana dajreniza el trabajo.
el sol sala y la l
```

Hay fragmentos reconocibles y errores evidentes. La pérdida ha descendido, pero el modelo no produce lenguaje general fiable. Mostrar el texto real, en lugar de seleccionar una frase artificialmente corregida, permite juzgar qué se obtuvo.

El corpus contiene seis frases originales repetidas veinte veces. La división contigua 90/10 deja contenido repetido en ambos lados. La cifra de validación es útil como comprobación del pipeline, pero no como evidencia de generalización a documentos independientes.

No llamar benchmark a un corpus de depuración

Un resultado sobre repeticiones puede demostrar que la cadena de entrenamiento funciona y memoriza patrones. No demuestra comprensión, cobertura lingüística ni superioridad frente a otro modelo. La advertencia se guarda también en el JSON del experimento.

35.5 Una ejecución anterior también enseña algo

Se conserva un reporte de 600 pasos con GEMM de referencia y atención densa. Obtuvo una pérdida final aproximada 0.31436 y validación 0.30030, con 8.14276 segundos de bucle

en aquella ejecución. No debe compararse con el nuevo tiempo como si fuera un benchmark controlado definitivo: cambian kernels y orden numérico, y las mediciones no forman una campaña estadística emparejada.

Los reportes históricos no se reescriben para que coincidan con el último código. Se identifican como experimentos anteriores. Para reanudar se usa el checkpoint del formato actual, que incluye configuración, vocabulario, tokenizador, hash del corpus, RNG y estado de AdamW.

35.6 Reanudar significa añadir pasos

La opción `-steps` indica cuántos pasos ejecutar en esta invocación, también cuando se reanuda. Para continuar diez actualizaciones desde el checkpoint anterior:

```
python examples/train_decoder.py --dim 64 --layers 2 --context 32 --steps 10 --gemm
    ↪ blocked --online --resume reports/decoder_final/checkpoint.npz --output reports/
    ↪ decoder_continuado
```

La historia nueva empieza en 601. No se reinicia en uno. La configuración del modelo y el corpus deben coincidir con los del checkpoint; un desacuerdo produce un error.

La prueba incluida compara una ejecución continua de 10 pasos con otra de 6 más 4. En el mismo entorno CPU, todos los campos guardados coincidieron bit a bit, incluidos pesos, momentos, RNG y metadatos. El archivo `reports/resume_check.json` conserva esa comprobación.

35.7 Entrenar con texto propio

La opción `-text` lee un archivo UTF-8. No descarga material ni verifica sus derechos de uso. Prepara un corpus cuya procedencia y permiso conozcas. El experimento debe registrar cómo se construyó, cuánto ocupa, cómo se dividió y qué transformaciones recibió.

```
python examples/train_decoder.py --text datos/corpus.txt --byte-tokens --bpe-merges 256
    ↪ --dim 128 --layers 4 --heads 4 --kv-heads 2 --context 64 --steps 1000 --gemm
    ↪ blocked --output reports/corpus_propio
```

Esta configuración es una propuesta de experimento, no un resultado ejecutado en esta edición. Antes de lanzarla, estima memoria y coste. No añadas ceros al número de pasos para compensar un corpus pequeño o contaminado.

El entrenador mantiene una división sencilla de un único texto. Para una evaluación seria por documentos, la extensión de datos del proyecto final reemplaza esa política por manifiestos explícitos. El libro no presenta el cargador de depuración como una plataforma de datos de producción.

35.8 La ruta GPU del mismo modelo

El mismo grafo puede enviarse al backend CUDA o HIP. En CUDA se exige declarar la arquitectura real. Un ejemplo para la RTX 5070 Ti Laptop validada, con destino `sm_120`,

sería:

```
export LUMBRE_CUDA_ARCH=sm_120
python examples/train_decoder.py --backend cuda --steps 60 --gemm blocked --output
    ↪ reports/decoder_cuda
```

No copies ese destino sin comprobar el dispositivo. El código GPU implementado incluye operaciones elementwise, reducciones y matmuls genéricos, así como una ruta de GEMM compartido. No utiliza automáticamente el ejemplo WMMA ni se presenta como un backend tensor-core de producción.

La revisión local ejecutó el decoder de depuración de 2.736 parámetros durante 20 pasos en CUDA, con atención online y GEMM bloqueada. La pérdida pasó de 3,1800122 a 2,9162602; se conservan checkpoint, evaluación y registro en `code/reports/validation_20261001/decoder_cuda/`. HIP sigue pendiente. La aceptación del laboratorio exige volver a comprobar los resultados y medir en el dispositivo del estudiante.

También se ejecutó el modelo principal de 75.584 parámetros durante 600 actualizaciones en CUDA. Una campaña posterior repite esa configuración en CPU y CUDA en el mismo host, con tareas secuenciales y carpetas de salida nuevas. El capítulo «CPU y CUDA en la misma máquina» incluye las órdenes completas, los resultados y el análisis de por qué 447 kernels pequeños pueden limitar la ventaja de una GPU.

35.9 Qué medir al aumentar el tamaño

Registra por separado construcción del grafo, compilación, primera ejecución, ejecución estable, copia del batch, checkpoint y evaluación. Cuenta tokens procesados y actualizaciones; no compares solo segundos entre configuraciones con contextos o batches distintos.

La arena estática de Lumbre informa de buffers administrados por el compilador. No incluye toda la memoria del compilador C, arrays de Python, bibliotecas cargadas o memoria interna del controlador. Presentarla como consumo total del ordenador sería incorrecto.

El primer objetivo del escalado es conservar corrección y observabilidad. Después se optimizan los cuellos de botella que revelan las medidas.

CAPÍTULO 36

Evaluar sin engañarse: datos, métricas y experimentos

36.1 Entrenamiento, validación y prueba responden preguntas distintas

Los datos de entrenamiento ajustan parámetros. La validación ayuda a escoger configuraciones y detener o comparar experimentos. Un conjunto de prueba reservado permite una evaluación final que no haya guiado repetidamente esas decisiones.

Si miramos el resultado de prueba después de cada cambio y elegimos el mejor, ya estamos utilizando ese conjunto como validación. El nombre de la carpeta no impide esa dependencia.

En un curso de compiladores, hay además otro eje: pruebas de corrección de operaciones frente a calidad del modelo. Un kernel correcto puede entrenar un modelo mediocre. Un modelo que parece bueno puede ocultar una operación incorrecta en casos que el corpus no visita.

36.2 Pérdida y perplejidad

Cuando la pérdida es una media de negativos logaritmos naturales por token, su exponencial se llama perplejidad. Una pérdida de $\log(V)$ corresponde a la predicción uniforme sobre V alternativas bajo ese planteamiento.

No compares perplejidades de tokenizadores diferentes como si tuvieran la misma unidad. Un token largo puede representar más texto que uno corto. También importan las máscaras, los tokens especiales y qué posiciones entran en la media.

Para comparar compiladores, usa exactamente el mismo modelo, pesos, datos, tokenizador, semántica numérica y reducción de la pérdida. Para comparar sistemas completos, explica qué diferencias forman parte de la propuesta.

36.3 Una pérdida finita puede estar mal normalizada

Supón dos secuencias: una tiene cuatro posiciones válidas y otra una. Si sumamos todas sus pérdidas y dividimos entre ocho posiciones de un tensor relleno, la pérdida parece artificialmente pequeña. El divisor correcto depende de la definición: cinco tokens válidos si buscamos una media por token.

Las posiciones de padding necesitan una máscara de pérdida distinta de la máscara causal. Una impide consultar el futuro; la otra decide qué etiquetas cuentan en la evaluación. Usar una no crea automáticamente la otra.

El entrenador didáctico usa ventanas completas sin padding de entrenamiento. Al extenderlo a documentos variables, se debe añadir esta distinción y probarla con ejemplos donde las cantidades válidas sean diferentes.

36.4 Contaminación y duplicados

Separar por filas al azar puede dejar copias del mismo documento en varios splits. También puede dejar fragmentos casi idénticos o versiones de un mismo texto. Una métrica favorable puede reflejar ese solapamiento.

Una práctica reproducible guarda identificadores de documentos y hashes, y define reglas de deduplicación antes de evaluar. El hash exacto detecta copias idénticas, pero no paráfrasis ni pequeñas ediciones. No se le atribuye una capacidad semántica que no tiene.

En nuestro corpus repetido el solapamiento es intencional y se declara. Convertirlo en un test de generalización exige cambiar el diseño de datos, no solo renombrar el campo `validation_loss`.

36.5 Comparación de implementaciones con los mismos pesos

Para aislar un cambio del compilador, primero compara un avance con pesos fijos. Después compara gradientes. Después una actualización. Finalmente compara una trayectoria corta de entrenamiento.

Pequeñas diferencias de redondeo pueden amplificarse tras muchos pasos. Que dos trayectorias de 600 pasos no sean bit a bit iguales no demuestra por sí solo un error. Pero tampoco autoriza a ignorar una discrepancia grande en el primer avance.

La estrategia consiste en localizar el primer nivel donde aparece una diferencia y cuantificarla. Las tolerancias se justifican por operaciones y tipos; no se aumentan indefinidamente hasta que cualquier resultado pase.

36.6 Ablaciones: cambiar una cosa con una razón

Una ablación elimina o sustituye un componente para investigar su efecto. En un compilador, podemos desactivar fusión, reutilización de memoria o GEMM bloqueado. En el modelo, podemos cambiar atención online por densa manteniendo la misma operación matemática.

Una tabla útil conserva varias dimensiones del resultado: error, tiempo, memoria y coste de compilación. Es posible mejorar una y empeorar otra. El proyecto de atención online mostrará precisamente ese caso.

Una conclusión que los datos no permiten

«La versión B es mejor porque usa menos memoria». ¿Qué falta para que esa frase responda a un objetivo de sistema?

Falta definir qué significa mejor: quizá el usuario necesita latencia, throughput, capacidad para un contexto mayor o menor coste total. Menos memoria puede permitir una configuración

antes imposible, pero también puede aumentar el tiempo de una que ya cabía. La conclusión debe nombrar el objetivo y las condiciones.

36.7 Semillas y variabilidad

Una semilla fija facilita repetir un caso, pero no convierte una única ejecución en una estimación completa de calidad. Para comparar recetas de entrenamiento conviene usar varias semillas y presentar dispersión, especialmente cuando las diferencias son pequeñas.

Para microbenchmarks, repeticiones dentro del mismo proceso estiman variación temporal local; procesos o sesiones distintas capturan otros efectos. Las dos clases de repetición no son intercambiables. Un millón de medidas casi idénticas de la misma condición no cubre un cambio de frecuencia o una actualización del controlador.

36.8 Informe final de un experimento

El informe debe permitir responder qué se cambió, por qué, con qué datos y entorno, qué se esperaba, qué se observó y qué sigue sin comprobarse. Incluye casos negativos y fallos: son información sobre el dominio de la implementación.

La última frase debe ser proporcional a la evidencia. «En esta CPU, para estas formas y esta precisión, la variante redujo el tiempo mediano» es una conclusión útil. «Hemos superado todas las bibliotecas de IA» no se deduce de una tabla de tres tamaños.

Atención online: resolver una fila sin guardar todos sus pesos

Una media que cambia de escala

Queremos calcular una mezcla ponderada mientras llegan nuevas puntuaciones. El problema es que softmax necesita una normalización global. La solución será mantener un resumen que podamos reescalar cuando aparezca una puntuación mayor.

37.1 La receta densa guarda más de lo necesario

La atención densa calcula una fila de puntuaciones, obtiene sus exponenciales normalizadas y multiplica esos pesos por los valores. Si solo necesitamos la mezcla final, quizá podamos evitar almacenar todos los pesos a la vez.

Para una fila, la salida puede escribirse como un cociente:

$$O = \frac{\sum_j e^{s_j} v_j}{\sum_j e^{s_j}}.$$

El numerador es un vector, porque cada valor v tiene varias componentes. El denominador es un escalar. Si las exponenciales fueran siempre manejables, podríamos acumular ambos conforme llegan términos. La dificultad es que e^{s_j} puede desbordarse.

37.2 Restar un máximo que todavía no conocemos

En softmax estable restábamos el máximo de todas las puntuaciones. En una lectura online, aún no sabemos si la próxima será mayor. Podemos usar el máximo visto hasta ahora y corregir la escala de lo acumulado cuando cambie.

Mantendremos tres objetos: m , máximo visto; l , suma de exponenciales en la escala de m ; y u , suma ponderada de valores en esa misma escala. La salida parcial es u/l .

Al recibir una puntuación s con valor v , el nuevo máximo es $m' = \max(m, s)$. Lo antiguo estaba medido respecto a m y ahora debe medirse respecto a m' . El factor de conversión es $\alpha = e^{m-m'}$. La contribución nueva pesa $\beta = e^{s-m'}$.

$$l' = \alpha l + \beta, \quad u' = \alpha u + \beta v, \quad m \leftarrow m'.$$

No hemos aproximado el denominador descartando términos. Hemos cambiado su escala común. En aritmética real, el cociente representa exactamente la misma atención sobre los

elementos visitados. En coma flotante, el orden de operaciones puede introducir diferencias que debemos medir.

37.3 Una ejecución entera con tres puntuaciones

Usamos puntuaciones 0, $\log 2$ y $\log 4$, y valores escalares 1, 3 y 5. Sus exponenciales sin normalizar serían 1, 2 y 4. La respuesta directa es $(1 + 6 + 20)/(1 + 2 + 4) = 27/7$.

Primera puntuación: $m=0$, $l=1$ y $u=1$. La salida parcial es 1.

Segunda puntuación: el máximo cambia a $\log 2$. El factor de lo antiguo es un medio y el nuevo peso es uno. Por tanto, $l=0.5 \cdot 1 + 1 = 1.5$ y $u=0.5 \cdot 1 + 3 = 3.5$. La salida parcial es $7/3$, exactamente la mezcla de las dos primeras contribuciones.

Tercera puntuación: el máximo cambia a $\log 4$. Volvemos a reescalar por un medio. Ahora $l=0.5 \cdot 1.5 + 1 = 1.75$ y $u=0.5 \cdot 3.5 + 5 = 6.75$. El cociente $6.75/1.75$ es $27/7$.

El invariante que demuestra la corrección

Después de visitar cualquier prefijo, m es su máximo, l es la suma de exponenciales de ese prefijo restando m y u es la suma de esas exponenciales multiplicadas por sus valores. Reescalar por el mismo factor mantiene ambos sumatorios en la escala nueva.

37.4 Inicialización y filas vacías

Podemos iniciar m en menos infinito, l en cero y u en cero. La primera puntuación finita produce factor antiguo cero y peso nuevo uno. Así no necesitamos un caso especial para el primer término, siempre que las operaciones y dominios estén controlados.

Una fila sin posiciones válidas no tiene una distribución softmax ordinaria: el denominador es cero. Hay que definir si se rechaza, se devuelve cero o se aplica otra convención. Nuestro operador causal de longitudes positivas incluye la posición propia, por lo que cada fila tiene al menos una contribución.

Si añadimos máscaras arbitrarias, ya no basta esa garantía. Los tests deben incluir una fila totalmente enmascarada y comprobar la política elegida.

37.5 De un elemento a un bloque

Para trabajar con tiles, un bloque de puntuaciones puede producir su máximo local, suma exponencial local y suma ponderada local. Se combinan dos resúmenes con un máximo común, reescalando cada uno.

Sean los resúmenes A y B. Tomamos m igual al máximo de sus máximos. Multiplicamos la suma y el numerador de A por $e^{m_A - m}$ y los de B por $e^{m_B - m}$. Después sumamos. Es la misma idea, ahora aplicada a grupos.

Esa propiedad permite organizar una reducción por bloques sin materializar la matriz completa. El tamaño del bloque se elige según recursos y patrón de cómputo, no porque la matemática exija un número concreto.

37.6 La operación nueva en Lumbre

La función `online_attention(q,k,v)` acepta tensores FP32 de igual forma terminados en $T \times D$, con T y D positivos. Aplica atención causal y devuelve la misma forma. La implementación materializa sus operandos y asigna un resultado por fila.

```
import numpy as np
from lumbre import param, Program, online_attention

q = param("q", (1,2,5,4))
k = param("k", (1,2,5,4))
v = param("v", (1,2,5,4))
o = online_attention(q,k,v)
rng = np.random.default_rng(63)
feed = {name:rng.normal(size=q.shape).astype("float32")
        for name in ("q","k","v")}
with Program([o], gemm="blocked") as p:
    result = p.run(feed)[0]
    assert result.shape == q.shape
    assert np.isfinite(result).all()
```

La prueba de finitud no basta para aceptar el operador. La batería compara además con atención densa y compara los gradientes de una pérdida cuadrática respecto a Q, K, V para varias formas, incluidas dimensiones irregulares.

37.7 Qué memoria ahorra exactamente

El forward online no crea un tensor global $T \times T$ de puntuaciones o probabilidades. Su salida y operandos siguen teniendo tamaño proporcional a $T \cdot D$. Nuestro helper usa el buffer de salida como acumulador de la fila.

Ese helper no es una implementación GPU de alto rendimiento: una ruta con un hilo por fila y escrituras frecuentes del acumulador no organiza la reutilización como un kernel FlashAttention especializado. La mejora de complejidad de almacenamiento no garantiza una buena realización física.

La diferencia entre algoritmo, mapping y hardware es justamente el tema del curso. La misma recurrencia puede formar parte de un kernel excelente o de uno lento según cómo se distribuyan sus datos y trabajo.

CAPÍTULO 38

Del algoritmo online a FlashAttention y sus derivados

38.1 Qué idea investigamos en los artículos

FlashAttention plantea la atención exacta con una organización consciente del tráfico entre memoria rápida y memoria de mayor capacidad. FlashAttention-2 estudia además el reparto del trabajo, y FlashAttention-3 adapta el diseño a mecanismos de Hopper, entre otros cambios. No tratamos sus resultados de rendimiento como mediciones propias. [41, 42, 43]

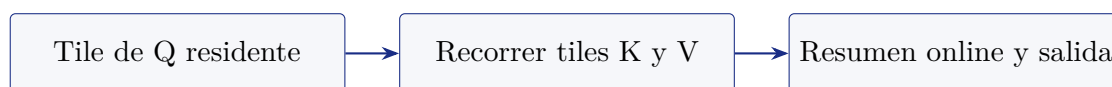
La lección común es que contar multiplicaciones no basta. Una implementación puede hacer un número similar de operaciones y tardar mucho menos si deja de materializar intermediarios y coordina mejor transferencia, cómputo y paralelismo.

El repositorio consultado en 2026 incluye rutas recientes basadas en CuTe DSL. Eso no autoriza a inventar una publicación científica para cada nombre de versión del código ni a suponer que toda GPU soporta las mismas rutas. [44]

38.2 Reutilizar Q mientras recorremos K y V

Supón que un bloque de trabajo posee varias filas de Q. Las mantiene cerca de la unidad de cómputo y recorre K,V por bloques. Cada bloque produce puntuaciones parciales, actualiza máximos y denominadores y añade una contribución al resultado.

La forma de los tiles controla varias cosas a la vez. Más filas de Q pueden aumentar reutilización de K,V, pero consumen más acumuladores. Un bloque más ancho en K puede aprovechar mejor una operación matricial, pero requiere más memoria temporal. Hay que equilibrar recursos, no maximizar cada dimensión por separado.



En la región causal, algunos bloques quedan completamente fuera del triángulo permitido y pueden omitirse. Los bloques que cruzan la diagonal necesitan una máscara por elemento. Separar esos dos casos evita realizar comprobaciones innecesarias en todo el dominio.

38.3 El coste no aritmético puede dominar

Una puntuación requiere productos y sumas, pero softmax requiere máximo, exponencial, suma y reescalado. Esas operaciones no tienen necesariamente el mismo throughput que la multiplicación matricial.

Si aceleramos mucho los GEMMs, el resto puede convertirse en el cuello de botella. La optimización deja de consistir en «hacer más tensor cores» y pasa a organizar el trabajo escalar o vectorial, reducir conversiones y solapar etapas.

La lectura de un kernel debe contar quién ejecuta cada parte. ¿Todos los warps participan en el GEMM? ¿Quién actualiza el máximo? ¿Dónde vive el acumulador de salida? ¿Hay participantes esperando mientras otros hacen softmax? Estas preguntas conectan el algoritmo con el perfil temporal.

38.4 Guardar log-sum-exp para el retroceso

Para reconstruir una probabilidad basta con conocer su puntuación y el logaritmo del denominador correspondiente. Si guardamos por fila $LSE = m + \log l$, podemos recuperar $P_{ij} = e^{S_{ij} - LSE_i}$ cuando volvemos a calcular un bloque de puntuaciones.

Así evitamos guardar toda P durante el avance. En el retroceso recomputamos bloques y acumulamos gradientes. Cambiamos memoria por cómputo adicional, pero ese cómputo puede ser barato comparado con trasladar una matriz grande.

No basta con eliminar P del checkpoint del forward. El backward debe estar diseñado para no volver a materializarla completa. Nuestra primera operación online todavía usa un VJP denso; por eso su ahorro de memoria durante entrenamiento es mucho menor que el de un forward aislado.

38.5 Una derivación que reduce un término del backward

Recordemos que $\bar{P} = \bar{O}V^T$. Para una fila i, el término de reducción de softmax es $D_i = \sum_j P_{ij} \bar{P}_{ij}$. Sustituyendo la definición y reordenando sumas obtenemos:

$$D_i = \sum_d \bar{O}_{id} \left(\sum_j P_{ij} V_{jd} \right) = \sum_d \bar{O}_{id} O_{id}.$$

Podemos calcular D a partir de la salida O y su adjunto, sin guardar todas las derivadas de P. Después cada bloque usa $\bar{S}_{ij} = P_{ij}(\bar{P}_{ij} - D_i)$.

Esta igualdad es una herramienta concreta para diseñar el backward. Es una transformación algebraica de sumas; en coma flotante, el nuevo orden puede cambiar el redondeo y necesita tolerancias acordes.

38.6 La propiedad de los gradientes importa

Un bloque de consultas puede acumular su propio gradiente de Q con facilidad. Pero varios bloques de consultas contribuyen a las mismas filas de K y V. Necesitamos una estrategia para reunir esas contribuciones sin carreras.

Una opción usa acumulación atómica. Otra asigna a un kernel la propiedad de un tile de K,V y recorre las consultas necesarias. Otra produce parciales y ejecuta una reducción posterior. Cada estrategia cambia memoria, paralelismo, determinismo y número de lanzamientos.

No existe una solución universalmente mejor. Para un proyecto de semestre, una reducción explícita de parciales puede ser más fácil de verificar antes de introducir atomics o kernels persistentes.

La carrera no se arregla con tolerancia numérica

Dos escrituras no coordinadas pueden perder contribuciones. El resultado quizá cambie entre ejecuciones y a veces parezca cercano. Aumentar `atol` no convierte una carrera en una aproximación legítima.

38.7 Dropout y reproducibilidad del retroceso

Si añadimos dropout a los pesos de atención, el backward necesita aplicar la misma selección aleatoria que el forward. Guardar toda la máscara consume memoria; regenerarla requiere un generador y un mapeo de contadores reproducibles.

Cambiar el tiling no debería cambiar sin explicación qué números aleatorios corresponden a cada posición si queremos comparar la misma ejecución. Una semilla global no basta si el orden de consumo del generador depende del scheduler.

El modelo incluido no usa dropout. Al incorporarlo, se añade una operación con estado o una generación funcional por contador, junto a sus pruebas. No se considera una modificación puramente elementwise sin consecuencias sobre el runtime.

38.8 Atención de entrenamiento y de generación

En entrenamiento suelen existir muchas consultas por secuencia. En generación incremental puede haber una consulta nueva y un historial largo de K,V. El patrón de reutilización y paralelismo cambia.

Una configuración excelente para prefill no tiene por qué serlo para decode. Si el número de filas de Q es pequeño, quizá no haya suficientes bloques para ocupar la máquina. Se pueden dividir otras dimensiones o usar varias particiones del historial con una combinación de resúmenes.

La máscara también necesita una definición para longitudes de Q y K diferentes. Nuestro operador educativo exige la misma longitud y causalidad simple; no se presenta como una API general de atención con caché o longitudes irregulares.

38.9 Reproducción de un artículo frente a inspiración

Implementar la recurrencia online y medirla en CPU es una reproducción de una idea algorítmica, no una reproducción completa de los resultados de un artículo GPU. Para esto último harían falta arquitectura, tipos, tamaños, baseline y protocolo compatibles, además de la implementación correspondiente.

En el proyecto final usamos tres etiquetas: derivación comprobada, implementación local validada y comparación externa pendiente. Mantener esas etiquetas separadas permite realizar investigación útil incluso con hardware limitado, sin inflar la conclusión.

Kernels persistentes, planificación y límites del autotuning

39.1 Cuando los tiles no tienen el mismo trabajo

En una matriz regular, podemos asignar un tile de salida a cada bloque. En una mezcla de expertos o una atención con longitudes distintas, algunos tiles pueden requerir mucho más trabajo que otros. Una partición estática puede dejar unidades ociosas mientras unas pocas terminan.

Un kernel persistente mantiene un conjunto de trabajadores que van tomando tareas. La idea se parece a una cola de pedidos: al terminar uno, el trabajador solicita otro. Así puede adaptarse mejor a una distribución irregular.

Pero la cola cuesta. Hay sincronización, metadatos, decisiones y recursos ocupados durante más tiempo. Para una operación pequeña, ese mecanismo puede ser más caro que el trabajo que reparte.

39.2 Un scheduler de tareas como modelo independiente

Antes de escribir un kernel persistente, crea un simulador pequeño que reciba tareas con costes conocidos y compare asignación fija con una cola. El simulador permite estudiar balance idealizado, no latencias reales de la GPU.

Para tareas de costes $[8,1,1,1,1]$ y dos trabajadores, una asignación por bloques contiguos puede dejar un reparto 9 frente 3. Una cola que toma la primera tarea larga y reparte las pequeñas entre el otro trabajador termina en 8, ignorando costes de coordinación. El máximo de las cargas limita la finalización.

Ahora añade un coste de 0.5 por tarea tomada. La ganancia cambia. Este ejemplo obliga a incluir overhead antes de asumir que una planificación dinámica siempre gana.

39.3 El orden de tareas también afecta a la caché

Dos tiles cercanos pueden reutilizar partes de A o B. Una cola que equilibra cómputo pero destruye localidad puede trasladar más datos. El scheduler debe considerar no solo cuánto cuesta una tarea aislada, sino qué datos comparte con otras.

En problemas agrupados, ordenar por tamaños o agrupar tareas compatibles puede reducir cambios de configuración. Sin embargo, ordenar también cuesta y puede aumentar la latencia de un pedido que esperaba ser atendido primero.

Un compilador de servicio tiene objetivos diferentes de un benchmark de throughput total. Es necesario especificar si optimizamos la suma de tiempos, el tiempo de la última tarea o una latencia por solicitud.

39.4 Especializar sin compilar infinitas variantes

Cada tamaño, dtype, layout y arquitectura podría producir una configuración distinta. Compilar todas las combinaciones no es viable. Se necesitan familias de especialización y una alternativa genérica.

Una política sencilla agrupa tamaños por intervalos y alinea tiles. Otra utiliza un pequeño conjunto de candidatos elegidos por un modelo de coste. La caché debe identificar todas las propiedades que cambian el significado del programa, no solo el tamaño total de elementos.

Si dos tensores tienen el mismo número de elementos pero strides distintos, reutilizar un kernel especializado para contigüidad puede ser incorrecto. Si el destino cambia de arquitectura, un binario anterior puede ser incompatible aunque los shapes coincidan.

39.5 Autotuning con un presupuesto explícito

Supongamos que probar cien candidatos tarda diez segundos y ahorra un microsegundo por ejecución. Harían falta unos diez millones de ejecuciones para recuperar ese coste, sin contar almacenamiento o compilación adicional. Es una cuenta de amortización, no una opinión sobre si el tuning es elegante.

Un autotuner puede decidir no buscar cuando la operación solo se ejecutará unas veces. En entrenamiento de muchos pasos, el mismo coste puede amortizarse bien. La frecuencia esperada de uso forma parte del modelo de decisión.

El proyecto de autotuning del libro exige registrar también candidatos descartados por incorrectos y por incompatibilidad. Seleccionar el más rápido entre programas que no calculan lo mismo es una optimización inválida.

39.6 El mejor candidato de una muestra puede sobreajustar

Si elegimos una configuración usando solo matrices cuadradas múltiplos de 128, quizá falle o pierda mucho en dimensiones de borde. Reservamos un conjunto de formas no usado durante selección y comprobamos allí la política.

También necesitamos estabilidad. Una configuración que gana por una diferencia menor que el ruido temporal no merece una conclusión fuerte. Podemos preferir la opción más simple o robusta cuando las medidas no distinguen claramente.

El resultado de esta semana

Propón kernels con guardas, selección y respaldo. Explica qué preserva su corrección y cuántas ejecuciones amortizan la búsqueda.

Escalar un modelo: primero contar memoria y trabajo

El tamaño de los pesos no es el tamaño del entrenamiento

Guardar un libro ocupa una estantería. Trabajar sobre él puede necesitar otra para anotaciones, borradores y copias temporales. Un entrenamiento mantiene pesos, gradientes, estados del optimizador, activaciones y buffers de ejecución.

40.1 Un presupuesto por categorías

Para P parámetros en FP32, los pesos ocupan $4P$ bytes. Si guardamos gradientes del mismo tipo, añadimos $4P$. Dos momentos FP32 de Adam añaden $8P$. Esa suma simple da $16P$ bytes antes de activaciones y otros temporales.

En precisión mixta puede haber pesos de trabajo de menos bytes, copias maestras FP32 y gradientes de un tipo elegido. No existe una cifra universal de bytes por parámetro para todas las implementaciones. Hay que enumerar lo que realmente se almacena y cuándo está vivo.

Lumbre conserva los parámetros y calcula propuestas de actualización como salidas separadas antes de asignarlas. Por tanto, un presupuesto basado solo en pesos y momentos no representa su pico completo. El planificador de arena da una cuenta más cercana de sus buffers, pero tampoco incluye toda la memoria del proceso.

Categoría	Pregunta de dimensionamiento	Estrategia posible
Pesos	¿Cuántos parámetros y qué tipo?	Tipos compactos o partición.
Gradientes	¿Todos simultáneos o por grupos?	Acumulación y liberación planificada.
Optimizador	¿Qué estados por parámetro?	Partición u offload.
Activaciones	¿Qué necesita el backward?	Recomputation o checkpointing.
Temporales	¿Qué kernels materializan?	Fusión, arena y algoritmos con menos memoria.

40.2 Una cuenta que descarta un plan antes de ejecutarlo

Con siete mil millones de parámetros y un presupuesto de 16 bytes por parámetro, el estado básico suma 112 GB decimales. Eso ya supera con mucho una tarjeta de 12 GB, antes de añadir activaciones.

No se arregla reduciendo únicamente el batch a uno. El batch afecta a activaciones, pero no elimina el estado de todos los parámetros entrenables. Cuantizar pesos para inferencia o entrenar solo un adaptador responde a tareas distintas de preentrenar todos los pesos.

La cuenta no impide trabajar con modelos grandes mediante partición, descarga a otra memoria o entrenamiento parcial. Obliga a nombrar esas decisiones y sus costes, en lugar de prometer un entrenamiento completo a partir del tamaño del archivo de inferencia.

40.3 Una arquitectura intermedia concreta

Entre 75,584 parámetros y un modelo de miles de millones hay etapas útiles. Una configuración hipotética con $V=32768$, $D=768$, $L=12$, $H=12$, $HK=4$, $F=3072$ tiene 128,994,048 parámetros con nuestra fórmula y pesos compartidos de entrada y salida.

```
from lumbre.nn import Config

config = Config(vocab=32768, batch=1, context=512,
               dim=768, heads=12, kv_heads=4,
               layers=12, hidden=3072, online=True)
```

Este bloque define una configuración; no informa de un entrenamiento ejecutado. Antes de construir todos sus arrays o compilar, estima el estado y prueba una versión menor. La implementación BPE didáctica tampoco está diseñada para aprender decenas de miles de fusiones con eficiencia industrial.

El objetivo de esta etapa es convertir el escalado en un proceso con puertas de aceptación: formas correctas, memoria dentro del presupuesto, paso numérico verificado, rendimiento estable y checkpoint probado. No se salta directamente a una escala enorme para descubrir errores básicos allí.

40.4 Activaciones: el contexto puede dominar

Un tensor de activaciones $B \times T \times D$ en FP32 ocupa $4BTD$ bytes. Si guardamos varios por capa, el factor del número de capas importa. La atención densa añade términos proporcionales a BHT^2 , que pueden dominar al aumentar el contexto.

Para $B=1$, $T=2048$, $D=768$, una sola activación $B \times T \times D$ ocupa aproximadamente 6 MiB. Multiplicar esa cifra por todas las activaciones vivas de varias capas produce una cantidad mucho mayor. No basta con contar la salida de cada bloque: el backward puede necesitar proyecciones, normalizaciones, puertas y estadísticas internas.

La fusión elimina algunos intermediarios; el checkpointing evita conservar otros; la atención online evita ciertas matrices cuadráticas. Son mecanismos diferentes y sus ahorros no deben sumarse sin analizar posibles solapamientos.

40.5 Checkpointing de activaciones no es guardar el entrenamiento en disco

El mismo término se usa para dos trabajos. Un checkpoint persistente permite recuperar un entrenamiento tras interrumpirlo. El checkpointing de activaciones guarda solo algunos estados del avance y recalcula los demás durante el retroceso.

Imagina una cadena de doce bloques. Podemos guardar la entrada de cada grupo de tres y recomputar su interior cuando toque derivarlo. Reducimos activaciones retenidas, pero repetimos cómputo. La política óptima depende de costes desiguales, memoria disponible y posibilidades de solapamiento.

Un compilador puede representar esa decisión insertando subgrafos de recomputación o cambiando la planificación. Debe conservar los valores aleatorios y efectos necesarios para repetir la misma función. No se puede recomputar libremente una operación que consume un estado externo distinto cada vez.

40.6 Cuánto trabajo aproxima un entrenamiento denso

Para una capa lineal grande, el avance cuesta aproximadamente dos operaciones por multiplicación-acumulación y el backward requiere productos adicionales. De ahí surge una estimación habitual del orden de $6 \cdot P \cdot N$ operaciones para entrenar un modelo denso sobre N tokens, bajo supuestos simplificados.

No es una ley exacta. La atención dependiente del contexto, embeddings, normalizaciones, activaciones, recomputación, comunicación y estructuras dispersas cambian la cuenta. En un MoE no se usa sin pensar el total de parámetros como si todos participaran en cada token.

Para una planificación inicial, escribe la estimación y sus términos omitidos. Después calibra con un paso real de una configuración pequeña o intermedia. Si la estimación y la observación difieren, investiga el cuello de botella antes de extrapolar varios órdenes de magnitud.

40.7 Datos y parámetros compiten por el presupuesto

Aumentar el modelo sin aumentar adecuadamente datos y entrenamiento puede no ser la mejor utilización de un presupuesto. Los trabajos de escalado compute-optimal estudian precisamente esa distribución de recursos; no ofrecen una regla universal independiente del dominio, la receta y los objetivos posteriores. [65]

Un curso de compiladores no necesita entrenar todos los puntos de una ley de escalado para aprender de ella. Sí necesita evitar la conclusión de que más parámetros implican automáticamente mejor experimento. Un modelo menor bien entrenado y evaluado puede ser una referencia mucho más útil para optimizar el sistema.

40.8 Puertas de aceptación antes de un entrenamiento largo

Primero, un paso completo produce valores finitos y gradientes contrastados. Segundo, varias decenas de pasos sobre un batch fijo reducen una pérdida controlada. Tercero, una ejecución con datos reales conserva throughput y memoria. Cuarto, guardar y reanudar produce el estado esperado. Quinto, el evaluador utiliza un split independiente y una métrica definida.

Solo después se reserva una ejecución larga. Si la preparación de datos tarda más que el kernel, optimizar el kernel no resuelve el sistema. Si el checkpoint tarda mucho y bloquea el dispositivo, debe medirse como parte de la operación real.

Precisión mixta y cuantización: ahorrar sin cambiar a ciegas el cálculo

41.1 Almacenar, multiplicar y acumular pueden usar tipos distintos

Un GEMM puede leer valores de baja precisión, multiplicarlos mediante una unidad especializada y acumular en una precisión mayor. El resultado puede volver a almacenarse en otro formato. Esos son tres puntos de redondeo potencialmente diferentes.

El dtype de un tensor de entrada no describe toda la semántica numérica del kernel. Para comparar implementaciones necesitamos tipos de almacenamiento, multiplicación, acumulación, epílogo y salida, además de escalas y modos de redondeo.

En Lumbre el camino principal es FP32. El ejemplo WMMA ilustra entradas FP16 y acumulación FP32, pero no está integrado como política automática de entrenamiento. El estudiante debe añadir una representación explícita de tipos antes de extender esa ruta de forma general.

41.2 Por qué el backward puede necesitar otro rango

Un valor de activación puede ser moderado mientras un gradiente es muy pequeño. Redondear ambos con el mismo formato puede tener efectos distintos. Una actualización menor que la separación entre números representables del peso puede desaparecer al almacenarla.

Una copia maestra de mayor precisión puede conservar actualizaciones pequeñas y producir pesos de trabajo de menor precisión para el avance. Eso aumenta memoria respecto a guardar solo los pesos compactos, pero evita perder sistemáticamente cambios.

La decisión debe verificarse con un problema donde las actualizaciones pequeñas importen. Un test que usa únicamente números enteros y pasos grandes no evalúa esa propiedad.

41.3 Escalado de la pérdida

Multiplicar la pérdida por un factor también multiplica sus gradientes. Podemos calcular esos gradientes escalados y dividirlos antes de aplicar la actualización. La idea ayuda cuando un formato tiene un rango insuficiente para gradientes pequeños, pero puede introducir desbordamiento si el factor es excesivo.

Un escalado dinámico necesita detectar valores no finitos, decidir si omite la actualización y ajustar el factor. El contador del optimizador y el scheduler de tasa deben tener un comportamiento definido en un paso omitido. No se limita a multiplicar una línea de código.

BF16 y FP16 tienen rangos y precisiones diferentes, por lo que no debe trasladarse mecánicamente una receta de uno a otro. Tampoco se supone que todo kernel de un dispositivo admita ambos tipos con la misma eficiencia.

41.4 Cuantización simétrica con una cuenta pequeña

En un esquema propio simple, aproximamos x por sq , donde q es un entero limitado y s una escala positiva. Elegimos q redondeando x/s y saturando al rango permitido.

Con $s=0.1$, el valor 0.26 se aproxima por 0.3 si el redondeo produce 3. Con $s=1$, se aproxima por 0. Ese ejemplo muestra el compromiso: una escala grande cubre magnitudes mayores, pero pierde detalle en valores pequeños.

Si un grupo contiene un valor 100 y muchos valores 0.01, una única escala puede representar mal la mayoría. Escalas por canal o por bloque reducen ese conflicto a cambio de metadatos y operaciones adicionales.

41.5 La calibración no debe mirar el examen

Un esquema puede elegir escalas a partir de estadísticas de datos de calibración. Esos datos deben estar definidos y separados de la evaluación final. Elegir la escala que produce mejor métrica en el conjunto de prueba es utilizarlo para ajustar el sistema.

El test del kernel cuantizado compara primero con una referencia que aplique exactamente la misma cuantización. Después se compara el modelo cuantizado con el modelo de mayor precisión para medir degradación de calidad. Son dos preguntas distintas: corrección de la implementación y efecto de la aproximación.

41.6 Formatos muy compactos y escalas por bloques

Los repositorios recientes de kernels incluyen formatos compactos y esquemas de escala por bloques. La compatibilidad depende de arquitectura, layout y operación; no se deduce únicamente del nombre FP8 o FP4. [11, 23, 24]

Para diseñar una IR, un tensor cuantizado debe transportar más que un array. Necesita identificar valores, escalas, ejes de agrupación, posible zero-point, formato y reglas de reconstrucción. Si se pierde esa información al hacer un reshape, el resultado puede seguir ocupando los mismos bytes y representar números diferentes.

Un movimiento es gratuito solo si conserva la interpretación completa del dato. Reorganizar escalas junto con valores puede requerir trabajo real o impedir una fusión.

41.7 Una política de error por operación

No todas las operaciones toleran el mismo error. Una suma larga acumula redondeo; una comparación cercana al umbral puede cambiar una decisión; un softmax puede amplificar diferencias de logs; una normalización tiene divisiones sensibles a magnitudes pequeñas.

Una política razonable empieza por conservar mayor precisión en reducciones y estadísticas críticas, y evalúa dónde reducir almacenamiento o multiplicación. El resultado final se comprueba en el modelo, no solo en kernels aislados.

Un falso ahorro

Un tensor baja de cuatro a un byte por elemento, pero el kernel lo convierte completo a FP32 en un buffer temporal y después ejecuta el mismo GEMM. ¿Qué ha mejorado y qué puede no haber mejorado?

Puede haber disminuido el almacenamiento persistente o el tráfico desde una fuente externa. Pero el pico temporal puede seguir siendo alto, y el coste de conversión puede empeorar la latencia. Para aprovechar cómputo de baja precisión necesitamos una ruta que lo use realmente, no solo un archivo comprimido.

Entrenamiento distribuido: repartir estado y reunir contribuciones

42.1 Dos estudiantes calculan mitades de un lote

Supón que un lote contiene ocho ejemplos y dos dispositivos procesan cuatro cada uno con los mismos pesos. Cada uno obtiene una media de gradientes local. Para reproducir la media de los ocho, promediamos ambas contribuciones porque los tamaños coinciden.

Si uno procesa seis y otro dos, las medias locales deben ponderarse $6/8$ y $2/8$. Promediarlas con peso un medio cambia el objetivo. La matemática de la reducción debe definirse antes de escoger una biblioteca de comunicación.

En paralelismo de datos, los parámetros suelen replicarse y los gradientes se combinan. La memoria de activaciones por dispositivo puede bajar al dividir el lote, pero los pesos replicados siguen ocupando espacio en cada uno.

42.2 All-reduce, reduce-scatter y all-gather

All-reduce combina contribuciones y entrega el resultado completo a todos los participantes. Reduce-scatter combina y reparte partes del resultado. All-gather reúne partes para que cada participante obtenga la colección completa.

Imagina dos vectores $[1,2]$ y $[3,4]$. Una suma all-reduce entrega $[4,6]$ a ambos. Una reduce-scatter puede entregar $[4]$ al primero y $[6]$ al segundo. Un all-gather posterior vuelve a construir $[4,6]$ en ambos.

Estas operaciones no son solo funciones matemáticas puras desde el punto de vista del runtime: los participantes deben ejecutarlas de forma compatible y sus buffers tienen dependencias de finalización. NCCL y RCCL son referencias de infraestructura colectiva; el repositorio de RCCL consultado remite a `rocm-systems`. [66, 67]

42.3 Particionar estados para no repetirlos

Si todos los dispositivos guardan los mismos momentos de Adam, parte de la memoria está duplicada. La familia de ideas de ZeRO reparte estados y, en distintas etapas, gradientes y parámetros, coordinando comunicación para conservar el entrenamiento. [62]

La cuenta ideal de memoria puede mejorar con el número de dispositivos, pero no desaparece el coste de mover datos ni los temporales necesarios. Un plan de partición debe explicar cuándo cada dispositivo necesita una parte que no posee y cómo la obtiene.

La elección también afecta al checkpoint: guardar únicamente el fragmento de un dispositivo no produce un modelo completo salvo que el formato y el manifiesto describan el resto. La reanudación con otro número de dispositivos puede necesitar redistribución.

42.4 Tensor parallelism con una multiplicación pequeña

Para $Y=XW$, podemos repartir columnas de W . Cada dispositivo calcula columnas distintas de Y . Si la operación siguiente acepta esa partición, quizá no sea necesario reunir Y inmediatamente.

También podemos repartir el eje de reducción de W y las columnas correspondientes de X . Cada dispositivo calcula una suma parcial de Y y hay que combinar esas parciales. Son particiones diferentes, con comunicaciones diferentes.

El trabajo de Megatron-LM es una referencia histórica importante para organizar paralelismo dentro de capas Transformer. La derivación de los dos casos anteriores es suficiente para empezar a razonar sobre dónde se necesitan colectivas en nuestro propio grafo. [63]

42.5 No reunir un tensor solo para volver a partirlo

Un compilador consciente de particiones puede conservar layouts distribuidos entre operaciones. Por ejemplo, una activación elementwise puede aplicarse a cada fragmento sin reunir todo el tensor. Una normalización sobre un eje partido necesita estadísticas globales o una estrategia equivalente.

Por eso el tipo de una IR distribuida debería incluir qué dimensiones están particionadas, qué valores son parciales y qué grupo de dispositivos participa. Un tensor local y una suma parcial de un tensor global no son intercambiables aunque tengan el mismo shape local.

Una suma parcial que parece una salida completa

Dos dispositivos calculan cada uno la mitad del eje K de un GEMM. Ambos producen arrays $M \times N$. Ninguno es todavía la salida Y : falta sumar las contribuciones. La forma por sí sola no expresa esa obligación.

42.6 Pipeline parallelism y burbujas

Podemos repartir capas en etapas. El primer dispositivo procesa las primeras y envía activaciones al segundo. Si solo hay un microbatch, algunos esperan mientras otros trabajan. Al introducir varios microbatches se puede llenar la tubería, pero aparecen más estados vivos y una planificación de avance y retroceso.

Una burbuja es un intervalo donde una etapa no tiene trabajo útil listo. Reducir burbujas puede aumentar memoria o complejidad de comunicación. La planificación debe respetar que un backward utiliza parámetros coherentes con el forward correspondiente según la receta elegida.

No basta con dibujar flechas diagonales bonitas. El calendario debe especificar qué microbatch, fase y versión de parámetros ocupa cada celda. Ese calendario puede comprobarse con una

simulación antes de desplegarlo.

42.7 Solapar comunicación y cómputo

Cuando un grupo de gradientes está listo, puede iniciarse su comunicación mientras se calculan otros. Para que el solapamiento sea correcto, el buffer no puede sobrescribirse ni leerse como resultado global antes de finalizar.

Agrupar muchos gradientes amortiza costes de lanzamiento de comunicación, pero retrasa el momento en que el primer grupo puede empezar. Agrupar muy poco aumenta overhead. La política de buckets es una decisión de scheduling con un compromiso medible.

El tiempo ideal de dos fases solapables se aproxima al máximo de sus duraciones, no a su suma. Sin embargo, comparten recursos y pueden interferir; un solapamiento dibujado en el grafo no garantiza el mismo solapamiento físico.

42.8 Fallos y orden colectivo

Si un participante ejecuta dos colectivas en un orden diferente al resto, puede haber bloqueo o resultados inválidos. Si un proceso falla, los demás necesitan detectar que la operación no completará normalmente. Un timeout es una señal de fallo, no una prueba automática de qué nodo lo causó.

Las pruebas distribuidas incluyen tamaños pequeños, participantes con datos vacíos según el contrato, orden de colectivas y reanudación. Se registran todos los ranks y no solo el proceso que imprime la pérdida.

Lumbre no incluye un runtime distribuido. Este capítulo define su diseño y criterios de aceptación como ampliación avanzada. Conectar una biblioteca colectiva sería una integración declarada, no una funcionalidad que exista por aparecer en la bibliografía.

CAPÍTULO 43

Mezclas de expertos y modelos de frontera en 2026

43.1 Más especialistas, no todos trabajando a la vez

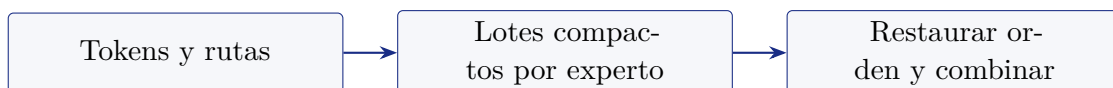
En una mezcla de expertos, un router selecciona qué subredes procesan cada token. El modelo puede contener muchos parámetros totales y activar solo una parte por token. Eso reduce el cómputo respecto a ejecutar todos los expertos, pero no elimina la necesidad de almacenar o distribuir sus pesos.

Imagina una biblioteca con varios especialistas. Cada consulta se envía a dos de ellos y después se combinan sus respuestas. Hay tres trabajos: elegir destinos, transportar consultas y ejecutar especialistas. Optimizar solo uno puede dejar los otros dominando la latencia.

43.2 Un ejemplo de dispatch completo

Tenemos cuatro tokens t_0, t_1, t_2, t_3 y dos expertos A,B. El router decide: t_0 va a A; t_1 a B; t_2 a A; t_3 a B. Podemos formar dos lotes compactos, $[t_0, t_2]$ y $[t_1, t_3]$, ejecutar cada experto y devolver resultados a las posiciones originales.

Si cada token usa dos expertos con pesos distintos, necesita dos contribuciones y una combinación ponderada. El manifiesto de dispatch debe conservar identificador del token, experto y peso. Perder el orden original produce una salida del shape correcto con contenido asignado a ejemplos equivocados.



43.3 Expertos vacíos y tamaños irregulares

Un experto puede no recibir tokens. Otro puede recibir casi todos. Un kernel de grouped GEMM debe tratar esos tamaños sin leer datos inexistentes ni gastar un gran bloque en trabajo vacío.

Las decisiones de capacidad y balance afectan al modelo, no solo al rendimiento. Descartar tokens que exceden una capacidad cambia el cálculo y debe formar parte de la receta de entrenamiento. No es una optimización transparente.

El routing top-k es discreto. El entrenamiento de un router necesita una definición de qué dependencias se derivan y qué funciones auxiliares se usan. No se obtiene una derivada de la decisión de selección simplemente porque el resto del grafo sea diferenciable.

43.4 Comunicación y cómputo son bibliotecas distintas por una razón

DeepEP se centra en mover y combinar trabajo de expertos; DeepGEMM en el cómputo matricial. Sus variantes Ascend muestran que el cambio de hardware afecta tanto al transporte y sincronización como a las multiplicaciones. [25, 23, 26, 24]

Una evaluación de MoE debe incluir el coste de empaquetado, dispatch, expertos y combine. Un GEMM muy rápido en un experto aislado no demuestra un paso completo rápido cuando los tokens se distribuyen mal o atraviesan una red.

43.5 Un mapa de diferencias respecto al decoder del curso

El decoder de Lumbre es denso en su feed-forward y utiliza atención causal estándar con GQA. No implementa routing de expertos, atención comprimida especializada, optimizador Muon ni una receta de entrenamiento de frontera.

El informe público de DeepSeek-V4 describe variantes con contexto de un millón de tokens y arquitecturas MoE; comunica, entre otros elementos, atención comprimida, mHC y uso de Muon. La ficha de Pro distingue 1.6 billones de parámetros totales y 49 mil millones activos; son cifras del autor, no resultados reproducidos aquí. [50]

Ese contraste evita llamar «réplica de DeepSeek» a un pequeño decoder que comparte solo algunas piezas. La utilidad del libro es que permite identificar qué habría que añadir al lenguaje de operaciones, al scheduler, al runtime y a la evaluación para acercarse a esas arquitecturas.

43.6 Qué exigiría una reproducción de calidad competitiva

Primero se fija un modelo y una versión de referencia, no una etiqueta genérica SOTA. Después se reproduce su tokenización, arquitectura, inicialización, datos y receta de entrenamiento. Se verifican operaciones y gradientes con pesos comparables. Se implementan los mecanismos de memoria y distribución necesarios. Finalmente se evalúa con protocolos y tareas definidos.

Una mejora de compilador puede evaluarse manteniendo la calidad constante y reduciendo coste. Una mejora de modelo necesita evaluar calidad y coste conjuntamente. Mezclar ambas sin control permite atribuir al compilador una ganancia que vino de cambiar el objetivo o usar más datos.

La escala de frontera requiere recursos y evidencia que no aporta el experimento local de este libro. El temario incluye sus técnicas y un itinerario de implementación; no se presenta un entrenamiento no realizado como si existiera.

CAPÍTULO 44

Generación eficiente: la caché KV es parte del sistema

44.1 Recalcular todo es correcto, pero caro

El ejemplo de muestreo vuelve a ejecutar una ventana completa cada vez que añade un token. Es sencillo de comprobar porque utiliza el mismo avance que ya conocemos. Pero repite proyecciones de tokens anteriores.

En generación causal, las claves y valores de posiciones pasadas pueden conservarse y reutilizarse. Para una nueva posición calculamos su consulta y sus nuevos K,V, añadimos estos al historial y atendemos sobre lo disponible. Esa estructura se llama caché KV.

La caché no contiene respuestas finales ni sustituye los pesos. Guarda activaciones de un historial concreto, por capa y cabeza KV. No puede reutilizarse entre secuencias diferentes sin una política de prefijos compartidos que demuestre compatibilidad.

44.2 Una cuenta de memoria

Para B secuencias, L capas, T posiciones almacenadas, HK cabezas KV y dimensión d , K y V ocupan aproximadamente $2BLTH_Kd$ elementos, antes de metadatos o padding. Multiplica por bytes del formato usado.

GQA reduce HK respecto al número de consultas, lo que puede disminuir esa memoria. Pero si una implementación materializa copias de K,V para cada cabeza de consulta, puede perder parte del beneficio de almacenamiento. La arquitectura y el layout de la caché deben diseñarse juntos.

44.3 Posiciones y desplazamientos

Al procesar solo el token nuevo, su posición no vuelve a cero. RoPE debe usar la posición correspondiente dentro de la secuencia o la política de ventana definida. Reiniciar el ángulo en cada llamada cambia la atención.

También hay que gestionar el límite de contexto. Un buffer circular necesita traducir posición lógica a posición física y conservar el orden semántico de las claves. Una máscara construida para memoria contigua puede ser incorrecta después de envolver el buffer.

Un test compara generación incremental con un avance completo de la misma secuencia y pesos. Se comprueban logits de cada posición, no solo el token muestreado: dos distribuciones distintas pueden producir el mismo token por casualidad.

44.4 Páginas para longitudes variables

Reservar el máximo contexto para todas las solicitudes desperdicia memoria cuando muchas son cortas. Una organización por bloques o páginas asigna capacidad conforme crece cada secuencia y mantiene una tabla que traduce posiciones lógicas a bloques físicos.

PagedAttention es una referencia de esta conexión entre gestión de memoria y servicio de LLM. Su contribución no equivale a sustituir la atención por una función de sistema operativo; reorganiza cómo se almacena y utiliza la caché. [64]

Un kernel que lee una caché paginada debe seguir esos índices correctamente. La indirección añade trabajo y puede cambiar la localidad, pero permite utilizar mejor la capacidad total y organizar más solicitudes simultáneas.

44.5 Prefill y decode necesitan métricas separadas

Prefill procesa el contexto inicial. Decode genera tokens nuevos de forma incremental. Una aplicación puede tener un prefill largo y muchos pasos de decode, o solicitudes cortas con pocos tokens de salida.

La latencia hasta el primer token, el tiempo por token posterior y el throughput agregado miden aspectos diferentes. Un cambio que mejora throughput mediante lotes mayores puede aumentar la espera de una solicitud individual.

Un compilador orientado a servicio debe conservar esa distinción en su selección de kernels. La configuración óptima de una matriz grande del entrenamiento no representa necesariamente el producto vector-matriz de decode.

44.6 Caché y entrenamiento no son la misma ruta

La caché KV de inferencia suele guardar valores para reutilizarlos sin construir un grafo de retroceso de todo el historial. El entrenamiento necesita dependencias y gradientes diferentes. Activar una API de caché de forward no garantiza soporte de backward.

Por eso el proyecto de caché del libro se plantea como una ampliación de inferencia con pruebas de equivalencia. No se utiliza para afirmar que el entrenamiento de contexto largo ya está resuelto.

Leer tinygrad sin perderse: del tensor al programa

No abras todos los archivos a la vez

Un repositorio grande se entiende siguiendo una pregunta pequeña. La nuestra será: «¿qué decisiones faltan entre escribir una multiplicación de matrices y enviar instrucciones a un dispositivo?». Cada archivo que abramos debe contestar una parte. Leer miles de líneas sin una pregunta no equivale a comprender la arquitectura.

45.1 George Hotz y la idea de estudiar una pila completa

George Hotz, conocido como geohot, describe en su retrospectiva de diciembre de 2025 el desarrollo de tinygrad desde su primer commit de octubre de 2020. Ese texto es una fuente histórica sobre las motivaciones del proyecto; sus cifras de equipo y tamaño corresponden a aquella fecha, no constituyen un censo de octubre de 2026. El sitio de Tiny Corp presenta tinygrad como una pila de cálculo que conecta operaciones tensoriales, compilación y dispositivos. [6, 3]

La lección que adoptamos no es «cuantas menos líneas, mejor». Es **hacer visible la distancia entre la intención matemática y la ejecución**. Un sistema corto puede tener conceptos difíciles; uno largo puede estar bien dividido. Para aprender, interesa que el estudiante pueda señalar dónde se decide una forma, dónde se produce un bucle y dónde se reserva una región de memoria. Lumbre organiza esas preguntas en módulos pequeños, pero no pretende reproducir todos los mecanismos de tinygrad.

Hay tres formas útiles de leer un proyecto. La lectura de usuario pregunta cómo obtener un resultado. La lectura de compilador pregunta qué representaciones atraviesa. La lectura de sistemas pregunta cómo ese resultado llega al dispositivo y cómo se detecta que ha terminado. En este curso se usan las tres; confundirlas provoca frases como «el compilador es el controlador» o «el kernel de la GPU es Linux».

45.2 Qué versión se está leyendo

El snapshot de tinygrad fijado para este libro es el commit `c3aec477b99d9bb87c54d91897cf60acd3f17441`, de 30 de septiembre de 2026. Se ha inspeccionado su pipeline de generación, además de documentación y páginas del proyecto. No se afirma haber ejecutado su batería completa ni auditado cada archivo del repositorio. Una ruta observada en una importación demuestra una dependencia del pipeline; no sustituye la lectura de la implementación importada. [1, 4]

Un procedimiento de lectura reproducible conserva el identificador del commit, el archivo y la pregunta respondida. Por ejemplo: «en este snapshot, ¿dónde se representa el tipo de eje?». La respuesta se conecta con `AxisType` y con los usos observados en `codegen/__init__.py`. Si una edición posterior reorganiza las carpetas, la pregunta sigue siendo válida aunque la ruta cambie.

Una ficha de lectura mejor que una captura

Pregunta: ¿cómo se convierte una reducción en estado de acumulación?

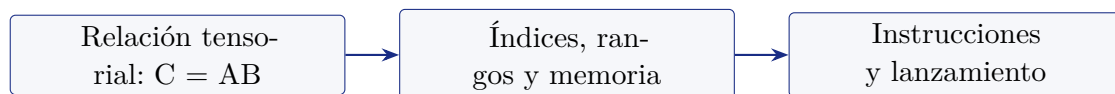
Evidencia inspeccionada: en el pipeline fijado aparece `reduce_ranges_to_acc`, que crea almacenamiento de registro, inicializa con el elemento identidad y construye actualizaciones y finalización de rangos.

Interpretación didáctica: una suma declarativa necesita un valor inicial, un orden de contribuciones y un punto donde el resultado esté disponible.

No demostrado por esa lectura: qué ensamblador exacto se obtiene para cualquier GPU, ni cuánto tarda.

45.3 Cuatro traducciones de una misma intención

Volvamos a $C = AB$. En la entrada, es una relación entre tensores. En una representación por índices, es una suma sobre k . En una organización por tiles, es un reparto de submatrices. En una instrucción matricial, ciertos fragmentos deben estar distribuidos entre hilos y registros como exige la máquina.



No todas las etapas tienen que usar clases de objetos diferentes. Una IR puede conservar un tipo de nodo general y cambiar los operadores que admite en cada fase. Lo importante es el **contrato de fase**: después de bajar una operación no debe reaparecer una forma que el siguiente paso no entiende. Por ejemplo, un renderer de C elemental no puede recibir una reducción abstracta sin una regla que la traduzca a bucles.

En el archivo fijado de `tinygrad` se observan familias de patrones para movimientos, operaciones, tipos, dimensiones GPU, coalescencia, linealización y asignación de registros. También se ven transformaciones de ejes `UNROLL` y `UPCAST`, así como tratamiento de `WMMA`. Esto muestra que «generar código» no es una sola interpolación de cadenas. [4]

Lumbre conserva parte de esa separación conceptual, pero su renderer es mucho más directo. La elección evita presentar al principiante un gran sistema de patrones antes de que comprenda un índice. Cuando Lumbre usa un bucle C explícito, el compilador de C todavía decide numerosas instrucciones. Por tanto, «nuestro compilador» no significa «hemos escrito también GCC, el ensamblador y el enlazador».

45.4 Cómo seguir una reducción agrupada

Supón que una fila tiene 1.024 valores y participan 256 hilos. Una organización posible entrega cuatro valores a cada hilo. Cada hilo obtiene una suma parcial; después se combinan esas

256 contribuciones. La primera reducción vive en el trabajo privado de cada hilo. La segunda necesita comunicación y una disciplina de sincronización.

En una representación abstracta, ambos pasos podrían seguir siendo reducciones. Al bajar a memoria, algunas contribuciones se materializan en almacenamiento local al grupo. Al bajar a control, se introducen condiciones y barreras. Es peligroso intentar deducir una barrera mirando solo el nombre SUM: hay que conocer **quién produce, quién consume y qué memoria comparten**.

Para leer el pipeline, anota junto a cada transformación qué información añade. «Añade un índice» es diferente de «añade un buffer». «Añade una dependencia de orden» es diferente de «reordena un cálculo puro». Esa anotación permite detectar una optimización sospechosa: si desaparece una dependencia sin una justificación de independencia, el resultado puede ser una carrera aunque la fórmula aritmética siga pareciendo correcta.

45.5 Qué copiar como idea y qué no copiar a ciegas

Es razonable aprender del uso de nodos inmutables, verificadores por fase, reglas locales y trazas de compilación. No es razonable copiar una transformación específica de una arquitectura sin conservar sus precondiciones. Una instrucción matricial puede necesitar alineamiento, tipos concretos y participación colectiva. Una regla que los omita no es una versión más sencilla: es una regla incorrecta fuera de un dominio accidentalmente favorable.

Tampoco se deben medir dos sistemas con contratos distintos. Si uno permite aproximaciones y el otro conserva un orden de suma, una diferencia de velocidad no se explica únicamente por la habilidad del compilador. La comparación debe declarar ambos contratos y, cuando proceda, ofrecer dos tablas: equivalencia estricta y modo de rendimiento con tolerancias explícitas.

Antes de cerrar el repositorio

Debes poder dibujar el recorrido de una operación, señalar dos lugares donde cambia su representación y explicar qué se verifica después. No necesitas memorizar los nombres de todas las funciones. Sí necesitas distinguir el dato del programa que lo calcula.

Tiny Corp más allá del compilador: sistema, firmware y colas

46.1 Un mapa sin mezclar responsabilidades

La organización pública de Tiny Corp contiene proyectos que no son bibliotecas de redes neuronales. `tinyos` se describe como un **constructor de imágenes de sistema para tinybox**, basado en `ubuntu-image`; no es, por ese nombre, un kernel de sistema operativo escrito desde cero. Su README también documenta etapas de actualización del sistema. [68]

`custom_mec_public` se presenta como una reimplementación del firmware MEC de GPUs AMD gfx1100, con firmware, emulador y pruebas. El README explica su posición entre paquetes de comandos y lanzamiento de trabajo, y atribuye a sus autores pruebas en hardware de agosto de 2026. Esas pruebas externas no se han repetido durante la elaboración del libro. [69]

Esta diversidad enseña una idea importante: acelerar una multiplicación no basta si el trabajo se envía mal, la imagen de sistema es irreproducible o la cola queda bloqueada. Pero no se arregla un fallo de GEMM reinstalando firmware al azar. Cada capa debe producir una evidencia que permita localizar el problema antes de modificar otra.

Capa	Objeto principal	Pregunta de diagnóstico
Modelo	Tensores, pérdida, parámetros	¿La operación pedida es la correcta?
Compilador	IR, bucles, layouts	¿El programa preserva esa operación?
Runtime	Buffers, módulos, lanzamientos	¿Reciben las funciones los argumentos correctos?
Controlador	Memoria y envío al dispositivo	¿Se autoriza y completa el trabajo?
Firmware	Protocolos del procesador de comandos	¿Se interpretan correctamente los paquetes?
Imagen de sistema	Servicios y versiones desplegadas	¿Se reproduce el entorno previsto?

46.2 Una cola explicada con pedidos de cocina

Imagina una cinta con pedidos numerados. El cocinero no recibe una llamada nueva por cada ingrediente: lee un pedido completo, usa su configuración y avisa al terminar. Una cola de comandos cumple una función parecida. El host escribe descriptores; un mecanismo

de notificación anuncia que hay trabajo; el dispositivo lo consume y publica una señal de finalización.

El modelo siguiente es **una simulación didáctica**, no una especificación de PM4, CUDA ni HSA. Sirve para razonar sobre orden y propiedad de los datos.

```
from dataclasses import dataclass
from collections import deque

@dataclass(frozen=True)
class Job:
    sequence: int
    operation: str
    inputs: tuple[str, ...]
    output: str

queue = deque()
completed = 0
queue.append(Job(1, "add", ("A", "B"), "C"))
queue.append(Job(2, "square", ("C",), "D"))

while queue:
    job = queue.popleft()
    # Ejecutar aquí la operación y publicar su salida.
    completed = job.sequence
```

La variable `completed` no debe cambiar antes de que la salida sea visible. En el modelo secuencial resulta obvio. En una máquina con ejecución asíncrona, cachés y varios motores, «he emitido el comando» y «sus escrituras son observables» dejan de coincidir. Ahí aparecen protocolos de espera, visibilidad y finalización.

46.3 Tres errores que parecen uno

Error de cálculo: C contiene números equivocados aun después de una finalización válida. Investiga índices, tipos y fórmula.

Error de vida de memoria: el buffer de A se reutiliza antes de que el dispositivo lo lea. El kernel puede ser matemáticamente correcto y recibir datos distintos a los esperados.

Error de observación: el host lee C antes de que el trabajo termine. Puede ver ceros, valores anteriores o una mezcla. Añadir un `sleep` cambia la probabilidad, pero no establece una dependencia correcta.

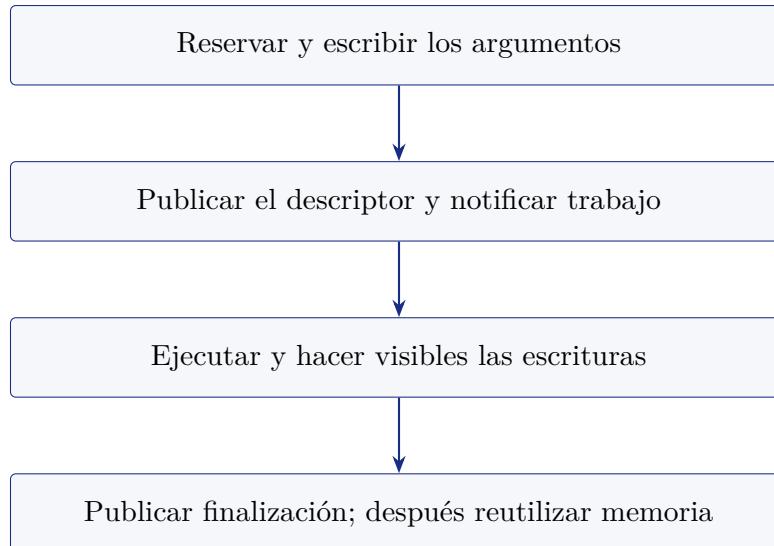
Una depuración cuidadosa conserva el mismo programa y modifica una sola hipótesis. Por ejemplo, esperar explícitamente la finalización puede separar un fallo de observación de uno aritmético. Esa prueba no convierte la espera global en la mejor solución de producción: más adelante se sustituye por dependencias precisas para conservar concurrencia.

46.4 Estado, comandos y eventos no son intercambiables

Un comando de configuración puede modificar cómo se interpretan los lanzamientos siguientes. Un comando de lanzamiento inicia trabajo. Un evento o señal permite observar un punto

del flujo. Si un optimizador reordena los tres como si fueran sumas puras, puede cambiar el significado completo de la secuencia.

En una IR de efectos conviene representar el estado que una operación consume y el estado que produce. No tiene por qué materializarse como un número físico; puede ser una dependencia de compilación. Así, dos operaciones aritméticas independientes se reordenan, pero una publicación de finalización queda detrás de las escrituras que certifica.



46.5 Un proyecto de firmware puede estudiarse sin instalarlo

El primer trabajo del estudiante es leer la documentación, identificar entradas y salidas del emulador y reproducir un modelo de estados con datos ficticios. No necesita escribir registros físicos, reemplazar módulos del sistema ni cargar una imagen en su GPU personal. Una máquina de desarrollo con trabajo académico no es un banco de pruebas desechable.

La prueba interesante en un emulador compara efectos observables: posiciones de lectura y escritura de la cola, señales, registros modelados y vida de los trabajos. Aun si dos implementaciones producen el mismo estado final en cien ejemplos, pueden discrepar en intercalaciones no examinadas. El informe debe indicar el alcance del modelo, qué eventos se simplificaron y qué propiedades se comprobaron exhaustivamente.

Diseña una prueba de cola, con solución

Se publican los trabajos 1 y 2. El segundo lee la salida del primero. Una implementación actualiza `completed=2` al sacar el pedido de la cola, antes de ejecutarlo. ¿Qué observación puede hacer el host?

Solución: puede interpretar que ambos trabajos terminaron y reutilizar sus buffers, aunque el segundo siga pendiente. El arreglo no consiste en cambiar el número a 1 por comodidad. La señal debe estar ligada al final real de las escrituras que representa. Si hay ejecución fuera de orden, una única frontera consecutiva exige además distinguir trabajos terminados de trabajos anteriores todavía pendientes.

CAPÍTULO 47

LLVM y MLIR: comprender SSA antes de aprender sus siglas

47.1 Dar un nombre nuevo a cada resultado

En un programa corriente, `x` puede cambiar varias veces. Para analizarlo, resulta útil dar un nombre distinto a cada definición. Esta idea se denomina **asignación única estática**, SSA. «Estática» se refiere a los nombres en la descripción del programa; no significa que un bucle solo se ejecute una vez. LLVM describe instrucciones y valores siguiendo esta organización. MLIR ofrece infraestructura para representar y transformar programas en distintos niveles. [36, 35]

Programa de partida:	Nombres separados:
<code>x = entrada</code>	<code>x0 = entrada</code>
<code>x = x + 1</code>	<code>x1 = x0 + 1</code>
<code>y = x * x</code>	<code>y0 = x1 * x1</code>

Ahora se ve sin ambigüedad que las dos entradas del producto proceden de la suma. En un DAG puro de Lumbre, la referencia al nodo cumple una función similar: no se pregunta «¿qué valor tiene actualmente la variable?», sino «¿qué resultado produjo este nodo?». La dificultad adicional aparece cuando hay bifurcaciones, bucles y memoria mutable.

47.2 Una bifurcación necesita reunir resultados

Considera una función que devuelve el valor absoluto. Una rama conserva el número; la otra cambia su signo. Al reunirse las ramas, hay que indicar qué resultado procede del camino tomado.

```
entrada(x):  
    si x >= 0: ir a positivo  
    si no:    ir a negativo  
positivo:  
    p = x  
    ir a union  
negativo:  
    n = -x  
    ir a union  
union:  
    resultado = elegir_segun_predecesor(p, n)
```

La selección no calcula arbitrariamente ambos valores y escoge uno por su contenido. Asocia cada entrada con una arista de control. LLVM puede representarla mediante `phi`; en otras

IR se usan argumentos de bloque. No se deben traducir todos esos mecanismos como una llamada ordinaria a una función de dos argumentos: su semántica depende del predecesor. [36, 34]

Dominancia significa que para llegar a un punto hay que haber pasado por otro. Si una definición domina un uso, ese uso no puede alcanzarse sin que la definición esté disponible. El nodo de suma del ejemplo lineal domina el producto. En la bifurcación, la definición *p* no domina por sí sola todos los caminos de la unión; de ahí la necesidad de la selección asociada al control.

47.3 Un bucle es una recurrencia, no un círculo arbitrario

Para sumar cuatro valores, la versión matemática dice «acumular contribuciones». Una IR con control necesita el valor inicial, la condición de continuación y el valor que vuelve por la arista del bucle.

```
inicio:
  ir a cabecera(i=0, acumulado=0)
cabecera(i, acumulado):
  si i == N: ir a salida(acumulado)
  valor = A[i]
  siguiente = acumulado + valor
  ir a cabecera(i+1, siguiente)
salida(resultado):
  devolver resultado
```

La pareja de argumentos de la cabecera cambia en cada iteración, aunque sus nombres estáticos sean los mismos. El ciclo tiene una estructura controlada y una condición. No equivale a insertar un nodo que se referencia a sí mismo en un DAG de expresiones puras, donde la evaluación topológica dejaría de estar definida.

Invariante completo del bucle

Antes de cada visita a la cabecera, *acumulado* es la suma de los elementos con índices menores que *i*, e *i* está entre 0 y *N*. Al principio se han sumado cero elementos. Una iteración añade exactamente *A[i]* y aumenta *i* en uno. Al terminar, *i=N*, por lo que se han incorporado todos los elementos y ninguno dos veces.

Para coma flotante, este invariante debe interpretarse como la secuencia de sumas redondeadas del programa. No autoriza automáticamente a sustituirla por cualquier árbol de reducción.

47.4 Dialectos: conservar información mientras sea útil

Una representación de alto nivel puede saber que una operación es una convolución. Después puede conocer bucles e índices afines. Más tarde, instrucciones vectoriales y accesos a memoria. Si se baja demasiado pronto a instrucciones escalares, reconocer otra vez la convolución resulta difícil. Si se conserva demasiado tiempo una abstracción que oculta la arquitectura, no se pueden tomar decisiones de registros o sincronización.

El diseño de MLIR permite dialectos y conversiones entre representaciones. El tutorial Toy desarrolla un lenguaje pequeño para enseñar ese recorrido. El objetivo de la lectura no es copiar todo MLIR en Lumbre, sino aprender que una transformación debería declarar qué información exige y qué información elimina. [34, 35]

Nuestro contrato de conversión imaginario será: «antes hay operaciones tensoriales con formas conocidas; después hay bucles y buffers; ninguna operación tensorial no legalizada puede quedar». El verificador revisa ese contrato. Si aparece una operación desconocida, debe producir un diagnóstico, no transformarla en un comentario de C ni ignorarla.

47.5 Alias: dos nombres pueden señalar la misma caja

SSA da nombres distintos a resultados, pero no elimina por magia el alias de memoria. Dos punteros pueden señalar la misma región. En ese caso, escribir mediante uno cambia lo que se lee mediante el otro. Reordenar un `load` alrededor de un `store` necesita demostrar que no interfieren, o representar la dependencia.

Por ejemplo, una función recibe `A` y `B`; calcula `t=A[0]`, escribe `B[0]=7` y vuelve a leer `A[0]`. Si los argumentos son alias, las dos lecturas pueden dar valores distintos. Una optimización que reutilice `t` como si la memoria fuera inmutable sería incorrecta. Este es uno de los motivos por los que el runtime educativo de Lumbre mantiene reglas simples de propiedad y actualización de parámetros.

Análisis y transformación deben distinguirse. El análisis obtiene una propiedad, como «estas dos regiones son disjuntas». La transformación utiliza esa propiedad, por ejemplo para adelantar una carga. Si la propiedad no se ha demostrado, la transformación no puede fingir que existe. En una primera implementación, ser conservador es preferible a aceptar resultados silenciosamente corruptos.

47.6 Diseñar diagnósticos para alguien que empieza

«IR inválida» es un mensaje pobre. «La suma esperaba operandos compatibles, pero recibió formas (2,3) y (4,3); el eje exterior 2 no coincide con 4» permite corregir el programa. Lo mismo ocurre con un error de dominancia: señala el uso, la definición y el camino por el que se alcanza el primero sin pasar por la segunda.

En un compilador educativo, cada fase puede devolver además una representación textual pequeña. El estudiante compara antes y después y busca la primera divergencia, igual que en una ejecución de un autómata. Una traza de compilación no es ruido de depuración: es la explicación concreta de cómo una promesa de alto nivel se convirtió en un programa ejecutable.

Triton, CuTe, TileLang y bibliotecas: elegir una abstracción

48.1 La pregunta no es quién tiene la sintaxis más corta

Tres programas pueden calcular la misma matriz y repartir el trabajo de maneras distintas. Uno expresa el resultado tensorial completo. Otro describe un tile y deja al compilador distribuirlo. Otro determina fragmentos y layouts cercanos al hardware. Comparar solo el número de líneas oculta qué decisiones ha tomado el programador y cuáles ha delegado.

Triton documenta programas de multiplicación matricial organizados por bloques; CUTLASS y CuTe ofrecen herramientas de composición y layouts; TileLang desarrolla un modelo de programación por tiles; ThunderKittens se centra en primitivas para construir kernels. Estas descripciones no implican que todos produzcan el mismo código ni soporten el mismo hardware. [32, 11, 31, 45]

48.2 Un contrato común para comparar soluciones

Definamos la tarea antes del lenguaje: A tiene forma (M, K) , B forma (K, N) , ambas contiguas por filas; acumulamos en FP32 y devolvemos C de forma (M, N) . Permitimos tamaños no múltiplos de tile, por lo que hay que proteger cargas y escrituras. La primera versión no admite alias entre entradas y salida.

Con ese contrato, una implementación debe contestar las mismas preguntas: cómo elige su tile de salida, cómo recorre K , qué sucede en los bordes, dónde conserva acumuladores y cuándo puede escribir. El lenguaje cambia la forma de expresar esas decisiones, no hace que desaparezcan.

El siguiente esquema **no es una API ejecutable de ningún proyecto**. Es una ficha de correspondencia para leer implementaciones reales sin confundir sus nombres.

```
programa(tile_fila, tile_columna):
    filas = tile_fila * BM + [0, ..., BM-1]
    columnas = tile_columna * BN + [0, ..., BN-1]
    acumulador = ceros(BM, BN)
    para inicio_k en 0, BK, 2*BK, ...:
        A_tile = cargar_con_mascara(A, filas, inicio_k)
        B_tile = cargar_con_mascara(B, inicio_k, columnas)
        acumulador += producto(A_tile, B_tile)
    guardar_con_mascara(C, filas, columnas, acumulador)
```

48.3 Triton: un programa procesa un bloque de datos

Al leer el tutorial oficial de GEMM de Triton, identifica primero los identificadores de programa y después los vectores de índices. Una expresión vectorial de índices no significa que haya un hilo de Python ejecutando cada elemento. Describe trabajo que el compilador bajará al dispositivo. Los parámetros de bloque afectan reutilización, ocupación y cantidad de programas. [32]

El error inicial más frecuente es interpretar el código como NumPy ejecutándose en la GPU. La sintaxis familiar no elimina el carácter compilado ni los requisitos de los punteros y máscaras. Otro error es copiar un tamaño de bloque óptimo para una forma grande y utilizarlo en matrices pequeñas: el lanzamiento puede crear demasiado trabajo inútil o demasiado pocos programas para ocupar la máquina.

Para estudiar un kernel, cambia solo una dimensión. Mantén M y N , aumenta K y observa qué bucle se alarga. Después deja K fijo y aumenta M : debería crecer el número de tiles de salida. Si ambas modificaciones cambian de la misma manera todos los recursos, revisa si has entendido qué dimensión se mapea al grid y cuál al bucle interno.

48.4 CuTe y CUTLASS: el layout también es un programa

CUTLASS no es solo una colección de funciones GEMM finales. Su organización permite componer componentes, y CuTe hace explícitas relaciones entre coordenadas y almacenamiento. La versión del README consultada incluye CuTe DSL en Python y secciones específicas de arquitecturas recientes. Eso no convierte las capacidades de una GPU de centro de datos en capacidades de cualquier tarjeta que comparta el nombre de familia. [11]

Para comprender un layout, empieza con una tabla de cuatro elementos. El layout lineal $[0, 1, 2, 3]$ y el layout $[0, 2, 1, 3]$ contienen los mismos valores, pero asignan coordenadas a posiciones distintas. Al componer un layout lógico con un reparto entre hilos, una coordenada termina en un par «hilo, registro». Una instrucción colectiva exige que ese reparto coincida con su contrato. No basta con que cada hilo posea el número correcto de elementos.

La pregunta útil es: «para la coordenada (i, j) , ¿quién la posee y en qué posición local?». Cuando puedes responderla para una matriz diminuta, un tipo de layout deja de parecer una fórmula ornamental. En hardware moderno, ciertos layouts además expresan descriptores, memoria compartida y operandos de instrucciones matriciales; esos detalles deben consultarse en la documentación de la arquitectura concreta. [10, 11]

48.5 TileLang: separar la intención del tile y su planificación

El trabajo de TileLang propone un modelo composable por tiles. Su interés didáctico está en poder razonar sobre regiones de datos y organización del cómputo sin escribir primero todas las instrucciones escalares. Las APIs evolucionan: la documentación consultada distingue interfaces dinámicas actuales y nombres usados por ejemplos antiguos. Un ejemplo encontrado

en una publicación debe ejecutarse con las versiones que declara, no mezclarse con la última instalación suponiendo compatibilidad. [31, 30]

Un port correcto conserva la semántica de bordes y reducción. Si una versión GPU usaba relleno cero para el borde de K , una traducción que lea memoria no inicializada puede fallar únicamente en tamaños impares. Si el lenguaje ofrece primitivas de pipeline, el estudiante todavía debe comprender las vidas de los buffers: reutilizar una etapa antes de que termine su consumidor no se vuelve seguro por estar escrito con una abstracción de alto nivel.

48.6 Bibliotecas de kernels frente a compiladores completos

DeepGEMM aporta implementaciones especializadas de multiplicación, incluidas variantes útiles para modelos con expertos. DeepEP se ocupa de comunicación para expertos. DeepJIT aborda piezas de compilación, carga y lanzamiento. No son nombres diferentes de una misma biblioteca: pertenecen a problemas complementarios. Los ports Ascend deben leerse con sus restricciones, versiones y estados de implementación. [23, 25, 27, 24, 26]

Un compilador puede decidir llamar a una biblioteca en lugar de generar toda la operación. Eso no es hacer trampa si la llamada se declara. Debe controlar el contrato de tipos, layouts, workspace, stream y propiedad de memoria. Para evaluar el compilador, conviene informar por separado qué operaciones se generan y cuáles se delegan. Una mejora atribuida al renderer podría proceder, en realidad, de una nueva versión de la biblioteca llamada.

En AMD, los avisos de migración de Composable Kernel y hipBLASLt remiten a ROCm/rocm-libraries; RCCL remite a ROCm/rocm-systems. El repositorio histórico sigue siendo útil para entender referencias antiguas, pero no se debe construir una guía de contribución actual ignorando esos avisos. [15, 14, 67]

48.7 Decisión de proyecto: tres rutas legítimas

Compilador desde cero: implementa el subconjunto semántico y genera C/CUDA/HIP propio. Aprendes cada frontera, pero el rendimiento inicial será modesto.

Frontend propio con backend existente: construye una IR didáctica y bájala a MLIR, Triton o TileLang. Aprendes integración y contratos de conversión; debes reconocer qué optimizaciones realiza el backend externo.

Especialización de un kernel: conserva el modelo y sustituye una operación por una implementación propia. El alcance es menor, pero permite una evaluación rigurosa y una contribución útil. Un kernel bien estudiado suele ser un proyecto más defendible que una promesa de reemplazar toda la pila sin pruebas.

Leer artículos de 2026 como investigador, no como espectador

49.1 Una afirmación necesita su dominio

Cuando un artículo dice que una solución es más rápida, pregunta: ¿para qué operaciones, formas, tipos, arquitecturas y presupuesto de ajuste? Una mejora sobre un conjunto no se transforma en una garantía para cualquier programa. Tampoco se puede comparar un candidato afinado durante días con una referencia tomada sin sus opciones habituales y llamarlo una comparación neutral.

El artículo **AI as a Compiler**, presentado el 29 de septiembre de 2026, estudia traducción mediante agentes de kernels Triton a PTX. Sus autores reportan resultados que incluyen tanto casos inferiores a la referencia como aceleraciones, y apoyan el proceso con un entorno de evaluación y extensiones de un verificador de PTX para capacidades recientes. Es evidencia sobre un método y un conjunto de experimentos, no una demostración de que cualquier LLM sustituya correctamente cualquier compilador. [51]

Nuestra interpretación para el curso es separar **proponer** de **aceptar**. Un generador puede ser creativo y producir candidatos poco obvios. La aceptación exige un contrato, pruebas, restricciones de recursos y, donde sea posible, verificación de propiedades. Cambiar el generador no elimina la obligación de explicar qué significa correcto.

49.2 Tres preguntas complementarias de la literatura reciente

TileSight aborda modelado analítico del rendimiento centrado en tiles. **Correct but Slow** estudia la distancia entre corrección y rendimiento en kernels de lenguajes específicos de dominio. **Characterizing Real-World Bugs in Tile Programs** examina fallos de programas por tiles para orientar su detección. Estas lecturas se incluyen como líneas de investigación de 2026; no se convierten aquí en una clasificación exhaustiva ni en benchmarks repetidos por el autor del libro. [52, 53, 54]

Podemos conectar esas tres preguntas con Lumbre sin copiar sus experimentos. Primera: ¿predice un modelo de bytes y operaciones qué versión conviene? Segunda: ¿cuántos candidatos pasan la comparación numérica, pero siguen siendo lentos? Tercera: ¿qué clases de errores detectan nuestras pruebas y cuáles podrían quedar ocultas? Son preguntas distintas. Tener una respuesta satisfactoria para la primera no resuelve las otras dos.

49.3 Diseñar una tabla que no oculte fracasos

Supón que se proponen diez kernels. Seis no compilan, uno produce resultados incorrectos, uno excede memoria y dos son correctos. Si se publica únicamente el más rápido de esos dos, falta información sobre el coste y fiabilidad del procedimiento. Una tabla útil conserva el estado de cada intento.

Estado	Qué se ha observado	Qué no se puede concluir
No compila	El toolchain rechaza el candidato	Que la idea matemática sea imposible
Compila, falla prueba	Un caso contradice el contrato	Que otros casos compensen el error
Correcto en la batería	Pasa las comprobaciones realizadas	Corrección universal sin más argumento
Tiempo excedido	No termina dentro del presupuesto	Su tiempo exacto ni su corrección
Más rápido	Mejora la métrica del ensayo	Mejora de todo el modelo o toda GPU

Esta tabla permite calcular tasas de éxito y costes de búsqueda, además de velocidad final. El presupuesto debe incluir compilaciones, pruebas fallidas y ajuste. Si el objetivo es amortizar una búsqueda durante millones de ejecuciones, el análisis puede favorecer un método caro; si la forma cambia cada segundo, quizá no. La conclusión depende de la carga de uso.

49.4 Separar prueba formal, comprobación diferencial y muestreo

Una **prueba formal** deriva una propiedad bajo un modelo y unas hipótesis. Si el modelo no incluye una característica del hardware, la garantía no puede extenderse silenciosamente a esa característica. Una **prueba diferencial** compara dos implementaciones para entradas concretas. Un **muestreo aleatorio** explora entradas, pero puede pasar por alto fronteras raras.

Las tres herramientas se complementan. Para una transposición, podemos demostrar la biyección de índices y además comparar resultados. Para una reducción, podemos justificar que cada elemento contribuye una vez, comprobar tolerancias y medir la sensibilidad al orden. Para un pipeline GPU, necesitamos razonar sobre estados y sincronización; comparar únicamente una salida en una ejecución puede no revelar una carrera intermitente.

Un resultado negativo que sí enseña

En las mediciones locales de este libro, la atención online evita materializar una matriz de puntuaciones, pero resulta más lenta que la alternativa densa bloqueada en dos tamaños examinados. No invalida FlashAttention. Muestra que una versión escalar por fila, una CPU y tamaños pequeños tienen un equilibrio diferente al de un kernel GPU optimizado por tiles.

La conclusión defendible es sobre esas implementaciones y condiciones. La siguiente hipótesis será reducir overhead, vectorizar o cambiar el reparto; no «el artículo es falso» ni «la reducción de memoria garantiza velocidad».

49.5 Elaborar una réplica de alcance honesto

Una réplica puede ser semántica, arquitectónica o de rendimiento. La réplica semántica implementa la misma operación o recurrencia y comprueba sus resultados. La arquitectónica reproduce organización relevante del cómputo, como tiles y pipeline. La de rendimiento intenta repetir condiciones y métricas publicadas. No deben usarse las tres etiquetas como sinónimos.

Lumbre implementa una recurrencia online de atención y prueba su equivalencia numérica con la versión densa en un dominio finito. Su backward recompone operaciones densas. Por eso no es una réplica integral del entrenamiento FlashAttention con memoria intermedia lineal. La etiqueta precisa permite saber qué está aprendido y cuál es el siguiente trabajo real.

Para tu informe, termina cada lectura con cuatro frases: qué problema define; qué mecanismo propone; qué evidencia presenta; qué experimento propio podría refutar tu interpretación. La última frase obliga a transformar admiración en una hipótesis comprobable.

CAPÍTULO 50

Laboratorio 1. Tu primera cadena completa: de cuatro números a C

50.1 Objetivo y preparación

Este laboratorio corresponde al comienzo de las semanas 1–2. La tarea es calcular $y = (x + 2)x$ para cuatro números. No vamos a empezar entrenando nada: queremos observar toda la cadena y saber qué parte se ejecuta en Python y qué parte en C. Necesitas el paquete de Lumbre, Python con NumPy y un compilador de C accesible como `cc`. En Windows, el runtime entregado se utiliza desde un entorno Linux, por ejemplo WSL; no es un cargador de DLL nativas de Windows.

Abre una terminal en la carpeta del código. Ejecuta la batería básica antes de cambiar archivos. Si falla por ausencia de compilador, todavía no es un fallo del grafo. Resuelve primero la dependencia y conserva el mensaje original para distinguir instalación de corrección.

```
python -m pytest -q
```

La primera compilación puede tardar más que las siguientes porque el runtime conserva bibliotecas por clave de caché. No compares el tiempo de esa primera ejecución con el de una función ya cargada como si fueran la misma fase del trabajo.

50.2 Predicción antes del programa

Toma $x = (-2, -1, 0, 3)$. Para cada elemento hay dos operaciones: sumar dos y multiplicar por el original. Los resultados deben ser $(0, -1, 0, 15)$. El valor original no se sobrescribe al sumar: el producto recibe tanto la suma como la entrada.

Posición	Entrada	Entrada más dos	Producto esperado
0	-2	0	0
1	-1	1	-1
2	0	2	0
3	3	5	15

Esta tabla ya detecta dos fallos frecuentes. Si el programa devuelve cuadrados de la suma, utilizó dos veces el resultado intermedio. Si devuelve $x + 2x$, cambió la agrupación. Un ejemplo con una sola entrada igual a cero no distinguiría bien esas equivocaciones.

50.3 Construcción y ejecución

Guarda el siguiente programa como `laboratorio_01.py` en la raíz del paquete y ejecútalo con Python. El nombre es un archivo que tú creas; no se presupone que exista en la entrega.

```
import numpy as np
from lumbre import param, Program

x = param("x", (4,))
y = (x + 2.0) * x
entrada = np.array([-2, -1, 0, 3], dtype=np.float32)

with Program([y]) as programa:
    salida = programa.run({"x": entrada})[0]
    print(salida)
    print(programa.report())
    print(programa.source)
    np.testing.assert_array_equal(
        salida, np.array([0, -1, 0, 15], dtype=np.float32)
    )
```

`param` crea una descripción de entrada, no un vector con números. `y` es otro nodo del grafo. `Program` decide materializaciones, genera código, lo compila y prepara el almacenamiento. `run` copia la entrada y llama a la función compilada. El arreglo devuelto es una copia de la salida declarada, de modo que no queda dependiendo de una región temporal que se vaya a reutilizar.

Busca en el C generado el acceso al elemento de `x`, la constante y la escritura de la salida. Los nombres automáticos pueden cambiar; no forman parte del resultado matemático. La propiedad importante es que cada posición válida produce exactamente un valor y que ninguna posición necesita el resultado de otra.

50.4 Comparar dos planificaciones sin cambiar la fórmula

Construye también `Program([y], fuse=False)`. La versión sin fusión materializa más pasos. Debe conservar los resultados para esta entrada. No afirmes que una variante es más rápida porque tiene menos líneas: mide después y separa compilación de ejecución.

```
with Program([y], fuse=True) as unido:
    a = unido.run({"x": entrada})[0]
    informe_a = unido.report()
with Program([y], fuse=False) as separado:
    b = separado.run({"x": entrada})[0]
    informe_b = separado.report()
np.testing.assert_array_equal(a, b)
print(informe_a["kernels"], informe_b["kernels"])
```

En CPU, «kernel» significa aquí una unidad generada del plan, no un lanzamiento GPU real. Esa precisión evita interpretar automáticamente el número de kernels como latencia de GPU. Las unidades pueden convertirse en funciones C llamadas secuencialmente dentro de una

ejecución.

50.5 Ejercicio de reescritura con límites explícitos

Sobre enteros matemáticos, $x+0 = x$. En coma flotante, algunas identidades necesitan cuidado con ceros con signo, NaN, infinitos y redondeo. Por eso el primer simplificador algebraico del laboratorio trabaja con expresiones enteras de índices, no activa indiscriminadamente reglas de números reales sobre todos los tensores.

Propón ahora la transformación de $(x+2)x$ a x^2+2x . Es algebraicamente correcta sobre reales exactos. ¿Es una regla que deba activarse siempre para FP32? La respuesta es no: cambia el orden y número de redondeos, puede alterar resultados extremos y podría aumentar operaciones. Para usarla en un modo aproximado habría que declarar la política numérica y justificar su utilidad, no solo escribir una igualdad simbólica.

Error intencional y reparación

Cambia la expresión a $(x + 2.0) * (x + 2.0)$ y ejecuta la prueba. Debe fallar. Localiza el primer valor que contradice la tabla y dibuja las dos aristas del producto.

Solución: con entrada -2, ambas fórmulas dan cero; ese caso no identifica el error. Con entrada -1, la fórmula pedida da -1 y la defectuosa da 1. El producto debe recibir el nodo de entrada en una de sus aristas, no dos copias de la suma. Reparar el resultado final con un signo añadido solo para esa posición sería un parche al ejemplo, no una corrección del programa.

50.6 Entrega y condición de avance

Conserva el programa, una copia del C generado y una explicación de diez líneas del recorrido. Añade una entrada que no aparezca en la tabla y calcúlala a mano. No hace falta adjuntar una biblioteca compilada dependiente de tu máquina.

Puedes pasar al laboratorio siguiente cuando distingas el momento de construir el grafo del momento de ejecutar los datos, y cuando sepas explicar por qué una operación elemento a elemento puede fusionarse sin comunicar posiciones. Un resultado correcto sin esa explicación todavía no muestra que controles el compilador.

CAPÍTULO 51

Laboratorio 2. Índices simbólicos, intervalos y máscaras

51.1 El problema que vamos a resolver

Tenemos 35 valores y queremos repartirlos en bloques de ocho posiciones. Habrá cinco bloques. Los primeros cuatro cubren 32 valores y el último contiene tres válidos y cinco sobrantes. La tarea es generar índices, demostrar qué posiciones son válidas y evitar leer los sobrantes.

El índice global se calcula como $i = 8b + t$, donde b identifica el bloque y t una posición local entre 0 y 7. No estamos hablando todavía de hilos físicos: primero probaremos una relación aritmética. Después podremos mapearla a un bucle, un vector o un hilo GPU.

51.2 Calcular una tabla completa del borde

Para el bloque cuatro, los índices son 32,33,34,35,36,37,38,39. La condición $i < 35$ acepta los tres primeros. La condición incorrecta $i \leq 35$ aceptaría también el 35, que queda fuera de un arreglo de 35 elementos, cuyos índices terminan en 34.

```
N, B = 35, 8
vistos = []
for bloque in range((N + B - 1) // B):
    for local in range(B):
        i = bloque * B + local
        if i < N:
            vistos.append(i)
assert vistos == list(range(N))
```

Esta comprobación exige orden y cobertura exacta, no únicamente cantidad. Un algoritmo que duplicase el índice 0 y perdiese el 34 también produciría 35 resultados; comparar solo la longitud no lo detectaría.

51.3 Construir la expresión en el módulo simbólico

```
from lumbre.symbolic import var, simplify, interval, render_c

b = var("b")
t = var("t")
indice = simplify((b * 8 + t) + 0)
print(indice.evaluate({"b": 4, "t": 2}))
print(interval(indice, {"b": (0, 4), "t": (0, 7)}))
print(render_c(indice))
```

La primera salida numérica es 34. El intervalo es $(0, 39)$. Eso nos dice que el índice generado **puede** exceder 34; no que todas las combinaciones lo hagan. El simplificador elimina una suma por cero dentro de su dominio entero. El renderer produce una expresión C, pero su contrato exige las condiciones indicadas para división y resto.

El módulo simbólico es una herramienta de laboratorio separada. El renderer tensorial de Lumbre construye sus expresiones de índices directamente. No se atribuye al compilador completo un sistema general de pruebas simbólicas por el hecho de incluir este pequeño módulo.

51.4 Probar una identidad y sus hipótesis

Para un entero no negativo i y un divisor positivo B , se cumple $i = (i // B)B + (i \bmod B)$. El cociente identifica el bloque y el resto la posición local. Con $i = 34$, obtenemos bloque 4 y resto 2: $4 \cdot 8 + 2 = 34$.

```
for i in range(40):
    bloque, local = i // 8, i % 8
    assert bloque * 8 + local == i
    assert 0 <= local < 8
```

La prueba por enumeración cubre esos 40 enteros. El argumento general es la división euclídea: el cociente y el resto se definen de forma que el resto está en el rango permitido. Al traducir a C, restringimos índices no negativos para evitar confundir su división truncada con la división suelo de Python cuando aparezcan valores negativos.

51.5 Una máscara debe proteger el acceso, no solo el resultado

Compara las dos formas siguientes, escritas como pseudocódigo:

```
Forma incorrecta:
    valor = A[i]
    resultado = valido ? valor : 0

Forma correcta para este contrato:
    resultado = valido ? A[i] : 0
```

En la primera ya se ha producido una lectura fuera de límites antes de descartar el valor. La segunda representa una lectura condicionada: su traducción debe preservar que la dirección inválida no se desreferencia. Una optimización no puede mover la carga fuera de la condición si carece de una garantía adicional de acceso seguro.

En una reducción por suma, cero es un relleno apropiado porque no cambia el acumulado. En una reducción por máximo, rellenar con cero puede ser erróneo si todos los valores válidos son negativos. La máscara depende de la operación, no únicamente del tamaño del arreglo.

Un borde que cambia el máximo

Los valores válidos del último bloque son -5,-2,-9. Su máximo es -2. Si las cinco posiciones sobrantes se rellenan con cero y participan, el resultado pasa a ser cero. El error no se arregla cambiando la tolerancia numérica: es una diferencia semántica. Debe usarse una identidad apropiada o excluir los elementos inválidos de la reducción.

51.6 Intervalos conservadores y precisión del análisis

Si sabemos $b \in [0, 4]$ y $t \in [0, 7]$, el intervalo de $8b + t$ es $[0, 39]$. Si añadimos la relación $b == 4$, el intervalo se estrecha a $[32, 39]$. Si añadimos además $t < 3$, queda $[32, 34]$. Cuanta más información relacional conservamos, más accesos pueden demostrarse seguros.

Pero el módulo de intervalos básico no representa todas las relaciones. Si a y b son realmente la misma variable, tratarlas como dos intervalos independientes puede producir una cota muy amplia. Una cota amplia no es un fallo de corrección si contiene todos los valores; es una pérdida de precisión que puede impedir optimizaciones. Lo peligroso es una cota demasiado estrecha que excluya valores posibles.

51.7 Entrega resuelta y variación

La entrega incluye una función que enumera índices para cualquier $N \geq 0$ y bloque positivo, junto con casos $N=0, 1, 7, 8, 9, 35$. Con $N=0$ no debe ejecutarse ningún acceso. Con $N=8$ hay un bloque completo; con $N=9$ aparece un borde de un solo elemento. Añade validación que rechace bloque cero antes de dividir.

Como variación, usa un tile bidimensional de 4×8 para una matriz 5×10 . La máscara correcta exige a la vez fila menor que 5 y columna menor que 10. Una única condición sobre el índice lineal puede aceptar elementos que cruzan indebidamente la frontera de una fila si la construcción de coordenadas no coincide con el layout declarado. Escribe primero las dos coordenadas y solo después la dirección.

CAPÍTULO 52

Laboratorio 3. Vistas sin copia y reducciones sin confusión

52.1 Del dibujo al almacenamiento

Tenemos una matriz de dos filas y tres columnas almacenada como `[0,1,2,3,4,5]`. Su primera fila es `[0,1,2]` y la segunda `[3,4,5]`. Queremos observar la transpuesta sin mover esos seis números. La vista debe traducir coordenadas, no modificar el contenido.

```
from lumbre.layout import View

original = View.contiguous((2, 3))
transpuesta = original.permute((1, 0))
print(original.shape, original.strides)
print(transpuesta.shape, transpuesta.strides)
assert transpuesta.address((2, 1)) == 5
```

La vista original tiene strides `(3,1)`; la transpuesta `(1,3)`. Para la coordenada `(2,1)` de la transpuesta, la dirección es $2 \cdot 1 + 1 \cdot 3 = 5$. Corresponde al elemento original `(1,2)`. La forma ha pasado a `(3,2)`, pero la memoria sigue siendo la misma lista.

52.2 Por qué aplanar no siempre es una vista contigua

El recorrido lógico por filas de la transpuesta sería `[0,3,1,4,2,5]`. Esa secuencia no está contigua en la memoria original. No basta con cambiar la forma a `(6,)` conservando stride uno: produciría `[0,1,2,3,4,5]`, que representa otro orden lógico.

El método `contiguous_reshape` de la vista del laboratorio rechaza esa conversión cuando necesita una materialización o una indexación más general. Ese rechazo es una propiedad de corrección, no una carencia que deba ocultarse devolviendo cualquier arreglo de seis elementos.

```
try:
    transpuesta.contiguous_reshape((6,))
except ValueError as error:
    print("Rechazo esperado:", error)
else:
    raise AssertionError("La vista no era contigua")
```

El tensor IR de Lumbre puede expresar composiciones lógicas mediante transformaciones de índices; eso no significa que su runtime acepte cualquier buffer externo con strides arbitrarios. Las entradas se copian a almacenamiento contiguo. Separar ambos contratos evita atribuir zero-copy a una copia de entrada que realmente existe.

52.3 Broadcasting como stride cero

Una fila de tres valores puede usarse como sesgo de muchas filas. En el modelo de vista, expandir un eje de tamaño uno permite usar stride cero: cambiar esa coordenada lógica no avanza en memoria. Así varias posiciones leen el mismo valor.

```
fila = View.contiguous((1, 3))
repetida = fila.expand((4, 3))
assert repetida.strides == (0, 1)
assert repetida.address((0, 2)) == 2
assert repetida.address((3, 2)) == 2
```

Leer es sencillo; escribir ya no lo es. Dos posiciones lógicas pueden señalar el mismo elemento físico. Si se permite una operación in-place sobre la vista expandida, distintas escrituras podrían entrar en conflicto. El laboratorio utiliza estas vistas para leer y razonar, no promete un sistema general de actualizaciones in-place sobre alias.

52.4 Reducir una dimensión y conservar las otras

Considera la matriz $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$. Sumar por el último eje produce $[6, 15]$: una contribución por fila. Sumar por el primer eje produce $[5, 7, 9]$: una contribución por columna. Sumar todo produce el escalar 21. El nombre «reducción» no especifica por sí solo qué dimensión desaparece.

```
import numpy as np
from lumbre import param, Program

x = param("datos", (2, 3))
filas = x.sum(axis=1)
columnas = x.sum(axis=0)
total = x.sum()
with Program([filas, columnas, total]) as p:
    a, b, c = p.run({"datos": [[1, 2, 3], [4, 5, 6]]})
np.testing.assert_array_equal(a, [6, 15])
np.testing.assert_array_equal(b, [5, 7, 9])
assert float(c) == 21.0
```

Escribe el C de referencia para cada caso antes de consultar el generado. Para sumar filas, el bucle exterior escoge la fila y el interior recorre columnas. Para sumar columnas, ocurre lo contrario. En ambos casos el acumulador se inicializa una vez por salida; si se inicializa dentro del bucle de reducción, solo queda la última contribución.

52.5 Composición que obliga a seguir índices

Transpone la matriz, selecciona las dos últimas filas de la transpuesta y suma cada fila. La transpuesta es $\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$; el recorte es $\begin{bmatrix} 2 & 5 \\ 3 & 6 \end{bmatrix}$; la salida debe ser $[7, 9]$.

```
y = x.permute(1, 0).slice(0, 1, 3).sum(axis=1)
with Program([y]) as p:
    salida = p.run({"datos": [[1, 2, 3], [4, 5, 6]]})[0]
```

```
np.testing.assert_array_equal(salida, [7, 9])
```

El propósito no es una operación compleja, sino comprobar que cada transformación compone el mapa anterior. Un fallo frecuente aplica el recorte sobre la forma original porque se conservó un eje antiguo. La forma de cada nodo debe describir su salida, no la del primer tensor de la cadena.

Gradiente del sesgo, adelantando la semana 9

Una matriz de cuatro filas suma el mismo sesgo de tres elementos. La pérdida es la suma de todos los resultados. ¿Qué gradiente recibe cada elemento del sesgo?

Solución: cada uno se utilizó cuatro veces y cada uso contribuye uno; el gradiente es $[4, 4, 4]$. El broadcasting que en forward repite lecturas se convierte en una suma de contribuciones en backward. No se devuelve una matriz de cuatro filas como gradiente de un parámetro cuya forma era $(3,)$.

52.6 Condición de éxito

Entrega los mapas de dirección de la vista original y transpuesta, la explicación del reshape rechazado y tres reducciones comprobadas. Añade una matriz no cuadrada: las cuadradas ocultan algunos intercambios de ejes porque la forma no cambia. Tu argumento debe explicar el resultado para una coordenada arbitraria válida, no solo para las cuatro direcciones que has probado.

CAPÍTULO 53

Laboratorio 4. Planificar kernels y reciclar memoria sin romper datos

53.1 Un grafo con una dependencia compartida

Construye $h = \tanh(x)$ y dos salidas: $a = h + 1$, $b = h \cdot h$. El nodo h se usa en varias aristas. Un plan puede calcularlo una vez y conservarlo, o recomputarlo dentro de cada consumidor. La decisión afecta trabajo y memoria; no puede cambiar el resultado matemático pedido.

```
import numpy as np
from lumbre import param, Program

x = param("x", (101,))
h = x.tanh()
a, b = h + 1.0, h * h
entrada = np.linspace(-2, 2, 101, dtype=np.float32)
with Program([a, b]) as p:
    ya, yb = p.run({"x": entrada})
    print(p.report())
np.testing.assert_allclose(ya, np.tanh(entrada) + 1, atol=1e-6)
np.testing.assert_allclose(yb, np.tanh(entrada)**2, atol=1e-6)
```

Que h tenga varios usos no obliga a materializarlo en todo compilador posible. Es una política razonable para evitar recomputaciones; una política más sofisticada compararía tamaño, coste y presión de memoria. Lumbre usa reglas simples y observables para que el estudiante pueda explicar su decisión.

53.2 Calcular vidas a mano

Usa este plan secuencial imaginario: K0 produce h ; K1 lee h y produce a ; K2 lee h y produce b ; al final se devuelven a y b . h debe vivir hasta K2. a debe conservarse hasta que el usuario lo lea, aunque ningún kernel posterior lo utilice. Las salidas declaradas tienen una vida diferente de un temporal interno.

Valor	Nace	Último uso necesario	¿Puede reciclarse antes de K2?
x	Antes de K0	K0	Solo bajo una política explícita de entradas
h	K0	K2	No
a	K1	Lectura final	No
b	K2	Lectura final	No

Un planificador que solo cuente consumidores dentro del grafo podría considerar a muerto justo después de producirlo y sobrescribirlo. Para evitarlo, las salidas deben actuar como raíces retenidas. La interfaz del programa, no solo las aristas internas, determina la vida de memoria.

53.3 Un ejemplo donde sí se puede reutilizar

En una cadena $u_1 = f(x)$, $u_2 = g(u_1)$, $u_3 = h(u_2)$, el almacenamiento de u_1 puede reutilizarse después de que K1 haya terminado de leerlo. No puede reutilizarse al inicio del mismo kernel si la escritura se solapa con lecturas futuras y no se ha demostrado seguridad in-place.

Una regla conservadora para kernels secuenciales exige que el último uso del buffer sea **estrictamente anterior** al kernel que lo recibe. La igualdad no basta. Esa desigualdad pequeña evita que una función sobrescriba datos que todavía está recorriendo. En ejecución asíncrona, además hay que traducir el último uso lógico a una garantía de finalización del dispositivo.

53.4 Comparación de las cuatro combinaciones

Construye una cadena de doce pasos $\tanh(0.9*u+0.1)$. Ejecuta versiones con fusión y reutilización activadas o desactivadas. Hay dos variables independientes; no cambies ambas y atribuyas toda diferencia a una sola.

```
u = x
for _ in range(12):
    u = (u * 0.9 + 0.1).tanh()
referencia = None
for fusion in (False, True):
    for reciclaje in (False, True):
        with Program([u], fuse=fusion, reuse=reciclaje) as p:
            resultado = p.run({"x": entrada})[0]
            print(fusion, reciclaje, p.report())
        if referencia is None:
            referencia = resultado
        else:
            np.testing.assert_allclose(
                resultado, referencia, rtol=1e-5, atol=1e-6
            )
```

La arena informada incluye categorías propias de la implementación, como entradas y temporales alineados. No es automáticamente el pico total del proceso Python ni la memoria reservada por un driver. Para comparar dos informes, utiliza la misma definición y explica qué queda fuera.

53.5 Una escritura de parámetros es una transacción

En entrenamiento, varias actualizaciones deben calcularse a partir del mismo estado antiguo. Si se actualiza una matriz mientras todavía se calcula el gradiente de otra que depende de

ella, el paso deja de corresponder al optimizador definido. Por eso Lumbre construye salidas de actualización y las aplica después de ejecutar el grafo completo.

No confundas esa organización con una base de datos transaccional general. Aquí la garantía es local y acotada: las entradas del paso permanecen separadas de sus salidas y se confirma la actualización cuando el cálculo ha terminado. Recuperarse de una interrupción del proceso exige además checkpoints completos y una escritura persistente segura.

Encuentra el fallo sin mirar números

Un planificador asigna el mismo offset a dos buffers de 256 bytes porque tienen el mismo tamaño. El primero se usa en K5; el segundo nace en K4. ¿Qué información falta?

Solución: el tamaño solo prueba que caben en la misma región, no que puedan compartirla en el tiempo. Sus vidas se solapan. El segundo no puede ocuparla hasta que termine el último uso del primero. También habría que revisar alineamiento, tipo y restricciones de propiedad; ninguno sustituye la condición temporal.

53.6 Entrega

Entrega un dibujo del DAG, una tabla de vidas y los cuatro informes. Explica qué cambio reduce kernels y cuál reduce bytes. Si un cambio empeora el tiempo, conserva la medida y busca una causa: mayor código, presión de registros, compilación o ruido. Una optimización tiene una intención, pero su nombre no es una prueba de que mejore todas las métricas.

Laboratorio 5. GEMM: mejorar sin esconder el coste

54.1 Una pregunta experimental concreta

Queremos saber si reutilizar pequeños bloques mejora la multiplicación de matrices de Lumbre en nuestra CPU. La hipótesis no es «hemos inventado la multiplicación más rápida». Es: **para las formas elegidas, el microkernel bloqueado reduce el tiempo medido frente al recorrido de referencia, con el mismo contrato de entrada y una comprobación numérica explícita.**

El paquete incluye `examples/bench_kernels.py`. Su informe conserva muestras, no únicamente el mejor número. Empieza por leer qué se cronometra. El camino medido utiliza datos residentes; no incluye necesariamente construir el grafo, copiar entradas, compilar o guardar resultados. Esas fases deben medirse aparte cuando se evalúe la experiencia completa de una aplicación.

```
python examples/bench_kernels.py
```

El script escribe su informe según la ruta establecida en el propio archivo. Conserva una copia con fecha y descripción de la máquina antes de repetir pruebas que puedan sobrescribirlo.

54.2 Cuenta de operaciones y bytes

Para matrices $M \times K$ y $K \times N$, la convención habitual cuenta aproximadamente $2MKN$ operaciones de coma flotante. Una suma y un producto se cuentan como dos, incluso si una instrucción FMA los combina. La cifra es una convención de rendimiento; no significa que el hardware ejecute exactamente ese número de instrucciones.

En una multiplicación 128^3 , son 4.194.304 operaciones. Los datos de entrada y salida ocupan $4(128^2 + 128^2 + 128^2) = 196.608$ bytes en FP32, sin contar tráfico repetido, cachés ni almacenamiento auxiliar. Dividir las operaciones entre ese mínimo de bytes da una intensidad idealizada. No demuestra el tráfico real de una implementación que relee elementos muchas veces.

El microkernel del paquete conserva un bloque de acumuladores de 4×8 y recorre K en tramos de 64. Es una decisión didáctica, no un parámetro universal. El compilador C puede vectorizar o no según el código y la plataforma. No se han añadido instrucciones intrínsecas específicas ni un sistema de packing comparable al de una BLAS madura.

54.3 Comprobar bordes antes de medir cuadrados bonitos

Una matriz 31×65 multiplicada por otra 65×47 obliga a tratar bordes en filas, columnas y reducción. Los tamaños potencia de dos son útiles, pero pueden ocultar un `min` ausente o un acumulador mal inicializado. La batería del paquete incluye formas no alineadas y comparación de estrategias.

```
import numpy as np
from lumbre import param, Program

rng = np.random.default_rng(5)
a = param("A", (31, 65))
b = param("B", (65, 47))
c = a @ b
A = rng.normal(size=a.shape).astype(np.float32)
B = rng.normal(size=b.shape).astype(np.float32)

resultados = []
for estrategia in ("reference", "blocked"):
    with Program([c], gemm=estrategia) as p:
        resultados.append(p.run({"A": A, "B": B})[0])
        print(p.report())
for resultado in resultados:
    np.testing.assert_allclose(resultado, A @ B,
                               rtol=3e-5, atol=1e-5)
```

NumPy sirve aquí como referencia de comprobación. No se está cronometrando NumPy como competidor, ni atribuyendo su backend a Lumbre. Si se quisiera comparar con una BLAS, habría que registrar también su implementación, número de hilos y configuración.

54.4 Entender una medición local real

Las medianas observadas en esta edición para referencia y bloqueado fueron aproximadamente 51,4 y 11,4 microsegundos en la forma irregular; 1.566 y 157 microsegundos en 128^3 ; 15.247 y 1.371 microsegundos en 256^3 . Son resultados de la CPU y el entorno del informe entregado. No son una predicción de tu portátil ni una comparación contra PyTorch.

El cociente referencia/bloqueado responde cuánto mejora esa sustitución dentro del ensayo. No se puede trasladar sin más al entrenamiento completo. Si GEMM ocupase solo la mitad del tiempo, incluso una aceleración enorme dejaría intacta la otra mitad. La ley de Amdahl del capítulo de medidas permite calcular esa limitación antes de hacer promesas.

54.5 Experimento de una sola variable

Cambia un tamaño de tile en una copia del microkernel, no en cinco lugares a la vez. Para cada candidato ejecuta primero corrección; solo los correctos pasan al cronómetro. Conserva la configuración original como baseline. Si un candidato falla, guarda su fuente y el caso mínimo que lo contradice.

Una exploración responsable puede usar tres valores para filas, tres para columnas y tres para tramo de reducción: 27 candidatos. Si cada compilación tarda un segundo y cada validación medio segundo, el coste ya no es despreciable. El autotuning debe contar esos intentos. Una mejora de un microsegundo que requiere un minuto de búsqueda necesita muchas ejecuciones para amortizarse.

Explica un resultado contrario a tu intuición

Aumentar el tile de salida reduce el número de iteraciones exteriores, pero empeora el tiempo. Propón dos mecanismos diferentes.

Solución: puede aumentar la presión de registros y provocar spills; también puede dificultar la vectorización o aumentar trabajo inútil en bordes. Otra posibilidad es que la medida incluya más compilación. Ninguna se acepta solo por parecer plausible: inspecciona ensamblador, recursos o fases de tiempo para distinguirlas.

54.6 Condición de entrega

Incluye una tabla con forma, estrategia, error máximo, mediana y número de muestras. Añade condiciones de compilación, hilos y datos residentes. La conclusión debe nombrar la referencia exacta. «Más rápido que nuestro bucle de referencia» es una afirmación útil y demostrable; «SOTA CPU» requiere una comparación mucho más amplia que este laboratorio no ha realizado.

Laboratorio 6. Convolución desde índices hasta gradientes

55.1 Una imagen diminuta y un filtro que puedes calcular

Trabajaremos con un solo canal, una imagen 3×3 y un filtro 2×2 , sin padding y con stride uno. La imagen contiene los números del 1 al 9 por filas. El filtro es $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$. Cada salida resta la esquina inferior derecha de la superior izquierda de una ventana.

Las cuatro ventanas producen -4. La forma de salida es 2×2 , porque el filtro puede comenzar en dos filas y dos columnas. No se invierten los coeficientes del filtro: la operación de las redes se formula aquí como correlación cruzada, siguiendo el contrato declarado en el capítulo de convoluciones.

```
import numpy as np
from lumbre import param, Program, gradients
from lumbre.nnops import conv2d

x = param("imagen", (1, 1, 3, 3))
w = param("filtro", (1, 1, 2, 2))
y = conv2d(x, w)
perdida = y.sum()
gx, gw = gradients(perdida, [x, w])
X = np.arange(1, 10, dtype=np.float32).reshape(1, 1, 3, 3)
W = np.array([1, 0, 0, -1], dtype=np.float32).reshape(1, 1, 2, 2)
with Program([y, gx, gw], gemm="blocked") as p:
    Y, dX, dW = p.run({"imagen": X, "filtro": W})
np.testing.assert_array_equal(Y, -4 * np.ones((1, 1, 2, 2)))
print(dX)
print(dW)
```

La implementación compuesta `nnops.conv2d` construye parches con operaciones ya presentes en la IR y usa multiplicaciones de matrices. Por tanto, los gradientes proceden de las reglas de esas operaciones, no de una función de convolución backward tomada de otro framework.

55.2 Resolver el gradiente del filtro a mano

Como la pérdida suma todas las salidas, cada salida aporta uno. El gradiente de un coeficiente del filtro es la suma de los píxeles que multiplicó. Para la esquina superior izquierda: $1 + 2 + 4 + 5 = 12$. Para la superior derecha: $2 + 3 + 5 + 6 = 16$. Para la inferior izquierda: $4 + 5 + 7 + 8 = 24$. Para la inferior derecha: $5 + 6 + 8 + 9 = 28$.

El gradiente esperado es $[[12, 16], [24, 28]]$. No depende aquí del valor actual del filtro, porque la salida es lineal en sus coeficientes y la pérdida es una suma. Cambiar la pérdida a cuadrados introduciría otra dependencia y cambiaría el resultado.

El gradiente de la imagen reúne filtros solapados. El píxel central participa como esquina inferior derecha de una ventana y superior izquierda de otra, con contribuciones -1 y $+1$ que se cancelan. La matriz resultante es $[[1, 1, 0], [1, 0, -1], [0, -1, -1]]$. Esta cancelación es una buena prueba: un scatter que sobrescriba en lugar de sumar puede devolver un valor distinto de cero en el centro.

55.3 Dos implementaciones con propósitos distintos

El archivo `kernels/conv.c` contiene una convolución directa con grupos, padding simétrico, stride y dilatación. Su prueba compila C y compara con un recorrido de referencia. `nnops.conv2d` expresa la operación mediante la IR para obtener autodiferenciación por composición. No hay que confundirlas: la primera es un kernel C independiente; la segunda conecta con el compilador y su grafo de gradientes.

La versión compuesta tiene una cota explícita de 1.024 posiciones espaciales de salida para impedir la construcción accidental de grafos enormes. Su estructura es apropiada para estudiar corrección y derivación, no para afirmar rendimiento de una implementación im2col optimizada. Una mejora futura consiste en representar la extracción de parches con un operador de índices compacto o generar directamente un kernel de convolución.

55.4 Padding y dilatación en coordenadas

Para una salida (o_y, o_x) y una posición del filtro (k_y, k_x) , la entrada correspondiente es $(o_y s_y - p_y + k_y d_y, o_x s_x - p_x + k_x d_x)$. Cada término tiene una función: stride desplaza el comienzo de ventana; padding cambia el origen lógico; dilatación separa muestras dentro del filtro.

No memorices la fórmula sin probarla. Con stride dos, padding uno y dilatación uno, la primera ventana comienza en coordenada -1 . Algunas posiciones quedan fuera y aportan cero. La segunda ventana comienza dos lugares después. Con dilatación dos, un filtro de tres elementos cubre un intervalo de cinco posiciones: no toma las cinco, sino tres separadas.

Grupos y forma de los pesos

Una entrada tiene cuatro canales y una salida seis. Se usan dos grupos. ¿Cuántos canales de entrada ve cada canal de salida? ¿Qué forma de pesos corresponde a filtros 3×3 ?

Solución: cada grupo recibe dos canales de entrada y produce tres de salida. Los pesos tienen forma $(6, 2, 3, 3)$, no $(6, 4, 3, 3)$. Los tres primeros canales de salida se conectan con el primer grupo de entrada y los otros tres con el segundo. La divisibilidad de canales de entrada y salida por grupos se valida antes de compilar.

55.5 Prueba numérica de un gradiente

Escoge un coeficiente, súmale y réstale una pequeña cantidad y calcula la diferencia central de pérdidas. Compara con el gradiente simbólico. Evita perturbaciones tan pequeñas que FP32 las redondee al mismo valor, y no confundas una coincidencia numérica en un coeficiente con la validación de todo el operador.

Las pruebas del paquete usan varias configuraciones de grupos, stride, padding y dilatación, además de perturbaciones de entradas, pesos y sesgo. El resultado local es evidencia de esas pruebas. El argumento general de composición explica por qué el mismo mecanismo produce contribuciones de todos los caminos, siempre que cada primitiva y su regla de gradiente sean correctas.

55.6 Entrega

Incluye las cuatro salidas, los dos gradientes calculados a mano y la comparación ejecutada. Después añade un caso con padding y otro con grupos. Señala qué implementación utilizaste y qué cota impide escalarla directamente. El laboratorio se completa comprendiendo el solapamiento; no basta con obtener la misma forma que una biblioteca externa.

Laboratorio 7. Llevar el mismo grafo a CUDA o HIP

Estado de validación de esta edición

Este laboratorio requiere una GPU y su toolchain. La revisión del 1 de octubre valida CUDA en una RTX 5070 Ti Laptop con WSL2, toolkit 12.9 y destino `sm_120`; HIP sigue pendiente. Los informes están en `code/reports/validation_20261001/`. Las salidas siguientes definen los requisitos de aceptación que debes volver a comprobar en tu dispositivo.

56.1 Preparar un inventario, no una instalación a ciegas

Antes de ejecutar, registra sistema operativo, dispositivo exacto, controlador, compilador y arquitectura destino. En CUDA, consulta la compute capability del modelo concreto. La tabla oficial distingue, por ejemplo, RTX 50 y las distintas GPUs Blackwell de centro de datos; compartir un nombre de generación no garantiza el mismo conjunto de instrucciones. En HIP, consulta la compatibilidad del dispositivo y del entorno en la documentación vigente. [55, 19]

En Windows, la guía CUDA para WSL explica un entorno específico; no debe mezclarse sin más con instrucciones de instalación de un controlador Linux físico. El runtime de Lumbre usa una biblioteca compartida POSIX. Comprueba primero un ejemplo del toolchain, después el programa elemento a elemento y solo entonces una matriz o un modelo. [7]

56.2 Primera prueba: 257 elementos

Usa un tamaño que no sea múltiplo del bloque de lanzamiento. El siguiente programa selecciona el backend por una variable de entorno y conserva exactamente el grafo de la CPU.

```
import os
import numpy as np
from lumbre import param, Program

backend = os.environ.get("LUMBRE_BACKEND", "cpu")
x = param("x", (257,))
y = x * 3.0 + 1.0
X = np.arange(257, dtype=np.float32)
with Program([y], backend=backend) as p:
    Y = p.run({"x": X})[0]
    np.testing.assert_array_equal(Y, X * 3 + 1)
    print(p.report())
```

Para una GPU NVIDIA con destino real `sm_120`, el entorno podría configurarse así; ese valor es un ejemplo condicionado al dispositivo y a soporte del toolkit, no un valor universal.

```
export LUMBRE_BACKEND=cuda
export LUMBRE_CUDA_ARCH=sm_120
python laboratorio_gpu.py
```

Para HIP, selecciona `LUMBRE_BACKEND=hip` y un entorno donde `hipcc` conozca la arquitectura apropiada. El ejemplo entregado depende de la selección del toolchain; un port de producción debe hacer explícita esa arquitectura también en el manifiesto y en la clave de caché.

56.3 Leer el kernel generado

Para contrastar la prueba aislada con la batería completa, ejecuta `python -m pytest -q -backend=cuda` desde `code/`. El capítulo «CPU y CUDA en la misma máquina» desarrolla el protocolo de comparación, muestra las medidas y conserva la distinción entre las pruebas tensoriales de GPU y las pruebas que siguen ejecutándose en el host.

Busca el índice global construido a partir de bloque e hilo. Después localiza la condición que protege `q<257`. Si el grid contiene más posiciones, no es necesariamente un error: el redondeo del número de bloques necesita la máscara. El error sería ejecutar cargas o escrituras de los hilos sobrantes sin protección.

Localiza también las llamadas del host. Una función `__global__` describe trabajo del dispositivo; la sintaxis de lanzamiento y el runtime son otra parte. El programa debe comprobar errores del lanzamiento y esperar antes de leer la salida. El runtime educativo sincroniza para simplificar la observación, lo que es correcto pero limita concurrencia.

56.4 Segunda prueba: GEMM con memoria compartida

La ruta GPU de Lumbre incluye un GEMM por tiles de 16×16 . Cada etapa carga regiones compartidas, sincroniza, acumula y vuelve a sincronizar antes de reutilizarlas. La segunda barrera es tan importante como la primera: impide sobrescribir una etapa que otros hilos todavía leen.

Prueba formas (17,19) @ (19,23) y luego una multiplicación por lotes. Cambia datos y repite muchas veces. Una sola ejecución puede no revelar una carrera. Utiliza las herramientas de diagnóstico del proveedor disponibles para tu versión y conserva sus opciones y resultados; no inventes un informe de ausencia de carreras a partir de una comparación numérica aislada.

Un hilo cuya salida está fuera de la matriz puede seguir necesitando participar en cargas y barreras del bloque. Por eso una salida temprana antes de una barrera colectiva puede ser incorrecta. El diseño debe permitir que todos los participantes requeridos sigan el protocolo, aunque algunos valores se enmascaren.

56.5 Tercera prueba: WMMA separado del compilador

`kernels/wmma_demo.cu` es un ejemplo completo para estudiar una operación matricial colectiva con entradas FP16 y acumulación FP32. Compílo para una arquitectura compatible y revisa todas las comprobaciones del host. Que este ejemplo funcione no demuestra que el renderer tensorial emita automáticamente tensor cores para todo GEMM: son hitos diferentes.

La entrega del laboratorio debe separar «he ejecutado el ejemplo WMMA» de «he integrado una transformación que reconoce GEMM y produce el layout WMMA correcto». La segunda requiere inspeccionar los operands y probar bordes, transpuestas y lotes. La mera presencia de la cadena `wmma` en un archivo no demuestra uso correcto en el entrenamiento.

56.6 Depuración en orden

Si no compila, conserva el archivo generado y el error exacto. Si no se puede cargar, revisa ABI y dependencias. Si el dispositivo rechaza el lanzamiento, revisa arquitectura, dimensiones y recursos. Si los valores son erróneos, empieza con tamaños pequeños y patrones distinguibles. Si solo falla al repetir, investiga vida de memoria y sincronización.

Un resultado correcto que no valida rendimiento

El kernel devuelve lo esperado, pero cada operación copia datos host-dispositivo y sincroniza globalmente. ¿Puede declararse competitivo frente a un entrenamiento con datos residentes?

Solución: no con esa comparación. Primero hay que igualar la frontera medida. Puede publicarse el tiempo extremo a extremo del programa educativo y el tiempo del kernel residente como métricas distintas. Ocultar las copias en un lado y contarlas en el otro sesga la conclusión.

Laboratorio 8. Autodiferenciación con una cuenta que puedes verificar

57.1 Una regresión de dos parámetros

Queremos ajustar una recta $\hat{y} = wx + b$ a tres parejas: $(0, 1)$, $(1, 3)$ y $(2, 5)$. La solución exacta es $w = 2, b = 1$. Empezaremos con ambos a cero y usaremos la media del error cuadrático. El objetivo inmediato no es converger: es comprobar el primer gradiente.

Al principio, los errores son $(-1, -3, -5)$. La pérdida vale $(1 + 9 + 25)/3 = 35/3$. El gradiente de w es $\frac{2}{3} \sum_i e_i x_i = -26/3$; el de b es $\frac{2}{3} \sum_i e_i = -6$. Con tasa 0,1, un paso debe producir aproximadamente $w = 0,8666667$ y $b = 0,6$.

57.2 Construir el paso como otro grafo

```
import numpy as np
from lumbre import param, gradients, Program

x = param("x", (3,))
t = param("objetivo", (3,))
w = param("w", ())
b = param("b", ())
error = x * w + b - t
perdida = (error * error).mean()
gw, gb = gradients(perdida, [w, b])
nuevo_w, nuevo_b = w - 0.1 * gw, b - 0.1 * gb

with Program([perdida, gw, gb, nuevo_w, nuevo_b]) as p:
    valores = p.run({"x": [0, 1, 2], "objetivo": [1, 3, 5],
                    "w": np.float32(0), "b": np.float32(0)})
    np.testing.assert_allclose(valores[1], -26/3, rtol=1e-6)
    np.testing.assert_allclose(valores[2], -6, rtol=1e-6)
    p.assign({"w": nuevo_w, "b": nuevo_b})
    siguiente = p.run(read=[perdida])[0]
    print(float(valores[0]), float(siguiente))
```

El gradiente se genera como operaciones de la misma IR. El compilador no llama a `torch.autograd`. Al aplicar `assign`, los nuevos parámetros se copian después de calcular todas las salidas; el segundo `run` usa el estado actualizado y conserva los datos que ya estaban inicializados.

57.3 Por qué una contribución no puede sobrescribir otra

En `error*error`, el mismo nodo aparece en dos aristas del producto. Cada arista aporta una derivada. La suma de ambas produce el factor dos. Un algoritmo que marque un nodo como visitado y descarte automáticamente la segunda contribución obtendría la mitad del gradiente.

El orden topológico inverso sirve para esperar a que estén reunidas las contribuciones antes de propagar a los padres. No significa que cada arista tenga una única ruta hasta la pérdida. Los parámetros compartidos en un decoder producen el mismo problema a mayor escala: una tabla usada como embedding y como proyección de salida recibe contribuciones de ambos usos.

57.4 Broadcasting en el backward

`b` es escalar, pero participa en tres predicciones. La derivada respecto a su valor es la suma de las tres contribuciones, dividida por tres por la media. La forma final debe ser `()`, no `(3,)`. `w` también es escalar, aunque cada contribución lleve multiplicado un dato distinto.

Una prueba de formas es tan importante como una de valores. Un gradiente con números plausibles y forma equivocada puede difundirse por broadcasting en la actualización y producir un programa que no representa los parámetros originales. El optimizador debe validar las formas antes de comprometer el estado.

57.5 Diferencias finitas: escoger una perturbación razonable

Calcula la pérdida en $w + \delta$ y $w - \delta$, dejando lo demás fijo. La diferencia dividida entre 2δ aproxima la derivada. Prueba varios valores de δ , como 10^{-2} y 10^{-3} , y observa cuándo el error deja de mejorar. En FP32, hacerlo arbitrariamente pequeño puede aumentar el error por cancelación y redondeo.

No utilices diferencias finitas en un punto no diferenciable esperando una derivada única. Para máximo con empate, el libro declara una convención de reparto. Una perturbación que rompe el empate puede observar una pendiente unilateral diferente sin demostrar que el código incumpla esa convención.

Diagnóstico de un gradiente reducido a la mitad

El gradiente calculado de `w` es aproximadamente -4,3333 en lugar de -8,6667, mientras que la pérdida es correcta. ¿Qué parte investigarías primero?

Solución: la acumulación de contribuciones del producto `error*error`. Si se propaga solo por una arista, se pierde el factor dos. También hay que descartar otra media introducida por error, pero la relación exacta de la discrepancia y el nodo compartido proporcionan una hipótesis concreta para empezar.

57.6 Entrega y ampliación

Entrega la cuenta del primer paso, la ejecución y una curva de pérdidas registrada durante varios pasos. La curva puede no ser monótona con cualquier tasa; no la alteres para que parezca ideal. Añade una prueba donde $\mathbf{w}$ se use en dos expresiones distintas y demuestra que recibe ambas contribuciones. Después cambia a AdamW utilizando el módulo del paquete y comprueba que guardas también sus momentos, no solo los pesos.

Laboratorio 9. Un decoder completo y un checkpoint que realmente reanuda

58.1 Qué vamos a demostrar

En este laboratorio se ejecuta entrenamiento real: se evalúa un decoder, se calcula una pérdida de siguiente token, se construyen gradientes y se actualizan parámetros. El tamaño es pequeño para que sea inspeccionable y ejecutable en CPU. No vamos a llamar «modelo de frontera» a un decoder de decenas de miles de parámetros, ni a confundir una pérdida baja sobre frases repetidas con comprensión general del lenguaje.

La arquitectura del paquete reúne componentes estudiados antes: embedding, RMSNorm, posiciones RoPE, atención causal con cabezas de consulta y de clave/valor, bloque SwiGLU, conexiones residuales y proyección de salida con pesos compartidos. La implementación exacta y sus formas aparecen en el módulo `nn.py` y en los capítulos del modelo. Los artículos de esos componentes son referencias del diseño, no una certificación de calidad del modelo entrenado aquí. [58, 59, 60, 61]

58.2 Primera ejecución pequeña

Empieza por una configuración breve para verificar instalación, compilación y guardado. No aumentes a la vez dimensiones, capas y contexto: si falla, perderás la pista de qué recurso o contrato cambió.

```
python examples/train_decoder.py --backend cpu --steps 20 \  
  --dim 16 --layers 1 --context 8 --batch 2 \  
  --heads 2 --kv-heads 1 --gemm blocked \  
  --output reports/mi_decoder_inicial
```

Abre `config.json` y comprueba que los valores corresponden a la orden. Después abre `report.json`: debe haber pérdidas finitas, un informe de compilación y una muestra. Que la muestra sea mala no implica automáticamente que el compilador falle; que parezca una frase tampoco demuestra corrección. Primero se validan operadores, causalidad, gradientes y actualización.

58.3 Reproducir la configuración principal

La ejecución principal de esta edición utilizó la siguiente configuración. Los tiempos pueden variar entre máquinas y según exista una entrada válida en caché.

```
python examples/train_decoder.py --backend cpu --steps 600 \  
  --dim 512 --layers 6 --context 1024 --batch 16 \  
  --heads 8 --kv-heads 2 --gemm blocked \  
  --output reports/mi_decoder_inicial
```

```
--dim 64 --layers 2 --context 32 --batch 2 \  
--heads 2 --kv-heads 1 --hidden 128 \  
--online --gemm blocked --output reports/mi_decoder_600
```

En el registro entregado hay 75.584 parámetros y 600 actualizaciones. La primera pérdida fue aproximadamente 3,2164 y la última 0,2747. La evaluación de cuatro lotes del fragmento reservado produjo aproximadamente 0,3322. El corpus de demostración repite seis frases: por tanto, esa evaluación no es un benchmark lingüístico independiente y puede compartir patrones casi idénticos con entrenamiento.

El tiempo registrado del bucle de entrenamiento fue aproximadamente 3,44 segundos en el entorno local, excluyendo compilación, evaluación posterior y generación. La compilación registrada en ese ensayo fue un acierto de caché. No se deben sumar o comparar esas cifras como si midieran una ejecución limpia completa en cualquier equipo.

58.4 Revisar un token de supervisión

Imprime una ventana de entrada y su objetivo. El objetivo debe estar desplazado una posición. Si la entrada contiene los tokens de «el sol», la posición del primer token intenta predecir el segundo, no reconstruirse a sí misma. Un fallo de desplazamiento puede producir una pérdida excelente para una tarea equivocada.

La máscara causal impide usar el futuro de la ventana para predecir el siguiente token. Para probarla, cambia tokens posteriores a una posición y compara sus logits anteriores. No deberían cambiar, salvo la tolerancia numérica pertinente. Esta prueba comprueba causalidad del programa; no mide habilidad de lenguaje.

58.5 La prueba fuerte de reanudación

Ejecuta diez pasos continuos en una carpeta. En otra, ejecuta seis pasos y reanuda cuatro más desde su checkpoint. Usa exactamente la misma configuración y corpus. El paquete guarda pesos, momentos del optimizador, número de paso, vocabulario/configuración y estado del generador aleatorio de lotes.

```
python examples/train_decoder.py --steps 10 --gemm blocked \  
--output reports/continuo  
python examples/train_decoder.py --steps 6 --gemm blocked \  
--output reports/parte  
python examples/train_decoder.py --steps 4 --gemm blocked \  
--resume reports/parte/checkpoint.npz --output reports/reanudado
```

Compara los archivos con `np.load(..., allow_pickle=False)`. Los campos numéricos deben coincidir para esta ejecución determinista en el mismo entorno. En la validación local, la comparación de diez pasos frente a seis más cuatro fue exacta en los campos guardados. En otros dispositivos o algoritmos con reducciones no deterministas, la especificación debe declarar si espera identidad bit a bit o una equivalencia tolerada.

Guardar únicamente los pesos no basta para esa prueba. AdamW necesita momentos y correcciones dependientes del paso. El muestreo de lotes necesita su estado aleatorio. Si alguno

falta, se puede continuar entrenando, pero no se está reproduciendo la misma trayectoria.

58.6 Interpretar fallos con orden

Si la pérdida no baja, primero intenta sobreajustar un único lote pequeño. Si tampoco funciona, revisa gradientes y actualización. Si solo falla con varias ventanas, revisa datos y objetivos. Si aparece NaN, guarda el primer paso afectado y localiza la primera operación no finita. No cambies simultáneamente tasa, precisión, arquitectura y datos esperando que desaparezca el síntoma.

Un checkpoint que parece completo

El archivo contiene todos los parámetros y el número de paso, pero omite los momentos de AdamW. ¿Por qué reanudar con el mismo número de paso no arregla el problema?

Solución: las correcciones del paso y las medias móviles describen conjuntamente el estado del optimizador. Usar el paso antiguo con momentos reiniciados combina dos historias incompatibles. Puede ser una decisión intencional de reinicio del optimizador, pero debe declararse y no presentarse como una reanudación exacta.

Laboratorio 10. Atención online: misma función, otra memoria

59.1 La recurrencia que queremos probar

Una fila de atención combina valores con pesos proporcionales a exponenciales de puntuaciones. La implementación densa guarda puntuaciones o probabilidades de toda la fila. La online conserva un máximo, un denominador y una suma ponderada que se actualizan al recibir cada bloque. Su interés es evitar una matriz intermedia cuadrática; no cambia la definición de la atención. [41]

Empieza con un ejemplo donde puedas usar fracciones exactas. Puntuaciones $(0, \log 2, \log 4)$ y valores $(1, 3, 5)$ producen pesos proporcionales a $(1, 2, 4)$. El resultado es $(1 + 6 + 20)/7 = 27/7$. Cuando cambia el máximo, las contribuciones anteriores se reescalan para seguir expresadas respecto al mismo origen.

59.2 Un programa de comparación

```
import numpy as np
from lumbre import param, Program, softmax, online_attention, gradients
from lumbre.ir import node

forma = (1, 2, 7, 4)
q, k, v = [param(nombre, forma) for nombre in ("q", "k", "v")]
puntuaciones = (q @ k.transpose()) / 2.0
mascara = node("causal", (7, 7))
densa = softmax(puntuaciones + mascara) @ v
online = online_attention(q, k, v)
rng = np.random.default_rng(8)
datos = {u.arg: rng.normal(size=forma).astype(np.float32)
         for u in (q, k, v)}
with Program([densa, online], gemm="blocked") as p:
    a, b = p.run(datos)
np.testing.assert_allclose(a, b, rtol=2e-5, atol=2e-5)
```

La raíz cuadrada de la dimensión de cabeza es dos porque $D = 4$. La máscara solo permite claves hasta la posición de la consulta. Las dos variantes deben usar el mismo convenio causal. Comparar una atención causal con otra bidireccional no es una prueba de aproximación: son operaciones distintas.

59.3 Comprobar backward sin prometer una memoria que no existe

Construye la suma de la salida de cada variante y pide gradientes respecto a $\mathbf{q}, \mathbf{k}, \mathbf{v}$. Compáralos en un programa aparte. También conviene usar un gradiente de salida no uniforme, por ejemplo ponderar cada posición por una constante diferente; la suma uniforme puede ocultar ciertos errores.

En Lumbre, la regla de gradiente de la operación online recompone operaciones densas para obtener un VJP correcto dentro de su dominio. Por eso la reducción de memoria observada en forward **no se extiende automáticamente a todo el entrenamiento**. Implementar backward por tiles con recomputación de probabilidades y acumulación adecuada es una extensión real, no un detalle que ya esté resuelto por usar la palabra FlashAttention.

59.4 Separar efecto algorítmico y efecto de implementación

Mide las variantes en programas separados. Si declaras ambas como salidas de un único programa, su arena y tiempo incluyen las dos; no puedes atribuir ese total a una de ellas. Usa la misma entrada residente, tres calentamientos y varias repeticiones. Conserva error máximo, mediana y bytes de arena.

En la medición local para forma $(1, 2, 128, 32)$, la arena de la variante densa fue 393.216 bytes y la online 131.072. Sin embargo, sus medianas fueron aproximadamente 437,5 y 624,6 microsegundos: la versión online fue más lenta. La reducción de almacenamiento es clara en ese ensayo; la aceleración no.

Las causas posibles incluyen trabajo escalar por fila, exponenciales frecuentes, falta de vectorización y tamaños donde la versión densa bloqueada reutiliza eficazmente la caché. Estas son hipótesis de análisis, no causas medidas de forma aislada. Para atribuir causalidad hace falta una ablación que cambie un mecanismo y mantenga lo demás.

Diseña la siguiente ablación

Quieres comprobar si el overhead de procesar elementos individualmente perjudica a la versión online. ¿Qué variante construirías?

Solución: una versión que procese bloques de claves y reúna máximos y sumas parciales antes de combinar estados, conservando la misma función y tipos. Se compararía contra la online escalar y la densa, con las mismas formas y verificación. Cambiar además precisión y hardware impediría atribuir la diferencia al tamaño del bloque.

59.5 Pruebas que no debes omitir

Incluye contexto uno, tamaños irregulares, valores repetidos, puntuaciones muy negativas y cambios del máximo. Cada fila causal conserva aquí la clave propia; una máscara que oculte todo exigiría definir otro comportamiento antes de normalizar.

Laboratorio 11. Tokenización, particiones y pérdidas comparables

60.1 Por qué los datos también forman parte del experimento

Un compilador correcto puede entrenar una tarea equivocada. Si el conjunto de evaluación aparece repetido en entrenamiento, la pérdida no mide generalización independiente. Si cambia la tokenización, cambia la unidad sobre la que se calcula la pérdida. Si se entrena el tokenizador con todo el corpus, el procedimiento ya ha observado información del conjunto reservado.

El objetivo del laboratorio es construir una tubería pequeña cuyo estado pueda auditarse. Usaremos el ByteBPE original del paquete. No pretende reproducir exactamente el tokenizador de un modelo comercial ni incluir normalización, pretokenización o todos sus tokens especiales. Su contrato es representar bytes UTF-8 y aprender fusiones ordenadas sobre texto de entrenamiento.

60.2 Una prueba de ida y vuelta

```
from lumbre.tokenizer import ByteBPE

texto_entrenamiento = "la casa y la carta. la casa y la calma."
codec = ByteBPE.fit(texto_entrenamiento, 12)
texto_prueba = "Otra casa: 123."
ids = codec.encode(texto_prueba)
assert codec.decode(ids) == texto_prueba
restaurado = ByteBPE.from_state(codec.state())
assert restaurado.encode(texto_prueba) == ids
```

La base de 256 bytes permite representar bytes no vistos durante el aprendizaje de fusiones. Eso es distinto de un vocabulario de caracteres creado solo con los caracteres observados y sin una política de desconocidos. En generación, una secuencia arbitraria de tokens puede formar bytes que no sean UTF-8 válido; el código de muestra utiliza reemplazo al decodificar para mostrarla sin bloquear el programa.

60.3 Fusiones solapadas: un caso pequeño pero decisivo

En una secuencia de cuatro símbolos iguales hay tres parejas adyacentes si contamos solapamientos, pero una pasada de reemplazo no puede usar el mismo símbolo en dos fusiones. El procedimiento debe especificar cómo recorre y reemplaza parejas. De lo contrario, aprender

una lista de fusiones y aplicarla puede producir segmentaciones diferentes.

La implementación del paquete utiliza una regla determinista y reemplazos no solapados. Sus pruebas verifican ida y vuelta y serialización. Ese determinismo importa para checkpoints: los identificadores de tokens no tienen significado fuera del vocabulario y orden de fusiones que los definió.

60.4 Dividir antes de aprender el vocabulario

Prepara dos archivos de documentos distintos, o separa grupos completos antes de entrenar el tokenizador. No cortes ventanas casi idénticas a ambos lados de una frontera y las llames datos independientes. En el script educativo, la ruta BPE hace el corte antes de ajustar fusiones; para un experimento científico, conviene sustituir el corte por documentos por una partición explícita y persistida.

La ruta de caracteres del corpus de demostración usa el alfabeto del texto completo, una simplificación declarada del ejemplo. No se debe presentar esa ruta como una tubería de evaluación rigurosa sin modificarla. El ejercicio consiste precisamente en separar el ejemplo de depuración de un experimento de lenguaje defendible.

60.5 Entrenar con tokens de bytes

```
python examples/train_decoder.py --steps 20 --dim 16 \  
  --layers 1 --context 8 --byte-tokens --bpe-merges 30 \  
  --gemm blocked --output reports/mi_decoder_bpe
```

Esta ruta fue comprobada localmente: construye un vocabulario, entrena y guarda su estado. Veinte pasos y una arquitectura mínima solo validan la tubería; no producen por ese hecho texto útil. La pérdida de ese ensayo no debe compararse directamente con la pérdida del modelo de caracteres como si ambos predijeran la misma unidad.

60.6 De nats por token a una unidad interpretable

Si la pérdida media usa logaritmo natural, está en nats por token. La perplejidad es su exponencial, pero depende de la tokenización y de cómo se ponderan secuencias. Para comparar segmentaciones, puede ser útil acumular la log-verosimilitud total y dividir por bytes del texto original, explicando los detalles de codificación y contexto. No basta con aplicar una conversión constante cuando cada token representa una cantidad variable de bytes.

Una media de medias puede sesgar el resultado. Dos lotes con diferente número de tokens válidos no deben pesar lo mismo si la métrica deseada es media por token. Conserva suma de pérdidas y número de tokens, y divide al final. El padding debe excluirse según un contrato explícito de máscara de pérdida.

Detectar una comparación inválida

El modelo A tiene pérdida 1 por carácter; B tiene pérdida 2 por token BPE, cuyos tokens cubren varias letras. ¿Demuestra la tabla que A predice mejor el texto?

Solución: no. Las unidades difieren. Hace falta evaluar sobre el mismo texto y una normalización comparable, además de controlar contexto y datos. La perplejidad tampoco arregla por sí sola esa diferencia: exponenciar conserva la dependencia de la unidad de tokenización.

60.7 Entrega

Incluye estado del tokenizador, hash del corpus, manifiesto de particiones, prueba de ida y vuelta y definición exacta de la métrica. Añade un texto con caracteres no presentes en el conjunto de entrenamiento. Tu informe debe poder explicar qué identificador representa cada token y cómo reconstruir la misma secuencia después de reanudar.

Laboratorio 12. Una entrega reproducible y una defensa técnica

61.1 El objetivo final no es una carpeta que solo funciona en tu sesión

Al terminar el semestre, otra persona debe poder reconstruir qué hiciste. Eso exige un entorno, órdenes, datos identificados, resultados y límites. No exige empaquetar controladores, gigabytes de caché o binarios incompatibles con su máquina. Una buena entrega separa fuente portable, datos permitidos y resultados específicos de una ejecución.

Crea una carpeta nueva y copia únicamente el paquete fuente. Instala las dependencias declaradas. Ejecuta pruebas, un entrenamiento pequeño y un benchmark. Esta reproducción limpia detecta importaciones accidentales desde tu directorio de trabajo, archivos omitidos y dependencias no registradas.

61.2 Qué pertenece al manifiesto

El manifiesto debe identificar código y datos, configuración, semilla, sistema, compilador y ámbito de la medida. Para GPU añade dispositivo, arquitectura destino, controlador y bibliotecas. Un nombre como «mi ordenador» no permite interpretar diferencias. Un archivo `requirements.txt` no sustituye al resto: dos máquinas con las mismas bibliotecas pueden generar instrucciones distintas.

```
{
  "experiment": "attention_cpu_comparison",
  "backend": "cpu",
  "dtype": "float32",
  "shape": [1, 2, 128, 32],
  "seed": 17,
  "warmup": 3,
  "repetitions": 20,
  "timed_region": "resident run; no compilation or result copy",
  "correctness": {"rtol": 0.00002, "atol": 0.00002},
  "status": "replace_with_observed_status"
}
```

El ejemplo es una **plantilla**, no un resultado. El campo de estado debe sustituirse por una observación real. Es preferible un estado «no ejecutado: falta dispositivo» a inventar ceros para rellenar una tabla. Los valores ausentes no son tiempos nulos ni errores perfectos.

61.3 Separar tres rutas de ejecución

La ruta de **corrección** utiliza tamaños pequeños, entradas diseñadas y comprobaciones estrictas. La ruta de **rendimiento** utiliza tamaños representativos, calentamiento y repeticiones, pero solo después de superar corrección. La ruta de **entrenamiento** comprueba estabilidad del sistema completo y conserva estado. Mezclarlas en un script gigantesco hace difícil interpretar un fallo.

También debe separarse tiempo frío y caliente. El tiempo frío incluye trabajo inicial que puede amortizarse, como compilación y carga. El caliente describe repeticiones con recursos preparados. Ninguno es más verdadero por definición; responden a preguntas de uso distintas. Una aplicación interactiva puede sufrir el frío, mientras un entrenamiento largo amortiza gran parte de él.

61.4 Preparar una defensa con un caso que no conozcas

Pide a otra persona que cambie una forma no fundamental: añade un elemento al contexto o usa una matriz irregular. Debes predecir qué máscaras y qué buffers cambian. Después ejecuta. Si solo sabes repetir la orden exacta del ejemplo, todavía no has construido una comprensión transferible.

La defensa puede comenzar con cuatro minutos de explicación sin pantalla: problema, representación, transformación y evidencia. Después muestra un resultado correcto y un fallo intencional detectado. Termina con una limitación concreta y una prueba que necesitarías para eliminarla. No hace falta afirmar que todo funciona; sí hace falta saber dónde termina la garantía.

Pregunta de defensa resuelta

«Tu compilador tarda menos con fusión. ¿Cómo sabes que no has eliminado trabajo necesario?»

Respuesta: comparo las salidas y gradientes dentro del dominio declarado, pruebo casos frontera y explico qué dependencia permite fusionar sin alterar efectos. Además inspecciono el plan para confirmar que todas las salidas se calculan. La mejora de tiempo por sí sola no sirve como prueba: un programa que no hace nada es rápido.

61.5 Rúbrica orientativa para el cierre del curso

Dimensión	Evidencia sólida	Señal de insuficiencia
Semántica	Contrato y casos que lo distinguen	«Se parece a NumPy» sin formas ni tipos
Compilación	IR antes/después y código generado	Solo llamada opaca a una biblioteca
Correctitud	Pruebas, invariantes, errores detectados	Una captura con una salida favorable
Rendimiento	Frontera medida, muestras y baseline	Único mejor tiempo sin configuración

Dimensión	Evidencia sólida	Señal de insuficiencia
Entrenamiento	Gradientes, pérdida, checkpoint reproducible	Texto de muestra como única evidencia
Proyecto	Hipótesis, ablación y límites	Promesas de trabajo futuro sin resultado

La rúbrica es una propuesta docente del libro, no una norma de una universidad. Un proyecto puede obtener un buen resultado científico aunque su variante sea más lenta, siempre que el experimento esté bien diseñado y explique el mecanismo y alcance del resultado. Lo que no debe premiarse es una afirmación más amplia que la evidencia.

61.6 Entrega final del estudiante

Conserva un README con una orden de inicio, el código modificado, pruebas, un informe de diez a veinte páginas y resultados en formato legible por máquina. El informe debe incluir al menos una tabla de corrección y una de coste. Adjunta el diff respecto a la base para que se pueda identificar la contribución propia.

No publiques datos privados ni redistribuyas material cuya autorización no hayas comprobado. Las licencias del código externo siguen siendo parte del proyecto, aunque el compilador propio tenga otra licencia. Para estudiar una idea no es necesario copiar su implementación: se puede derivar una versión original y citar la fuente conceptual.

Proyecto final A. Atención con memoria acotada: una réplica semántica completa

Pregunta de investigación

¿Podemos sustituir la atención causal densa de nuestro compilador por una recurrencia online que conserve resultados y reduzca almacenamiento? Después, una pregunta distinta: ¿esa sustitución mejora también el tiempo en la CPU disponible?

62.1 Alcance y contribución del proyecto de referencia

Este es un proyecto desarrollado, no únicamente una lista de tareas futuras. El paquete contiene el operador `online_attention`, su generación de C, una regla de gradiente por recomposición, pruebas numéricas y un benchmark con resultados locales. La contribución es una **implementación didáctica original de una recurrencia conocida**, integrada en un compilador propio y estudiada con hipótesis separadas de corrección, memoria y velocidad.

La fuente conceptual de atención eficiente por reorganización del tráfico es la familia FlashAttention. La réplica de este proyecto es semántica y acotada: no reproduce íntegramente sus kernels GPU, su partición entre warps ni su backward optimizado. Esa distinción es fundamental para atribuir correctamente qué se ha conseguido. [41, 42, 43]

62.2 Definir la función exacta

Las entradas Q, K, V tienen la misma forma (B, H, T, D) , tipos FP32 y dimensiones positivas. Cada fila de consulta en posición i utiliza claves $0 \dots i$, con factor de escala $1/\sqrt{D}$. La salida tiene la misma forma. No se admite en este operador una máscara arbitraria ni longitudes diferentes de consulta y clave.

La referencia densa calcula puntuaciones, añade una máscara causal, normaliza mediante softmax estable y multiplica por los valores. El candidato procesa claves por fila sin conservar toda la matriz de puntuaciones. La equivalencia buscada es numérica dentro de tolerancias justificadas para el conjunto de prueba; no se exige identidad bit a bit entre órdenes de reducción distintos.

62.3 Derivación del estado que basta conservar

Para un conjunto de claves ya procesadas, guardamos m , el máximo de puntuaciones; l , la suma de exponenciales desplazadas; y a , el vector de suma ponderada de valores. La salida parcial normalizada sería a/l . Si llega una puntuación s con valor v , el nuevo máximo es $m' = \max(m, s)$.

Todas las contribuciones antiguas estaban expresadas respecto a m . Para expresarlas respecto a m' , se multiplican por $\alpha = \exp(m - m')$. La nueva contribución pesa $\beta = \exp(s - m')$. Por tanto:

$$l' = \alpha l + \beta, \quad a' = \alpha a + \beta v.$$

No hemos aproximado la normalización: hemos cambiado una representación intermedia. El factor común que se introduce en numerador y denominador se cancela. En FP32 aparecen redondeos, de ahí la comparación tolerada. La estabilidad mejora frente a calcular exponenciales de puntuaciones sin desplazar, pero no elimina todos los problemas posibles de precisión.

Una prueba inductiva, con su base

Antes de procesar claves, toma denominador cero y acumulador cero. La primera clave válida establece el máximo y aporta peso uno en la escala elegida. Después de cada incorporación, el denominador y acumulador representan exactamente, en aritmética real, todas las claves procesadas respecto al máximo actual.

El paso inductivo es la reescala anterior: transforma cada contribución antigua por el mismo factor y añade la nueva. Al procesar la última clave permitida, dividir acumulador entre denominador produce la atención causal definida. La prueba requiere al menos una clave válida por fila; una fila completamente enmascarada necesitaría otro contrato.

62.4 Integración en el compilador

La operación online actúa como una frontera de kernel. Sus operandos se materializan contiguos para que la función C reciba el layout que espera. El renderer genera un recorrido por filas y, dentro de cada una, un bucle sobre claves permitidas. El acumulador de salida mantiene D componentes y se normaliza al final.

Esta elección conserva la comprensión del algoritmo, pero renuncia a algunas optimizaciones. No hay un reparto avanzado entre hilos CPU, ni tiles vectoriales de claves, ni pipeline GPU de copias. El mismo operador tiene una ruta GPU simple en el renderer, pero no se ha validado en dispositivo. Ninguna tabla de este proyecto debe mezclar esa ruta no ejecutada con resultados CPU.

El backward del operador se expresa con primitivas densas. Esa decisión permite integrar entrenamiento y comprobar gradientes sin escribir de inmediato otro kernel complejo. El precio es que el grafo de gradientes puede recuperar almacenamiento cuadrático. La memoria de forward y la de un paso completo se informan por separado.

62.5 Plan de pruebas que acompaña al resultado

La batería compara forward con atención densa para varias formas y compara gradientes respecto a las tres entradas. Los tamaños pequeños permiten identificar una primera coordenada incorrecta. También se verifica causalidad en el decoder: cambiar el futuro no debe alterar salidas anteriores.

Para fortalecer el proyecto, el estudiante puede añadir entradas construidas para hacer cambiar varias veces el máximo, valores grandes con signos opuestos y longitudes cercanas al límite de un tile. Cada caso debe responder a una hipótesis de fallo. Repetir mil veces una forma cómoda con valores normales no sustituye una prueba donde la primera clave domina y las siguientes son muy negativas.

En una réplica más avanzada, se guarda también el logaritmo del denominador en la escala original, conocido como log-sum-exp. Permite reconstruir probabilidades en backward sin conservarlas todas. La fórmula $p_j = \exp(s_j - L)$ requiere que L corresponda exactamente a la fila y máscara del forward. Un error de índice de fila puede producir probabilidades plausibles que suman mal o mezclan ejemplos.

62.6 Resultados locales del forward

La tabla se obtiene del informe `reports/kernels.json`. Los tiempos son medianas de veinte repeticiones, después de tres calentamientos, con entradas residentes. La arena es la definida por Lumbre; no es el pico total del proceso. Todas las filas usan $(B,H,D)=(1,2,32)$.

Contexto T	Densa: microsegundos / bytes	Online: microsegundos / bytes
16	10,15 / 22.528	9,70 / 16.384
64	106,44 / 131.072	136,31 / 65.536
128	437,53 / 393.216	624,61 / 131.072

En contexto 128, la arena online es un tercio de la densa, pero el tiempo es aproximadamente 1,43 veces el de la densa. En contexto 64 ocurre también una pérdida de velocidad. El primer tamaño es demasiado pequeño para convertir una diferencia reducida de tiempo en una conclusión amplia sin estudiar dispersión y más repeticiones.

La conclusión válida es doble: **la implementación evita el intermedio denso y reduce la arena observada; esa mejora no garantiza aceleración en esta CPU**. No se publican cifras de ancho de banda efectivo ni ocupación GPU porque no se midieron. Tampoco se atribuye una causa concreta sin una ablación adicional.

62.7 Qué experimento realizar después

La siguiente variante debería procesar varias claves juntas. Puede calcular un máximo de bloque, un denominador de bloque y una suma ponderada, y combinar ese estado con el anterior. Así se mantiene la prueba por composición de estados, pero cambia la granularidad del trabajo y se abren oportunidades de vectorización.

Otra variante conserva forward y cambia únicamente la organización de las componentes D , buscando que el compilador genere instrucciones vectoriales. Se inspecciona el ensamblador y se miden varios valores de D . Si ambas variantes cambian a la vez tamaño de bloque, precisión y número de hilos, ya no se puede identificar qué mecanismo explica el resultado.

Para GPU, la extensión debe diseñar memoria compartida, participantes y barreras antes de copiar un código de atención. La verificación de resultados es necesaria, pero no suficiente para carreras. La ruta de tensor cores requiere además distinguir tipos de entrada, acumulación y layout colectivo. Se utiliza hardware compatible y herramientas de diagnóstico reales; un emulador lógico no certifica esas propiedades.

62.8 Entregables y defensa del proyecto

La memoria del proyecto contiene función, derivación, integración, pruebas, tabla y discusión. El código debe incluir un modo de referencia que no desaparezca cuando el candidato empiece a funcionar. Los resultados en JSON conservan muestras y configuración. Una tabla negativa se mantiene, porque es parte del hallazgo.

La defensa se aprueba cuando el estudiante explica por qué la reescala no pierde contribuciones, por qué su backward todavía puede usar memoria cuadrática y por qué una CPU no reproduce automáticamente los resultados de una publicación GPU. Esas tres respuestas demuestran comprensión de matemática, compilación y evaluación, respectivamente.

Proyecto final B. Pasaporte de índices para un acelerador desconocido

63.1 La idea original

Antes de portar un compilador a una máquina que no conocemos, podemos separar una pregunta que sí sabemos resolver: **¿el reparto lógico escribe cada salida válida exactamente una vez y evita direcciones fuera de rango?** El proyecto construye un pasaporte verificable del mapa de índices. No inventa una GPU virtual completa ni simula ciclos; comprueba una propiedad finita y útil antes de gastar tiempo en el hardware.

El paquete incluye `lumbre/passport.py`, pruebas y `examples/verify_launch.py`. Se ejecutó una colección de quince mapas válidos y se rechazó una variante con stride incorrecto. Este resultado es validación del mapa lógico por enumeración CPU, no ejecución CUDA/HIP/Ascend ni una prueba de ausencia de carreras de memoria compartida.

63.2 Definición del mapa

Una matriz tiene M filas y N columnas. Los tiles tienen T_M filas y T_N columnas. Para un tile (b_y, b_x) y una posición local (t_y, t_x) , la salida lógica es $r = b_y T_M + t_y$, $c = b_x T_N + t_x$. Si $r < M$ y $c < N$, su dirección contigua es $rN + c$.

El pasaporte enumera participantes lógicos, aplica la máscara y cuenta escritores de cada dirección. Deben cumplirse tres condiciones: ninguna dirección válida sin escritor, ninguna con varios escritores y ninguna escritura fuera de la región. El conteo permite distinguir pérdida de cobertura, duplicación y desbordamiento.

```
from lumbre.passport import Launch, verify

correcto = verify(Launch(17, 23, 16, 16))
assert correcto["status"] == "LOGICAL_MAP_VALIDATED"
assert correcto["valid_writes"] == 17 * 23

erroneo = verify(Launch(3, 5, 2, 4),
                 lambda fila, columna, s: fila * s.rows + columna)
assert erroneo["status"] == "FAILED"
print(erroneo["missing"], erroneo["duplicate_writes"])
```

El segundo mapa multiplica por número de filas cuando debía multiplicar por número de columnas. En una matriz cuadrada ambos números coinciden y el fallo queda oculto. Por eso las formas rectangulares son esenciales en una prueba de layouts.

63.3 Resolver el contraejemplo completo

Para $M = 3$, $N = 5$, la dirección correcta de la segunda fila empieza en 5. El mapa defectuoso la empieza en 3. La primera fila escribe 0,1,2,3,4; la segunda escribe 3,4,5,6,7; la tercera 6,7,8,9,10. Se repiten 3,4,6,7 y quedan sin escribir 11,12,13,14.

No hace falta ejecutar aritmética para encontrar el fallo: la propiedad de propiedad de salida ya es falsa. Si después un kernel devolviese algunos números correctos, sería accidental. Aumentar tolerancias no repara direcciones duplicadas. La corrección es sustituir el stride por N , y después volver a comprobar la colección completa.

63.4 Del ensayo finito a un argumento general

La enumeración prueba las formas elegidas. Podemos añadir una demostración para el mapa correcto. Cada fila r tiene una descomposición única en cociente y resto respecto a T_M ; cada columna c , respecto a T_N . Por tanto, cada coordenada válida corresponde a un único tile y una única posición local.

Además, para $0 \leq r < M$ y $0 \leq c < N$, la dirección $rN + c$ queda entre 0 y $MN - 1$. Si dos coordenadas producen la misma dirección, sus cocientes y restos respecto a N coinciden, luego fila y columna son iguales. Hay cobertura, unicidad y rango. La máscara excluye participantes que no corresponden a una coordenada válida.

Qué no demuestra el argumento

No demuestra que un compilador GPU traduzca bien los índices, que todos los hilos lleguen a una barrera, que una instrucción tensorial tenga el layout correcto ni que el dispositivo admita el número de participantes. La prueba se refiere al mapa matemático definido. Un port debe conectar esa prueba con el código generado y con capacidades verificadas del hardware.

63.5 Por qué sirve para hardware extraño

Un acelerador puede organizar trabajo en warps, wavefronts, grupos o unidades matriciales con nombres diferentes. Antes de utilizar sus características, el pasaporte conserva una representación neutral de qué salidas posee cada participante lógico. Después una capa de mapeo asigna participantes a unidades reales.

Si una unidad real procesa varios participantes lógicos, debe explicar cómo los recorre. Si varios participantes cooperan para una salida, el pasaporte simple de un escritor por elemento deja de ser suficiente: hay que representar contribuciones y una reducción o una escritura final única. El contrato debe evolucionar; no basta con desactivar la comprobación de duplicados para que aparezca verde.

En un port Ascend, por ejemplo, habrá que estudiar por separado las unidades de cómputo, buffers y mecanismos de sincronización de la versión concreta del entorno. El material de Huawei y los repositorios Ascend de DeepSeek ayudan a identificar esos contratos, pero no se consideran una interfaz intercambiable con CUDA. [28, 24, 26]

63.6 TDD del proyecto

La primera prueba exige que una matriz vacía no produzca accesos. La segunda utiliza un elemento. La tercera coincide con un tile completo. La cuarta obliga a enmascarar bordes. La quinta usa una forma rectangular grande respecto al tile. Después se añaden fallos intencionales: stride incorrecto, desplazamiento de uno y dirección fuera de rango.

El verificador tiene límites explícitos de tamaño para evitar enumeraciones accidentales enormes. Un rechazo por límite de recursos no debe etiquetarse como mapa incorrecto: es una solicitud fuera del dominio del verificador. Del mismo modo, los límites didácticos de tile no afirman cuáles son los límites físicos de CUDA o HIP.

```
python examples/verify_launch.py --output reports/mi_pasaporte.json
python -m pytest tests/test_passport.py -q
```

El informe conserva el alcance de la comprobación en cada resultado. La etiqueta `LOGICAL_MAP_VALIDATED` es deliberadamente más estrecha que «backend validado». Una buena nomenclatura evita que un informe de simulación se convierta después, por descuido, en una afirmación de pruebas en hardware.

63.7 Extensión a layouts de fragmentos

Una ampliación interesante describe la propiedad de cada elemento como un par «participante, posición local». Para una instrucción matricial, el verificador puede comprobar que la distribución de fragmentos cubre las coordenadas esperadas y que las transformaciones entre layouts son biyectivas dentro de su dominio.

No conviene empezar por una instrucción enorme. Construye una matriz 4×4 y dos repartos diferentes entre cuatro participantes. Escribe las tablas completas y su conversión. Después añade un tamaño no divisible y decide si hay padding, máscara o una ruta de fallback. Esta metodología convierte el port en una secuencia de contratos pequeños en lugar de una búsqueda ciega de código que compile.

63.8 Entrega del proyecto

El resultado mínimo es el verificador ejecutado, quince casos válidos, tres clases de fallos y la demostración del mapa contiguo. El resultado avanzado conecta el pasaporte con un renderer de Lumbre y compara sus expresiones de índices con la tabla esperada. El nivel de hardware exige además compilación, ejecución y herramientas de diagnóstico del dispositivo, y permanece pendiente hasta contar con esa evidencia.

Proyecto final C. Competitividad CPU y GPU con un contrato medible

64.1 Convertir «competitivo» en una pregunta que pueda fallar

El temario propone producir código competitivo con implementaciones modernas. Es un objetivo exigente y valioso, pero necesita una definición operacional. Este proyecto no declara que Lumbre ya iguale a PyTorch, BLAS o los mejores kernels CUDA. Diseña la comparación necesaria y proporciona una base CPU mejorada que el estudiante puede extender.

Elige una familia de carga: GEMM de proyecciones de un decoder, reducciones de normalización o convoluciones de una red pequeña. Define formas y tipos antes de ajustar. Una colección útil mezcla matrices pequeñas, grandes, irregulares y relaciones entre dimensiones distintas. Una suite formada solo por la forma que mejor le va a tu kernel no permite una conclusión general.

64.2 Baseline de tres niveles

El primer baseline es el bucle de referencia del compilador, porque ayuda a aislar el efecto de cada transformación propia. El segundo es una implementación madura de la misma operación en el entorno disponible. El tercero es el paso del modelo completo, donde importan fusión, llamadas y memoria.

Estas comparaciones responden a preguntas diferentes. Ganar al bucle propio muestra una mejora interna. Acercarse a una biblioteca madura muestra competitividad de kernel. Mejorar el paso completo muestra impacto de sistema. Ninguna implica automáticamente las otras. Una biblioteca puede ganar el GEMM aislado y perder tiempo de conversión de layout al integrarse en una cadena concreta.

Para una referencia PyTorch, conserva código, versión y configuración, y comprueba si se utiliza ejecución eager o compilada. No compares una versión compilada del candidato con una referencia que incluye transferencias o inicialización sin declararlo. La referencia debe tener una oportunidad razonable de usar sus rutas habituales.

64.3 Una escalera de optimizaciones CPU

La base entregada ofrece un microkernel con acumuladores y bloques de reducción. La primera extensión puede separar packing de la ejecución: reorganizar B en un layout que permita

accesos contiguos en el microkernel. Mide packing por separado y extremo a extremo; si la matriz cambia cada llamada, no puedes amortizarlo como si fuera un peso constante.

La segunda extensión usa vectorización explícita o verifica la autovectorización del compilador. La tercera reparte tiles entre hilos sin que dos escriban la misma región. La cuarta estudia afinidad, memoria NUMA y tamaño de trabajo. No implementes todas antes de obtener una referencia correcta de una sola hebra: perderías un punto de comparación interpretable.

Un diseño de packing debe describir su dirección. Para un bloque de $K_B \times N_B$, un formato simple copia filas contiguas de B y rellena el borde con cero. El microkernel usa el mismo orden de K si se quiere conservar cierta comparabilidad numérica. Formatos más complejos pueden intercalar grupos para SIMD, pero requieren otra prueba de mapa.

64.4 Repartir trabajo sin false sharing innecesario

Si cada hilo recibe tiles de salida completos, las escrituras no se solapan lógicamente. Aun así, dos tiles pequeños podrían compartir líneas de caché en sus bordes y perjudicar rendimiento. Esa interferencia no es una carrera de datos si las posiciones son distintas, pero puede aumentar coherencia y tráfico.

No confundas las dos clases de problema: una carrera compromete corrección; false sharing puede comprometer tiempo. El remedio depende del diagnóstico. Añadir un mutex a todas las escrituras puede evitar una carrera real, pero destruir paralelismo y no ser la solución apropiada a un reparto mal diseñado.

64.5 Escalera GPU y puertas de aceptación

Empieza por un hilo por salida, después tiling compartido, luego microtiles por hilo y finalmente instrucciones matriciales si el dispositivo las admite. Cada nivel mantiene la versión anterior como oráculo de comparación y fallback. No actives el nivel siguiente solo porque compila.

Puerta	Evidencia exigida	Motivo de rechazo
Semántica	Valores y formas en una batería diseñada	Error fuera de tolerancia
Memoria	Bordes, alias y sincronización revisados	Acceso inválido o carrera
Recursos	Arquitectura y lanzamiento compatibles	Tile que excede recursos
Rendimiento	Mediana y dispersión frente a baseline justo	Mejora no reproducible
Integración	Paso completo y gradientes correctos	Kernel aislado que rompe el modelo

Una instrucción tensorial no es una victoria si obliga a convertir repetidamente datos para una matriz pequeña. La comparación debe incluir la ruta de conversión cuando forme parte del uso real. También debe mantener la misma precisión de acumulación o informar explícitamente el cambio de contrato.

64.6 Función de puntuación sin premiar errores

Una propuesta es calcular la media geométrica de aceleraciones por forma solo entre candidatos que superen todas las pruebas requeridas. Si una forma obligatoria falla, el proyecto no puede ocultarla eliminándola de la media. Se informa cobertura, tasa de fallos y tiempos por forma, además del agregado.

La media geométrica evita que una aceleración enorme en una sola forma domine linealmente el resumen, pero tampoco sustituye la tabla. Para una aplicación con frecuencias conocidas, una métrica ponderada por el tiempo real de la carga puede ser más relevante. El peso debe fijarse antes de ajustar, no después de ver qué favorece al candidato.

Umbral de competitividad elegido por el curso

Como criterio docente, se puede exigir estar dentro de un factor dos de una referencia fuerte en al menos el 80 por ciento de la suite, sin fallos de corrección, y explicar los peores casos. Ese umbral es una propuesta de evaluación, no una definición universal de SOTA. Un proyecto que lo alcance todavía debe declarar hardware, suite y precisión.

64.7 Calendario de cuatro semanas

En la primera semana se fija la suite y se reproduce la referencia. En la segunda se implementa una transformación con pruebas y se registra un baseline intermedio. En la tercera se estudia el recurso limitante y se realiza una ablación. En la cuarta se integra en el modelo y se redactan resultados, incluidos los negativos.

Cada semana termina con una decisión verificable. Si no se consigue reproducir la referencia, no se dedica la siguiente a ajustar contra una medición dudosa. Si el candidato es incorrecto, no se aumenta el presupuesto de búsqueda de velocidad. Si mejora el kernel pero no el modelo, se estudia Amdahl y overhead, no se oculta la diferencia.

64.8 Qué resultado haría original el trabajo

La originalidad puede estar en un selector de kernels sensible a formas irregulares, una política de packing que se amortice con pesos reutilizados o un análisis de por qué una transformación pierde en una región de tamaños. No necesita afirmar una técnica inédita mundialmente. Sí necesita una pregunta propia, una implementación identificable y evidencia que vaya más allá de ejecutar un tutorial ajeno.

La entrega debe permitir activar y desactivar la contribución con una opción, de modo que se pueda realizar una ablación limpia. Un conjunto de cambios inseparables dificulta aprender cuál importa. Esa disciplina es especialmente valiosa en compiladores, donde una mejora aparente puede proceder de un detalle accidental de generación.

Proyecto final D. Un agente propone kernels; el contrato decide

65.1 Motivación científica y límite de la automatización

Los trabajos recientes que exploran generación o bajada de kernels con modelos de lenguaje sugieren que un proponente puede encontrar transformaciones distintas de las de un pipeline tradicional. El artículo *AI as a Compiler* estudia precisamente una ruta de Triton a PTX acompañada de evaluación y verificación. *KernelBench* ofrece otra referencia útil para pensar tareas, comprobación y rendimiento de kernels generados. [51, 48]

El proyecto del estudiante consiste en separar el proponente del evaluador. El proponente puede ser un LLM, una búsqueda enumerativa o un conjunto de reglas. El evaluador no acepta un resultado porque venga explicado con confianza: verifica el contrato y mide en un entorno definido. El núcleo del diseño es esa separación, no el nombre del modelo utilizado.

65.2 Elegir un espacio de búsqueda seguro y pequeño

Para empezar, no permitas programas arbitrarios que puedan leer archivos, acceder a la red o ejecutar comandos del host. Una opción es permitir únicamente parámetros de un generador de kernels conocido: tamaños de tile, orden de ejes y grado de desenrollado dentro de límites. Así el candidato es un objeto estructurado, no código con autoridad ilimitada.

```
{
  "operation": "gemm_fp32",
  "tile_rows": 4,
  "tile_columns": 8,
  "reduction_block": 64,
  "strategy": "blocked",
  "numeric_contract": "fp32_reference_tolerance"
}
```

El validador comprueba tipos, rangos y combinaciones legales antes de generar fuente. Los campos desconocidos se rechazan. No basta con pedir al agente que sea cuidadoso: la herramienta debe impedir que una respuesta inesperada cambie el contrato, borre resultados o ejecute una acción fuera del espacio permitido.

65.3 Un evaluador en fases

Primero valida el esquema. Después construye el código con un generador controlado. Compila con límite de tiempo. Ejecuta en un proceso separado y entorno restringido. Comprueba

formas y números. Solo si pasa, mide varias repeticiones. Finalmente conserva candidato, fuente, diagnóstico y resultado.

El pseudocódigo siguiente describe arquitectura, no una función ya integrada en Lumbre:

```
evaluar(candidato):  
    comprobar_esquema_y_limites(candidato)  
    fuente = generar_desde_parametros_conocidos(candidato)  
    binario = compilar_con_presupuesto(fuente)  
    resultado = probar_en_proceso_aislado(binario, casos_ocultos)  
    si resultado no es correcto:  
        registrar_fallo_y_terminar  
    medidas = cronometrar_con_calentamiento(binario)  
    registrar(candidato, fuente, resultado, medidas)
```

Un proceso separado no es por sí solo una frontera de seguridad completa. El proyecto debe usar el aislamiento y permisos que ofrezca su entorno. Para un primer laboratorio local, es preferible el espacio estructurado de parámetros a compilar código libre de origen no confiable dentro del mismo proceso del estudiante.

65.4 No enseñar todas las pruebas al proponente

Si un candidato conoce únicamente tres entradas fijas, puede especializarse indebidamente a ellas. Separa casos de desarrollo y casos reservados. Cambia semillas y valores, pero mantén casos estructurales: tamaños cero donde proceda, uno, bordes, negativos, magnitudes variadas y formas irregulares.

Tampoco permitas que el candidato cambie tolerancias, elimine pruebas o modifique la referencia. La evaluación debe tener un hash o versión independiente. Un agente que corrige su propia nota cambiando el examen no ha mejorado el kernel. El informe debe distinguir propuestas, fallos de compilación, fallos de corrección y candidatos válidos.

65.5 Un presupuesto que no desaparece del artículo

Fija número máximo de candidatos, tiempo de compilación y tiempo total de búsqueda. Conserva intentos fallidos. Compara con una búsqueda aleatoria o una cuadrícula del mismo presupuesto. Si el agente solo gana porque prueba cien veces más variantes, la conclusión sobre su estrategia es distinta.

El rendimiento final y el coste de encontrarlo son dos ejes. Una búsqueda cara puede ser rentable para una operación que se repetirá millones de veces. Para formas efímeras, puede perder frente a una heurística rápida. Calcula el punto de amortización: coste adicional de búsqueda dividido entre ahorro por ejecución, siempre que ambos estén medidos en unidades compatibles.

65.6 Detectar soluciones tramposas o frágiles

Un kernel que devuelve constantes aprendidas de entradas públicas no implementa la operación. Uno que utiliza una biblioteca prohibida incumple el alcance de la tarea aunque sea correcto.

Uno que omite sincronización puede pasar ocasionalmente y fallar en otra intercalación. El evaluador debe revisar tanto resultados como condiciones del experimento.

Las restricciones deben declararse antes de la búsqueda. No prohíbas después la técnica del ganador porque no era la que esperabas, ni aceptes una precisión inferior sin reflejarla. Si se permite llamar a una biblioteca, el resultado se etiqueta como selección o integración de biblioteca, no como generación completa de un kernel propio.

Experimento resuelto de control

Un agente encuentra un tile más rápido que el inicial. Para comprobar si su razonamiento aporta algo, se ejecuta una búsqueda aleatoria con el mismo número de propuestas legales y el mismo evaluador. Si ambas encuentran rendimientos similares, ¿ha fracasado el proyecto?

Solución: no. Se ha obtenido evidencia de que, en ese espacio y presupuesto, no se observa una ventaja clara del agente sobre el control. La comparación evita atribuir a inteligencia del proponente lo que podría explicar una exploración sencilla. El proyecto puede aportar un evaluador robusto y una caracterización del espacio de búsqueda.

65.7 Entrega y conexión con los LLM del curso

El estudiante entrega el generador controlado, evaluador, suite separada, historial completo y una comparación con control. No necesita entrenar un LLM para generar kernels si el proyecto estudia evaluación; puede usar una API o un modelo local autorizado, registrando versión y configuración. Esas llamadas no forman parte de las pruebas ejecutadas de esta edición.

La conexión con el decoder propio puede ser usarlo como carga objetivo: escoger una de sus operaciones y mejorarla sin cambiar la pérdida ni los gradientes. Así el proyecto une semántica, generación, validación y entrenamiento, en lugar de terminar en una demostración que solo produce código bonito.

Proyecto final E. Del decoder pequeño al entrenamiento de un LLM mayor

66.1 Qué significa ampliar sin cambiar de problema

El núcleo del curso ya entrena un decoder pequeño mediante C generado. Este proyecto desarrolla la ruta hacia una escala mayor, sin fingir que aumentar dos números en la configuración resuelve memoria, rendimiento, datos y evaluación. El resultado esperado del estudiante es un hito de escala medido y una arquitectura de entrenamiento reproducible, no una promesa de competir con modelos de frontera sin recursos ni evidencia.

Las técnicas de paralelismo de modelos, partición de estados y planificación de cómputo se estudian en Megatron-LM, ZeRO y trabajos sobre asignación de presupuesto de entrenamiento. Son herramientas para razonar sobre la escala, no órdenes mágicas que conviertan el runtime educativo en un sistema distribuido completo. [63, 62, 65]

66.2 Tres tamaños con objetivos distintos

El tamaño de **depuración** usa pocas capas y dimensiones para probar todos los gradientes y reproducir checkpoints. El tamaño de **ingeniería** debe ocupar una parte relevante del dispositivo disponible y revelar costes reales de memoria y lanzamiento. El tamaño de **calidad** necesita suficientes datos y presupuesto para evaluar generalización en una tarea lingüística definida.

No impongas que los tres tamaños coincidan. Un modelo suficientemente grande para saturar un acelerador puede ser demasiado caro para diferencias finitas de todos sus parámetros. Un modelo diminuto puede validar el optimizador sin decir nada sobre eficiencia de memoria a gran escala. Cada hito usa el tamaño adecuado a su pregunta.

66.3 Un presupuesto de memoria antes de reservar

Cuenta pesos, gradientes, momentos, copias maestras cuando existan, activaciones, buffers temporales y estado del runtime. Si se usa AdamW con cuatro arreglos FP32 por parámetro entre pesos, gradientes y dos momentos, solo ese conjunto ocupa aproximadamente 16 bytes por parámetro, antes de activaciones. En una configuración mixta, las copias y tipos cambian el cálculo; no se reutiliza ciegamente el mismo coeficiente.

Para 100 millones de parámetros, 16 bytes por parámetro son 1.600 millones de bytes, aproximadamente 1,49 GiB. Esa cifra no demuestra que el modelo quepa en una GPU de dos GiB: faltan activaciones, logits, atención, workspace y reservas. El presupuesto debe incluir un margen operativo y contrastarse con el pico observado.

Construye una tabla por tensor y por fase del paso. La arena estática de Lumbre da una primera estimación para su propio grafo, pero no resuelve automáticamente buffers externos de bibliotecas o comunicación. Un modelo de memoria debe reconciliar estimación y medición, no declarar victoria cuando una de las dos es menor.

66.4 Orden de las extensiones

Primero reduce materializaciones innecesarias y fusión de operaciones elemento a elemento. Después mejora GEMM y normalizaciones. Luego implementa atención eficiente también en backward, si esa matriz domina el pico. Añade checkpoint de activaciones cuando recomputar sea preferible a conservar. Solo entonces decide qué estado repartir entre dispositivos.

La precisión mixta se introduce con pruebas de estabilidad: tipos de almacenamiento, acumulación, escalado y detección de valores no finitos. FP8 o FP4 requieren contratos y escalas específicos; no son una sustitución textual de `float` por un tipo más pequeño. Los repositorios modernos de DeepSeek y NVIDIA muestran implementaciones especializadas, pero sus restricciones deben conservarse al integrarlas. [23, 11, 49, 50]

66.5 Datos y evaluación que no se puedan confundir con la demo

Sustituye el corpus repetido por documentos con autorización de uso y una partición por unidades independientes. Guarda hashes o un manifiesto reproducible, reglas de limpieza y tokenizador. Define qué evaluación se ejecutará antes de comenzar la búsqueda de hiperparámetros. Mantén un conjunto final reservado para la conclusión.

Además de pérdida, registra número de tokens procesados, tokens por segundo, tiempo frío, tiempo por paso y pico de memoria. Para comparar dos compiladores, mantén arquitectura, inicialización, datos, orden de lotes y optimizador. Si cambias todo a la vez, una diferencia de calidad no puede atribuirse al compilador.

La muestra de texto se conserva como ilustración cualitativa. No sustituye métricas ni análisis de contaminación. Un modelo puede producir una frase convincente y tener un entrenamiento incorrecto; también puede tener un pipeline correcto y producir texto pobre por escala o datos insuficientes.

66.6 Distribuido: un paso debe conservar su significado

Con paralelismo de datos, cada réplica calcula contribuciones de un lote local. Hay que definir si las pérdidas son medias locales o sumas y cómo se combinan gradientes. Un promedio doble puede reducir indebidamente la magnitud; una suma sin normalización puede cambiar la tasa efectiva al aumentar dispositivos.

Con partición tensorial, una operación se divide entre dispositivos y aparecen colectivos en puntos específicos. Su orden debe coincidir entre participantes y sus buffers vivir hasta finalización. Un fallo de colectivo puede parecer un bloqueo del modelo cuando realmente hay

secuencias distintas entre procesos. Las bibliotecas NCCL y RCCL pertenecen a esta capa, no al algoritmo de autodiferenciación. [66, 67]

El primer ensayo distribuido debe ser minúsculo y compararse con una ejecución de un solo dispositivo que use el mismo lote global. No se comienza con el modelo máximo esperando depurar a partir de un timeout. Se comprueban pérdidas, gradientes y una actualización antes de medir escalabilidad.

66.7 Puertas de aceptación del proyecto LLM

La primera puerta exige que el modelo pequeño conserve pruebas de causalidad, gradientes y checkpoint. La segunda exige una ejecución mayor sin valores no finitos y con presupuesto de memoria explicado. La tercera compara rendimiento justo contra una referencia definida. La cuarta evalúa calidad con datos independientes. Ninguna puerta se sustituye por haber completado un número de epochs.

Una configuración que cabe pero tarda demasiado aporta información: identifica qué operación domina. Una que es rápida pero cambia la trayectoria por un error numérico no supera corrección. Una que reduce pérdida en entrenamiento y empeora evaluación obliga a analizar datos y sobreajuste, no a celebrar automáticamente que el compilador «entrena SOTA».

66.8 Informe final y posible contribución

Un proyecto sólido puede aportar un backward de atención por tiles, un planificador de activaciones, una integración de GEMM con conversiones amortizadas o un runtime que reusa lanzamientos. Debe mostrar el efecto en un paso completo y conservar equivalencia semántica dentro del contrato elegido.

La meta de entrenar LLM actuales se cubre mediante arquitectura, autodiferenciación, kernels, datos, precisión, memoria, paralelismo y evaluación. El resultado local de este libro sigue siendo el modelo pequeño documentado. El estudiante que avance la escala debe añadir su propia evidencia a esa cadena, no heredar una afirmación de rendimiento que todavía no ha medido.

Boletín de integración: resolver antes de ejecutar

67.1 Problema 1. Un paso completo de aprendizaje con matrices pequeñas

Enunciado. Sean $X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $W = I$ y $b = (1, -1)$. Calcula $Y = XW + b$, la pérdida media de los cuatro elementos de Y^2 , sus gradientes respecto a X, W, b y una actualización de descenso de gradiente de tasa 0,1 sobre W, b . El sesgo se comparte entre filas.

Resolución del forward. Multiplicar por identidad conserva X . Sumar el sesgo aumenta la primera columna en uno y reduce la segunda en uno. Así $Y = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}$. Los cuadrados son 4,1,16,9, cuya suma es 30. La media es 7,5. Dividir por cuatro, y no por dos, corresponde a la media de todos los elementos.

Resolución del backward. La derivada de cada cuadrado es dos veces su entrada y la media introduce un cuarto. Por tanto, $G = \partial L / \partial Y = Y/2$. El sesgo recibe suma de filas: $(1 + 2, 0,5 + 1,5) = (3,2)$. Como W es identidad, el gradiente de X coincide con G . El gradiente de pesos es $X^T G$:

$$\frac{\partial L}{\partial W} = \begin{pmatrix} 7 & 5 \\ 10 & 7 \end{pmatrix}, \quad \frac{\partial L}{\partial X} = \begin{pmatrix} 1 & 0.5 \\ 2 & 1.5 \end{pmatrix}.$$

Actualización. Los pesos nuevos son $\begin{pmatrix} 0,3 & -0,5 \\ -1 & 0,3 \end{pmatrix}$ y el sesgo nuevo $(0,7, -1,2)$. La notación decimal con coma no debe confundirse con cuatro componentes: son dos valores, 0,7 y -1,2. Al repetir forward se obtiene $\begin{pmatrix} -1 & -1,1 \\ -2,4 & -1,5 \end{pmatrix}$ y la pérdida es 2,555. En este ejemplo el paso reduce la pérdida; una tasa arbitraria no tendría esa garantía.

```
import numpy as np
from lumbre import param, Program, gradients

x, w, b = param("X", (2,2)), param("W", (2,2)), param("b", (2,))
y = x @ w + b
loss = (y * y).mean()
g = gradients(loss, [x, w, b])
with Program([y, loss, *g], gemm="blocked") as p:
    outputs = p.run({"X": [[1,2],[3,4]], "W": np.eye(2), "b": [1,-1]})
np.testing.assert_allclose(outputs[1], 7.5)
np.testing.assert_allclose(outputs[3], [[7,5],[10,7]])
np.testing.assert_allclose(outputs[4], [3,2])
```

La prueba integra matmul, broadcasting, reducción, reutilización de un nodo en dos aristas y transposición en backward. Si una implementación falla, compara los intermedios en ese

orden. Un gradiente final incorrecto puede originarse en un eje de reducción equivocado, no necesariamente en la regla del producto matricial.

67.2 Problema 2. Fusión que duplica trabajo

Enunciado. Una operación cara $h = f(x)$ alimenta dos consumidores. Fusionarla dentro de ambos elimina un buffer de un millón de elementos, pero ejecuta f dos veces. ¿Debe fusionarse?

Solución razonada. Falta información sobre coste de f , tráfico del buffer y recursos. Si f es una suma sencilla y el buffer se escribiría y leería desde memoria lenta, recomputar puede convenir. Si f es una reducción costosa o una función trascendental compleja, duplicarla puede perjudicar. La decisión también cambia si uno de los consumidores apenas usa una parte de h .

Formula dos estimaciones: coste de calcular una vez más escribir y leer, frente a coste de calcular dos veces sin intermedio. Después mide ambas con el mismo contrato. La respuesta correcta no es una regla universal, sino un procedimiento que conserva semántica y compara costes. El número de kernels por sí solo no determina el ganador.

67.3 Problema 3. Una vista invertida y una dirección válida

Enunciado. Una fila de cinco elementos tiene stride uno y offset cero. Se invierte lógicamente. Escribe nuevo offset, stride y direcciones de las posiciones 0 y 4. Explica por qué no puede usarse un puntero al comienzo con stride negativo sin cambiar offset.

Solución. El nuevo offset es cuatro y el stride -1. La dirección de la posición lógica cero es 4; la de la posición cuatro es $4 - 4 = 0$. Si se conservase offset cero, la segunda posición lógica tendría dirección -1, fuera de la región. Invertir no es solamente cambiar el signo de un stride: hay que situar el origen lógico en el extremo correcto.

Para una dimensión vacía no existe un último elemento que consultar. El modelo de vista debe evitar fabricar un acceso a una posición anterior al buffer. Es otro motivo para probar formas vacías por separado, incluso cuando no producen salida.

67.4 Problema 4. Un techo de rendimiento no es una medida

Enunciado. Una operación realiza 200 millones de operaciones y mueve como mínimo 100 millones de bytes. La máquina ofrece un techo de 1 billón de operaciones por segundo y 100 mil millones de bytes por segundo, en unidades decimales. Calcula la cota idealizada del modelo roofline y explica sus límites.

Solución. La intensidad idealizada es dos operaciones por byte. El techo por memoria sería 200 mil millones de operaciones por segundo, inferior al techo de cómputo de un billón. El

tiempo mínimo por datos es un milisegundo; el mínimo por cómputo, 0,2 milisegundos. El modelo ideal toma el mayor, un milisegundo.

No se ha medido un tiempo de un milisegundo. Puede haber más tráfico que el mínimo, latencia, falta de paralelismo, conversiones y overhead. Tampoco se deduce automáticamente que aumentar operaciones mejore rendimiento: cambiar el algoritmo puede modificar bytes, instrucciones y recursos. El modelo sirve para formular hipótesis y cotas, no para sustituir el cronómetro. [39]

67.5 Problema 5. Dos barreras y un hilo sin salida

Enunciado. Un bloque carga un tile compartido, calcula con él y vuelve a usar la misma memoria para el siguiente tile. Un hilo no tiene salida válida y retorna antes de la primera barrera. Otro elimina la segunda barrera porque «ya hemos calculado». Identifica los dos riesgos.

Solución. El retorno puede impedir la participación colectiva requerida y, además, omitir una carga que otros hilos necesitan. El hilo sin salida final no es necesariamente un hilo sin responsabilidad. Debe respetar el protocolo del bloque y enmascarar sus accesos apropiadamente.

Eliminar la segunda barrera permite que hilos rápidos sobrescriban memoria compartida mientras otros siguen leyendo la etapa anterior. La primera protege que la etapa esté lista antes de consumir; la segunda protege que haya terminado de consumirse antes de reutilizar. Son relaciones temporales distintas. Una ejecución favorable no demuestra ausencia de carrera.

67.6 Problema 6. Dos lotes y una media mal calculada

Enunciado. Un lote tiene dos tokens válidos con pérdidas 1 y 3; otro tiene seis tokens con pérdida 2 cada uno. Calcula la media global y la media de medias. Después cambia las pérdidas del segundo lote a 4 y repite.

Solución. En el primer caso ambos lotes tienen media 2, de modo que las dos formas dan 2. Ese ejemplo no detecta el error. En el segundo, la media global es $(1 + 3 + 6 \cdot 4)/8 = 3,5$, mientras la media de medias es $(2 + 4)/2 = 3$.

La diferencia aparece porque se dio el mismo peso a grupos con distinta cantidad de datos. Para una métrica media por token se acumula suma de pérdidas y número de tokens válidos. En entrenamiento distribuido hay que aplicar la misma disciplina al combinar contribuciones; una normalización equivocada puede cambiar la magnitud del gradiente al variar el tamaño del lote o el número de réplicas.

67.7 Problema 7. Un caché aparentemente válido

Enunciado. Se guarda una biblioteca compilada usando como clave únicamente el texto C. Después se cambia de compilador y de flags, pero el texto no cambia. ¿Es correcto reutilizarla?

Solución. No está justificado por esa clave. El artefacto depende también del toolchain, opciones, arquitectura y ABI relevantes. Incluso si la biblioteca pudiera cargarse, ya no

demostraría el experimento que se cree estar realizando. Una clave de caché debe representar las entradas que afectan al resultado, y un manifiesto debe permitir saber cuál se utilizó.

Tampoco conviene escribir directamente sobre un archivo final que otro proceso puede estar cargando. La compilación se realiza en un temporal y se publica al terminar de forma segura. El sistema de caché forma parte de la reproducibilidad, no es únicamente una optimización de comodidad.

67.8 Problema 8. Qué significa un máximo con empate

Enunciado. La pérdida es el máximo de $(2, 2, 1)$. El sistema reparte el gradiente a partes iguales entre máximos. ¿Cuál es el vector devuelto? ¿Demuestra un error que una perturbación positiva del primer elemento observe pendiente uno?

Solución. La convención devuelve $(0, 5; 0, 5; 0)$. En el empate no existe una derivada ordinaria única. Una perturbación positiva del primero rompe el empate y selecciona otra región, donde la pendiente respecto a ese elemento es uno. Esa observación no contradice la convención de subgradiente declarada en el punto de empate.

La prueba debe separar puntos suaves, donde diferencias finitas son una herramienta directa, de fronteras no diferenciables, donde hay que comprobar la política elegida. Si una biblioteca usa otra convención, ambas pueden ser defendibles; no se exige igualdad entre políticas distintas sin acordar el contrato.

67.9 Problema 9. Compresión de pesos y memoria de entrenamiento

Enunciado. Un modelo guarda pesos en dos bytes por parámetro. Una persona multiplica ese tamaño por el número de parámetros y afirma que ha calculado toda la memoria necesaria para entrenar con AdamW. ¿Qué falta?

Solución. Como mínimo hay que considerar gradientes, momentos, posibles pesos maestros, activaciones guardadas, workspace y estado del runtime. Los tipos de cada categoría pueden ser distintos. La compresión de pesos reduce una categoría; no garantiza la misma reducción del total.

También hay que distinguir el pico de una fase del tamaño final de un checkpoint. Un checkpoint puede omitir activaciones que existen durante el paso. Un proceso puede reservar una arena mayor que los tensores lógicamente vivos. Medir ambas cosas con el nombre «memoria del modelo» crea confusión.

67.10 Problema 10. Interpretar una tabla de un artículo

Enunciado. Un artículo reporta aceleraciones entre 0,8 y 3,2 en su suite. Alguien resume «es tres veces más rápido». Da una reformulación precisa y una pregunta que harías antes de portarlo.

Solución. La reformulación conserva variabilidad: «en las operaciones y condiciones estudiadas, el candidato incluye casos más lentos que la referencia y casos con mejoras de hasta 3,2 veces». Hace falta conocer agregado, formas, precisión y hardware antes de atribuir una mejora típica.

Una pregunta de port es si el mecanismo ganador depende de una instrucción o layout exclusivo de la arquitectura evaluada. Otra es si el coste de búsqueda y compilación está amortizado. La interpretación científica conserva el dominio; no sustituye una distribución por su máximo favorable.

CAPÍTULO 68

Glosario razonado y conexiones que conviene conservar

68.1 De la intención al programa

Tensor. Datos organizados por ejes y forma. En este curso no significa automáticamente un objeto físico multidimensional en memoria: su almacenamiento puede ser lineal y su vista traducir coordenadas.

Operación elemento a elemento. Cada salida se calcula con elementos correspondientes de entradas, después de aplicar broadcasting. No necesita por definición mezclar distintas posiciones de una reducción.

Reducción. Combina contribuciones de uno o más ejes. Requiere identidad o política para el caso vacío, orden numérico y una salida por combinación de los ejes conservados.

IR. Representación intermedia del programa. Debe tener semántica, tipos y reglas de validez; no es simplemente cualquier estructura de datos que contenga nombres de operaciones.

UOp. Nombre usado aquí para un nodo de operación de bajo nivel o de una IR unificada, inspirado en el recorrido del curso. El UOp educativo de Lumbre no es la clase exacta de `tinygrad` ni tiene toda su semántica.

DAG. Grafo dirigido sin ciclos. Permite ordenar dependencias de expresiones puras. Los bucles de control necesitan una representación de recurrencia, no un ciclo arbitrario introducido en el evaluador topológico.

SSA. Organización donde cada nombre estático tiene una definición. Facilita seguir productores y usos; no elimina alias ni convierte memoria mutable en una expresión pura.

Reescritura. Sustitución de una parte de la representación por otra que preserva el contrato bajo hipótesis. Una identidad real no es automáticamente una reescritura válida de FP32 estricto.

Lowering o bajada. Traducción a una representación con decisiones más concretas. Debe definir qué operaciones desaparecen, cuáles aparecen y qué invariantes conserva.

Renderer. Parte que escribe una representación destino, como C. Puede apoyarse en análisis anteriores y en un compilador externo que todavía realizará selección de instrucciones.

68.2 De las coordenadas a la máquina

Shape. Tamaño de cada eje lógico. Dos tensores con el mismo número de elementos pueden tener formas diferentes y admitir operaciones distintas.

Stride. Avance en almacenamiento al aumentar una coordenada. Un stride cero permite lecturas repetidas; uno negativo exige un origen lógico apropiado.

Layout. Regla completa que relaciona coordenadas con almacenamiento o propiedad por participantes. Incluye más información que la forma.

Tile. Región de trabajo o datos que se procesa como unidad. Su tamaño afecta reutilización, bordes y recursos; no es necesariamente un bloque físico de GPU.

Upcasting. En el contexto del compilador, convertir parte de un eje de iteración en trabajo explícito de varios valores o registros. No debe confundirse con convertir un número a un tipo de más precisión.

SIMD. Una instrucción opera sobre varias posiciones de datos. Tener un bucle con varios elementos no demuestra que el compilador haya generado una instrucción vectorial.

SIMT. Modelo donde varios hilos ejecutan de forma coordinada instrucciones sobre sus datos. La divergencia y las operaciones colectivas requieren comprender quién participa.

Memoria compartida o local al grupo. Almacenamiento usado para cooperación de un grupo de ejecución. El nombre concreto depende del entorno; no equivale siempre a «local» en otros lenguajes o APIs.

Barrera. Relación de sincronización entre participantes bajo un alcance. No es un remedio universal: debe corresponder a los productores y consumidores correctos.

Tensor core o instrucción matricial. Capacidad especializada para operaciones sobre fragmentos. Impone contratos de tipos, layouts y participación; no es una llamada mágica para cualquier matriz.

Kernel de cálculo. Unidad de código que realiza trabajo numérico, a menudo en un acelerador. No es sinónimo del kernel de Linux ni de un controlador.

Runtime. Gestiona buffers, módulos, argumentos, ejecución y finalización. Su corrección es necesaria aunque la fórmula del kernel sea perfecta.

Firmware. Software de control próximo al dispositivo. Puede interpretar protocolos o inicializar componentes; no reemplaza automáticamente las funciones de una biblioteca de ML.

68.3 Del programa al aprendizaje

Autodiferenciación. Transformación que aplica reglas de derivación a un programa. No es estimar todos los gradientes mediante diferencias finitas ni pedirlos a un LLM.

VJP. Producto de un vector de contribuciones de salida por el jacobiano en la orientación del modo inverso. Permite propagar una pérdida sin materializar todo el jacobiano.

Gradiente acumulado. Suma de contribuciones por todos los caminos de uso. Es indispensable para nodos compartidos, broadcasting y embeddings repetidos.

Checkpoint. Estado persistido suficiente para el objetivo de recuperación declarado. Reproducir una trayectoria exige más que los pesos: optimizador, paso, datos y azar pueden importar.

Token. Unidad de la secuencia del modelo. Puede representar un carácter, bytes o una pieza aprendida. Las pérdidas por token no son comparables entre tokenizaciones sin más análisis.

Logit. Puntuación anterior a normalizar probabilidades. No es ya una probabilidad ni tiene que estar entre cero y uno.

Atención causal. Atención que restringe las claves disponibles a posiciones permitidas por el pasado. La máscara debe coincidir en forward, backward e inferencia.

KV cache. Estado de claves y valores conservado durante generación para evitar recomputación. No es el mismo caché que guarda una biblioteca compilada.

Prefill y decode. Procesar inicialmente una secuencia y generar posteriormente nuevos tokens. Tienen formas de trabajo y límites de memoria diferentes.

MoE. Arquitectura que selecciona expertos para parte del cómputo. Añade decisiones de agrupación y comunicación; parámetros totales y activos dejan de ser la misma cifra.

68.4 De la observación a la conclusión

Benchmark. Experimento con frontera medida, carga y referencia. Un número sin esas condiciones no permite interpretar rendimiento.

Ablación. Comparación que retira o cambia un mecanismo para estudiar su efecto. Modificar a la vez cinco componentes no aísla una causa.

Oráculo. Referencia usada para comprobar una salida. También puede tener limitaciones; conviene combinar implementaciones y argumentos cuando la propiedad importa.

Prueba diferencial. Comparación de implementaciones sobre entradas concretas. Detecta discrepancias, pero no garantiza por sí sola todos los casos posibles.

Resultado negativo. Evidencia de que una hipótesis no se cumple en las condiciones estudiadas. Bien documentado puede ser más informativo que una mejora seleccionada sin controles.

SOTA. Estado del arte respecto a una tarea, fecha, suite y condiciones. No es una propiedad que se obtenga por usar una arquitectura moderna o una GPU reciente.

El mapa mental final

Una intención matemática se expresa como programa; el compilador la transforma; el runtime la ejecuta sobre una máquina; las pruebas y medidas permiten evaluar qué se ha conservado y qué se ha mejorado. El entrenamiento añade gradientes, estado y datos, pero no elimina ninguna de esas obligaciones.

CAPÍTULO 69

CPU y CUDA en la misma máquina: de la prueba a la medida

Disponer de una GPU cambia las preguntas que podemos contestar. Ya podemos compilar el mismo grafo para dos destinos, comprobar sus resultados y observar cuánto cuesta ejecutarlo en cada uno. El objetivo de este capítulo es construir una comparación que puedas repetir y explicar. Los informes que acompañan al libro incluyen tanto la primera validación de CUDA como una campaña posterior con tareas ejecutadas en secuencia.

69.1 El entorno que realmente ejecutó el código

La máquina de validación ofrece una NVIDIA GeForce RTX 5070 Ti Laptop de 12 GB. Dentro de Ubuntu 24.04 en WSL2 usamos Python 3.12.3, GCC 13.3.0, NumPy 2.3.5 y pytest 9.0.2. El controlador observado es 591.86; el compilador CUDA es `nvcc` 12.9.86 y el destino es `sm_120`. Puedes consultar el inventario completo en `code/reports/validation_20261001/environment.json`.

Hay tres piezas que conviene distinguir al diagnosticar una ejecución. El controlador permite utilizar el dispositivo. El toolkit aporta el compilador y sus cabeceras y bibliotecas. El runtime de Lumbre genera y carga una biblioteca anfitriona que reserva memoria, copia datos y lanza kernels. Ver una GPU en `nvidia-smi` solo comprueba una parte de esa cadena.

En esta máquina también existía un toolkit Windows 12.4. La validación de Lumbre utilizó el toolkit Linux 12.9 dentro de WSL, porque el runtime suministrado carga bibliotecas POSIX. Un `nvcc` encontrado en una terminal Windows no identifica por sí mismo el compilador que utilizará Python dentro de WSL. Registra el ejecutable que resuelve cada entorno.

Desde una terminal WSL y con el entorno Python del proyecto activado, comprueba:

```
python --version
cc --version
command -v nvcc
nvcc --version
nvidia-smi
```

Si usas el toolkit aislado de la validación local, su directorio es `~/.local/opt/cuda-12.9.1/bin`. En otra instalación utiliza la ruta de tu propio toolkit. El valor de `LUMBRE_CUDA_ARCH` debe corresponder a tu dispositivo; el valor siguiente describe la RTX validada.

```
export LUMBRE_CUDA_ARCH=sm_120
export LUMBRE_CACHE="$HOME/.cache/lumbre-comparacion"
```

Qué significa seleccionar un backend

La variable `LUMBRE_CUDA_ARCH` configura el destino del compilador CUDA. Para seleccionar el dispositivo también debes pasar `backend=cuda` a `Program`, `-backend cuda` a los ejemplos o `-backend=cuda` a `pytest`. Una variable de arquitectura no convierte una ejecución CPU en una ejecución CUDA.

69.2 La escalera de aceptación

El primer escalón comprueba tamaños pequeños y bordes. Con bloques de 128 hilos, 257 elementos requieren tres bloques: se lanzan 384 hilos y 127 posiciones deben quedar enmascaradas. También interesa probar 127, 128 y 129 para cruzar el límite del bloque. El tamaño cero comprueba que el renderer omite el kernel cuando la salida está vacía; lanzar una grid de tamaño cero sería otra operación.

El segundo escalón comprueba matrices irregulares. En una GEMM por tiles, un hilo cuya salida queda fuera de la matriz puede seguir participando en cargas compartidas y barreras. Las pruebas deben ejercer bordes en M, N y K, broadcasting entre lotes y una dimensión de reducción vacía admitida por el contrato. El kernel compartido de Lumbre usa tiles de 16 por 16 y enmascara las cargas de los tiles incompletos.

El tercer escalón comprueba operaciones compuestas y gradientes. Las pruebas existentes incluyen movimientos, reducciones, gather/scatter, softmax, convolución por composición, atención online y causalidad del decoder. Finalmente, el entrenamiento debe mantener pérdidas y gradientes finitos, guardar sus pesos y reanudar el estado del optimizador y del generador de números aleatorios.

Desde `code/`, ejecuta cada backend de forma explícita:

```
python -m pytest -q --backend=cpu
python -m pytest -q --backend=cuda
python examples/validate_book.py --backend cpu \
    --output reports/comparacion/ejemplos_cpu.json
python examples/validate_book.py --backend cuda \
    --output reports/comparacion/ejemplos_cuda.json
```

La batería registrada superó 61 pruebas en cada selección de backend, sin omisiones. Esa cifra corresponde a la batería completa: el tokenizador, los mapas lógicos y la convolución C independiente mantienen su trabajo en el host. Los `Program` tensoriales seleccionan CUDA cuando se pide ese backend. El archivo `tests/conftest.py` implementa esta selección; un compilador o dispositivo ausente provoca fallos en las pruebas que lo necesitan.

69.3 Comparar valores y gradientes antes de comparar segundos

Para un mismo grafo, utiliza las mismas entradas, dtypes y formas. La comprobación elemento a elemento de una referencia finita suele expresar dos márgenes:

$$|y_{\text{CUDA}} - y_{\text{CPU}}| \leq \text{atol} + \text{rtol}|y_{\text{CPU}}|.$$

La tolerancia absoluta protege la comparación cerca de cero. La relativa escala con la magnitud de la referencia. Una tolerancia debe apoyarse en la operación, la precisión y la longitud de la reducción. Aumentarla hasta ocultar un índice incorrecto destruye el valor de la prueba.

En el decoder de depuración de 2.736 parámetros, las historias de veinte pasos registradas en CPU y CUDA difieren como máximo en $4.76837158203125\text{e-}7$ en la pérdida. Ese resultado pertenece a esa configuración y esas entradas. Las reducciones y las transformaciones de coma flotante pueden producir diferencias que crezcan a lo largo de un entrenamiento de muchas actualizaciones.

La igualdad exacta tiene otro papel en la prueba de reanudación. Dentro de cada backend, compara diez actualizaciones continuas con seis seguidas de cuatro al cargar el checkpoint. Deben coincidir los pesos, los momentos de AdamW, el contador de pasos, la configuración y el estado del RNG. Los informes `resume_cpu.json` y `resume_cuda.json` de la primera revisión registran esa igualdad. No se exige identidad bit a bit entre los checkpoints CPU y CUDA de dos entrenamientos completos.

Localizar una divergencia

Un modelo guarda pesos finitos, pero su pérdida CUDA se separa mucho de la CPU desde el primer paso. Compara primero el forward con los mismos pesos y el mismo batch. Después compara la pérdida, los gradientes por parámetro y la actualización del optimizador. Si el forward ya diverge, inspeccionar el muestreo de texto no resolverá la primera diferencia.

69.4 Medir con una frontera explícita

El benchmark de Lumbre construye el grafo, compila el programa y carga entradas antes de medir. Ejecuta tres calentamientos y después cronometra `run(read=[])`. La medida incluye la llamada Python y la sincronización que hace el runtime; excluye la compilación, la carga inicial de entradas y la copia de los resultados al host. Su unidad de observación es una llamada residente al programa.

En CUDA, una medida de host que solo cronometra el lanzamiento puede terminar antes que el trabajo de la GPU. Lumbre sincroniza al final de `run`, de modo que esta medida incluye la finalización del programa. Aun así, no equivale al tiempo aislado de una instrucción del dispositivo ni al de todo el proceso Python.

Para repetir la campaña, desde `code/` y con la arquitectura configurada:

```
python examples/bench_kernels.py --backend cpu --repeat 30 \
--output reports/comparacion/kernels_cpu.json
python examples/bench_kernels.py --backend cuda --repeat 30 \
--output reports/comparacion/kernels_cuda.json
```

Espera a que termine una orden antes de lanzar la siguiente. La campaña incorporada a este capítulo ejecutó sus tareas en secuencia en el mismo host WSL. No fijó frecuencias, potencia ni carga del escritorio: las cifras son observaciones locales que conviene repetir antes

de defender una mejora de rendimiento. Los JSON conservan las treinta muestras, mínimo, máximo, mediana, error numérico y metadatos del programa.

En GEMM, el campo **shape** usa el orden **[M,N,K]**: la entrada A tiene forma **[M,K]** y B tiene forma **[K,N]**. En atención la forma es **[B,H,T,D]**. La tabla utiliza microsegundos por llamada residente y redondea la mediana para facilitar la lectura.

Operación y forma	Variante	CPU (us)	CUDA (us)
gemm 31x47x65	reference	329.94	464.43
gemm 31x47x65	blocked	97.92	205.93
gemm 128x128x128	reference	12840.66	364.82
gemm 128x128x128	blocked	1644.77	423.72
gemm 256x256x256	reference	106237.30	415.52
gemm 256x256x256	blocked	12203.57	426.81
attention 1x2x16x32	dense	49.18	1583.98
attention 1x2x16x32	online	40.28	581.51
attention 1x2x64x32	dense	828.06	954.86
attention 1x2x64x32	online	671.03	550.33
attention 1x2x128x32	dense	3143.98	1313.48
attention 1x2x128x32	online	2369.11	896.29

Las formas pequeñas permiten estudiar cuánto pesa el coste de llamada y sincronización frente a la cantidad de aritmética. Al aumentar una dimensión, vuelve a medir: la relación entre memoria, paralelismo y trabajo cambia. Una variante denominada **blocked** tampoco tiene que ganar en todas las formas o en todos los dispositivos. La implementación CPU bloquea bucles; la CUDA utiliza memoria compartida y barreras. Comparten el contrato matemático, pero realizan trabajos de gestión distintos.

Para calcular un cociente comparable, divide la mediana CPU por la mediana CUDA de la misma fila. Un resultado mayor que uno indica menor tiempo CUDA en ese ensayo; uno menor que uno indica menor tiempo CPU. Conserva ambas latencias y las muestras originales. Un cociente aislado omite la escala del trabajo y la variación de las medidas.

69.5 Entrenar la misma configuración en ambos destinos

La configuración principal tiene vocabulario 24, lote 2, contexto 32, dimensión 64, dos capas, dos cabezas de consulta, una cabeza de claves/valores y dimensión intermedia 128. Usa atención online y GEMM bloqueada: 75.584 parámetros, 447 unidades de kernel y una arena planificada de 2.100.160 bytes.

```
python examples/train_decoder.py --backend cpu --steps 600 \
  --dim 64 --layers 2 --context 32 --batch 2 \
  --heads 2 --kv-heads 1 --hidden 128 \
  --online --gemm blocked --output reports/comparacion/decoder_cpu
python examples/train_decoder.py --backend cuda --steps 600 \
  --dim 64 --layers 2 --context 32 --batch 2 \
  --heads 2 --kv-heads 1 --hidden 128 \
  --online --gemm blocked --output reports/comparacion/decoder_cuda
```


Los reportes de esta campaña están en `code/reports/book_gpu_extension/`. Cada ejecución conserva su historia de entrenamiento, configuración, checkpoint y evaluación. El tiempo del bucle excluye compilación, evaluación y generación posteriores; incluye el trabajo que realiza el entrenador para alimentar los lotes y actualizar los parámetros.

Backend	Pasos	Pérdida final	Evaluación	Bucle (s)
cpu	600	0.2746565	0.3321578	18.744
cuda	600	0.2508018	0.3293002	28.685

El plan contiene muchos kernels pequeños. Tras cada `run`, el runtime sincroniza; las asignaciones del optimizador también requieren operaciones explícitas. Una GPU puede resolver una GEMM grande con rapidez y dedicar una fracción considerable del paso a gestionar cientos de lanzamientos y transferencias pequeñas. Para explicar el tiempo total, inspecciona el plan, el tamaño de cada operación y las copias del entrenador. El número de parámetros por sí solo no identifica el cuello de botella.

La arena contabiliza los buffers que administra Lumbre. No es la memoria total que verás en `nvidia-smi`: ese inventario incluye otros usos del dispositivo y del runtime. Del mismo modo, un entrenamiento que reduce su pérdida sobre las seis frases repetidas del corpus de demostración verifica aprendizaje y ejecución, pero requiere otra evaluación para estudiar generalización lingüística.

69.6 WMMA: una comprobación con alcance propio

La demostración `kernels/wmma_demo.cu` multiplica un tile de 16 por 16 con entradas FP16 y acumulación FP32. Un warp participa en las operaciones colectivas de carga, multiplicación y escritura. El programa comprueba errores de reserva, transferencia, lanzamiento y sincronización, y compara la salida con una referencia calculada en el host.

La RTX de validación ejecutó este programa con error absoluto máximo cero para los datos deterministas de la demostración. Esos valores tienen una estructura numérica sencilla; el cero observado no garantiza error cero para cualquier entrada FP16. Cambiar valores, distribuciones y escalas forma parte de la siguiente prueba.

El entrenamiento de Lumbre utiliza el renderer tensorial y su GEMM compartida. La demostración WMMA es un programa separado. Integrarla requiere reconocer la operación, adaptar formas y tipos, resolver los tiles de borde y verificar el error de la nueva ruta. Conserva una comparación contra el kernel anterior para detectar una conversión o un layout incorrectos.

69.7 Qué entregar al terminar la comparación

Tu informe debe permitir que otra persona reconstruya el experimento. Incluye el commit del código, el entorno, el destino CUDA, las formas, dtypes y semillas, las órdenes exactas y la política de caché. Adjunta resultados numéricos y muestras de tiempo. Identifica qué incluye cada reloj y qué trabajo quedó fuera.

Un experimento siguiente puede estudiar una sola hipótesis: fusionar operaciones elemento a

elemento, agrupar transferencias del optimizador o reducir sincronizaciones conservando las dependencias. Mantén el caso original, comprueba valores y gradientes y repite la medida. Una mejora que solo funciona al reutilizar por accidente un buffer no supera la prueba de corrección.

Puerta de salida

Puedes completar este capítulo cuando reproduces una prueba de bordes, explicas una comparación numérica CPU/CUDA, localizas las treinta muestras de un benchmark y distingues el tiempo de una llamada residente del tiempo de entrenamiento. También debes poder señalar qué evidencia valida WMMA y qué parte del renderer sigue utilizando la GEMM compartida.

Kernels portables y estado persistente: dos lecturas de DeepSeek

Los capítulos anteriores permiten formular dos preguntas separadas: cómo ejecutar una multiplicación con eficiencia en otra arquitectura y cuánto estado conservar al generar texto. Las dos afectan al coste de un sistema, pero requieren contratos y pruebas diferentes. Este capítulo propone una lectura guiada y un experimento pequeño que puedes reproducir en CPU.

70.1 DeepGEMM-Ascend: el contrato público y la representación física

DeepGEMM-Ascend se publicó para Ascend 950. Su documentación declara compatibilidad de API con DeepGEMM y kernels BF16, FP8 y FP4, además de operaciones de atención y MoE. Requiere CANN 9.20, `torch_npu`, Python y herramientas C++20. La revisión consultada es 8491bbb4b8c02a094a2318965f50c70438a3e73c. [24]

El repositorio advierte que los factores de escala UE8M0 se empaquetan por pares a lo largo de K dentro de enteros de 16 bits, con almacenamiento MN-major. Compartir una función Python no garantiza que el layout de sus buffers sea idéntico entre arquitecturas. [24]

Aquí solo disponemos de una GPU NVIDIA. La lectura del código Ascend no constituye una ejecución de esos kernels. Las medidas publicadas por sus autores pertenecen a su plataforma; no se incorporan como resultados locales.

Separar tres contratos

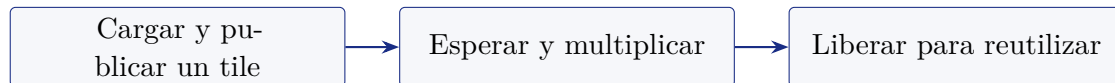
El contrato matemático define formas, tipos y operación. El de almacenamiento añade strides, alineaciones y metadatos de cuantización. El de ejecución determina quién produce un buffer, quién lo consume y cuándo puede reutilizarse. Una API común necesita preservar los tres, aunque oculte decisiones del dispositivo.

70.2 Leer un kernel industrial sin empezar por todos sus detalles

Abre `deep_gemm/include/deep_gemm/bf16_gemm.hpp` en el commit citado. Sus parámetros separan dimensiones de bloque, dimensiones de las operaciones MAD y cantidades de etapas. El fuente declara almacenamiento L1 y L0, comprueba capacidades mediante `static_assert` y utiliza un scheduler para repartir bloques. El bucle de reducción coordina copias y multiplicación mediante esperas y notificaciones. [70]

Relaciona esa estructura con la GEMM compartida del tutorial CUDA: primero localiza el dominio lógico [M,N,K]; después identifica dónde se coloca una pieza de A y B, quién la consume y qué autoriza a sobrescribirla. El código consultado distingue copias hacia L1, movimientos hacia L0 y acumulación matricial. El epílogo gestiona la salida y admite configuraciones diferentes. [70]

No necesitas memorizar cada plantilla para dibujar una dependencia. Para un tile, marca tres estados: libre, cargado y consumido. Con dos buffers, el productor puede preparar el siguiente tile mientras el consumidor trabaja sobre el anterior, siempre que exista un protocolo de propiedad que impida adelantarse al uso del buffer.



El archivo `ascend/sync.hpp` ofrece helpers de señales entre pipelines y barreras entre partes del kernel. Sus identificadores describen participantes del hardware Ascend. No deben sustituirse mecánicamente por una barrera CUDA de bloque: primero hay que reconstruir qué dependencia expresa cada señal. [71]

Diseñar el descriptor de un kernel externo

Imagina que Lumbre delega una GEMM a una biblioteca. Escribe los campos del descriptor: formas, dtype de entradas y acumulador, layout, stride, factores de escala, scratch necesario y stream o cola. Añade qué comprobación rechaza una entrada incompatible antes de llamar al kernel. La solución debe incluir también vida de los buffers y estado de error.

70.3 Presupuestar KV antes de hablar de compresión

En atención autoregresiva, guardar claves y valores anteriores evita recalcularlos en cada paso. Para un modelo hipotético con L capas, T tokens, H cabezas KV, dimensión D y b bits por elemento, el almacenamiento denso ideal de K y V es:

$$B_{KV} = 2LTHD \frac{b}{8}.$$

El factor dos corresponde a K y V. Esta cuenta no incluye escalas, alineación, tablas de páginas ni buffers de trabajo. Tampoco representa automáticamente una arquitectura que comprime secuencias o comparte estado entre capas.

Para L=24, T=32768 y D=64, pasar de 16 a cuatro cabezas KV reduce por cuatro la cuenta ideal a igual precisión. Pasar después de 16 a cuatro bits reduce otra vez por cuatro el contenido numérico. El programa `tutorials/research/cache_lab.py` comprueba:

Cabezas KV	Bits	Bytes ideales	MiB
16	16	3221225472	3072
4	16	805306368	768
4	4	201326592	192

MiB significa 2^{20} bytes. La fila de cuatro bits es aritmética de almacenamiento, no una

implementación FP4. Una cuantización real necesita especificar cómo codifica valores y escalas, cuánto error introduce y qué instrucciones realizan la conversión.

70.4 Qué añade la lectura de DeepSeek-V4.1-Flash

El informe DeepSeek-V4.1-Flash, arXiv:2609.19969v1, combina una arquitectura causal encoder-decoder, reutilización de KV entre capas en CSA2 y caché FP4. Sus autores reportan 890 bytes por token para la caché global residente en HBM. Esa cifra no representa toda la memoria de inferencia. [72]

El trabajo también propone SWA Bounded Replay: reconstruir estados de atención de ventana mediante un sufijo acotado y aceptar una aproximación. Los autores distinguen esa reconstrucción del estado exacto y señalan posibles límites en casos no evaluados. Nos interesa esa distinción entre ahorrar almacenamiento y preservar semántica exacta; aquí no reproducimos el modelo ni sus resultados. [72]

Para tu compilador, separa la memoria de pesos, KV global, ventanas locales, activaciones temporales y metadatos. Anota además si un dato vive en GPU, host o almacenamiento persistente. Reducir una partida no elimina las otras. El coste de recuperar una conversación puede incluir transferencia y recomputación, aunque el siguiente token utilice pocos kernels.

70.5 Un experimento de reconstrucción exacta y aproximada

Construiremos un sistema propio, mucho más pequeño que un Transformer. Cada capa sustituye un valor por la media causal de hasta W valores de la capa anterior. Con L capas, la última salida depende de un máximo de $1 + L(W - 1)$ entradas originales. Cada capa amplía el alcance hacia atrás en $W-1$ posiciones.

Para $L=3$ y $W=4$, el sufijo necesario para la última salida contiene diez entradas. El programa compara el resultado de una secuencia completa de 24 cuadrados con reconstruir solo sus diez últimas entradas o sus cuatro últimas. El operador y sus condiciones de borde se definen en el fuente; no simulan CSA2.

```
python tutorials/research/cache_lab.py
```

La secuencia completa y el sufijo de diez producen 384. El sufijo de cuatro produce aproximadamente 457,347222; su error absoluto es 73,347222. Las aserciones exigen identidad dentro de tolerancia para el sufijo suficiente y una diferencia observable para el sufijo corto.

El ejemplo muestra por qué el contexto necesario puede acumularse a través de capas. Una ventana local de cuatro no implica que un sufijo original de cuatro reconstruya el estado profundo. Si aceptas una aproximación, necesitas medir su impacto sobre las salidas que importan, no describirla como una optimización que conserva el programa exacto.

Una política de caché con pruebas

Supón que un runtime expulsa estados de una conversación y luego la reanuda. Define una prueba de salida exacta si promete reconstrucción exacta. Si utiliza una aproximación, registra el punto de reanudación, el contexto rehecho, el error frente a la referencia y una métrica de la tarea. Incluye casos donde cambie el punto de corte: una política puede comportarse de forma distinta en dos prefijos de la misma conversación.

70.6 Un proyecto que sí cabe en esta entrega

Puedes extender el decoder educativo con KV incremental exacta antes de explorar compresión. Empieza con una capa y pesos fijos. Compara los logits de cada posición calculados con el prefijo completo frente a los de la ruta incremental. Después añade capas, GQA y reanudación de la caché. Mantén el tamaño pequeño hasta entender las discrepancias.

La integración de DeepGEMM-Ascend requeriría hardware Ascend y una validación propia. El experimento de medias causales solo necesita NumPy. Los tutoriales CUDA y tinygrad aportan prácticas ejecutadas en los dispositivos disponibles; estos tres alcances quedan separados en los informes del repositorio.

Contrato de UOps: de la especificación a una IR verificable

71.1 Tres representaciones con responsabilidades distintas

El syllabus define `UOp(op, src:tuple[UOp, ...], arg)`. Este capítulo explica cómo razonar sobre ese contrato sin imponer clases, lenguaje ni biblioteca. Las propiedades `dtype`, `shape` y `addrspace` se infieren a partir de la operación, sus fuentes y su argumento. Una implementación puede memorizarlas, pero no debe permitir que una caché contradiga el cálculo de tipos.

Conviene distinguir el grafo puro con formas, el grafo que hace explícitos índices y efectos, y la secuencia final del renderer. El primero expresa el modelo y permite autodiff; el segundo conserva dependencias de memoria y rangos; la última decide el orden de emisión. Confundir esas responsabilidades hace que una mejora del renderer termine cambiando el significado del modelo.

Lumbre tiene una IR propia con forma y dtype almacenados, operaciones tensoriales y planificación específica. No implementa literalmente todo el catálogo del syllabus ni un lector universal `uop v1`. Aquí se especifica el trabajo que debe realizar el estudiante y se explica su relación con los mecanismos del libro. Los ejemplos de UOps son notación didáctica salvo cuando se reproduce expresamente el fragmento de intercambio original.

71.2 Hash-consing con tipos, ámbitos y efectos

Dos nodos puros con la misma operación, fuentes canónicas y argumento semántico pueden compartir identidad. Esto permite que $(x+2)*(x+2)$ contenga una sola suma. El argumento debe preservar toda distinción relevante: un literal booleano no es un entero, aunque el lenguaje anfitrión compare `True` e `1` como iguales. Una representación robusta de constantes incluye etiqueta de tipo y representación del valor.

Si el contrato conserva cero con signo, las constantes FP32 deben distinguir sus bits. Para NaN hay que decidir qué representaciones se admiten y si se canonicalizan. No uses igualdad ordinaria de coma flotante como definición universal de identidad estructural: NaN no es igual a sí mismo y ceros de signo distinto pueden compararse iguales.

`PARAM` necesita identidad de ámbito y posición. Los parámetros cero de dos funciones diferentes no se vuelven el mismo argumento por compartir nombre. El syllabus no detalla `ParamArg` ni `CallArg`; el estudiante debe fijar sus campos y comprobarlos. Una posibilidad es que `ParamArg` identifique ámbito, slot y tipo esperado, con tamaño y espacio de direcciones cuando proceda. Esa es una decisión de implementación, no un campo añadido al estándar original.

La tabla de internring no puede borrar eventos distintos de memoria. Dos **ALLOC** de igual tamaño pueden requerir almacenamiento diferente. Dos escrituras textualmente iguales pueden ocurrir en momentos diferentes. Para mantener hash-consing, la identidad o las dependencias del evento deben formar parte de su estructura; alternatively, la implementación debe representar efectos en una capa que no los confunda con expresiones puras. En ambos casos, explica cómo se conserva esa distinción.

Una clave que parece suficiente

Una caché utiliza (**op**, **src**, **arg**) y **arg=1** tanto para un booleano verdadero como para el entero uno. ¿Qué prueba revela el fallo?

Solución. Construye ambos literales y consulta su **dtype**; deben seguir siendo distintos. Después construye **MUL** booleano e integer con entradas del tipo correspondiente. La regla de internring tiene que incluir la etiqueta semántica del literal, aunque la sintaxis pública del nodo siga teniendo tres campos.

71.3 Semana 1: operaciones puras y sus dominios

Familia	Operaciones	Obligación del verificador
Entradas y aplicación	PARAM, CALL, CONST	Resolver ámbito y aridad; CALL.src[0] es el cuerpo y el resto son argumentos
Unarias	CAST, NEG, RECIP, EXP2, LOG2	Fijar tipo de resultado, conversiones y dominios admitidos
Binarias	ADD, MUL, CMPLT, CMPNE, MAX	Validar tipos y producir bool en comparaciones
División entera	FLOORDIV, FLOORMOD	Definir redondeo, divisor cero y desbordamiento
Selección	WHERE	Condición booleana y ramas con tipos compatibles

El syllabus enumera operaciones y tipos, pero no publica todas las sobrecargas posibles. No deduzcas, por ejemplo, que **ADD** booleano significa **OR**: solo establece las equivalencias **MUL=AND**, **MAX=OR** y **CMPNE=XOR**. Las combinaciones adicionales deben rechazarse o especificarse expresamente. Tampoco aceptes conversiones implícitas diferentes en el evaluador y en el código C.

El plegado de constantes debe seguir la precisión del destino. Si **f32** significa redondeo binario32 tras cada operación, evaluar una cadena en doble precisión y convertir al final puede dar un resultado diferente. El tratamiento de FMA, infinitos, NaN, subnormales y ceros con signo se documenta antes de introducir reglas que dependan de él. Para un primer entregable, es válido limitar el dominio si el rechazo es verificable y está declarado.

RECIP(x) expresa el recíproco. **EXP2(x)** y **LOG2(x)** usan base dos. Reducir **LOG2(EXP2(x))** a **x** necesita hipótesis sobre rango, precisión y dominio; no es una identidad universal del programa FP32. **WHERE** selecciona valores, pero el syllabus no determina por sí solo si sus

ramas se evalúan de forma perezosa. Esa distinción será decisiva para una carga enmascarada de la semana 4.

71.4 División hacia menos infinito y C

Para enteros con divisor distinto de cero, una definición de división por suelo exige $a = bq + r$, con $q = \lfloor a/b \rfloor$. Así, para $a = -7$ y $b = 3$, resultan $q = -3$ y $r = 2$. Con $a = 7$ y $b = -3$, resultan $q = -3$ y $r = -2$. El resto tiene el signo del divisor o es cero.

La división entera de C trunca hacia cero. Por eso imprimir `/` y `%` directamente no implementa esos ejemplos. Si se parte de cociente y resto truncados, y el resto es no nulo con signo distinto del divisor, la corrección es restar uno al cociente y sumar el divisor al resto. Antes debe tratarse el divisor cero y el caso del mínimo entero dividido entre menos uno, cuyo resultado no cabe en el mismo entero con signo.

Tabla mínima del evaluador

Para $(7,3)$, $(-7,3)$, $(7,-3)$ y $(-7,-3)$, los pares cociente/resto por suelo son $(2,1)$, $(-3,2)$, $(-3,-2)$ y $(2,-1)$. Comprueba también divisiones exactas. Las cuatro filas deben coincidir entre plegador y programa generado.

71.5 Reescribir de abajo arriba y saber cuándo detenerse

Visita primero las fuentes, reconstruye el nodo con sus versiones normalizadas y aplica reglas locales orientadas. Si una regla crea otro nodo reducible, vuelve a considerar el resultado con una política explícita. Conserva memoización por nodo y detecta ciclos o agotamiento de un presupuesto de reescritura. Un presupuesto sirve como diagnóstico; no demuestra que las reglas terminen.

Para una familia sencilla, puede justificarse terminación mediante una medida que disminuya: cantidad de operaciones constantes pendientes y, después, tamaño de una representación canónica. No mezcles $a+b \rightarrow b+a$ con su inversa. Ordenar operandos con una clave estable evita ese ciclo cuando la conmutatividad esté permitida por el contrato numérico.

Una regla como $x*0 \rightarrow 0$ requiere atención en flotantes: con x infinito, la expresión original puede producir NaN. Cambiar asociación de sumas también modifica redondeos. Mantén reglas exactas para el dominio elegido y reglas bajo relajación numérica identificadas por separado. Si el modo cambia, debe cambiar la clave del artefacto compilado.

El plegador puede evaluar un `CALL` puro con argumentos constantes ligando los `PARAM` del cuerpo al entorno correspondiente. Evita sustituir parámetros de una llamada anidada por coincidencia de nombre. La recursión general, si se admite, necesita su propia semántica; no se obtiene gratuitamente de un evaluador recursivo de DAG.

71.6 Semana 2: tipo de valor, forma y dirección

`dtype` responde qué clase de dato representa un nodo; `shape` describe sus ejes lógicos; `addrspace` distingue memoria de valores de cálculo. Una dirección a un escalar `f32` y un valor

f32 no son intercambiables aunque compartan forma (). void representa ausencia de valor utilizable, como una escritura.

Operación	Contrato del syllabus	Consecuencia práctica
BUFFER	Buffer global persistente, nunca definido dentro de CALL	Su vida supera una invocación
ALLOC	Almacenamiento local de llamada	No puede escaparse una dirección que ya no sea válida
STACK	Nuevo eje inicial; fuentes de igual forma	Con m fuentes de forma S, la salida tiene forma (m,) + S
INDEX	Consume ejes por la izquierda mediante fuentes índice	Con k índices válidos, quedan los ejes desde k en adelante
LOAD	Una fuente	Obtiene un valor desde memoria válida
STORE	Dos fuentes	Escribe un valor en el destino compatible
AFTER	AFTER(buf, store)	Hace depender el uso posterior del buffer de la escritura
LINEAR	Fuentes en orden antes del renderer	La emisión debe respetar la secuencia elegida

La tabla no impone una sintaxis textual para cada constructor. En particular, el orden destino/valor de STORE, los campos de tipos y el tratamiento exacto de STACK en cada espacio deben declararse al completar la especificación del proyecto. En los ejemplos siguientes usaremos destino primero y valor después.

Si x tiene forma (2,3), indexarlo con i produce forma (3,); indexarlo con i,j produce (). Hay que verificar índices enteros y, cuando se conocen estáticamente, sus límites. Una forma escalar tiene un elemento. Una forma (0,) tiene cero. Confundirlas provoca reservas y accesos incorrectos.

71.7 Una función C sin bucles y una dependencia real

El primer programa de memoria puede tener esta salida C, bajo el contrato de que p apunta a un elemento válido y el llamador admite modificarlo:

```
float incrementar(float *p) {
    float anterior = p[0];
    p[0] = anterior + 1.0f;
    return p[0];
}
```

En una notación conceptual, indexa el buffer, carga el valor anterior, construye la suma, crea STORE(destino, suma) y alimenta la lectura final desde una dirección obtenida del buffer dependiente AFTER(buffer, escritura). Así la lectura posterior tiene una dependencia explícita. Volver a cargar desde la dirección original sin ese vínculo permite que una optimización la trate indebidamente como la primera lectura.

Con entrada 4, la primera llamada devuelve 5 y deja 5 en memoria; la segunda devuelve 6. Esta prueba detecta un runtime que no conserva buffers o un compilador que congela entradas como constantes. También debe existir una prueba con dos direcciones alias si la ABI permite aliasing. Si lo prohíbe, ese contrato debe verificarse o mantenerse como precondition pública, no esconderse en el renderer.

Un orden topológico cualquiera conserva dependencias del grafo, pero no inventa las que falten. **LINEAR** es una lista final de emisión, no una reparación de aliasing. El verificador comprueba que toda fuente necesaria aparece antes, que los efectos ordenados conservan su relación y que ninguna operación fuera de ámbito usa un local de una llamada terminada.

71.8 Semana 3: rangos, cierre y acumulación

RANGE(end, nombre) representa el intervalo semiabierto desde cero hasta end. **END**(passthrough, range) vincula un resultado o efecto con el cierre de ese rango. El nombre por sí solo no debe mezclar dos ámbitos. Verifica correspondencia entre aperturas y cierres, anidamiento y disponibilidad de los valores usados después.

La reducción suma exige inicialización una sola vez por salida. Este pseudocódigo fija el significado de una GEMM rectangular; no prescribe una representación interna de acumuladores:

```
para i en [0, M):
  para j en [0, N):
    acc = 0
    para k en [0, K):
      acc = acc + A[i,k] * B[k,j]
    C[i,j] = acc
```

Inicializar acc dentro del rango k dejaría solo el último producto; inicializarlo antes del rango j mezclaría columnas. La prueba con K igual a cero, si se admite, obliga a producir ceros para la suma sin leer entradas. La prueba con M o N igual a cero no debe escribir salida alguna.

Para una convolución 1D válida sin invertir el filtro, usa $x=[1,2,3,4]$ y $w=[2,-1]$. La suma de productos sobre ventanas produce $[0,1,2]$. Esta convención es correlación cruzada, habitual en redes neuronales; si se quiere convolución matemática, hay que invertir el filtro. El nombre de la operación no sustituye esa decisión.

71.9 El fragmento de intercambio proporcionado

El syllabus aporta este ejemplo literal:

```
# uop v1 256x256 GEMM
%0 = buffer 65536 : dtype=f32 slot=0
%1 = buffer 65536 : dtype=f32 slot=1
%2 = reshape %0, (256, 256, 1)
%3 = reshape %1, (256, 1, 256)
%4 = mul %2, %3
%5 = reduce %4 : op=add pop=1
```

Son seis nodos. Las dos vistas conservan 65.536 elementos cada una. Bajo broadcasting, el

producto lógico tiene forma (256,256,256): contiene 16.777.216 valores si se materializa, 64 MiB en FP32. Una bajada que acumule directamente puede evitar esa materialización.

Para interpretar la cuenta necesitamos declarar qué ejes elimina `pop=1` y cómo se interpretan los buffers. **En este desarrollo didáctico adoptamos reducción de los ejes iniciales.** Si `%0` contiene $A[k, i]$ y `%1` contiene $B[k, j]$, el producto es $A[k, i] * B[k, j]$ y la salida es la suma sobre k . Con ambos buffers vistos como matrices fila a fila, esto representa $A^T B$. Para representar AB con A almacenada como $[i, k]$, se requiere la permutación correspondiente. El nombre GEMM sigue siendo adecuado, pero las dimensiones cuadradas no revelan esta diferencia.

El fragmento también presupone broadcasting en `mul`. Una IR que exija formas idénticas debe insertar `EXPAND` durante la normalización. El ejemplo no especifica una gramática completa, escapes, codificación de literales, salidas, ABI ni política de errores. El alumno debe acordar esos detalles antes de prometer interoperabilidad.

71.10 Un intercambio que se pueda comprobar

Como extensión docente, el lector admite solo versiones declaradas, identificadores definidos una vez y referencias a nodos ya definidos. El impresor recorre un orden topológico estable y conserva tipos y argumentos. Las salidas se declaran con una convención documentada; no se asume para todos los programas que el último nodo sea la única salida.

La prueba de ida y vuelta compara estructura semántica: leer, imprimir, volver a leer y comprobar operación, fuentes, argumentos, tipos y salidas. Los números `%0`, `%1` pueden cambiar por una reenumeración válida. No deben cambiar slots externos, constantes, relaciones de dependencia ni disposición lógica de datos.

Añade casos inválidos: versión desconocida, referencia inexistente, identificador duplicado, operación no admitida, tipo incompatible, reshape que cambia elementos y slot incompatible con la firma. El diagnóstico debe señalar nodo y propiedad. Estos casos prueban el verificador; no son excusas para aceptar silenciosamente una interpretación distinta en cada backend.

CAPÍTULO 72

De rangeify al entrenamiento: preservar el contrato

72.1 Movimientos como mapas de coordenadas

RESHAPE cambia la descomposición de un índice lineal conservando el número de elementos. EXPAND replica lógicamente un eje de tamaño uno; su coordenada de entrada siempre es cero. PERMUTE reordena ejes mediante una permutación válida. FLIP convierte la coordenada i de un eje de longitud n en $n-1-i$. SHRINK desplaza la coordenada por el inicio de una región. PAD amplía el dominio y distingue posiciones interiores de posiciones de relleno.

Estas operaciones describen valores, no necesariamente nuevas reservas. Por ejemplo, permutar una matriz $(2,3)$ a $(3,2)$ transforma la coordenada de salida (j,i) en la entrada (i,j) . Al consumirla en un cálculo, el compilador puede componer ese mapa con el acceso original. No es obligatorio copiar la matriz para obtener su traspuesta lógica.

Para PAD, el predicado de validez forma parte del mapa. Con x de longitud tres, un cero a la izquierda y dos a la derecha, la salida tiene longitud seis. Su coordenada de entrada es $i-1$, válida solo si $1 \leq i < 4$. En otro caso el resultado es cero. No ejecutes primero una carga fuera de límites y después un WHERE: si las ramas se evalúan ávidamente, el acceso inválido ya ocurrió. La bajada debe producir carga predicada o control de flujo que garantice seguridad.

72.2 Reducción por número de ejes

El syllabus define REDUCE mediante cantidad de ejes y operación, sin completar aquí todos sus detalles. Adoptamos para los ejemplos una reducción de los p ejes iniciales. Una suma sobre (K,M,N) con $p=1$ da (M,N) . Para reducir otro conjunto, primero permuta los ejes deseados a la izquierda. Mantener o eliminar dimensiones unitarias es una decisión que debe reflejarse en el tipo de salida.

Una reducción suma tiene identidad cero. Para máximo, el dominio y el tipo determinan si existe una identidad admisible o si se rechaza un eje vacío. No inicialices indiscriminadamente el máximo a cero: sobre $[-4,-2]$ daría cero en lugar de -2 . Tampoco cambies orden de acumulación FP32 sin considerar la política numérica.

72.3 Derivar GEMM con un ejemplo rectangular

Sean $A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ y $B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}$. Para calcular AB con reducción inicial, transforma A de (M,K) a (K,M) , añade un eje final y transforma B de (K,N) a $(K,1,N)$. Expande ambos a (K,M,N) , multiplica y reduce k .

```

A: (2,3) -> PERMUTE(1,0) -> (3,2) -> RESHAPE(3,2,1)
B: (3,2) -> RESHAPE(3,1,2)
EXPAND ambos a (3,2,2)
MUL -> REDUCE(add, pop=1) -> (2,2)
resultado: [[58,64], [139,154]]

```

La coordenada lógica (k, i, j) se traduce a $A[i \cdot K + k]$ y $B[k \cdot N + j]$ en almacenamiento contiguo. Rangeify introduce los rangos i, j y k y un acumulador por salida. El producto lógico de doce elementos no obliga a reservar doce posiciones: los productos pueden consumirse inmediatamente.

Esta prueba rectangular detecta intercambios de ejes que una multiplicación de matrices cuadradas podría ocultar. Amplíala con valores distintos por fila y columna, K igual a uno y dimensiones no múltiplos del tile. Los ejemplos numéricos son deducciones exactas con estos enteros pequeños, no mediciones de rendimiento.

72.4 Derivar convolución sin un operador privilegiado

Una convolución 1D de entrada de longitud L y filtro de longitud F , sin padding y con paso uno, puede expresarse como suma de F vistas desplazadas multiplicadas por cada peso. Para el desplazamiento r , SHRINK selecciona $x[r:r+L-F+1]$. STACK agrupa esas vistas en forma $(F, L-F+1)$. El filtro se reinterpreta como $(F, 1)$, se expande, se multiplica y se reduce el eje inicial.

Así se obtiene el ejemplo $[0, 1, 2]$ del capítulo anterior sin introducir una primitiva opaca de convolución. El uso explícito de STACK puede hacer crecer el grafo con F ; una extensión con índices simbólicos evita desplegar filtros grandes. El ejercicio inicial busca demostrar semántica y composición, no escoger ya el mejor kernel industrial.

Con padding, dilatación o paso distinto de uno, escribe primero la coordenada `salida*paso + r*dilatacion - padding`. Después determina su máscara. El gradiente del filtro suma contribuciones de todas las ventanas y el de la entrada acumula las ventanas que la tocan; una copia de los índices del forward no basta para construir ambos.

72.5 Qué elimina rangeify y qué debe conservar

El resultado del pase expresa el cálculo con valores escalares, índices y rangos. Las operaciones aritméticas ya no portan los ejes tensoriales originales como una ejecución implícita. Las direcciones conservan información suficiente para generar accesos y el runtime conserva tamaños de buffers y restricciones. «Eliminar shapes de todas las ops» no significa olvidar cuántos bytes necesita el programa.

Valida tres aspectos por separado: toda coordenada admitida encuentra el valor correcto, ninguna coordenada inválida produce una carga o escritura, y cada salida se calcula la cantidad de veces exigida. Un programa puede respetar límites y aun así escribir dos veces una posición dejando otra sin calcular.

Conserva un dump antes y después del pase para un caso pequeño. La prueba diferencial ejecuta el grafo puro con un evaluador sencillo y el programa bajado con los mismos datos.

El evaluador de referencia no debería llamar al mismo generador de índices que pretende comprobar, porque reproduciría sus errores.

72.6 STAGE: pagar memoria para evitar trabajo repetido

Si un intermedio caro tiene dos consumidores, fusionarlo en ambos puede duplicar su cálculo. STAGE permite materializarlo y hacer que los consumidores lo indexen. Decide su tamaño, productor, consumidores, ubicación y vida. La reserva solo puede reutilizarse cuando todos los consumidores han terminado.

Supón que un intermedio cuesta C , y que escribirlo y leerlo para ambos consumidores cuesta T . Recomponerlo dos veces cuesta aproximadamente $2C$; materializarlo cuesta $C+T$. Si C supera T , materializar podría resultar favorable. Son costes orientativos: lanzamientos, caché, vectorización y capacidad de memoria pueden cambiar la decisión, por lo que el modelo debe contrastarse con medidas.

Una barrera de materialización no es equivalente a una regla de autodiff que detenga el gradiente. Mantén el grafo matemático puro para diferenciar; decide stages y efectos al bajar forward y backward. Si se inserta una anotación de stage antes, debe aclararse que no cambia la función y cómo la trata el diferenciador.

Dos consumidores de una reducción

Sea $s = \text{sum}(x \cdot x)$ y dos salidas $s+1$ y $s+2$. ¿Es mejor generar un kernel por salida?

Solución. Si cada kernel vuelve a recorrer x , duplica la reducción. Un plan calcula s una vez y lo comparte; otro fusiona las dos salidas en un productor común. Elegir depende de la representación y del destino. La prueba de rendimiento debe contar recorridos, bytes y lanzamientos, además del número de kernels.

72.7 Autodiff antes de introducir efectos

Recorre el grafo puro en orden topológico inverso y acumula contribuciones para cada entrada diferenciable. En $y = a \cdot a$, a recibe dos contribuciones. Sustituir la suma de gradientes por una asignación conserva solo una y produce un error por factor dos. Los tipos enteros y booleanos se usan para índices y condiciones; la entrega debe declarar qué nodos admiten derivación.

Para $z = x + b$ con x de forma (B, D) y b de forma $(D,)$, el gradiente de b suma sobre el eje de lote. En un EXPAND, el backward reduce ejes replicados. En una permutación, aplica la inversa. En un reshape, restaura la forma. En un flip, invierte el mismo eje. En padding y shrink, el backward recorta o rellena según corresponda. Acumula las contribuciones de vistas solapadas.

En un máximo con empate o una función no diferenciable, documenta la convención. No compruebes diferencias finitas exactamente en el punto de quiebre y después atribuyas el desacuerdo al compilador. Para funciones suaves y entradas pequeñas, compara gradientes analíticos, autodiff y diferencias centrales con una perturbación y tolerancia justificadas.

STORE, CALL, AFTER, RANGE y END quedan fuera del autodiff descrito por el syllabus. Aunque

un CALL matemáticamente puro pudiera diferenciarse mediante otra extensión, esa no es la ruta exigida aquí. Expande o expresa el modelo en el grafo puro admitido antes de diferenciar, y vuelve a aplicar planificación y rangeify al grafo resultante.

72.8 El primer entrenamiento: MNIST como entrega

Una propuesta mínima de laboratorio es un clasificador lineal con imágenes aplanadas de 784 componentes, pesos de forma (784,10) y sesgo (10,). Con lote B, los logits son $\mathbf{XW}+\mathbf{b}$, de forma (B,10). Este modelo deja visibles GEMM, broadcasting, reducción y gradientes. Una capa oculta puede añadirse después como extensión.

Usa entropía cruzada estable. Para una fila de logits $\mathbf{z}$ y etiqueta y , resta su máximo m antes de exponenciar y calcula $m + \log(\text{sum}(\exp(\mathbf{z}-m))) - \mathbf{z}[y]$. Con las primitivas del syllabus, $\exp(x)$ puede expresarse como $\text{EXP2}(x*\log2(e))$ y $\log(x)$ como $\text{LOG2}(x)*\ln(2)$, bajo la precisión acordada. La resta se expresa con ADD y NEG.

La selección de etiqueta puede formularse con una máscara one-hot externa y una reducción si todavía no existe gather diferenciable. Los datos y etiquetas son entradas; no se necesita derivar respecto a ellos para optimizar los pesos. Normaliza la pérdida una sola vez por lote y aplica una actualización simple antes de introducir un optimizador más complejo.

Primero comprueba un lote artificial, luego sobreajusta un conjunto diminuto de ejemplos y finalmente utiliza una partición de entrenamiento y evaluación. Registra procedencia de datos, normalización, semilla, tamaño de lote, pasos, pérdida y exactitud. No uses evaluación para ajustar indefinidamente el modelo. El libro no fija una exactitud conseguida: el estudiante aporta su resultado y la configuración que lo produce.

72.9 Semana 6: LOOP, UPCAST, THREAD y GROUP

Un rango LOOP se ejecuta mediante iteración. UPCAST expone varias posiciones como operaciones o acumuladores distintos, permitiendo desenrollado y posible vectorización. THREAD reparte trabajo entre trabajadores. Esta clasificación describe cómo ejecutar un eje; no autoriza a repartir una reducción con escrituras concurrentes sin combinar sus parciales.

Para sumar un vector, una propuesta segura reparte bloques disjuntos, calcula una suma privada por trabajador y combina las sumas en un orden definido. Usar `total += x[i]` desde todos los hilos sin sincronización introduce una carrera. Si cambia el orden FP32, la prueba compara según el contrato numérico, no exige identidad de bits accidental entre planes diferentes.

GROUP reúne múltiples resultados void sin establecer orden entre sus fuentes; su forma es (). Si dos salidas escriben posiciones independientes, pueden agruparse. Si una lee lo que otra escribe, hace falta una dependencia explícita. Ni el orden textual de argumentos de GROUP ni un renderer determinista sustituyen ese vínculo.

72.10 Contar operaciones y localizar el límite

Con la convención habitual de contar multiplicación y suma como dos operaciones, una GEMM M por K por N realiza aproximadamente $2MKN$ FLOPs. Para 256 en los tres ejes son 33.554.432 FLOPs. FLOPs es cantidad de trabajo; FLOP/s es tasa. Divide por segundos de la fase medida, no por tiempo de compilación salvo que estudies expresamente el coste completo.

Un tile de salida de 16 por 16 que recorre K necesita reutilizar cada valor cargado tantas veces como permita su organización. Aumentar tile puede mejorar reutilización y a la vez consumir más registros o caché. Mide formas pequeñas, grandes e irregulares; un único cuadrado no describe la carga del modelo.

La comparación CPU debe fijar hilos, precisión, biblioteca de referencia y fronteras de medición. Separa ejecución residente, preparación y extremo a extremo. Para declarar competitividad con el estado del arte, la referencia debe ser pertinente y los casos desfavorables deben permanecer en el informe. Los resultados CPU existentes de Lumbre no se convierten en esa afirmación por añadir una tabla de metas.

72.11 Atención online y el objetivo FlashAttention

El artículo FlashAttention estudia atención exacta organizada por tiles para reducir tráfico entre niveles de memoria. Ese principio conecta directamente con la semana de jerarquías de memoria. Implementar la recurrencia en CPU permite comprobar la matemática antes de abordar un kernel GPU especializado. [41]

Para una fila de scores, mantén máximo m , suma normalizadora l y numerador vectorial o . Al recibir otro bloque, actualiza el máximo, reescala el estado anterior y añade las contribuciones del bloque. El resultado final es o dividido por l . Atiende explícitamente el primer bloque no vacío y las filas completamente enmascaradas para evitar operaciones indeterminadas con infinitos.

La forma densa es un buen oráculo para longitudes pequeñas. Compara salidas y gradientes con máscara causal y longitudes no múltiplos del bloque. Un forward con menos memoria no garantiza un backward con la misma propiedad: la validación actual de Lumbre recompone operaciones densas en backward. Para alcanzar el hito completo hay que diseñar y medir también esa fase.

72.12 GPU: la dependencia lógica se convierte en ejecución

En las semanas 7–8, un eje de salidas independientes puede mapearse a bloques e hilos. Un eje de reducción necesita coordinación y una estrategia de parciales. Las dependencias AFTER y los tiempos de vida deben conservarse cuando un lanzamiento es asíncrono: regresar al host no implica que el dispositivo haya terminado de leer un temporal.

Una barrera de bloque solo coordina ese bloque. Las dependencias entre kernels necesitan el orden que establezca el runtime en sus colas o eventos. Las operaciones matriciales espe-

cializadas añaden restricciones de tipos, layouts y participación de hilos. El tutorial CUDA desarrolla ejercicios para convertir estos requisitos en criterios de selección y pruebas.

72.13 Formas simbólicas y el paso de entrenamiento

Una dimensión simbólica representa una familia de tamaños con restricciones. Si el kernel requiere K múltiplo de ocho, el compilador puede demostrarlo, comprobarlo al lanzar o escoger otro kernel. No debe reutilizar una variante suponiendo que el tamaño anterior sigue vigente. Incluye en la caché las decisiones que afectan al código: destino, tipos, especializaciones y modo numérico.

Antes de entrenar un LLM, fija el contrato de un paso: entradas y etiquetas desplazadas, máscara causal, pérdida normalizada, gradientes, actualización y contador. Un checkpoint de reanudación conserva pesos, estados del optimizador, contador y estados aleatorios o del flujo de datos necesarios. Guardar solo pesos permite inferencia o reinicio, pero no acredita continuar exactamente la misma trayectoria.

La aceptación de las semanas 9–10 incluye una prueba de causalidad, comprobación de gradientes en un caso pequeño, reanudación comparada y medición del paso completo. El objetivo de escala se elige según memoria y tiempo disponibles. Se conserva el modelo pequeño como regresión incluso cuando el proyecto final trabaje con un modelo mayor.

CAPÍTULO A

Código completo de Lumbre y sus pruebas

El código siguiente coincide con los archivos del paquete de esta edición. Los comentarios del software se conservan en inglés para facilitar trabajo técnico; el desarrollo didáctico del libro está en español. La revisión local valida CUDA en una RTX 5070 Ti Laptop; HIP sigue pendiente de hardware compatible.

A.1 examples/bench_kernels.py

```
"""Correctness first; resident microbenchmarks, not claims of SOTA.
Compile, transfers and result copies excluded from the resident hot timing.
"""

import argparse,json,platform,time,statistics
from pathlib import Path
import numpy as np
from lumbre import param,Program,online_attention,softmax
from lumbre.ir import node

def measure(p,repeat):
    for _ in range(3): p.run(read=[])
    ts=[]
    for _ in range(repeat):
        t=time.perf_counter_ns();p.run(read=[]);ts.append((time.perf_counter_ns()-t)*1e
        ↪ -9)
    return {'median_seconds':statistics.median(ts),'min_seconds':min(ts),'max_seconds':
    ↪ max(ts),'samples_seconds':ts}

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--repeat',type=int,default=20)
    ap.add_argument('--backend',choices=['cpu','cuda','hip'],default='cpu')
    ap.add_argument('--output',type=Path,default=Path('reports/kernels.json'));args=ap.
    ↪ parse_args()
    if args.repeat<3: raise ValueError('use at least 3 samples')
    rng=np.random.default_rng(17);records=[]
    for M,N,K in [(31,47,65),(128,128,128),(256,256,256)]:
        a=param('a',(M,K));b=param('b',(K,N));out=a@b
        data={'a':rng.normal(size=a.shape).astype('float32'),'b':rng.normal(size=b.shape)
        ↪ .astype('float32')}
        reference=data['a']@data['b']
        for strategy in ['reference','blocked']:
            with Program([out],backend=args.backend,gemm=strategy) as p:
                got=p.run(data)[0];np.testing.assert_allclose(got,reference,atol=1e-4,
                ↪ rtol=1e-4)
```

```

        timing=measure(p,args.repeat)
        records.append({'op':'gemm','shape':[M,N,K],'strategy':strategy,'
↪ max_abs_error':float(np.max(np.abs(got-reference))),**timing,'gflops':2*M*N*K/
↪ timing['median_seconds']/1e9,'compiler':p.report()})
    for T in [16,64,128]:
        D=32;q,k,v=[param(n,(1,2,T,D)) for n in ('q','k','v')]
        dense=softmax((q@k.transpose()/(D**0.5)+node('causal',(T,T)))@v
        online=online_attention(q,k,v)
        data={n:rng.normal(size=q.shape).astype('float32') for n in ('q','k','v')}
        with Program([dense],backend=args.backend,gemm='blocked') as p: reference=p.run(
↪ data)[0]
        for name,out in [('dense',dense),('online',online)]:
            with Program([out],backend=args.backend,gemm='blocked') as p:
                got=p.run(data)[0];np.testing.assert_allclose(got,reference,atol=2e-5,
↪ rtol=2e-5)
            records.append({'op':'attention','shape':[1,2,T,D],'strategy':name,'
↪ max_abs_error':float(np.max(np.abs(got-reference))),**measure(p,args.repeat),'
↪ compiler':p.report()})
    result={'system':platform.platform(),'backend':args.backend,'records':records,'scope'
↪ ':f'local {args.backend} resident microbenchmark; no cross-system comparison'}
    args.output.parent.mkdir(parents=True,exist_ok=True);args.output.write_text(json.
↪ dumps(result,indent=2))
    for r in records:print(r['op'],r['shape'],r['strategy'],r['median_seconds'],r['
↪ compiler'],['arena_bytes'])

if __name__=='__main__':main()

```

A.2 examples/train_decoder.py

```

"""Train a small real decoder with Lumbre-generated C; CUDA/HIP are optional.
The supplied original corpus is a debugging corpus, not evidence of useful LLM quality.
"""

import argparse
from dataclasses import asdict
import json
import hashlib
from pathlib import Path
import time
import numpy as np
from lumbre import Program
from lumbre.nn import Decoder,Config,adamw
from lumbre.tokenizer import ByteBPE

CORPUS = """el sol sale y la luna descansa.\nla casa tiene una puerta azul.\nel gato
↪ mira al rio y escucha el viento.\nuna idea se convierte en un programa.\nprimero
↪ comprobamos el resultado y despues medimos el tiempo.\nla memoria guarda datos y
↪ el compilador organiza el trabajo.\n"""

def main():
    ap=argparse.ArgumentParser()
    ap.add_argument("--backend",choices=["cpu","cuda","hip"],default="cpu")
    ap.add_argument("--steps",type=int,default=60)
    ap.add_argument("--dim",type=int,default=16)
    ap.add_argument("--layers",type=int,default=1)

```

```

ap.add_argument("--context",type=int,default=8)
ap.add_argument("--text",type=Path)
ap.add_argument("--byte-tokens",action="store_true")
ap.add_argument("--bpe-merges",type=int,default=0)
ap.add_argument("--batch",type=int,default=2)
ap.add_argument("--heads",type=int,default=2)
ap.add_argument("--kv-heads",type=int,default=1)
ap.add_argument("--hidden",type=int,default=0)
ap.add_argument("--online",action="store_true")
ap.add_argument("--gemm",choices=["reference","blocked"],default="reference")
ap.add_argument("--resume",type=Path)
ap.add_argument("--output",type=Path,default=Path("reports/decoder"))
args=ap.parse_args()
if args.steps < 1: raise ValueError("steps must be positive")
text=args.text.read_text(encoding="utf-8") if args.text else CORPUS*20
if args.bpe_merges<0: raise ValueError("bpe-merges must be nonnegative")
if args.byte_tokens or args.bpe_merges:
    split=int(0.9*len(text)); raw_train,raw_val=text[:split],text[split:]
    codec=ByteBPE.fit(raw_train,args.bpe_merges)
    alphabet=[piece.hex() for piece in codec.vocab]
    encode_text=codec.encode
    decode_tokens=lambda tokens:codec.decode(tokens,errors="replace")
    tokenizer_state=codec.state()
    train=np.asarray(codec.encode(raw_train),dtype=np.int32)
    val=np.asarray(codec.encode(raw_val),dtype=np.int32)
else:
    alphabet=sorted(set(text)); encode={c:i for i,c in enumerate(alphabet)}
    encode_text=lambda value:[encode.get(c,0) for c in value]
    decode_tokens=lambda tokens:"".join(alphabet[int(t)] for t in tokens)
    tokenizer_state={"kind":"character","alphabet":alphabet}
    ids=np.asarray([encode[c] for c in text],dtype=np.int32)
    split=int(0.9*len(ids)); train,val=ids[:split],ids[split:]
if min(len(train),len(val)) <= args.context+1: raise ValueError("corpus is too small
↳ for train/validation")
cfg=Config(vocab=len(alphabet),dim=args.dim,heads=args.heads,kv_heads=args.kv_heads,
↳ layers=args.layers,hidden=args.hidden or 2*args.dim,context=args.context,batch=
↳ args.batch,online=args.online)
metadata={"model":asdict(cfg),"alphabet":alphabet,"tokenizer":tokenizer_state,
        "corpus_sha256":hashlib.sha256(text.encode("utf-8")).hexdigest()}
model=Decoder(cfg)
outputs,updates,optinit=adamw(model)
print("Compiling",model.parameter_count(),"parameters...",flush=True)
program=Program(outputs,backend=args.backend,gemm=args.gemm)
rng=np.random.default_rng(19)
def batch(data):
    starts=rng.integers(0,len(data)-cfg.context,size=cfg.batch)
    x=np.stack([data[s:s+cfg.context] for s in starts])
    y=np.stack([data[s+1:s+cfg.context+1] for s in starts])
    return x,y
for name,value in {**model.initial,**optinit}.items():
    if name in program.params: program.set(name,value)
start_step=0
if args.resume:
    archive=np.load(args.resume,allow_pickle=False)
    expected=json.dumps(metadata,sort_keys=True)
    if "configuration" not in archive or str(archive["configuration"])!=expected:
        raise ValueError("checkpoint configuration or vocabulary mismatch")

```

```

    for name in list(model.weights)+list(optinit): program.set(name,archive[name])
    start_step=int(archive["step"])
    rng.bit_generator.state=json.loads(str(archive["rng_state"]))
    args.output.mkdir(parents=True,exist_ok=True)
    history=[]; start=time.perf_counter()
    for step in range(start_step+1,start_step+args.steps+1):
        x,y=batch(train)
        feed={"tokens":x,"labels":y,"learning_rate":np.array(0.004,dtype="float32"),
              "adam_bc1":np.array(1-0.9**step,dtype="float32"),"adam_bc2":np.array
↪ (1-0.999**step,dtype="float32")}
        loss,norm=program.run(feed,read=outputs[:2])
        if not np.isfinite(loss) or not np.isfinite(norm): raise RuntimeError("nonfinite
↪ training state")
        program.assign(updates)
        row={"step":step,"loss":float(loss),"grad_norm":float(norm)};history.append(row)
        if step==1 or step%10==0: print(row,flush=True)
    elapsed=time.perf_counter()-start
    # Reading persistent weights is an explicit checkpoint operation, not training
↪ through NumPy.
    saved={}
    for name in list(model.weights)+list(optinit):
        u=program.params[name]
        if args.backend=="cpu": saved[name]=program._view(u).copy()
        else:
            import ctypes
            a=np.empty(u.shape,dtype=u.dtype)
            program._check(program.lib.lm_copy(a.ctypes.data,program.address+program.
↪ offsets[u],a.nbytes,2))
            saved[name]=a
    saved["step"]=np.array(start_step+args.steps)
    saved["configuration"]=np.array(json.dumps(metadata,sort_keys=True))
    saved["rng_state"]=np.array(json.dumps(rng.bit_generator.state))
    np.savez(args.output/"checkpoint.npz",**saved)
    # A separate inference graph has no optimizer and does not modify weights.
    inference=Program([model.loss,model.logits],backend=args.backend,gemm=args.gemm)
    for name,value in model.initial.items():
        if name in inference.params: inference.set(name,saved.get(name,value))
    vals=[]
    for _ in range(4):
        x,y=batch(val)
        v=inference.run({"tokens":x,"labels":y},read=[model.loss])[0]; vals.append(float
↪ (v))
    prompt=encode_text("el sol ")
    for _ in range(80):
        ctx=([0]*cfg.context+prompt)[-cfg.context:]
        x=np.tile(np.asarray(ctx,dtype=np.int32),(cfg.batch,1))
        logits=inference.run({"tokens":x,"labels":x},read=[model.logits])[0][0,-1]
        p=np.exp((logits-logits.max())/0.8); p/=p.sum()
        prompt.append(int(rng.choice(len(alphabet),p=p)))
    report={"config":asdict(cfg),"parameters":model.parameter_count(),"backend":args.
↪ backend,
           "steps":args.steps,"train_seconds_excluding_compile":elapsed,"history":
↪ history,
           "validation_loss":float(np.mean(vals)),"sample":decode_tokens(prompt),
           "compiler":program.report(),"corpus":"external UTF-8" if args.text else "
↪ original repeated teaching corpus",

```

```

        "warning":"debugging-scale training; not SOTA quality, not an independent
        ↪ language benchmark"}
        (args.output/"report.json").write_text(json.dumps(report,ensure_ascii=False,indent=2)
        ↪ ,encoding="utf-8")
        (args.output/"config.json").write_text(json.dumps(metadata,ensure_ascii=False,indent
        ↪ =2),encoding="utf-8")
        print("Validation",report["validation_loss"],"seconds",elapsed,flush=True)
        print(report["sample"],flush=True)
        inference.close();program.close()

if __name__=="__main__": main()

```

A.3 examples/validate_book.py

```

"""Execute the small worked numerical examples printed in the book."""
import argparse
import json
from pathlib import Path
import numpy as np
from lumbre import param, Program, gradients
from lumbre.nnops import conv2d

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--backend', choices=('cpu', 'cuda', 'hip'), default='cpu')
    ap.add_argument('--output', type=Path, default=Path('reports/book_examples.json'))
    args = ap.parse_args()
    records = []
    x, w, b = param("book_X", (2, 2)), param("book_W", (2, 2)), param("book_b", (2,))
    y = x @ w + b
    loss = (y * y).mean()
    gs = gradients(loss, [x, w, b])
    with Program([y, loss, *gs], backend=args.backend, gemm="blocked") as p:
        out = p.run({"book_X": [[1, 2], [3, 4]], "book_W": np.eye(2), "book_b": [1, -1]})
        ↪
    np.testing.assert_allclose(out[1], 7.5)
    np.testing.assert_allclose(out[2], [[1, .5], [2, 1.5]])
    np.testing.assert_allclose(out[3], [[7, 5], [10, 7]])
    np.testing.assert_allclose(out[4], [3, 2])
    records.append({"example": "linear_mse", "status": "PASS", "loss": float(out[1])})

    x, w = param("book_image", (1, 1, 3, 3)), param("book_filter", (1, 1, 2, 2))
    y = conv2d(x, w)
    gs = gradients(y.sum(), [x, w])
    with Program([y, *gs], backend=args.backend, gemm="blocked") as p:
        out = p.run({"book_image": np.arange(1, 10).reshape(1, 1, 3, 3),
                    "book_filter": np.array([1, 0, 0, -1]).reshape(1, 1, 2, 2)})
    np.testing.assert_allclose(out[0], -4)
    np.testing.assert_allclose(out[1][0, 0], [[1, 1, 0], [1, 0, -1], [0, -1, -1]])
    np.testing.assert_allclose(out[2][0, 0], [[12, 16], [24, 28]])
    records.append({"example": "convolution_gradients", "status": "PASS"})

    x, t = param("book_rx", (3,)), param("book_rt", (3,))
    w, b = param("book_rw", ()), param("book_rb", ())

```

```
e = x * w + b - t
loss = (e * e).mean()
gw, gb = gradients(loss, [w, b])
uw, ub = w - .1 * gw, b - .1 * gb
with Program([loss, gw, gb, uw, ub], backend=args.backend) as p:
    old = p.run({"book_rx": [0, 1, 2], "book_rt": [1, 3, 5], "book_rw": 0, "book_rb":
↪ 0})
    np.testing.assert_allclose(old[1], -26 / 3, rtol=1e-6)
    np.testing.assert_allclose(old[2], -6, rtol=1e-6)
    p.assign({"book_rw": uw, "book_rb": ub})
    new = p.run(read=[loss])[0]
    assert new < old[0]
records.append({"example": "regression_step", "status": "PASS", "before": float(old
↪ [0]), "after": float(new)})
path = args.output
path.parent.mkdir(parents=True, exist_ok=True)
path.write_text(json.dumps(records, indent=2), encoding="utf-8")
print("Three worked examples: PASS")

if __name__ == "__main__":
    main()
```


A.4 examples/verify_launch.py

```

"""Write a finite mapping study; this does not run GPU hardware."""
import argparse
import json
from pathlib import Path
from lumbre.passport import Launch, verify

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--output", type=Path, default=Path("reports/passport.json"))
    args = ap.parse_args()
    good = [verify(Launch(m, n, tm, tn))
             for m, n in [(0, 7), (1, 1), (16, 16), (17, 23), (31, 65)]
             for tm, tn in [(4, 8), (8, 8), (16, 16)]]
    assert all(r["status"] == "LOGICAL_MAP_VALIDATED" for r in good)
    bad = verify(Launch(3, 5, 2, 4), lambda y, x, s: y * s.rows + x)
    assert bad["status"] == "FAILED"
    report = {"valid_cases": good, "deliberate_bug": bad,
             "scope": "CPU enumeration of index maps; not a GPU backend port"}
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print("15 valid maps; wrong-stride bug rejected; no GPU execution")

if __name__ == "__main__":
    main()

```

A.5 kernels/conv.c

```

/* Original teaching reference: NCHW x OIHW grouped convolution.
   FP32, positive sizes/strides/dilations, symmetric non-negative padding.
   Caller owns allocation. Returns zero, or -1 for an invalid contract.
   Size products must fit size_t; callers must check allocation products. */
#include <stddef.h>
int conv2d_nchw(const float *x, const float *w, const float *bias,
               float *out, int B, int C, int H, int W, int O,
               int KH, int KW, int SH, int SW, int PH, int PW,
               int DH, int DW, int groups) {
    if (!x || !w || !out || B<=0 || C<=0 || H<=0 || W<=0 || O<=0 ||
        KH<=0 || KW<=0 || SH<=0 || SW<=0 || DH<=0 || DW<=0 ||
        PH<0 || PW<0 || groups<=0 || C%groups || O%groups) return -1;
    const long long eh=(long long)DH*(KH-1)+1;
    const long long ew=(long long)DW*(KW-1)+1;
    const long long nh=(long long)H+2LL*PH-eh;
    const long long nw=(long long)W+2LL*PW-ew;
    if (nh<0 || nw<0) return -1;
    const size_t OH=(size_t)(nh/SH+1), OW=(size_t)(nw/SW+1);
    const int CG=C/groups, OG=O/groups;
    for (int b=0; b<B; b++) for (int o=0; o<O; o++)
        for (size_t y=0; y<OH; y++) for (size_t z=0; z<OW; z++) {
            float acc=bias ? bias[o] : 0.0f;
            const int group=o/OG;

```

```

    for (int ci=0;ci<CG;ci++) for (int ky=0;ky<KH;ky++)
        for (int kx=0;kx<KW;kx++) {
            const long long iy=(long long)y*SH-PH+(long long)ky*DH;
            const long long ix=(long long)z*SW-PW+(long long)kx*DW;
            if (iy>=0 && iy<H && ix>=0 && ix<W) {
                const size_t xi=(((size_t)b*C+group*CG+ci)*H+(size_t)iy)*W+(size_t)ix;
                const size_t wi=(((size_t)o*CG+ci)*KH+ky)*KW+kx;
                acc += x[xi]*w[wi];
            }
        }
    out[(((size_t)b*O+o)*OH+y)*OW+z]=acc;
}
return 0;
}

```

A.6 kernels/wmma_demo.cu

```

// Complete one-warp WMMA example. Requires a compatible NVIDIA GPU/toolkit.
// Validated on RTX 5070 Ti Laptop (sm_120), CUDA 12.9; see validation reports.
#include <cuda_runtime.h>
#include <cuda_fp16.h>
#include <mma.h>
#include <cmath>
#include <cstdio>
#include <stdexcept>
#include <string>

static void check(cudaError_t status, const char* operation) {
    if (status != cudaSuccess)
        throw std::runtime_error(std::string(operation)+" : "+cudaGetErrorString(status));
}

__global__ void tile16(const half* A, const half* B, float* C) {
    using namespace nvcuda;
    // Every lane of the only warp must execute the same collective operations.
    wmma::fragment<wmma::matrix_a,16,16,16,half,wmma::row_major> a;
    wmma::fragment<wmma::matrix_b,16,16,16,half,wmma::row_major> b;
    wmma::fragment<wmma::accumulator,16,16,16,float> c;
    wmma::fill_fragment(c, 0.0f);
    wmma::load_matrix_sync(a, A, 16);
    wmma::load_matrix_sync(b, B, 16);
    wmma::mma_sync(c, a, b, c);
    wmma::store_matrix_sync(C, c, 16, wmma::mem_row_major);
}

int main() {
    half *A=nullptr,*B=nullptr;
    float* C=nullptr;
    try {
        cudaDeviceProp property{};
        check(cudaGetDeviceProperties(&property,0),"device properties");
        if (property.major < 7) {
            std::fprintf(stderr,"This teaching example needs compatible WMMA support.\n")
        }
    }
}

```

```

        return 77;
    }
    std::printf("Device: %s; compute capability %d.%d\n",property.name,property.
↪ major,property.minor);
    half ha[256],hb[256]; float hc[256];
    for (int i=0;i<256;i++) {
        ha[i]=__float2half(((i*7) %13-6)*0.25f);
        hb[i]=__float2half(((i*3) %11-5)*0.125f);
    }
    check(cudaMalloc((void**)&A,sizeof(ha)),"allocate A");
    check(cudaMalloc((void**)&B,sizeof(hb)),"allocate B");
    check(cudaMalloc((void**)&C,sizeof(hc)),"allocate C");
    check(cudaMemcpy(A,ha,sizeof(ha),cudaMemcpyHostToDevice),"copy A");
    check(cudaMemcpy(B,hb,sizeof(hb),cudaMemcpyHostToDevice),"copy B");
    tile16<<<1,32>>>(A,B,C);
    check(cudaGetLastError(),"launch");
    check(cudaDeviceSynchronize(),"synchronize");
    check(cudaMemcpy(hc,C,sizeof(hc),cudaMemcpyDeviceToHost),"copy C");
    double error=0;
    for (int i=0;i<16;i++) for (int j=0;j<16;j++) {
        double reference=0;
        for (int k=0;k<16;k++)
            reference+=(double)__half2float(ha[i*16+k])*__half2float(hb[k*16+j]);
        error=std::fmax(error,std::fabs(hc[i*16+j]-reference));
    }
    check(cudaFree(A),"free A"); A=nullptr;
    check(cudaFree(B),"free B"); B=nullptr;
    check(cudaFree(C),"free C"); C=nullptr;
    std::printf("Maximum absolute error: %.9g\n",error);
    return error <= 1e-4 ? 0 : 1;
} catch(const std::exception& e) {
    if(A) cudaFree(A); if(B) cudaFree(B); if(C) cudaFree(C);
    std::fprintf(stderr,"%s\n",e.what());
    return 1;
}
}

```

A.7 lumbre/___init___py

```

from .ir import UOp, param, tensor, gradients, concat, gather, softmax, cross_entropy,
↪ online_attention
from .compiler import Program

```

A.8 lumbre/compiler.py

```

"""C/CUDA/HIP renderer, static scheduler, arena planner and JIT runtime.
Linux/POSIX host. CUDA/HIP routes need the corresponding toolkit and device.
Device validation records live in reports/validation_20261001.
"""
from __future__ import annotations
from collections import Counter
import ctypes as C

```

```

import hashlib
import json
import os
from pathlib import Path
import platform
import shutil
import subprocess
import tempfile
import time
import numpy as np
from .ir import UOp, topo, size

FUSIBLE = {"add","sub","mul","div","exp","log","sqrt","tanh","eq","lt","where",
           "reshape","expand","permute","slice","pad","concat","detach","onehot","causal"
           ↪ "}
CTYPES = {"float32":"float", "int32":"int32_t"}

def coordinates(shape, q):
    return [f"(({q})/{max(1,size(shape[i+1:])))}%{max(1,d)})" for i,d in enumerate(
    ↪ shape)]

def flatten(shape, coords):
    return "(" + "+".join(f"({c})*{size(shape[i+1:])}" for i,c in enumerate(coords)) + "
    ↪ " if shape else "0"

def bindex(out_shape, in_shape, q):
    cs = coordinates(out_shape,q)[len(out_shape)-len(in_shape):]
    return flatten(in_shape,["0" if d == 1 else c for d,c in zip(in_shape,cs)])

def align(x, a=64): return ((x+a-1)//a)*a

class Program:
    def __init__(self, outputs, backend="cpu", fuse=True, reuse=True, cache=None, gemm="
    ↪ reference"):
        self.outputs = tuple(outputs)
        if not self.outputs: raise ValueError("at least one output is required")
        if backend not in ("cpu","cuda","hip"): raise ValueError("backend must be cpu,
    ↪ cuda or hip")
        self.backend = backend
        if gemm not in ("reference","blocked"): raise ValueError("unknown GEMM strategy")
        ↪
        self.gemm = gemm
        order = topo(self.outputs)
        names = [u.arg for u in order if u.op == "param"]
        if len(set(names)) != len(names): raise ValueError("same input name used with
    ↪ different shapes/dtypes")
        self.params = {u.arg:u for u in order if u.op == "param"}
        users = Counter(s for u in order for s in u.src)
        mats, depth = set(self.outputs), {}
        for u in order:
            if u.op == "const" and u not in mats:
                depth[u] = 0; continue

```

```

        d = 1+max((depth[s] for s in u.src), default=0)
        if not fuse or u.op not in FUSIBLE or users[u] > 1 or d >= 4 or u in mats:
            mats.add(u); depth[u] = 0
        else: depth[u] = d
    # Calls consume contiguous materialized operands. Their boundaries are explicit.
    for u in order:
        if u.op == "attention" or (u.op=="matmul" and gemm=="blocked"):
            mats.update(u.src)
    self.materialized = mats
    self.kernels = [u for u in order if u in mats and u.op != "param"]
    self.ids = {u:i for i,u in enumerate(order)}
    self.inputs = {u:self.dependencies(u) for u in self.kernels}
    self.offsets, self.bytes = self.plan_arena(reuse)
    self.source = self.render()
    self.cache = Path(cache or os.getenv("LUMBRE_CACHE", Path.home()/.cache/lumbre")
    ↪ )
    self.cache.mkdir(parents=True,exist_ok=True)
    compiler = {"cpu":os.getenv("CC","cc"),"cuda":"nvcc","hip":"hipcc"}[backend]
    path = shutil.which(compiler)
    if not path: raise RuntimeError(f"required compiler not found: {compiler}")
    version = subprocess.run([path,"--version"],capture_output=True,text=True,check=
    ↪ True).stdout
    if backend == "cpu": flags = ["-O3","-std=c11","-shared","-fPIC","-ffp-contract=
    ↪ off"]
    elif backend == "cuda":
        arch = os.getenv("LUMBRE_CUDA_ARCH")
        if not arch: raise RuntimeError("set LUMBRE_CUDA_ARCH, e.g. sm_80, for your
    ↪ actual device")
        flags = ["-O3","-std=c++17","-shared","-Xcompiler","-fPIC",f"-arch={arch}"]
        else: flags = ["-O3","-std=c++17","-shared","-fPIC"]
        identity = json.dumps([self.source,path,version,flags,platform.machine(),
    ↪ platform.system()])
        self.digest = hashlib.sha256(identity.encode()).hexdigest()
        self.source_path = self.cache/(self.digest+{"cpu":".c","cuda":".cu","hip":".cpp"
    ↪ }[backend])
        self.library_path = self.cache/(self.digest+".so")
        self.source_path.write_text(self.source,encoding="utf-8")
        self.compile_seconds = 0.0
        if not self.library_path.exists():
            start = time.perf_counter()
            with tempfile.TemporaryDirectory(dir=self.cache) as temp:
                target = Path(temp)/"module.so"
                command = [path,*flags,str(self.source_path),"-o",str(target)]
                if backend == "cpu": command += ["-lm"]
                result = subprocess.run(command,capture_output=True,text=True,timeout
    ↪ =180)
                if result.returncode:
                    raise RuntimeError(f"compiler failed ({self.source_path}):\n{result.
    ↪ stderr[-12000:]}")
                os.replace(target,self.library_path)
                self.compile_seconds = time.perf_counter()-start
            self.lib = C.CDLL(str(self.library_path))
            self.lib.run.argtypes = [C.c_void_p]; self.lib.run.restype = C.c_int
            self._closed = False
            if backend == "cpu":
                self._storage = np.zeros(self.bytes+64,dtype=np.uint8)
                self.address = align(self._storage.ctypes.data)

```

```

    else:
        self.lib.lm_alloc.argtypes=[C.c_size_t]; self.lib.lm_alloc.restype=C.
↪ c_void_p
        self.lib.lm_free.argtypes=[C.c_void_p]; self.lib.lm_free.restype=C.c_int
        self.lib.lm_copy.argtypes=[C.c_void_p,C.c_void_p,C.c_size_t,C.c_int]; self.
↪ lib.lm_copy.restype=C.c_int
        self.lib.lm_sync.argtypes=[]; self.lib.lm_sync.restype=C.c_int
        self.address = self.lib.lm_alloc(max(self.bytes,64))
        if not self.address: raise RuntimeError("device allocation failed")
        self.initialized = set()
        self.ran = False

def dependencies(self, root):
    result, seen = [], set()
    def visit(u):
        if u in seen: return
        seen.add(u)
        if u is not root and u in self.materialized:
            result.append(u)
        else:
            for s in u.src: visit(s)
    visit(root)
    return tuple(result)

def plan_arena(self, reuse):
    offsets, end = {}, 0
    for u in self.params.values():
        offsets[u] = end; end += align(max(1,size(u.shape))*4)
    last = {u:i for i,u in enumerate(self.kernels)}
    for i,u in enumerate(self.kernels):
        for s in self.inputs[u]: last[s] = max(last.get(s,-1),i)
    for u in self.outputs: last[u] = len(self.kernels)
    active, free = [], []
    for i,u in enumerate(self.kernels):
        remaining = []
        for old,off,n in active:
            if reuse and last[old] < i: free.append((n,off))
            else: remaining.append((old,off,n))
        active = remaining
        need = align(max(1,size(u.shape))*4)
        candidates = sorted((n,off,j) for j,(n,off) in enumerate(free) if n >= need)
        if candidates:
            n,off,j = candidates[0]; free.pop(j)
            if n > need: free.append((n-need,off+need))
        else: off = end; end += need
        offsets[u] = off; active.append((u,off,need))
    return offsets, max(end,64)

def pointer(self,u): return f"p{self.ids[u]}"

def read(self,u,q,root):
    if u is not root and u in self.materialized: return f"{self.pointer(u)}[{q}]"
    op,s = u.op,u.src
    if op == "const": return float.fromhex(u.arg).hex()+"f"
    if op == "causal":
        T=u.shape[-1]
        return f"((({q})%{T})<=(({q})/{T}) ? 0.0f : -INFINITY)"

```

```

if op == "param": return f"{self.pointer(u)}[{q}]"
if op in ("reshape","detach"): return self.read(s[0],q,root)
if op == "expand": return self.read(s[0],bindex(u.shape,s[0].shape,q),root)
if op == "permute":
    cs = coordinates(u.shape,q)
    back = [cs[u.arg.index(i)] for i in range(len(u.shape))]
    return self.read(s[0],flatten(s[0].shape,back),root)
if op == "slice":
    cs = coordinates(u.shape,q); axis,start,_ = u.arg
    cs[axis] = f"({cs[axis]}+{start})"
    return self.read(s[0],flatten(s[0].shape,cs),root)
if op == "pad":
    cs = coordinates(u.shape,q)
    test = " && ".join(f"({c})>={l} && {c}<{l+d})" for c,d,(l,r) in zip(cs,s[0].
↪ shape,u.arg)) or "1"
    back = [f"({c})-{p[0]}" for c,p in zip(cs,u.arg)]
    v = self.read(s[0],flatten(s[0].shape,back),root)
    return f"(({test}) ? ({v}) : 0)"
if op == "concat":
    cs = coordinates(u.shape,q); axis=u.arg; cut=s[0].shape[axis]
    a = self.read(s[0],flatten(s[0].shape,cs),root)
    bs = cs.copy(); bs[axis]=f"({bs[axis]}-{cut})"
    b = self.read(s[1],flatten(s[1].shape,bs),root)
    return f"(({cs[axis]}<{cut}) ? ({a}) : ({b}))"
if op == "onehot":
    v = self.read(s[0],f"(({q})/{u.arg})",root)
    return f"(({v}) == (({q})%{u.arg}) ? 1.0f : 0.0f)"
args = [self.read(x,bindex(u.shape,x.shape,q),root) for x in s]
if op in ("add","sub","mul","div","eq","lt"):
    symbol={"add":"+","sub":"-","mul":"*","div":"/","eq":"==","lt":"<"}[op]
    return f"(({args[0]}){symbol}({args[1]}))"
if op in ("exp","log","sqrt","tanh"): return f"{op}f({args[0]})"
if op == "where": return f"(({args[0]}) ? ({args[1]}) : ({args[2]}))"
raise NotImplementedError(f"cannot inline {op}")

def body(self,u):
    out = self.pointer(u); op=u.op; s=u.src
    if op in FUSIBLE or op == "const": return f"{out}[q] = {self.read(u,'q',u)};"
    if op in ("sum","max"):
        shape = s[0].shape
        cs = coordinates(u.shape,"q")
        redshape = tuple(shape[i] for i in u.arg)
        rs = coordinates(redshape,"r")
        back = [rs[u.arg.index(i)] if i in u.arg else cs[i] for i in range(len(shape)
↪ )]
        v = self.read(s[0],flatten(shape,back),u)
        initial = "0.0f" if op == "sum" else "-INFINITY"
        update = f"acc += ({v});" if op == "sum" else f"acc = fmaxf(acc,({v}));"
        return f"float acc={initial};\nfor(int64_t r=0;r<{size(redshape)};r++) {{ {
↪ update} }}\n{out}[q]=acc;"
    if op == "matmul":
        a,b=s; M,K=a.shape[-2:]; N=b.shape[-1]
        batch=u.shape[:-2]
        ab = bindex(batch,a.shape[:-2],f"q/{max(1,M*N)}")
        bb = bindex(batch,b.shape[:-2],f"q/{max(1,M*N)}")
        ai = f"(({ab})*{M*K}+((q/{max(1,N)})%{max(1,M)})*{K}+k)"
        bi = f"(({bb})*{K*N}+k*{N}+q%{max(1,N)})"

```

```

        av,bv=self.read(a,ai,u),self.read(b,bi,u)
        return f"float acc=0.0f;\nfor(int64_t k=0;k<{K};k++) acc+={({av})*({bv});\n{
↪ out}[q]=acc;"
        if op == "gather":
            table,ids=s; V,D=table.shape
            iv=self.read(ids,f"q/{max(1,D)}",u)
            tv=self.read(table,f"((int64_t)row*{D}+q%{max(1,D)})",u)
            return f"int32_t row={iv};\n{out}[q]=(row>=0 && row<{V}) ? {tv} : NAN;"
        if op == "scatter":
            ids,g=s; V,D=u.shape
            iv=self.read(ids,"r",u); gv=self.read(g,f"(r*{D}+q%{max(1,D)})",u)
            return f"float acc=0.0f;\nfor(int64_t r=0;r<{size(ids.shape)};r++) if(({iv})
↪ ==q/{max(1,D)}) acc+={({gv});\n{out}[q]=acc;"
            raise NotImplementedError(op)

def render(self):
    gpu = self.backend != "cpu"
    header = "#include <stdint.h>\n#include <stddef.h>\n#include <math.h>\n"
    prefix = "extern \"C\" " if gpu else ""
    if gpu:
        p="cuda" if self.backend == "cuda" else "hip"
        header += "#include <cuda_runtime.h>\n" if p=="cuda" else "#include <hip/
↪ hip_runtime.h>\n"
        header += f'''extern "C" void* lm_alloc(size_t n) {{ void* p=nullptr; return
↪ {p}Malloc(&p,n)={{p}Success ? p : nullptr; }}
extern "C" int lm_free(void* p) {{ return (int){p}Free(p); }}
extern "C" int lm_sync() {{ return (int){p}DeviceSynchronize(); }}
extern "C" int lm_copy(void* d,const void* s,size_t n,int kind) {{
    auto k=kind==1 ? {p}MemcpyHostToDevice : (kind==2 ? {p}MemcpyDeviceToHost : {p}
↪ MemcpyDeviceToDevice);
    return (int){p}Memcpy(d,s,n,k);
}}
'''
        header += self.call_helpers(gpu)
        definitions,run=[],[]
        for u,off in self.offsets.items():
            run.append(f"{CTYPES[u.dtype]}* {self.pointer(u)}={({CTYPES[u.dtype]}*)(base+{
↪ off});")
        for i,u in enumerate(self.kernels):
            count=size(u.shape)
            if count==0: continue
            if u.op == "attention":
                T,D=u.shape[-2:]; rows=size(u.shape)//D
                q,k,v=map(self.pointer,u.src); out=self.pointer(u)
                if not gpu:
                    run.append(f"for(int64_t r=0;r<{rows};r++) lm_attention_row({q},{k},{
↪ v},{out},r,{T},{D});")
                else:
                    definitions.append(f"__global__ void k{i}(const float* Q,const float*
↪ K,const float* V,float* O) {{ int64_t r=(int64_t)blockIdx.x*blockDim.x+threadIdx.
↪ x; if(r<{rows}) lm_attention_row(Q,K,V,O,r,{T},{D}); }}")
                    run.append(f"k{i}<<<{rows+127}//128,128>>>({q},{k},{v},{out});")
                continue
            if u.op == "matmul" and self.gemm=="blocked":
                a,b=u.src; M,K=a.shape[-2:]; N=b.shape[-1]
                B=size(u.shape[:-2]); ab=bindindex(u.shape[:-2],a.shape[:-2],"batch")
                bb=bindindex(u.shape[:-2],b.shape[:-2],"batch")

```



```

        A,BB,0=self.pointer(a),self.pointer(b),self.pointer(u)
        if not gpu:
            run.append(f"for(int64_t batch=0;batch<{B};batch++) lm_gemm({A}+({ab
↪ })*{M*K},{BB}+({bb})*{K*N},{0}+batch*{M*N},{M},{N},{K});")
        else:
            if B>65535: raise ValueError("tiled GPU GEMM batch exceeds this
↪ renderer's grid-z limit")
            definitions.append(f"__global__ void k{i}(const float* AA,const
↪ float* BB,float* CC) {{
int64_t batch=blockIdx.z; const float* A=AA+({ab})*{M*K}; const float* B=BB+({bb})*{K*
↪ N}; float* C=CC+batch*{M*N};
__shared__ float a[16][16], b[16][16];
int tx=threadIdx.x,ty=threadIdx.y; int64_t row=(int64_t)blockIdx.y*16+ty,col=(int64_t)
↪ blockIdx.x*16+tx;
float acc=0.0f;
for(int64_t k0=0;k0<{K};k0+=16) {{
    a[ty][tx]=(row<{M} && k0+tx<{K}) ? A[row*{K}+k0+tx] : 0.0f;
    b[ty][tx]=(k0+ty<{K} && col<{N}) ? B[(k0+ty)*{N}+col] : 0.0f;
    __syncthreads();
    for(int t=0;t<16;t++) acc+=a[ty][t]*b[t][tx];
    __syncthreads();
}}
if(row<{M} && col<{N}) C[row*{N}+col]=acc;
}}""")
            run.append(f"k{i}<<<dim3({(N+15)//16},{(M+15)//16},{B}),dim3(16,16)
↪ >>>({A},{BB},{0});")
            continue
        if not gpu:
            run.append(f"/* kernel {i}: {u.op} {u.shape} */\nfor(int64_t q=0;q<{
↪ count};q++) {{\n{self.body(u)}\n}}")
        else:
            args=(u,)+self.inputs[u]
            signature=", ".join(f"{CTYPES[x.dtype]}* {self.pointer(x)}" for x in
↪ args)
            definitions.append(f"__global__ void k{i}({signature}) {{\nint64_t q=(
↪ int64_t)blockIdx.x*blockDim.x+threadIdx.x;\nif(q<{count}) {{\n{self.body(u)}\n}}\n
↪ n}}")
            run.append(f"k{i}<<<{((count+127)//128),128}>>>({'','.join(self.pointer(x)
↪ for x in args));")
            ret="0" if not gpu else "(int)cudaGetLastError()" if self.backend=="cuda" else
↪ "(int)hipGetLastError()"
            return header+"\n".join(definitions)+f"\n{prefix}int run(unsigned char* base) {{\n
↪ n"+"\\n".join(run)+f"\nreturn {ret};\n}}\n"

def call_helpers(self,gpu):
    prefix="__device__ " if gpu else ""
    parts=[]
    if any(u.op=="attention" for u in self.kernels):
        parts.append(prefix+"static void lm_attention_row(const float* Q,const
↪ float* K,const float* V,float* O,int64_t r,int64_t T,int64_t D) {{
int64_t i=r/T,base=(r/T)*T*D;
const float* q=Q+r*D; float* o=O+r*D;
float m=-INFINITY,den=0.0f,scale=1.0f/sqrtf((float)D);
for(int64_t d=0;d<D;d++) o[d]=0.0f;
for(int64_t j=0;j<=i;j++) {{
    float score=0.0f;
    for(int64_t d=0;d<D;d++) score+=q[d]*K[base+j*D+d];

```

```

    score*=scale;
    float nm=fmaxf(m,score),alpha=expf(m-nm),beta=expf(score-nm);
    for(int64_t d=0;d<D;d++) o[d]=alpha*o[d]+beta*V[base+j*D+d];
    den=alpha*den+beta; m=nm;
}
for(int64_t d=0;d<D;d++) o[d]/=den;
}
"""
    if self.gemm=="blocked" and not gpu:
        parts.append("""static void lm_gemm(const float* A,const float* B,float* C,
↪ int64_t M,int64_t N,int64_t K) {
for(int64_t ii=0;ii<M;ii+=4) for(int64_t jj=0;jj<N;jj+=8) {
    float acc[4][8]={0};
    for(int64_t kk=0;kk<K;kk+=64) {
        int64_t end=kk+64<K ? kk+64 : K;
        for(int64_t k=kk;k<end;k++) for(int i=0;i<4 && ii+i<M;i++) {
            float av=A[(ii+i)*K+k];
            for(int j=0;j<8 && jj+j<N;j++) acc[i][j]+=av*B[k*N+jj+j];
        }
    }
    for(int i=0;i<4 && ii+i<M;i++) for(int j=0;j<8 && jj+j<N;j++) C[(ii+i)*N+jj+j]=acc[i
↪ ][j];
}
}
"""
    return "".join(parts)

def _view(self,u):
    buf=(C.c_uint8*(size(u.shape)*4)).from_address(self.address+self.offsets[u])
    return np.frombuffer(buf, dtype=u.dtype).reshape(u.shape)

def set(self,name,value):
    if self._closed: raise RuntimeError("program is closed")
    u=self.params[name]; a=np.asarray(value,dtype=u.dtype)
    if a.shape != u.shape: raise ValueError(f"{name}: expected {u.shape}, got {a.
↪ shape}")
    a=np.array(a,dtype=u.dtype,order="C",copy=True)
    if self.backend=="cpu": self._view(u)[...]=a
    else: self._check(self.lib.lm_copy(self.address+self.offsets[u],a.ctypes.data,a.
↪ nbytes,1))
    self.initialized.add(name)

def run(self,feed=None,read=None):
    if self._closed: raise RuntimeError("program is closed")
    for name,value in (feed or {}).items(): self.set(name,value)
    missing=set(self.params)-self.initialized
    if missing: raise ValueError(f"uninitialized parameters: {sorted(missing)}")
    self._check(self.lib.run(self.address)); self.ran=True
    if self.backend!="cpu": self._check(self.lib.lm_sync())
    selected=self.outputs if read is None else tuple(read)
    return [self.read_output(u) for u in selected]

def read_output(self,u):
    if not self.ran or u not in self.outputs: raise ValueError("read requires an
↪ evaluated declared output")
    if self.backend=="cpu": return self._view(u).copy()
    out=np.empty(u.shape,dtype=u.dtype)

```

```

        self._check(self.lib.lm_copy(out.ctypes.data,self.address+self.offsets[u],out.
↪ nbytes,2))
        return out

def assign(self,updates):
    """Commit only after the complete graph has read the old parameter state."""
    if not self.ran: raise RuntimeError("run before assign")
    for name,u in updates.items():
        target=self.params[name]
        if u not in self.outputs or u.op=="param" or u.shape!=target.shape or u.
↪ dtype!=target.dtype:
            raise ValueError(f"invalid update for {name}")
        # Input tensors live outside the scratch region: updates cannot overwrite
↪ outputs.
        for name,u in updates.items():
            target=self.params[name]
            if self.backend=="cpu": self._view(target)[...]=self._view(u)
            else:
                self._check(self.lib.lm_copy(self.address+self.offsets[target],self.
↪ address+self.offsets[u],size(u.shape)*4,3))
                self.initialized.add(name)

@staticmethod
def _check(status):
    if status: raise RuntimeError(f"device/runtime error code {status}")

def close(self):
    if not self._closed and self.backend!="cpu": self._check(self.lib.lm_free(self.
↪ address))
    self._closed=True

def __enter__(self): return self
def __exit__(self,*args): self.close()

def report(self):
    return {"backend":self.backend,"kernels":len(self.kernels),"arena_bytes":self.
↪ bytes,
           "gemm_strategy":self.gemm,"compile_seconds":self.compile_seconds,"sha256
↪ ":self.digest,"source":str(self.source_path)}
```

A.9 lumbre/ir.py

```

"""Lumbre: static tensor IR and symbolic reverse-mode differentiation.
Original teaching implementation. No PyTorch/tinygrad computation is called.
All floating tensor operations are lowered by compiler.py to C/CUDA/HIP.
"""

from __future__ import annotations
from dataclasses import dataclass
from functools import lru_cache
import itertools
import ctypes
import math

Shape = tuple[int, ...]
_counter = itertools.count()
```

```

def size(shape: Shape) -> int:
    return math.prod(shape)

def valid_shape(shape) -> Shape:
    s = tuple(int(x) for x in shape)
    if any(x < 0 for x in s):
        raise ValueError(f"negative dimension: {s}")
    if size(s) > 2**40:
        raise ValueError("teaching implementation limits tensor size to 2**40")
    return s

def broadcast(a: Shape, b: Shape) -> Shape:
    a, b = (1,) * max(0, len(b)-len(a)) + a, (1,) * max(0, len(a)-len(b)) + b
    out = []
    for x, y in zip(a, b):
        if x != y and x != 1 and y != 1:
            raise ValueError(f"cannot broadcast {a} with {b}")
        out.append(y if x == 1 else x)
    return tuple(out)

@dataclass(frozen=True, eq=False)
class UOp:
    op: str
    shape: Shape
    src: tuple[UOp, ...] = ()
    arg: object = None
    dtype: str = "float32"
    uid: int = -1

    def __add__(self, other): return binary("add", self, other)
    def __radd__(self, other): return binary("add", other, self)
    def __sub__(self, other): return binary("sub", self, other)
    def __rsub__(self, other): return binary("sub", other, self)
    def __mul__(self, other): return binary("mul", self, other)
    def __rmul__(self, other): return binary("mul", other, self)
    def __truediv__(self, other): return binary("div", self, other)
    def __rtruediv__(self, other): return binary("div", other, self)
    def __neg__(self): return self * -1.0
    def __matmul__(self, other): return matmul(self, other)
    def exp(self): return unary("exp", self)
    def log(self): return unary("log", self)
    def sqrt(self): return unary("sqrt", self)
    def tanh(self): return unary("tanh", self)
    def detach(self): return node("detach", self.shape, (self,), dtype=self.dtype)
    def eq(self, other): return binary("eq", self, other)
    def lt(self, other): return binary("lt", self, other)
    def where(self, yes, no):
        yes, no = tensor(yes), tensor(no)
        s = broadcast(self.shape, broadcast(yes.shape, no.shape))
        return node("where", s, (self, yes, no))

    def reshape(self, *shape):

```

```

    if len(shape) == 1 and isinstance(shape[0], (tuple, list)):
        shape = tuple(shape[0])
    s = valid_shape(shape)
    if size(s) != size(self.shape):
        raise ValueError(f"reshape changes size: {self.shape} -> {s}")
    return node("reshape", s, (self,), dtype=self.dtype)

def permute(self, *axes):
    if len(axes) == 1 and isinstance(axes[0], (tuple, list)):
        axes = tuple(axes[0])
    if sorted(axes) != list(range(len(self.shape))):
        raise ValueError(f"invalid permutation {axes}")
    return node("permute", tuple(self.shape[i] for i in axes), (self,), tuple(axes),
    ↪ self.dtype)

def transpose(self):
    if len(self.shape) < 2:
        raise ValueError("transpose needs at least two axes")
    p = list(range(len(self.shape)))
    p[-1], p[-2] = p[-2], p[-1]
    return self.permute(p)

def expand(self, shape):
    shape = valid_shape(shape)
    if broadcast(self.shape, shape) != shape:
        raise ValueError("expand target is not the broadcast result")
    return node("expand", shape, (self,), dtype=self.dtype)

def sum(self, axis=None, keepdim=False): return reduce("sum", self, axis, keepdim)
def max(self, axis=None, keepdim=False): return reduce("max", self, axis, keepdim)
def mean(self, axis=None, keepdim=False):
    axes = normalize_axes(axis, len(self.shape))
    n = math.prod(self.shape[i] for i in axes)
    if n == 0: raise ValueError("mean of an empty reduction is undefined here")
    return self.sum(axes, keepdim) / float(n)

def slice(self, axis, start, stop):
    axis %= len(self.shape)
    if not 0 <= start <= stop <= self.shape[axis]:
        raise ValueError("slice requires 0 <= start <= stop <= dimension")
    shape = list(self.shape)
    shape[axis] = stop-start
    return node("slice", tuple(shape), (self,), (axis, start, stop), self.dtype)

def pad(self, pads):
    pads = tuple(tuple(int(x) for x in p) for p in pads)
    if len(pads) != len(self.shape) or any(len(p) != 2 or min(p) < 0 for p in pads):
        raise ValueError("pad expects one nonnegative (left,right) pair per axis")
    shape = tuple(d+l+r for d, (l,r) in zip(self.shape, pads))
    return node("pad", shape, (self,), pads, self.dtype)

@lru_cache(maxsize=None)
def node(op, shape, src=(), arg=None, dtype="float32"):
    if dtype not in ("float32", "int32"):
        raise ValueError(f"unsupported dtype {dtype}")
    return UOp(op, valid_shape(shape), tuple(src), arg, dtype, next(_counter))

```

```

def param(name, shape, dtype="float32"):
    if not isinstance(name, str) or not name:
        raise ValueError("parameter name must be a nonempty string")
    return node("param", valid_shape(shape), arg=name, dtype=dtype)

def tensor(x) -> UOp:
    if isinstance(x, UOp): return x
    v = ctypes.c_float(float(x)).value
    if not math.isfinite(v):
        raise ValueError("use finite scalar constants; padding/masks need explicit
        ↪ semantics")
    return node("const", (), arg=v.hex())

def unary(op, x):
    if x.dtype != "float32": raise TypeError("floating operation on integer tensor")
    return node(op, x.shape, (x,))

def binary(op, a, b):
    a, b = tensor(a), tensor(b)
    if a.dtype != "float32" or b.dtype != "float32":
        raise TypeError("binary arithmetic only supports float32")
    return node(op, broadcast(a.shape, b.shape), (a, b))

def normalize_axes(axis, ndim):
    if axis is None: return tuple(range(ndim))
    axes = (axis,) if isinstance(axis, int) else tuple(axis)
    if any(not -ndim <= x < ndim for x in axes): raise ValueError("axis out of bounds")
    axes = tuple(sorted(x % ndim for x in axes))
    if len(set(axes)) != len(axes): raise ValueError("duplicate reduction axis")
    return axes

def reduce(op, x, axis, keepdim):
    axes = normalize_axes(axis, len(x.shape))
    if x.dtype != "float32": raise TypeError("reductions require float32")
    if op == "max" and any(x.shape[i] == 0 for i in axes):
        raise ValueError("empty max is not defined by this API")
    if not axes: return x
    s = tuple(1 if i in axes else d for i, d in enumerate(x.shape))
    out = node(op, s, (x,), axes)
    return out if keepdim else out.reshape(tuple(d for i, d in enumerate(s) if i not in
    ↪ axes))

def matmul(a, b):
    if min(len(a.shape), len(b.shape)) < 2: raise ValueError("matmul requires rank >= 2")
    ↪
    if a.shape[-1] != b.shape[-2]: raise ValueError("matmul K dimensions disagree")
    if a.dtype != "float32" or b.dtype != "float32": raise TypeError("matmul requires
    ↪ float32")
    batch = broadcast(a.shape[:-2], b.shape[:-2])

```

```

    return node("matmul", batch + (a.shape[-2], b.shape[-1]), (a,b))

def concat(a, b, axis=-1):
    if len(a.shape) != len(b.shape): raise ValueError("concat ranks disagree")
    axis %= len(a.shape)
    if any(x != y for i,(x,y) in enumerate(zip(a.shape,b.shape)) if i != axis):
        raise ValueError("concat dimensions disagree")
    if a.dtype != b.dtype: raise TypeError("concat dtypes disagree")
    shape = list(a.shape); shape[axis] += b.shape[axis]
    return node("concat", tuple(shape), (a,b), axis, a.dtype)

def gather(table, ids):
    if len(table.shape) != 2 or table.dtype != "float32" or ids.dtype != "int32":
        raise TypeError("gather requires float32[V,D] and int32 ids")
    return node("gather", ids.shape + (table.shape[1],), (table, ids))

def one_hot(ids, classes):
    if ids.dtype != "int32" or classes <= 0: raise ValueError("invalid one_hot")
    return node("onehot", ids.shape + (int(classes),), (ids,), int(classes))

def topo(outputs):
    """Iterative DFS; each input is emitted before each use."""
    done, out = set(), []
    for root in outputs:
        stack = [(root, False)]
        while stack:
            u, exit = stack.pop()
            if u in done: continue
            if exit:
                done.add(u); out.append(u)
            else:
                stack.append((u, True))
                stack.extend((s, False) for s in reversed(u.src) if s not in done)
    return out

def unbroadcast(g, shape):
    aligned = (1,) * (len(g.shape)-len(shape)) + shape
    axes = tuple(i for i,(a,b) in enumerate(zip(aligned,g.shape)) if a == 1 and b != 1)
    if axes: g = g.sum(axes, keepdim=True)
    return g.reshape(shape)

def gradients(loss, wrt):
    """Construct derivative graphs, not numerical finite differences."""
    grad = {loss: tensor(1.0).expand(loss.shape)}
    for u in reversed(topo((loss,))):
        if u not in grad: continue
        g = grad[u]; s = u.src; op = u.op
        gs = []
        if op == "add": gs = [g,g]
        elif op == "sub": gs = [g,-g]
        elif op == "mul": gs = [g*s[1], g*s[0]]

```

```

        elif op == "div": gs = [g/s[1], -g*s[0]/(s[1]*s[1])]
        elif op == "exp": gs = [g*u]
        elif op == "log": gs = [g/s[0]]
        elif op == "sqrt": gs = [g/(2.0*u)]
        elif op == "tanh": gs = [g*(1.0-u*u)]
        elif op == "where": gs = [None, s[0].where(g,0.0), s[0].where(0.0,g)]
        elif op == "reshape": gs = [g.reshape(s[0].shape)]
        elif op == "permute": gs = [g.permute(tuple(u.arg.index(i) for i in range(len(u.
↪ arg))))]
        elif op == "expand": gs = [unbroadcast(g, s[0].shape)]
        elif op == "sum": gs = [g.expand(s[0].shape)]
        elif op == "max":
            mask = s[0].eq(u.expand(s[0].shape))
            gs = [g.expand(s[0].shape)*mask/mask.sum(u.arg,keepdim=True)]
        elif op == "matmul": gs = [g @ s[1].transpose(), s[0].transpose() @ g]
        elif op == "slice":
            axis,start,stop = u.arg
            pads = tuple((start,s[0].shape[i]-stop) if i == axis else (0,0) for i in
↪ range(len(u.shape)))
            gs = [g.pad(pads)]
        elif op == "pad":
            back = g
            for i,(left,right) in enumerate(u.arg): back = back.slice(i,left,left+s[0].
↪ shape[i])
            gs = [back]
        elif op == "concat":
            axis = u.arg; split = s[0].shape[axis]
            gs = [g.slice(axis,0,split), g.slice(axis,split,u.shape[axis])]
        elif op == "gather":
            gs = [node("scatter", s[0].shape, (s[1],g)), None]
        elif op == "scatter": gs = [None, gather(g,s[0])]
        elif op == "attention":
            q,k,v = s; T,D = q.shape[-2:]
            p = softmax((q @ k.transpose())/(D**0.5) + node("causal", (T,T)))
            dp = g @ v.transpose()
            ds = p*(dp-(dp*p).sum(-1,keepdim=True))
            gs = [(ds @ k)/(D**0.5), (ds.transpose() @ q)/(D**0.5), p.transpose() @ g]
        elif op in ("param","const","detach","eq","lt","onehot","causal"): continue
        else: raise NotImplementedError(f"gradient for {op}")
        for x,gx in zip(s,gs):
            if gx is not None and x.dtype == "float32":
                gx = unbroadcast(gx,x.shape)
                grad[x] = grad[x]+gx if x in grad else gx
    return [grad.get(x,tensor(0.0).expand(x.shape)) for x in wrt]

def softmax(x, axis=-1):
    m = x.max(axis,keepdim=True).detach()
    e = (x-m).exp()
    return e/e.sum(axis,keepdim=True)

def cross_entropy(logits, labels):
    if labels.shape != logits.shape[:-1]: raise ValueError("label shape mismatch")
    m = logits.max(-1,keepdim=True).detach()
    z = logits-m
    logp = z-z.exp().sum(-1,keepdim=True).log()

```



```

    return -(logp*one_hot(labels,logits.shape[-1])).sum(-1).mean()

def online_attention(q,k,v):
    """Causal attention with a linear-memory forward and a dense symbolic VJP.
    This is an educational online-softmax implementation, not FlashAttention's
    complete tiled GPU algorithm. q/k/v must share a static [...,T,D] shape.
    """
    if q.shape != k.shape or q.shape != v.shape or len(q.shape)<2:
        raise ValueError("attention requires equal [...,T,D] shapes")
    if min(q.shape[-2:])<=0 or any(x.dtype!="float32" for x in (q,k,v)):
        raise ValueError("attention requires nonempty float32 rows")
    return node("attention",q.shape,(q,k,v))

```

A.10 lumbré/layout.py

```

"""Zero-copy layout model for the movement-operations laboratory.
This is a separate oracle/lab module, not a claim that Program accepts arbitrary strides.
↪
"""
from dataclasses import dataclass
import math

@dataclass(frozen=True)
class View:
    shape: tuple[int,...]
    strides: tuple[int,...]
    offset: int=0

    def __post_init__(self):
        if len(self.shape)!=len(self.strides) or any(d<0 for d in self.shape): raise
        ↪ ValueError("invalid view")

    @staticmethod
    def contiguous(shape):
        shape=tuple(shape)
        return View(shape,tuple(math.prod(shape[i+1:]) for i in range(len(shape))))

    def address(self,index):
        if len(index)!=len(self.shape) or any(not 0<=i<d for i,d in zip(index,self.shape)
        ↪ ): raise IndexError(index)
        return self.offset+sum(i*s for i,s in zip(index,self.strides))

    def permute(self,axes):
        if sorted(axes)!=list(range(len(self.shape))): raise ValueError("invalid
        ↪ permutation")
        return View(tuple(self.shape[a] for a in axes),tuple(self.strides[a] for a in
        ↪ axes),self.offset)

    def expand(self,shape):
        shape=tuple(shape)
        if len(shape)<len(self.shape): raise ValueError("cannot remove axes")
        old=(1,)*(len(shape)-len(self.shape))+self.shape
        strides=(0,)*(len(shape)-len(self.shape))+self.strides

```

```

        if any(a!=b and a!=1 for a,b in zip(old,shape)): raise ValueError("invalid
↪ broadcast")
        return View(shape,tuple(0 if a==1 else s for a,s in zip(old,strides)),self.
↪ offset)

def flip(self,axis):
    axis%=len(self.shape);s=list(self.strides)
    off=self.offset+(self.shape[axis]-1)*s[axis] if self.shape[axis] else self.
↪ offset
    s[axis]=-s[axis]
    return View(self.shape,tuple(s),off)

def contiguous_reshape(self,shape):
    shape=tuple(shape)
    if math.prod(shape)!=math.prod(self.shape): raise ValueError("size changes")
    expected=1
    for d,s in reversed(list(zip(self.shape,self.strides))):
        if d>1 and s!=expected: raise ValueError("noncontiguous reshape needs
↪ general indexing or materialization")
        expected*=d
    v=View.contiguous(shape)
    return View(v.shape,v.strides,self.offset)

```

A.11 lumbre/nn.py

```

"""Decoder-only Transformer: RMSNorm, RoPE, GQA, SwiGLU, tied embeddings."""
from dataclasses import dataclass
import math
import numpy as np
from .ir import param, tensor, concat, gather, softmax, cross_entropy, gradients,
↪ online_attention

@dataclass(frozen=True)
class Config:
    vocab: int = 256
    batch: int = 2
    context: int = 16
    dim: int = 32
    heads: int = 4
    kv_heads: int = 2
    layers: int = 2
    hidden: int = 64
    rope_base: float = 10000.0
    eps: float = 1e-5
    online: bool = False

    def __post_init__(self):
        if min(self.vocab,self.batch,self.context,self.dim,self.heads,self.kv_heads,self.
↪ layers,self.hidden)<=0:
            raise ValueError("all dimensions must be positive")
        if self.dim%self.heads or self.heads%self.kv_heads or (self.dim//self.heads)%2:
            raise ValueError("dim/heads must be even; heads must be divisible by
↪ kv_heads")

```

```

class Decoder:
    def __init__(self, config=Config(), seed=7):
        self.c=config; self.weights={}; self.initial={}
        self.rng=np.random.default_rng(seed)
        c=config; B,T,D,H,HK=c.batch,c.context,c.dim,c.heads,c.kv_heads
        dh=D//H
        self.tokens=param("tokens", (B,T), "int32")
        self.labels=param("labels", (B,T), "int32")
        inv=c.rope_base*(-np.arange(0,dh,2,dtype=np.float32)/dh)
        angle=np.arange(T,dtype=np.float32)[:,:None]*inv[None,:]
        self.cos=self.fixed("rope_cos", np.cos(angle).reshape(1,1,T,dh//2))
        self.sin=self.fixed("rope_sin", np.sin(angle).reshape(1,1,T,dh//2))
        mask=np.where(np.arange(T)[None,:] <= np.arange(T)[:,:None], 0.0, -1e9).astype("
    ↪ float32")
        self.mask=self.fixed("causal_mask", mask.reshape(1,1,T,T))
        embed=self.weight("embedding", (c.vocab,D), scale=0.02)
        x=gather(embed, self.tokens)
        for i in range(c.layers):
            tag=f"layer{i}"
            z=self.norm(x, self.weight(tag+".attn_norm", (D,), ones=True))
            q=z@self.weight(tag+".wq", (D,H*dh))
            k=z@self.weight(tag+".wk", (D,HK*dh))
            v=z@self.weight(tag+".wv", (D,HK*dh))
            q=q.reshape(B,T,H,dh).permute(0,2,1,3)
            k=k.reshape(B,T,HK,dh).permute(0,2,1,3)
            v=v.reshape(B,T,HK,dh).permute(0,2,1,3)
            q,k=self.rope(q), self.rope(k)
            if HK!=H:
                k=k.reshape(B,HK,1,T,dh).expand((B,HK,H//HK,T,dh)).reshape(B,H,T,dh)
                v=v.reshape(B,HK,1,T,dh).expand((B,HK,H//HK,T,dh)).reshape(B,H,T,dh)
            if c.online:
                attended=online_attention(q,k,v)
            else:
                a=softmax((q@k.transpose())/math.sqrt(dh)+self.mask)
                attended=a@v
            y=attended.permute(0,2,1,3).reshape(B,T,D)
            x=x+y@self.weight(tag+".wo", (D,D), scale=1/math.sqrt(D*c.layers))
            z=self.norm(x, self.weight(tag+".ffn_norm", (D,), ones=True))
            gate=z@self.weight(tag+".gate", (D,c.hidden))
            up=z@self.weight(tag+".up", (D,c.hidden))
            silu=gate/(1.0+(-gate).exp())
            x=x+(silu*up)@self.weight(tag+".down", (c.hidden,D), scale=1/math.sqrt(c.
    ↪ hidden*c.layers))
            x=self.norm(x, self.weight("final_norm", (D,), ones=True))
            self.logits=x@embed.transpose()
            self.loss=cross_entropy(self.logits, self.labels)

    def weight(self, name, shape, scale=None, ones=False):
        u=param(name, shape); self.weights[name]=u
        if ones: a=np.ones(shape, dtype=np.float32)
        else: a=(self.rng.standard_normal(shape)*(scale or 1/math.sqrt(shape[0]))).
    ↪ astype(np.float32)
        self.initial[name]=a
        return u

    def fixed(self, name, value):

```

```

        value=np.asarray(value,dtype=np.float32); self.initial[name]=value
        return param(name,value.shape)

    def norm(self,x,w): return x/(x*x).mean(-1,keepdim=True).__add__(self.c.eps).sqrt()*
    ↪ w

    def rope(self,x):
        half=x.shape[-1]//2
        a=x.slice(-1,0,half); b=x.slice(-1,half,2*half)
        return concat(a*self.cos-b*self.sin,a*self.sin+b*self.cos,-1)

    def parameter_count(self): return sum(math.prod(w.shape) for w in self.weights.
    ↪ values())

def adamw(model,weight_decay=0.01,clip_norm=1.0,beta1=0.9,beta2=0.999,eps=1e-8):
    """Return compiled update outputs, assignment map, and initial optimizer state."""
    weights=list(model.weights.values())
    gs=gradients(model.loss,weights)
    norm2=sum((g*g).sum() for g in gs)
    norm=(norm2+1e-12).sqrt()
    ratio=clip_norm/(norm+1e-6)
    scale=ratio.lt(1.0).where(ratio,1.0)
    lr=param("learning_rate",())
    bc1=param("adam_bc1",()); bc2=param("adam_bc2",())
    updates={}; initial={}
    for (name,w),g in zip(model.weights.items(),gs):
        g=g*scale
        m=param("m."+name,w.shape); v=param("v."+name,w.shape)
        m1=beta1*m+(1-beta1)*g
        v1=beta2*v+(1-beta2)*g*g
        new=w*(1-lr*weight_decay)-lr*(m1/bc1)/((v1/bc2).sqrt()+eps)
        updates[name]=new; updates[m.arg]=m1; updates[v.arg]=v1
        initial[m.arg]=np.zeros(w.shape,dtype=np.float32)
        initial[v.arg]=np.zeros(w.shape,dtype=np.float32)
    return [model.loss,norm,*updates.values()],updates,initial

```

A.12 lumbre/nnops.py

```

"""Composite CNN operation using only existing differentiable IR primitives.
This explicit patch construction is for small correctness laboratories. It grows
with the number of output positions; it is not an optimized im2col lowering.
"""

from functools import reduce
from .ir import concat

def join(values,axis):
    if not values: raise ValueError('cannot concatenate an empty sequence')
    return reduce(lambda a,b:concat(a,b,axis),values)

def pair(value):
    if isinstance(value,int): return (value,value)
    if len(value)!=2: raise ValueError('expected an integer or pair')

```

```

return tuple(int(x) for x in value)

def conv2d(x,w,bias=None,stride=1,padding=0,dilation=1,groups=1):
    """NCHW x OIHW -> NCHW, grouped, symmetric padding, static FP32.
    Gradients come from composition; no handwritten convolution VJP is used.
    A 1024-position cap prevents accidentally building huge teaching graphs.
    """
    if len(x.shape)!=4 or len(w.shape)!=4: raise ValueError('conv2d needs rank 4')
    if x.dtype!='float32' or w.dtype!='float32': raise TypeError('conv2d needs FP32')
    B,C,H,W=x.shape;O,CG,KH,KW=w.shape
    sh,sw=pair(stride);ph,pw=pair(padding);dh,dw=pair(dilation)
    if min(B,C,H,W,O,CG,KH,KW,sh,sw,dh,dw,groups)<=0 or min(ph,pw)<0:
        raise ValueError('invalid convolution dimensions')
    if C%groups or O%groups or CG!=C//groups: raise ValueError('invalid groups')
    OH=(H+2*ph-dh*(KH-1)-1)//sh+1
    OW=(W+2*pw-dw*(KW-1)-1)//sw+1
    if min(OH,OW)<=0 or OH*OW>1024: raise ValueError('output outside teaching limits')
    if bias is not None and (bias.shape!=(O,) or bias.dtype!='float32'):
        raise ValueError('bias must be FP32[O]')
    xp=x.pad(((0,0),(0,0),(ph,ph),(pw,pw)))
    OG=O//groups;outputs=[]
    for group in range(groups):
        weight=w.slice(0,group*OG,(group+1)*OG).reshape(OG,CG*KH*KW).transpose()
        rows=[]
        for y in range(OH):
            columns=[]
            for z in range(OW):
                entries=[]
                for ci in range(CG):
                    for ky in range(KH):
                        for kx in range(KW):
                            iy=y*sh+ky*dh;ix=z*sw+kx*dw;c=group*CG+ci
                            entries.append(xp.slice(1,c,c+1).slice(2,iy,iy+1).slice(3,ix,
↪ ix+1).reshape(B,1))
                        columns.append((join(entries,1)@weight).reshape(B,OG,1,1))
            rows.append(join(columns,3))
        outputs.append(join(rows,2))
    result=join(outputs,1)
    return result if bias is None else result+bias.reshape(1,O,1,1)

```

A.13 lumbr/pasport.py

```

"""Logical launch verification for a teaching portability project.
It verifies a finite index map, NOT GPU instructions, races or device support.
"""

from dataclasses import dataclass
from collections import Counter
from typing import Callable

@dataclass(frozen=True)
class Launch:
    rows: int
    columns: int

```

```

tile_rows: int = 16
tile_columns: int = 16

def __post_init__(self):
    values = (self.rows, self.columns, self.tile_rows, self.tile_columns)
    if any(type(v) is not int for v in values):
        raise TypeError("launch dimensions must be integers")
    if min(self.rows, self.columns) < 0 or min(self.tile_rows, self.tile_columns) <=
↪ 0:
        raise ValueError("negative extent or nonpositive tile")
    if self.rows * self.columns > 1_000_000:
        raise ValueError("finite verifier limited to one million outputs")
    if self.tile_rows * self.tile_columns > 4096:
        raise ValueError("finite verifier tile limit exceeded")

@property
def grid(self):
    return ((self.rows + self.tile_rows - 1) // self.tile_rows,
            (self.columns + self.tile_columns - 1) // self.tile_columns)

def row_major(row: int, column: int, launch: Launch) -> int:
    return row * launch.columns + column

def verify(launch: Launch,
           address: Callable[[int, int, Launch], int] = row_major) -> dict:
    """Enumerate logical participants, mask edges and verify output ownership.
    Each valid logical output must have exactly one writer. An out-of-range
    address is a failure even when another address happens to be missing.
    """
    counts = Counter()
    masked = 0
    outside = []
    gy, gx = launch.grid
    for by in range(gy):
        for bx in range(gx):
            for ty in range(launch.tile_rows):
                for tx in range(launch.tile_columns):
                    row = by * launch.tile_rows + ty
                    column = bx * launch.tile_columns + tx
                    if row >= launch.rows or column >= launch.columns:
                        masked += 1
                        continue
                    index = address(row, column, launch)
                    if type(index) is not int:
                        raise TypeError("address map must return an integer")
                    if not 0 <= index < launch.rows * launch.columns:
                        outside.append((row, column, index))
                    else:
                        counts[index] += 1
    missing = [i for i in range(launch.rows * launch.columns) if counts[i] == 0]
    duplicates = {i: n for i, n in counts.items() if n > 1}
    ok = not (missing or duplicates or outside)
    return {"status": "LOGICAL_MAP_VALIDATED" if ok else "FAILED",
            "rows": launch.rows, "columns": launch.columns,
            "tile": [launch.tile_rows, launch.tile_columns],

```

```

"grid": list(launch.grid), "valid_writes": sum(counts.values()),
"masked_participants": masked, "missing": missing,
"duplicate_writes": duplicates, "out_of_range": outside,
"scope": "finite logical output map only; no device execution or race proof"}

```

→

A.14 lumbr/symbolic.py

```

"""Exact integer expressions for index arithmetic; NOT float rewrite rules."""
from __future__ import annotations
from dataclasses import dataclass
from functools import lru_cache

@dataclass(frozen=True)
class Expr:
    op: str
    src: tuple[Expr,...]=()
    arg: int|str|None=None
    def __add__(self,x): return make("add",self,expr(x))
    def __radd__(self,x): return expr(x)+self
    def __mul__(self,x): return make("mul",self,expr(x))
    def __rmul__(self,x): return expr(x)*self
    def __floordiv__(self,x): return make("div",self,expr(x))
    def __mod__(self,x): return make("mod",self,expr(x))
    def evaluate(self,values):
        if self.op=="const": return self.arg
        if self.op=="var": return int(values[self.arg])
        a,b=(x.evaluate(values) for x in self.src)
        if self.op=="add": return a+b
        if self.op=="mul": return a*b
        if b<=0: raise ValueError("index division requires a positive denominator")
        return a//b if self.op=="div" else a%b

@lru_cache(None)
def var(name): return Expr("var",arg=name)
@lru_cache(None)
def const(n): return Expr("const",arg=int(n))
def expr(x): return x if isinstance(x,Expr) else const(x)

@lru_cache(None)
def make(op,a,b):
    if op not in ("add","mul","div","mod"): raise ValueError(op)
    return Expr(op,(a,b))

def simplify(root):
    memo={}
    def walk(u):
        if u in memo: return memo[u]
        if not u.src: out=u
        else:
            a,b=map(walk,u.src)
            out=make(u.op,a,b)
            ac=a.op=="const"; bc=b.op=="const"
            if ac and bc:

```

```

        out=const(out.evaluate({}))
    elif u.op=="add" and bc and b.arg==0: out=a
    elif u.op=="add" and ac and a.arg==0: out=b
    elif u.op=="mul" and bc and b.arg==1: out=a
    elif u.op=="mul" and ac and a.arg==1: out=b
    elif u.op=="mul" and ((ac and a.arg==0) or (bc and b.arg==0)): out=const(0)
    elif u.op=="div" and bc and b.arg==1: out=a
    elif u.op=="mod" and bc and b.arg==1: out=const(0)
    memo[u]=out; return out
return walk(root)

def interval(u,bounds):
    if u.op=="const": return u.arg,u.arg
    if u.op=="var": return bounds[u.arg]
    (lo,hi),(l2,h2)=[interval(s,bounds) for s in u.src]
    if u.op=="add": return lo+l2,hi+h2
    if u.op=="mul":
        vals=(lo*l2,lo*h2,hi*l2,hi*h2);return min(vals),max(vals)
    if l2!=h2 or l2<=0: raise ValueError("only positive constant divisor supported")
    if u.op=="div": return lo//l2,hi//l2
    return (lo%l2,hi%l2) if lo//l2==hi//l2 else (0,l2-1)

def render_c(u):
    if u.op=="const": return str(u.arg)
    if u.op=="var":
        if not str(u.arg).isidentifier() or not str(u.arg).isascii(): raise ValueError("
↳ unsafe variable name")
        return u.arg
    a,b=map(render_c,u.src)
    # Contract: numerator nonnegative, divisor positive; C / then matches floor.
    return f"({a}){ {'add': '+', 'mul': '*', 'div': '/', 'mod': '%'}[u.op] }{b})"

```

A.15 lumbre/tokenizer.py

```

"""Deterministic byte BPE for teaching, not a production-speed tokenizer.
No pretokenizer, normalization or special tokens. Training is O(tokens * merges).
Fit only on the training split. Any UTF-8 input is encodable through 256 bytes.
"""

from collections import Counter
from dataclasses import dataclass

def replace_pair(ids, pair, new_id):
    out=[]; i=0
    while i<len(ids):
        if i+1<len(ids) and (ids[i],ids[i+1])==pair:
            out.append(new_id); i+=2
        else:
            out.append(ids[i]); i+=1
    return out

@dataclass(frozen=True)

```



```

class ByteBPE:
    merges: tuple[tuple[int,int], ...] = ()

    def __post_init__(self):
        for rank,pair in enumerate(self.merges):
            if len(pair)!=2 or any(not isinstance(x,int) or x<0 or x>=256+rank for x in
↪ pair):
                raise ValueError('a merge may only reference previously defined tokens')
            if len(set(self.merges))!=len(self.merges):
                raise ValueError('duplicate merge')

    @classmethod
    def fit(cls,text,max_merges=128):
        if max_merges<0: raise ValueError('max_merges must be nonnegative')
        ids=list(text.encode('utf-8')); merges=[]
        for _ in range(max_merges):
            counts=Counter(zip(ids,ids[1:]))
            if not counts: break
            pair,count=min(counts.items(),key=lambda item:(-item[1],item[0]))
            if count<2: break
            new_id=256+len(merges); merges.append(pair)
            ids=replace_pair(ids,pair,new_id)
        return cls(tuple(merges))

    @property
    def vocab(self):
        pieces=[bytes([i]) for i in range(256)]
        for a,b in self.merges: pieces.append(pieces[a]+pieces[b])
        return tuple(pieces)

    def encode(self,text):
        ids=list(text.encode('utf-8'))
        for rank,pair in enumerate(self.merges): ids=replace_pair(ids,pair,256+rank)
        return ids

    def decode(self,ids,errors='strict'):
        pieces=self.vocab; result=[]
        for index in ids:
            index=int(index)
            if index<0 or index>=len(pieces): raise ValueError('token id out of
↪ vocabulary')
            result.append(pieces[index])
        return b''.join(result).decode('utf-8',errors=errors)

    def state(self):
        return {'kind':'byte_bpe','merges':[list(pair) for pair in self.merges]}

    @classmethod
    def from_state(cls,state):
        if state.get('kind')!='byte_bpe': raise ValueError('unsupported tokenizer format
↪ ')
        return cls(tuple(tuple(pair) for pair in state['merges']))

```

A.16 tests/conftest.py

```

"""Run the numerical tests on an explicitly selected device backend."""
from functools import partial
import pytest
from lumbr import Program

def pytest_addoption(parser):
    parser.addoption('--backend', choices=('cpu', 'cuda', 'hip'), default='cpu',
                     help='Backend for Lumbr Programs; requested devices must work.')

@pytest.fixture(autouse=True)
def select_backend(request, monkeypatch):
    backend = request.config.getoption('--backend')
    if backend != 'cpu' and hasattr(request.module, 'Program'):
        monkeypatch.setattr(request.module, 'Program', partial(Program, backend=backend))
    ↪

```

A.17 tests/test_conv.py

```

"""Compile and validate the standalone C reference, not a claimed fast kernel."""
import ctypes as ct
from pathlib import Path
import shutil
import subprocess
import numpy as np
import pytest

@pytest.fixture(scope='module')
def conv(tmp_path_factory):
    cc=shutil.which('cc') or shutil.which('gcc')
    if cc is None: pytest.skip('C compiler unavailable')
    source=Path(__file__).resolve().parents[1]/'kernels'/ 'conv.c'
    target=tmp_path_factory.mktemp('conv')/'conv.so'
    subprocess.run([cc, '-std=c11', '-O2', '-shared', '-fPIC', str(source), '-o', str(target)],
    ↪ check=True)
    lib=ct.CDLL(str(target)); f=lib.conv2d_nchw
    f.argtypes=[ct.POINTER(ct.c_float)]*4+[ct.c_int]*14
    f.restype=ct.c_int
    return f

@pytest.mark.parametrize('groups, stride, dilation, pad', [(1,1,1,0), (1,2,1,1), (2,1,2,2)
    ↪ , (4,1,1,1)])
def test_conv_reference(conv, groups, stride, dilation, pad):
    B,C,H,W,O,KH,KW=2,4,7,8,3,2
    OH=(H+2*pad-dilation*(KH-1)-1)//stride+1
    OW=(W+2*pad-dilation*(KW-1)-1)//stride+1
    rng=np.random.default_rng(7)
    x=rng.normal(size=(B,C,H,W)).astype('float32')
    w=rng.normal(size=(O,C//groups,KH,KW)).astype('float32')
    bias=rng.normal(size=O).astype('float32'); out=np.empty((B,O,OH,OW), dtype='float32')
    ptr=lambda a: a.ctypes.data_as(ct.POINTER(ct.c_float))

```

```

assert conv(*map(ptr, (x,w,bias,out)),B,C,H,W,O,KH,KW,stride,stride,pad,pad,dilation,
↪ dilation,groups)==0
want=np.zeros_like(out, dtype='float64')
for b,o,y,z in np.ndindex(want.shape):
    want[b,o,y,z]=bias[o]
    group=o//(O//groups)
    for ci,ky,kx in np.ndindex((C//groups,KH,KW)):
        iy=y*stride-pad+ky*dilation;ix=z*stride-pad+kx*dilation
        if 0<=iy<H and 0<=ix<W:
            want[b,o,y,z]+=float(x[b,group*(C//groups)+ci,iy,ix])*float(w[o,ci,ky,kx
↪ ])
np.testing.assert_allclose(out,want,rtol=2e-5,atol=2e-5)

```

A.18 tests/test_conv__ir.py

```

import numpy as np
import pytest
from lumbr import param,Program,gradients
from lumbr.nnops import conv2d

@pytest.mark.parametrize('groups,stride,pad,dilation',[(1,1,0,1),(2,2,1,1),(2,1,1,2)])
def test_composite_conv_and_gradient(groups,stride,pad,dilation):
    rng=np.random.default_rng(93)
    x=param('cx',(1,2,4,5));w=param('cw',(4,2//groups,2,2));b=param('cb',(4,))
    out=conv2d(x,w,b,stride=stride,padding=pad,dilation=dilation,groups=groups)
    loss=(out*out).mean();gx,gw,gb=gradients(loss,[x,w,b])
    feed={u.arg:(rng.normal(size=u.shape)*.2).astype('float32') for u in (x,w,b)}
    with Program([out,loss,gx,gw,gb],gemm='blocked') as p:
        result=p.run(feed);want=np.empty(out.shape,dtype='float64')
        for batch,o,y,z in np.ndindex(out.shape):
            acc=float(feed['cb'][o]);group=o//(4//groups)
            for ci,ky,kz in np.ndindex(w.shape[1:]):
                iy=y*stride-pad+ky*dilation;iz=z*stride-pad+kz*dilation
                if 0<=iy<4 and 0<=iz<5:
                    acc+=float(feed['cx'][batch,group*(2//groups)+ci,iy,iz])*float(feed[
↪ 'cw'][o,ci,ky,kz])
            want[batch,o,y,z]=acc
    np.testing.assert_allclose(result[0],want,rtol=2e-5,atol=2e-5)
    for u,analytic in zip((x,w,b),result[2:]):
        index=tuple(0 for _ in u.shape); plus=feed[u.arg].copy();minus=plus.copy()
        plus[index]+=1e-3;minus[index]-=1e-3
        hi=p.run(**feed,u.arg:plus,read=[loss])[0]
        lo=p.run(**feed,u.arg:minus,read=[loss])[0]
        np.testing.assert_allclose(analytic[index],(hi-lo)/2e-3,rtol=4e-3,atol=4e-4)

```

A.19 tests/test_lumbr.py

```

import math
import numpy as np
import pytest
from lumbr import *
from lumbr.ir import one_hot, broadcast

```

```

from lumbre.nn import Decoder, Config, adamw
from lumbre.layout import View
from lumbre.symbolic import var, simplify, interval

rng=np.random.default_rng(23)

@pytest.mark.parametrize("shape_a, shape_b", [((2,3), (3,)), ((2,1,4), (1,3,1)), (( ), (3,))
    ↪ , ((0,3), (1,3))])
@pytest.mark.parametrize("fuse", [True, False])
def test_elementwise(shape_a, shape_b, fuse):
    a,b=param("a", shape_a), param("b", shape_b)
    x=rng.normal(size=shape_a).astype("float32"); y=rng.normal(size=shape_b).astype("
    ↪ float32")
    out=(a+b)*a-b
    got=Program([out], fuse=fuse).run({"a":x, "b":y})[0]
    np.testing.assert_allclose(got, (x+y)*x-y, rtol=1e-6, atol=1e-6)

@pytest.mark.parametrize("a_shape, b_shape", [((2,3), (3,4)), ((2,2,3), (1,3,5)), ((1,3,2,4)
    ↪ , (2,1,4,3)), ((2,0), (0,3))])
def test_matmul(a_shape, b_shape):
    a,b=param("ma", a_shape), param("mb", b_shape)
    x=rng.normal(size=a_shape).astype("float32"); y=rng.normal(size=b_shape).astype("
    ↪ float32")
    got=Program([a@b]).run({"ma":x, "mb":y})[0]
    np.testing.assert_allclose(got, x@y, rtol=2e-5, atol=1e-6)

@pytest.mark.parametrize("axis", [0,1,(0,2),None])
def test_reduce(axis):
    a=param("r", (2,3,4)); x=rng.normal(size=a.shape).astype("float32")
    p=Program([a.sum(axis), a.max(axis), a.mean(axis)])
    for got,want in zip(p.run({"r":x}), [x.sum(axis=axis), x.max(axis=axis), x.mean(axis=
    ↪ axis)]):
        np.testing.assert_allclose(got,want, rtol=2e-5, atol=1e-6)

@pytest.mark.parametrize("reuse", [True, False])
def test_movement(reuse):
    a=param("v", (2,3)); x=np.arange(6, dtype="float32").reshape(2,3)
    z=a.permute(1,0).slice(0,1,3).pad(((1,2), (2,0)))
    y=concat(z,z,-1)
    got=Program([y], reuse=reuse).run({"v":x})[0]
    expected=np.pad(x.T[1:3], ((1,2), (2,0))); expected=np.concatenate((expected, expected),
    ↪ axis=-1)
    np.testing.assert_equal(got, expected)

def test_gather():
    a=param("table", (4,3)); ids=param("ids", (2,2), "int32")
    x=np.arange(12, dtype="float32").reshape(4,3); i=np.array([[1,1], [0,3]], dtype="int32")
    y=gather(a, ids); g=gradients(y.sum(), [a])[0]
    got, grad=Program([y,g]).run({"table":x, "ids":i})
    np.testing.assert_equal(got, x[i]); np.testing.assert_equal(grad, np.array
    ↪ ([[1]*3, [2]*3, [0]*3, [1]*3]))

@pytest.mark.parametrize("op", ["exp", "log", "sqrt", "tanh"])
def test_unary_gradient(op):
    a=param("u", (3,)); x=np.array([0.5, 0.9, 1.4], dtype="float32")
    y=getattr(a, op)().sum(); g=gradients(y, [a])[0]

```

```

p=Program([y,g]);base,analytic=p.run({"u":x});numeric=np.empty_like(x)
for i in range(3):
    xp=x.copy();xm=x.copy();xp[i]+=1e-3;xm[i]-=1e-3
    numeric[i]=(p.run({"u":xp},read=[y])[0]-p.run({"u":xm},read=[y])[0])/(2e-3)
np.testing.assert_allclose(analytic,numeric,rtol=2e-3,atol=2e-3)

def test_composite_gradient():
    a=param("g_a",(2,3));b=param("g_b",(3,4));bias=param("g_bias",(4,))
    y=((a@b+bias).tanh())**1 if False else (a@b+bias).tanh().sum()
    gs=gradients(y,[a,b,bias]);p=Program([y,*gs])
    feed={u.arg:(rng.normal(size=u.shape)*0.2).astype("float32") for u in (a,b,bias)}
    vals=p.run(feed)
    for u,analytic in zip((a,b,bias),vals[1:]):
        x=feed[u.arg];numeric=np.empty_like(x)
        for idx in np.ndindex(x.shape):
            plus=x.copy();minus=x.copy();plus[idx]+=1e-3;minus[idx]-=1e-3
            f1=p.run(**feed,u.arg:plus,read=[y])[0]
            f0=p.run(**feed,u.arg:minus,read=[y])[0]
            numeric[idx]=(f1-f0)/(2e-3)
        np.testing.assert_allclose(analytic,numeric,rtol=3e-3,atol=3e-3)

def test_softmax_and_cross_entropy():
    a=param("logits",(2,3));t=param("target",(2,),"int32")
    x=np.array([[1000,1001,999],[-1000,-1002,-999]],dtype="float32");labels=np.array
    ↪ ([1,2],dtype="int32")
    loss=cross_entropy(a,t);g=gradients(loss,[a])[0]
    got,grad=Program([loss,g]).run({"logits":x,"target":labels})
    z=x-x.max(-1,keepdims=True);probs=np.exp(z)/np.exp(z).sum(-1,keepdims=True)
    target=np.eye(3)[labels]
    np.testing.assert_allclose(got,-np.log(probs[np.arange(2),labels]).mean(),rtol=1e-5)
    np.testing.assert_allclose(grad,(probs-target)/2,rtol=1e-5,atol=1e-6)

def test_arena_reuse_and_fusion():
    a=param("chain",(101,));u=a
    for _ in range(12): u=(u*0.9+0.1).tanh()
    x=rng.normal(size=a.shape).astype("float32")
    p=Program([u],reuse=True,fuse=True);q=Program([u],reuse=False,fuse=False)
    np.testing.assert_allclose(p.run({"chain":x})[0],q.run({"chain":x})[0],rtol=1e-5,
    ↪ atol=1e-6)
    assert len(p.kernels)<len(q.kernels);assert p.bytes<q.bytes

def test_views():
    v=View.contiguous((2,3)).permute((1,0))
    assert v.address((2,1))==5
    assert v.flip(0).address((0,1))==5
    with pytest.raises(ValueError): v.contiguous_reshape((6,))
    assert View.contiguous((1,3)).expand((4,3)).address((3,2))==2

def test_symbolic():
    i=var("i");e=(i*1+0)*3
    assert simplify(e)==i*3
    assert interval((i+3)*2,{"i":(0,7)})==(6,20)

```

```

    for n in range(100): assert e.evaluate({"i":n})==simplify(e).evaluate({"i":n})

def test_shapes_and_errors():
    with pytest.raises(ValueError): broadcast((2,3),(4,3))
    with pytest.raises(ValueError): param("bad",(-1,))
    with pytest.raises(ValueError): param("s",(2,3)).reshape(5)
    with pytest.raises(ValueError): Program([param("unset",(1,))+1]).run()

def test_decoder_causality():
    m=Decoder(Config(vocab=7,batch=1,context=4,dim=8,heads=2,kv_heads=1,layers=1,hidden
    ↪ =16))
    p=Program([m.logits])
    for name,value in m.initial.items():
        if name in p.params:p.set(name,value)
    x=np.array([[1,2,3,4]],dtype="int32");y=x.copy();y[0,3]=6
    a=p.run({"tokens":x})[0];b=p.run({"tokens":y})[0]
    np.testing.assert_allclose(a[:, :3],b[:, :3],rtol=1e-6,atol=1e-6)

def test_adam_learning_and_assignment():
    w=param("learn",());loss=(w-3)*(w-3)
    g=gradients(loss,[w])[0];new=w-0.1*g
    p=Program([loss,new]);p.set("learn",np.array(0,dtype="float32"))
    first=float(p.run(read=[loss])[0])
    for _ in range(25): p.run(read=[]);p.assign({"learn":new})
    last=float(p.run(read=[loss])[0]);assert last<first*1e-4

@pytest.mark.parametrize('shape',[ (3,5,7),(8,16,64),(1,1,0),(9,17,65)])
def test_blocked_gemm(shape):
    M,N,K=shape; rng=np.random.default_rng(32)
    a=param('a',(2,M,K));b=param('b',(1,K,N));out=a@b
    av=rng.normal(size=a.shape).astype('float32');bv=rng.normal(size=b.shape).astype('
    ↪ float32')
    with Program([out],gemm='blocked') as p:
        np.testing.assert_allclose(p.run({'a':av,'b':bv})[0],av@bv,atol=2e-5,rtol=2e-5)

@pytest.mark.parametrize('T,D',[ (1,2),(5,4),(17,7)])
def test_online_attention_and_gradients(T,D):
    from lumbre import online_attention,softmax
    from lumbre.ir import node
    rng=np.random.default_rng(63)
    q,k,v=[param(n,(1,2,T,D)) for n in ('q','k','v')]
    online=online_attention(q,k,v)
    dense=softmax((q@k.transpose())/(D**0.5)+node('causal',(T,T)))@v
    g1=gradients((online*online).sum(),[q,k,v]);g2=gradients((dense*dense).sum(),[q,k,v])
    ↪
    values={n:rng.normal(size=q.shape).astype('float32') for n in ('q','k','v')}
    with Program([online,dense,*g1,*g2],gemm='blocked') as p:
        results=p.run(values)
    np.testing.assert_allclose(results[0],results[1],atol=2e-5,rtol=2e-5)
    for i in range(3): np.testing.assert_allclose(results[2+i],results[5+i],atol=4e-5,
    ↪ rtol=4e-5)

def test_assign_rejects_aliasing_parameter_outputs():

```

```

a=param('a',(2,)); b=param('b',(2,))
with Program([a,b]) as p:
    p.run({'a':np.array([1,2]),'b':np.array([3,4])})
    with pytest.raises(ValueError): p.assign({'a':b,'b':a})

```

A.20 tests/test_passport.py

```

import pytest
from lumbre.passport import Launch, verify

@pytest.mark.parametrize("shape", [(0, 7), (1, 1), (16, 16), (17, 23), (31, 65)])
def test_logical_map(shape):
    r = verify(Launch(*shape))
    assert r["status"] == "LOGICAL_MAP_VALIDATED"
    assert r["valid_writes"] == shape[0] * shape[1]

def test_wrong_stride_detected():
    r = verify(Launch(3, 5, 2, 4), lambda y, x, s: y * s.rows + x)
    assert r["status"] == "FAILED"
    assert r["missing"] and r["duplicate_writes"]

def test_outside_detected():
    r = verify(Launch(5, 3), lambda y, x, s: y * s.rows + x)
    assert r["out_of_range"]

def test_one_origin_bug():
    r = verify(Launch(2, 3), lambda y, x, s: y * s.columns + x + 1)
    assert r["missing"] == [0]
    assert r["out_of_range"] == [(1, 2, 6)]

def test_invalid_launch():
    for args in [(-1, 3), (2, 3, 0, 8), (1001, 1001)]:
        with pytest.raises(ValueError):
            Launch(*args)
    with pytest.raises(TypeError):
        Launch(2.5, 3)

```

A.21 tests/test_tokenizer.py

```
import pytest
from lumbre.tokenizer import ByteBPE, replace_pair

@pytest.mark.parametrize('text', ['', 'aaaaaaa', 'el gato mira el gato', 'á, ñ, \u4e2d\u6587',
    ↪ y un símbolo: \U0001f408'])
def test_bpe_roundtrip(text):
    t=ByteBPE.fit(text,25)
    assert t.decode(t.encode(text))==text
    assert ByteBPE.from_state(t.state())==t
    unseen='otra cadena: \u03bb'
    assert t.decode(t.encode(unseen))==unseen

def test_nonoverlap():
    assert replace_pair([1,1,1],(1,1),256)==[256,1]

def test_bpe_validation():
    with pytest.raises(ValueError): ByteBPE(((256,0),))
    with pytest.raises(ValueError): ByteBPE().decode([256])
    with pytest.raises(ValueError): ByteBPE.fit('x',-1)
```


CAPÍTULO B

Registro de validación y alcance de la entrega

Cómo leer las evidencias

Se separan hechos ejecutados en esta edición, derivaciones del texto, resultados de autores externos y extensiones propuestas. Un archivo de código presente no se convierte automáticamente en una prueba de ejecución. Este registro describe el paquete entregado, no todas las capacidades de los proyectos citados.

B.1 Entorno observado

Las pruebas originales se ejecutaron en Linux 6.18.44, x86_64, glibc 2.41, con Python 3.13.5, NumPy 2.3.5, pytest 9.0.2 y GCC 14.2.0. El comando `cc` resolvía al compilador C del entorno. En aquel entorno no estaban disponibles `nvcc`, `hipcc` ni `nvidia-smi`; el archivo `reports/environment.json` conserva ese inventario histórico.

La revisión local del 1 de octubre utiliza Ubuntu 24.04 bajo WSL2, Python 3.12.3, GCC 13.3.0, NumPy 2.3.5 y pytest 9.0.2. El dispositivo es una NVIDIA GeForce RTX 5070 Ti Laptop de 12 GB, controlador 591.86, CUDA Toolkit 12.9.86 y destino `sm_120`. Se conservaron los registros originales y se guardaron las nuevas evidencias en `reports/validation_20261001/`. HIP sigue pendiente de hardware compatible.

El entorno virtualizado no ofrece una base suficiente para atribuir los resultados a un modelo comercial concreto de procesador o predecir el rendimiento del ordenador del lector. Por esa razón, la tabla local compara implementaciones en el mismo entorno y no presume equivalencia con una CPU o GPU del estudiante.

B.2 Batería y ejemplos numéricos

La ejecución original con una caché nueva produjo **61 pruebas superadas**. El tiempo informado por pytest fue 7,03 segundos; esa cifra es de la batería, no del entrenamiento ni de un kernel aislado. Comprende comprobaciones de operaciones elemento a elemento, formas vacías admitidas, broadcasting, reducciones, vistas lógicas, `matmul`, gradientes, `arena`, atención, convoluciones, tokenizador y pasaporte de índices.

La revisión local repite la batería y los tres ejemplos numéricos, con selección explícita del backend mediante `pytest -backend=cuda` y `validate_book.py -backend cuda`. Los casos de memoria, índices, tokenización y la convolución C independiente conservan su ejecución anfitriona; los programas tensoriales seleccionan CUDA. También se verifican longitudes 0, 1, 127, 128, 129 y 257, doce mediciones residentes por backend y la igualdad exacta de

checkpoints de diez pasos frente a seis más cuatro, tanto en CPU como en CUDA. Los archivos `tests_cpu.log`, `tests_cuda.log` y `resume_cuda.json` están en la carpeta de la revisión.

El programa `examples/validate_book.py` verificó además los tres ejemplos numéricos de integración: pérdida matricial y gradientes; convolución y gradientes calculados a mano; y un paso de regresión escalar. Su informe es `reports/book_examples.json`. Estas comprobaciones no se presentan como ejecución automática de cada fragmento aislado del texto: algunos son pseudocódigo, contratos o ejercicios que requieren completar un contexto anterior.

Componente	Evidencia de esta edición	Límite que se conserva
Compilador C y runtime	Pruebas y ejemplos ejecutados	Subconjunto estático; no todo operador de cualquier framework.
Autodiferenciación	Comparación analítica, diferencial y entrenamiento	Convenciones explícitas en máximos; no todos los puntos son suaves.
GEMM bloqueada	Pruebas y tiempos CPU/CUDA	No se midió competitividad frente a BLAS o PyTorch.
Convolución C	Compilación y comparación numérica	No es una biblioteca de convoluciones optimizada completa.
Convolución por IR	Pruebas forward y gradientes	Límite didáctico de posiciones; grafos costosos al crecer.
Atención online	Comparación forward y gradientes; benchmark	Backward recompone operaciones densas.
Decoder	600 actualizaciones; checkpoint y evaluación	Corpus repetido, escala pequeña, no calidad de frontera.
ByteBPE	Ida y vuelta, estado y entrenamiento breve	Tokenizador educativo, no réplica de uno comercial.
Pasaporte de índices	15 mapas y rechazo de un error intencional	Enumeración lógica CPU; no ejecución de hardware.
CUDA y WMMA	Compilación y ejecución en RTX 5070 Ti Laptop; WMMA con error cero	Alcance local, sin comparación industrial; HIP pendiente.
ZeRO, MoE distribuido, FP8/FP4	Teoría y diseño de proyectos	No son funcionalidades ejecutadas del runtime entregado.

B.3 Entrenamiento principal

La configuración fue vocabulario 24, lote 2, contexto 32, dimensión 64, dos capas, dos cabezas de consulta, una cabeza de clave/valor y dimensión intermedia 128. Se activaron atención online y GEMM bloqueada. Hay 75.584 parámetros, 447 unidades de kernel en el plan de entrenamiento y una arena de 2.100.160 bytes bajo la definición del compilador.

El registro contiene 600 actualizaciones. Primera pérdida: 3,2164369. Última pérdida: 0,2746565. Pérdida de evaluación: 0,3321578, media de cuatro lotes del fragmento reservado del corpus

repetido. El tiempo del bucle fue 3,4371843 segundos, sin compilación ni la evaluación y generación posteriores. La compilación de ese registro fue un acierto de caché. Todos esos campos se pueden inspeccionar en `reports/decoder_final/report.json`.

La comparación entre diez actualizaciones continuas y una ejecución de seis más cuatro reanudadas coincidió exactamente en los campos guardados del checkpoint, en el mismo entorno y configuración. El resultado se conserva en `reports/resume_check.json`. No se extrapola identidad bit a bit a otros dispositivos, tipos o órdenes de reducción.

La revisión CUDA repite las 600 actualizaciones de la configuración de 75.584 parámetros. La pérdida final es 0,2508018 y la pérdida de evaluación 0,3293002. El bucle de entrenamiento tarda 32,493 segundos, sin compilación, evaluación ni generación posteriores. Su checkpoint e informe se guardan en `reports/validation_20261001/decoder_cuda_75584/`. Los tiempos proceden de entornos diferentes y no definen una comparación de velocidad CPU/GPU. Las diferencias numéricas entre backends no invalidan la comprobación exacta de reanudación dentro de cada backend.

Una evaluación reservada no siempre es independiente

El corpus de demostración repite frases. Un corte del texto puede dejar secuencias prácticamente iguales en entrenamiento y evaluación. La pérdida demuestra que el sistema aprende ese material de depuración; no demuestra generalización lingüística independiente. La ruta de experimentación con documentos separados se desarrolla en los capítulos de datos y proyectos.

B.4 Campaña secuencial CPU/CUDA

El capítulo 69, p.244, amplía la validación con doce casos de kernels por backend y treinta muestras residentes por caso. Repite también las 600 actualizaciones del modelo principal en CPU y CUDA dentro del mismo host WSL. Las tareas se ejecutan en secuencia; los informes no presuponen aislamiento de frecuencias, potencia o carga del escritorio.

Los archivos de `reports/book_gpu_extension/` conservan muestras de tiempo, referencias numéricas, metadatos de compilación, configuraciones, historias y checkpoints. `campaign.json` documenta el orden de las tareas. El capítulo desarrolla cómo interpretar las diferencias entre una llamada residente, la compilación y el bucle de entrenamiento; las tablas proceden de esos JSON.

B.5 Cómo distinguir reportes históricos

El paquete conserva un registro de la configuración final y uno de la ruta BPE de depuración. No se mezclan sus pérdidas ni vocabularios. Las muestras de generación son resultados del modelo, no ejemplos editados para mejorar su aspecto. Los archivos de reportes pueden contener rutas de caché del entorno original: son procedencia del ensayo; el runtime creará sus propias rutas al reproducirlo.

No se suministran bibliotecas nativas compiladas, controladores ni firmware para instalar. Los repositorios externos se enlazan en la bibliografía. El código completo del apéndice anterior y los archivos del directorio `code/` son las fuentes del compilador educativo. El manifiesto del

paquete permite comprobar que no se han modificado accidentalmente.

CAPÍTULO C

Reproducir, modificar y reconstruir el material

C.1 Primera instalación con una ruta única

Descomprime el paquete y abre una terminal en su directorio. Entra en `code/`. Utiliza un entorno virtual para no mezclar las dependencias de este curso con las de otros proyectos. El compilador nativo debe estar disponible antes de las pruebas. En Windows, utiliza la ruta Linux/WSL explicada en el libro; cambiar extensiones de bibliotecas no adapta un ABI.

```
cd code
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[test]'
python -m pytest -q
python examples/validate_book.py
python examples/verify_launch.py
```

La instalación editable hace visible `lumbre` a los programas de `examples/`. Sin ella, antepone `PYTHONPATH=.` a las órdenes ejecutadas desde la raíz del código. No interpretes un fallo de importación como un fallo del algoritmo de autodiferenciación: todavía no se ha llegado a ese algoritmo.

C.2 Reproducción fría y caliente

Para estudiar una compilación limpia, utiliza una carpeta de caché nueva en lugar de borrar indiscriminadamente archivos de otros proyectos. Para estudiar tiempo caliente, conserva esa caché y registra que se reutilizó. El programa guarda la fuente generada junto al artefacto compilado.

```
LUMBRE_CACHE="$PWD/cache_del_ensayo" python -m pytest -q
python examples/bench_kernels.py --repeat 20 \
  --output reports/mi_benchmark.json
```

El benchmark utiliza entradas residentes y excluye la copia de las salidas dentro de la región cronometrada. Si se desea tiempo extremo a extremo, se crea otro ensayo y se nombra de forma diferente. No se cambia silenciosamente la frontera medida conservando la misma etiqueta en una tabla.

C.3 Modificar el compilador por capas

Para añadir una operación, escribe primero su contrato: forma, tipo, valores especiales y efectos. Añade una referencia sencilla y pruebas. Después incorpora el nodo, su generación C y, cuando proceda, su VJP. Finalmente estudia fusión, planificación y una ruta GPU. Una operación que compila pero carece de una política para formas inválidas aún no tiene una interfaz completa.

Para cambiar planificación, conserva un modo sin fusión ni reciclaje y compara resultados. Para cambiar precisión, no reutilices tolerancias sin examinarlas. Para añadir una llamada externa, declara la biblioteca, versiones, layout, workspace y stream. Esa llamada puede ser una excelente decisión de ingeniería, pero no debe atribuirse a código generado íntegramente por Lumbre.

C.4 Diagnóstico de problemas frecuentes

Síntoma	Primera comprobación útil
No se importa <code>lumbre</code>	Confirma instalación editable o <code>PYTHONPATH</code> ; comprueba el intérprete y directorio actual.
No se encuentra <code>cc</code>	Comprueba el compilador nativo y <code>PATH</code> . Aún no es una discrepancia de resultados tensoriales.
Se rechaza una forma	Lee la forma esperada y la recibida. No fuerces un reshape que cambie el número de elementos.
Checkpoint incompatible	Compara arquitectura, corpus y vocabulario. El rechazo evita interpretar pesos con otra semántica.
Pérdida no finita	Conserva el primer paso y reduce el caso; localiza la primera operación no finita antes de cambiar varias opciones.
CPU correcta, GPU falla	Registra compilación, arquitectura y ejecución por separado; no traslades automáticamente la validación CPU.
Primera ejecución muy lenta	Separa construcción, compilación, carga y ejecución. Revisa si las posteriores usan caché.
Atención ahorra memoria pero pierde tiempo	Conserva el resultado y realiza ablaciones de granularidad, vectorización y tamaño; no alteres la referencia para ocultarlo.

C.5 Compilar el libro

Para reconstruir el PDF y eliminar los auxiliares al terminar, usa desde la raíz del repositorio:

```
python build_book.py --pdf --report code/reports/book_gpu_extension/book_build.json
```

La opción `-pdf` compila una copia autocontenida en una carpeta temporal. Ejecuta tres pasadas, comprueba referencias y cajas desbordadas y copia el PDF final al repositorio solo si supera esas comprobaciones. Al terminar elimina la carpeta temporal y sus archivos auxiliares,

incluso si una pasada falla. El informe JSON conserva el resultado, los hashes y los tiempos sin acumular archivos `.aux`, `.toc`, `.out` o registros LaTeX. Ejecutar el constructor sin `-pdf` regenera únicamente el fuente. Las órdenes directas de LaTeX que se muestran a continuación permiten estudiar las pasadas por separado y dejan sus auxiliares en el directorio de trabajo.

El archivo principal `Compiladores_ML_Desde_Cero_2026.tex` contiene los listados incrustados y los diagramas vectoriales; no necesita copiar fuentes tipográficas del paquete. Requiere una distribución LaTeX con los paquetes declarados en su preámbulo. Dos o tres pasadas actualizan referencias, páginas e índice.

```
pdflatex -interaction=nonstopmode -halt-on-error \  
  Compiladores_ML_Desde_Cero_2026.tex  
pdflatex -interaction=nonstopmode -halt-on-error \  
  Compiladores_ML_Desde_Cero_2026.tex
```

El paquete incluye además los capítulos fuente y `build_book.py`. Al ejecutar ese script desde su ubicación original se regenera el `.tex` con las fuentes del código incluidas. Si se modifica el software, conviene regenerar también los listados para que libro y paquete no diverjan. El PDF suministrado es la versión maquetada de referencia de esta edición.

CAPÍTULO D

Mapa de cobertura y puertas del semestre

D.1 Correspondencia con el temario solicitado

Este mapa sirve para localizar los contenidos, no para sustituir su lectura. El programa completo aparece en la p.4; el contrato de UOps, en la p.254; y su desarrollo hasta entrenamiento, en la p.260. Las páginas son referencias internas del PDF y pueden cambiar al recompilar con otra edición o tipografía.

Bloque	Núcleo del libro	Práctica y evaluación
Preparación desde cero	Programas y taller, p.9; tensores, p.19; números, p.24.	Predicciones manuales y programas mínimos.
Semanas 1–2: UOps, elementwise, symbolic, rewrites, renderer	UOps, p.29; C, p.33; índices, p.37; reescrituras, p.41.	Laboratorios 1–2, pp.185 y 188.
Semanas 3–4: loops, movement, reduction, rangeify	Vistas, p.45; recorridos, p.49; reducción, p.53; kernels, p.57.	Laboratorios 3–4, pp.191 y 194.
Semanas 5–6: fisión, fusión, autodiff, MNIST y CPU rápida	Kernels, p.57; autodiff, p.119; memoria, p.69; SIMD, p.73; GEMM y conv.	Laboratorios 4–6 y 8. MNIST es una entrega propuesta, no un resultado local acreditado.
Semanas 7–8: call, GPU, aceleradores, tensor cores	GPU, p.89; runtime, p.93; memoria, p.97; tensor cores, p.101; AMD y Ascend.	Laboratorio 7, p.203; pasaporte, p.224; CUDA validada, HIP pendiente.
Semanas 9–10: modelos reales y formas simbólicas	Contrato simbólico, p.260; decoder, p.135; entrenamiento, p.141.	Laboratorios 9–11; reutilización de autodiff, entrenamiento y checkpoint.
Escala LLM moderna	Atención eficiente, p.151; memoria, p.156; precisión, p.160; distribuido, p.163; MoE y KV cache.	Proyecto LLM, p.233; separar arquitectura estudiada de escala ejecutada.
Semanas 11–final: proyectos	Atención, p.220; pasaporte, p.224; rendimiento, p.227; agentes, p.230; LLM mayor.	Hipótesis, implementación, pruebas, ablación, informe y defensa.

Bloque	Núcleo del libro	Práctica y evaluación
Investigación y repositorios	tinygrad, p.170; Tiny Corp, p.173; LLVM/MLIR, p.176; lenguajes, p.179; artículos, p.182.	Lectura por contratos y atribución de evidencia; bibliografía enlazada.

D.2 Qué tiene que saber explicar el estudiante

Al terminar las semanas 1–2 debe construir un grafo, justificar una reescritura y localizar su efecto en el C. Al terminar las semanas 3–4 debe obtener índices de una composición de vistas y reducciones. En las semanas 5–6 debe planificar kernels, diferenciar el grafo puro, entrenar su primer MNIST y medir una mejora CPU con un baseline explicado. En las semanas 7–8 debe distinguir cálculo, lanzamiento y sincronización, y validar en hardware las rutas que vaya a afirmar como ejecutadas.

En las semanas 9–10 debe comprobar una pérdida, generar gradientes y reanudar entrenamiento conservando el estado. El proyecto final añade una hipótesis propia y una contribución identificable. Un resultado negativo con explicación puede cumplir ese objetivo. Una afirmación de SOTA sin suite, fecha, contrato y comparación no lo cumple.

D.3 Frontera entre libro y proyecto de producción

El libro explica mecanismos más amplios que los implementados en el pequeño compilador de referencia. Los capítulos de precisión mixta, distribución, MoE y entrenamiento a gran escala enseñan lo que habría que diseñar y validar; no convierten esas capacidades en funciones ya presentes. A la inversa, el código de CPU, los gradientes y el entrenamiento pequeño no son meros planes: se han ejecutado y sus reportes se entregan.

Conservar esta frontera permite utilizar la obra como curso completo y como punto de partida técnico sin confundir una herramienta pedagógica con una plataforma de producción ya certificada. La siguiente edición de un proyecto debe añadir pruebas y resultados concretos, no solo cambiar su descripción.

Bibliografía y recursos verificables

Todas las páginas web se consultaron para el corte del 1 de octubre de 2026. Una URL de rama principal puede cambiar; únicamente se presenta como snapshot el commit indicado explícitamente. Las referencias no son una clasificación de calidad ni una validación independiente de las afirmaciones de rendimiento de sus autores.

- [1] tinygrad. Snapshot de código, commit `c3aec477b99d9bb87c54d91897cf60acd3f17441`, 30-09-2026. <https://github.com/tinygrad/tinygrad/tree/c3aec477b99d9bb87c54d91897cf60acd3f17441>.
- [2] tinygrad. *Developer documentation*. <https://docs.tinygrad.org/developer/developer/>.
- [3] Tiny Corp. Sitio y explicación de operaciones. <https://tinygrad.org/>.
- [4] tinygrad. Pipeline de generación del snapshot. https://github.com/tinygrad/tinygrad/blob/c3aec477b99d9bb87c54d91897cf60acd3f17441/tinygrad/codegen/__init__.py.
- [5] Tiny Corp. Organización pública y repositorios. <https://github.com/tinygrad/>.
- [6] George Hotz. *Five years of tinygrad*, 29-12-2025. <https://geohot.github.io/blog/jekyll/update/2025/12/29/five-years-of-tinygrad.html>.
- [7] NVIDIA. *CUDA on WSL User Guide*. <https://docs.nvidia.com/cuda/wsl-user-guide/index.html>.
- [8] NVIDIA. *Floating Point and IEEE 754 Compliance for NVIDIA GPUs*. <https://docs.nvidia.com/cuda/floating-point/index.html>.
- [9] NVIDIA. *CUDA Programming Guide*. <https://docs.nvidia.com/cuda/cuda-programming-guide/>.
- [10] NVIDIA. *Parallel Thread Execution ISA*. <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>.
- [11] NVIDIA. CUTLASS, README y código; sección 4.8.0 de septiembre de 2026. <https://github.com/NVIDIA/cutlass>.
- [12] NVIDIA. cuTile Python. <https://github.com/NVIDIA/cutile-python>.
- [13] NVIDIA. Open GPU Kernel Modules. <https://github.com/NVIDIA/open-gpu-kernel-modules>.
- [14] AMD. hipBLASLt, aviso de migración al monorepo. <https://github.com/ROCm/hipBLASLt>.
- [15] AMD. Composable Kernel, aviso de migración. https://github.com/ROCm/composable_kernel.
- [16] AMD. ROCm Libraries. <https://github.com/ROCm/rocm-libraries>.

- [17] AMD. AITER. <https://github.com/ROCm/aiter>.
- [18] AMD. ROCm LLVM Project. <https://github.com/ROCm/llvm-project>.
- [19] AMD. HIP documentation. <https://rocm.docs.amd.com/projects/HIP/en/latest/>.
- [20] Linux kernel documentation. AMDGPU driver. <https://docs.kernel.org/gpu/amdgpu/index.html>.
- [21] Linux kernel documentation. DRM memory management. <https://docs.kernel.org/gpu/drm-mm.html>.
- [22] Linux. Fuente del kernel. <https://github.com/torvalds/linux>.
- [23] DeepSeek. DeepGEMM. <https://github.com/deepseek-ai/DeepGEMM>.
- [24] DeepSeek-AI. *DeepGEMM-Ascend*. Revisión 8491bbb4b8c0, 30-09-2026; hash completo en el enlace. Requisitos e interfaces. <https://github.com/deepseek-ai/DeepGEMM-Ascend/tree/8491bbb4b8c02a094a2318965f50c70438a3e73c>.
- [25] DeepSeek. DeepEP. <https://github.com/deepseek-ai/DeepEP>.
- [26] DeepSeek. DeepEP-Ascend: README y restricciones de la entrega preliminar. <https://github.com/deepseek-ai/DeepEP-Ascend>.
- [27] DeepSeek. DeepJIT. <https://github.com/deepseek-ai/DeepJIT>.
- [28] Huawei Ascend. Samples. <https://github.com/Ascend/samples>.
- [29] Tile-AI. TileLang. <https://github.com/tile-ai/tilelang>.
- [30] TileLang. Documentación y API. <https://www.tilelang.com/autoapi/>.
- [31] Wang et al. *TileLang: A Composable Tiled Programming Model for AI Systems*. 2025. <https://arxiv.org/abs/2504.17577>.
- [32] Triton. Tutorial de multiplicación de matrices. <https://triton-lang.org/main/getting-started/tutorials/03-matrix-multiplication.html>.
- [33] Triton. Compilador y lenguaje. <https://github.com/triton-lang/triton>.
- [34] LLVM. MLIR Toy Tutorial. <https://mlir.llvm.org/docs/Tutorials/Toy/>.
- [35] Lattner et al. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. 2020. <https://arxiv.org/abs/2002.11054>.
- [36] LLVM. Language Reference Manual. <https://llvm.org/docs/LangRef.html>.
- [37] Chen et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. 2018. <https://arxiv.org/abs/1802.04799>.
- [38] Willsey et al. *egg: Fast and Extensible Equality Saturation*. Trabajo original POPL 2021; presentación CACM 2026. <https://doi.org/10.1145/3815481>.
- [39] Williams, Waterman y Patterson. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. 2009. <https://doi.org/10.1145/1498765.1498785>.
- [40] Vaswani et al. *Attention Is All You Need*. 2017. <https://arxiv.org/abs/1706.03762>.

- [41] Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. 2022. <https://arxiv.org/abs/2205.14135>.
- [42] Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. 2023. <https://arxiv.org/abs/2307.08691>.
- [43] Shah et al. *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision*. 2024. <https://arxiv.org/abs/2407.08608>.
- [44] Dao-AILab. FlashAttention: implementaciones y requisitos por backend, incluida FA4 con CuTe DSL. <https://github.com/Dao-AILab/flash-attention>.
- [45] HazyResearch. ThunderKittens. <https://github.com/HazyResearch/ThunderKittens>.
- [46] Karpathy et al. llm.c. <https://github.com/karpathy/llm.c>.
- [47] Karpathy. nanoGPT. <https://github.com/karpathy/nanoGPT>.
- [48] ScalingIntelligence. KernelBench. <https://github.com/ScalingIntelligence/KernelBench>.
- [49] DeepSeek-AI. *DeepSeek-V3 Technical Report*. 2024. <https://arxiv.org/abs/2412.19437>.
- [50] DeepSeek-AI. DeepSeek-V4-Pro, model card y reporte enlazado. <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>. Reporte: <https://arxiv.org/abs/2606.19348>.
- [51] Costa et al. *AI as a Compiler: Compiling Triton kernels without the Triton compiler*. 29-09-2026. <https://arxiv.org/abs/2609.36800>.
- [52] Mo et al. *TileSight: A First-Principles Tile-Centric Analytical GPU Performance Model from Cores to Clusters*. 2026. <https://arxiv.org/abs/2607.22432>.
- [53] *Correct but Slow: An Empirical Study of the GPU Kernel Evaluation Gap in Modern Domain-Specific Languages*. 2026. <https://arxiv.org/abs/2607.04454>.
- [54] *Characterizing Real-World Bugs in Tile Programs for Automated Bug Detection*. 2026. <https://arxiv.org/abs/2605.19652>.
- [55] NVIDIA. CUDA GPU Compute Capability. <https://developer.nvidia.com/cuda/gpus>.
- [56] D. P. Kingma y J. Ba. *Adam: A Method for Stochastic Optimization*. 2014, ICLR 2015. <https://arxiv.org/abs/1412.6980>.
- [57] I. Loshchilov y F. Hutter. *Decoupled Weight Decay Regularization*. ICLR 2019. <https://arxiv.org/abs/1711.05101>.
- [58] J. Ainslie y colaboradores. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. <https://arxiv.org/abs/2305.13245>.
- [59] J. Su y colaboradores. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021. <https://arxiv.org/abs/2104.09864>.
- [60] B. Zhang y R. Sennrich. *Root Mean Square Layer Normalization*. 2019. <https://arxiv.org/abs/1910.07467>.

- [61] N. Shazeer. *GLU Variants Improve Transformer*. 2020. <https://arxiv.org/abs/2002.05202>.
- [62] S. Rajbhandari y colaboradores. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. 2019/2020. <https://arxiv.org/abs/1910.02054>.
- [63] M. Shoeybi y colaboradores. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. 2019/2020. <https://arxiv.org/abs/1909.08053>.
- [64] W. Kwon y colaboradores. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. <https://arxiv.org/abs/2309.06180>.
- [65] J. Hoffmann y colaboradores. *Training Compute-Optimal Large Language Models*. 2022. <https://arxiv.org/abs/2203.15556>.
- [66] NVIDIA. NCCL. <https://github.com/NVIDIA/nccl>.
- [67] AMD. RCCL: aviso de migración a ROCm/rocm-systems. <https://github.com/ROCm/rccl>.
- [68] Tiny Corp. tinyos: constructor de imágenes de sistema para tinybox. <https://github.com/tinygrad/tinyos>.
- [69] Tiny Corp. custom_mec_public: firmware MEC, emulador y pruebas para gfx1100. https://github.com/tinygrad/custom_mec_public.
- [70] DeepSeek-AI. Kernel BF16 de DeepGEMM-Ascend, misma revisión. https://github.com/deepseek-ai/DeepGEMM-Ascend/blob/8491bbb4b8c02a094a2318965f50c70438a3e73c/deep_gemm/include/deep_gemm/bf16_gemm.hpp.
- [71] DeepSeek-AI. Sincronización de pipelines Ascend, misma revisión. https://github.com/deepseek-ai/DeepGEMM-Ascend/blob/8491bbb4b8c02a094a2318965f50c70438a3e73c/deep_gemm/include/deep_gemm/ascend/sync.hpp.
- [72] DeepSeek-AI. *DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression*. 2026, arXiv:2609.19969v1, secciones 2, 3.2 y 6. <https://arxiv.org/abs/2609.19969v1>.

Índice detallado

Empieza aquí	1
0.1. Qué significa completar el curso	1
0.2. Un semestre con punto de partida honesto	2
0.3. Cómo utilizar el código entregado	3
0.4. Fuentes, fechas y límites de esta investigación	3
Programa del semestre: Compilers for Machine Learning	4
0.5. El encargo y la forma de trabajar	4
0.6. El semestre de un vistazo	5
0.7. Semana 1. UOps y reescrituras	5
0.8. Semana 2. Renderer, runtime y memoria	6
0.9. Semana 3. Bucles, GEMM y convolución	6
0.10. Semana 4. Movimientos y rangeify	6
0.11. Semana 5. Fisión, fusión y autodiferenciación	6
0.12. Semana 6. Memoria, multihilo y kernels rápidos en CPU	7
0.13. Semanas 7–8. GPU y aceleradores	7
0.14. Semanas 9–10. Modelos reales y dimensiones simbólicas	7
0.15. Semanas 11 en adelante. Proyecto y defensa	8
0.16. Qué se conserva entre entregas	8
1. Un programa que fabrica otro programa	9
1.1. Cuatro temperaturas y una regla	9
1.2. Variables: etiquetas para valores	9
1.3. De la lista al array de C	10
1.4. Compilar y ejecutar no son la misma acción	11
1.5. Qué hace un compilador de aprendizaje automático	11
1.6. Un generador de texto ya es un primer paso	11
1.7. Un problema, tres tiempos	12
1.8. Ejercicios con respuesta explicada	12
2. Preparar el taller y aprender a leer un fallo	14
2.1. Una carpeta es parte del experimento	14
2.2. Terminal sin conocimientos previos	14
2.3. Entorno Python aislado	15
2.4. Funciones: una tarea con entradas y salida	15
2.5. Excepciones y códigos de salida	16
2.6. La primera prueba automatizada	16
2.7. Tres clases de prueba	16
2.8. Leer el código generado	17
2.9. Seguridad básica del JIT	17
2.10. Git como cuaderno de decisiones	17

3.	Tensores: números con una organización	19
3.1.	Escalar, vector y matriz	19
3.2.	Forma y número de elementos	19
3.3.	Operaciones elemento a elemento	20
3.4.	Broadcasting: reutilizar sin inventar dimensiones	20
3.5.	Reducciones: muchas posiciones producen una	21
3.6.	Producto escalar: comparar dos listas de características	21
3.7.	Multiplicación matricial desde una sola casilla	21
3.8.	Transponer no es invertir	22
3.9.	Lotes de matrices	22
3.10.	Ejercicios resueltos	22
4.	Los números de la máquina no son los números de la pizarra	24
4.1.	Una regla con pocas marcas	24
4.2.	Bits, bytes y tipos	24
4.3.	Signo, exponente y fracción	24
4.4.	El orden de una suma puede cambiar el resultado	25
4.5.	Infinito, NaN y ceros con signo	25
4.6.	FMA: una multiplicación y una suma con un único redondeo	26
4.7.	Error absoluto y relativo	26
4.8.	Reducciones largas y acumulación	27
4.9.	Softmax y el truco que evita un desbordamiento	27
4.10.	Qué comprobar antes de optimizar	27
5.	UOps: construir un grafo de intenciones	29
5.1.	De una fórmula escrita a piezas conectadas	29
5.2.	Qué información necesita un nodo	29
5.3.	Un grafo dirigido sin ciclos	30
5.4.	Orden topológico explicado como lista de tareas	30
5.5.	Identidad frente a igualdad estructural	31
5.6.	El frontend verifica el significado	31
5.7.	Por qué la forma es información del compilador	31
5.8.	Parámetros y constantes	32
5.9.	Representación impresa y representación interna	32
5.10.	Ejercicios de construcción	32
6.	Del grafo al C: renderer, ABI y primera ejecución	33
6.1.	Traducir sin cambiar de tarea	33
6.2.	Qué es un puntero sin metáforas misteriosas	33
6.3.	Firmas y ABI	33
6.4.	Compilar una biblioteca compartida	34
6.5.	Un renderer recursivo reducido	34
6.6.	Paréntesis y precedencia	35
6.7.	Fusión: un recorrido en lugar de varios	35
6.8.	Primera ejecución completa con Lumbre	35
6.9.	Caché de compilación	36
6.10.	Errores que deben resultar visibles	36

7. Índices simbólicos: calcular con lo que todavía no sabemos	37
7.1. Una casilla cuyo número depende de otra	37
7.2. Variables, constantes y operaciones	37
7.3. Por qué los intervalos son útiles	37
7.4. División y resto: encontrar fila y columna	38
7.5. Simplificaciones que necesitan hipótesis	38
7.6. Máscaras como proposiciones	39
7.7. Aritmética matemática y anchura de máquina	39
7.8. Rangos relacionados: lo que un intervalo pierde	39
7.9. Expresiones afines y transformaciones de bucles	39
7.10. Cómo probar un simplificador simbólico	40
8. Reescrituras: mejorar el programa conservando su contrato	41
8.1. Cambiar el plan, no el resultado pedido	41
8.2. Pattern matching explicado con piezas	41
8.3. Reescribir de abajo hacia arriba	41
8.4. Punto fijo y riesgo de bucles	42
8.5. Constant folding	42
8.6. Eliminar trabajo muerto	43
8.7. Equivalencia: qué debemos conservar	43
8.8. E-graphs y saturación de igualdades	43
8.9. Costes que no se suman de manera local	44
8.10. Pruebas diferenciales y metamórficas	44
8.11. Un expediente de regla	44
9. Movement ops: cambiar el mapa sin mover las cajas	45
9.1. Una estantería y dos maneras de leerla	45
9.2. Strides: cuánto avanzar al cambiar una coordenada	45
9.3. Transposición como intercambio de strides	45
9.4. Reshape no siempre es una vista simple	46
9.5. Expand y stride cero	46
9.6. Slicing: cambiar el inicio y la extensión	47
9.7. Invertir un eje y strides negativos	47
9.8. Padding: una lectura condicional	47
9.9. Componer movimientos	47
9.10. El backward de un mapa	48
10. Loops y rangeify: convertir un tensor en recorridos	49
10.1. La receta tensorial necesita índices	49
10.2. Un bucle lineal puede recorrer varios ejes	49
10.3. Bucles anidados y equivalencia con el índice lineal	49
10.4. Ejes de salida y ejes de reducción	50
10.5. Invariantes de bucle	50
10.6. División de rangos en bloques	50
10.7. Intercambiar bucles	51
10.8. Dependencias de memoria	51
10.9. Lowering por etapas	51
10.10. Verificar que no faltan ni sobran posiciones	52

11. Reducciones: identidad, orden y cooperación	53
11.1. La suma de una caja vacía	53
11.2. Una reducción por fila	53
11.3. Árbol de reducción	53
11.4. Particionar y combinar	54
11.5. Atomics no significan determinismo numérico	54
11.6. Máximo y empates	54
11.7. Promedio y ejes de tamaño uno	55
11.8. Log-sum-exp	55
11.9. RMSNorm como combinación de reducciones y operaciones locales	55
11.10. Pruebas que separan mecanismos	55
12. Un modelo se convierte en varios kernels	57
12.1. La diferencia entre fórmula y plan de ejecución	57
12.2. Materializar: dar una ubicación a un resultado	57
12.3. Fronteras naturales y fronteras elegidas	57
12.4. Dependencias de un kernel fusionado	58
12.5. Orden topológico del plan	58
12.6. Por qué una fusión grande puede ser peor	58
12.7. Epílogos de GEMM	58
12.8. Scheduling con efectos	59
12.9. Programas con varias salidas	59
12.10. Un experimento de fusión	59
12.11. Preguntas de revisión de un scheduler	60
13. Memoria del programa: reservar, reutilizar y no pisar el pasado	61
13.1. Un almacén compartido por tareas sucesivas	61
13.2. Vida útil y último uso	61
13.3. Arena y offsets	61
13.4. Best fit y fragmentación	62
13.5. Alias: dos nombres, una dirección	62
13.6. Actualizaciones de entrenamiento	62
13.7. Memoria asíncrona	62
13.8. Pico de memoria y memoria acumulada	63
13.9. Recomputation frente a almacenamiento	63
13.10. Verificar un plan de memoria	63
13.11. Un certificado sencillo de no solapamiento	63
14. Descomponer modelos: operaciones generales y límites explícitos	65
14.1. Un catálogo pequeño puede expresar muchas redes	65
14.2. Capa lineal	65
14.3. Activaciones	65
14.4. Convolución unidimensional desde una ventana	66
14.5. Dimensiones de salida de una convolución	66
14.6. Convolución bidimensional	66
14.7. Im2col: hacer explícitas las ventanas	66
14.8. Gather y embedding	67
14.9. Formas dinámicas y guardias	67

14.10.	Control de flujo	67
14.11.	Un manifiesto de operaciones soportadas	68
15.	La jerarquía de memoria: por qué multiplicar no es todo el trabajo . .	69
15.1.	La cocina y el almacén	69
15.2.	Latencia y ancho de banda	69
15.3.	Caché y localidad	69
15.4.	Líneas de caché y accesos dispersos	70
15.5.	Intensidad aritmética	70
15.6.	El modelo roofline	70
15.7.	Contar tráfico con cuidado	70
15.8.	Blocking: trabajar por porciones que se reutilizan	71
15.9.	Prefetch y predicción de accesos	71
15.10.	NUMA y afinidad	71
15.11.	Una investigación guiada	71
16.	Upcasting, unrolling y SIMD: varias casillas por instrucción	73
16.1.	Ocho lápices que avanzan juntos	73
16.2.	Del bucle escalar al vector	73
16.3.	Unrolling no es necesariamente vectorización	73
16.4.	Qué significa upcasting en este contexto	74
16.5.	Acumuladores múltiples	74
16.6.	Alineación	74
16.7.	Aliasing como obstáculo y como contrato	74
16.8.	Auto-vectorización y ensamblador	75
16.9.	Intrinsics y despacho por ISA	75
16.10.	Reducir presión de registros	75
16.11.	Un ejercicio de predicción de colas	75
17.	GEMM rápida: del triple bucle al microkernel	77
17.1.	Empezar por una salida correcta	77
17.2.	Reutilizar una entrada de A	77
17.3.	Tres niveles de blocking	77
17.4.	Packing	78
17.5.	El microkernel incluido en Lumbre	78
17.6.	Colas y dimensiones cero	78
17.7.	Orden de acumulación	79
17.8.	Matrices por lotes y transpuestas	79
17.9.	Medidas locales de esta edición	79
17.10.	Cuándo decir «competitivo»	79
17.11.	Progresión de ingeniería	80
18.	Convoluciones y kernels de reducción rápidos	81
18.1.	Reutilización que se solapa entre ventanas	81
18.2.	Contrato completo de conv2d	81
18.3.	Separar interior y bordes	82
18.4.	Transformar a GEMM paso a paso	82
18.5.	Convolución implícita	82

18.6.	Layouts de canales	82
18.7.	Reducciones especializadas	83
18.8.	Fusionar estadísticas y epílogos	83
18.9.	Convolución y backward	83
18.10.	Pruebas indispensables	83
18.11.	Objetivo de rendimiento	84
19.	Medir, perfilar y autotunear sin engañarse	85
19.1.	El cronómetro también tiene un contrato	85
19.2.	Calentamiento	85
19.3.	Sincronizar lo que se mide	85
19.4.	Muestras y estadísticas	85
19.5.	Corrección antes de tiempo	86
19.6.	Throughput y latencia	86
19.7.	El comparador PyTorch	86
19.8.	Autotuning	86
19.9.	Coste de buscar	87
19.10.	Overfitting del autotuner	87
19.11.	Perfilado	87
19.12.	Energía y coste	88
19.13.	Una tabla de resultados honesta	88
20.	GPU: repartir posiciones entre miles de tareas	89
20.1.	Una GPU no adivina el paralelismo	89
20.2.	Hilo, bloque y grid	89
20.3.	La guardia de la cola	89
20.4.	Mapeo bidimensional	90
20.5.	SIMT y grupos de ejecución	90
20.6.	Divergencia	90
20.7.	Registros y memoria local	90
20.8.	Ocupación	91
20.9.	Memoria del host y del dispositivo	91
20.10.	Primer hito GPU	91
20.11.	CUDA frente a HIP	91
21.	Call y runtime: hacer que el código llegue al dispositivo	93
21.1.	Un kernel compilado todavía no se ha ejecutado	93
21.2.	Un contrato de llamada	93
21.3.	Cargar un módulo y resolver una función	93
21.4.	Comprobar el target	94
21.5.	Streams	94
21.6.	Eventos	94
21.7.	Errores asíncronos	94
21.8.	Vida de buffers y módulos	95
21.9.	Caché del JIT y capacidades	95
21.10.	Bibliotecas externas y fallback	95
21.11.	Un pasaporte de llamada	95

22. Coalescing, shared memory y sincronización	97
22.1. Una fila de hilos y una fila de datos	97
22.2. Elegir qué eje sigue al hilo x	97
22.3. Shared memory como mesa de trabajo del bloque	97
22.4. Dos barreras por iteración	97
22.5. La trampa del retorno temprano	98
22.6. GEMM compartida de 16×16	98
22.7. Bancos de shared memory	98
22.8. Swizzling	98
22.9. Registros compartidos por una operación colectiva	99
22.10. Double buffering	99
22.11. Profundidad de pipeline	99
22.12. Pruebas de concurrencia	99
23. Tensor cores: una operación colectiva, no una multiplicación mágica	101
23.1. De una celda a un mosaico	101
23.2. Fragmentos: cada participante guarda solo parte	101
23.3. Las cuatro preguntas antes de compilar	102
23.4. De la demostración a un GEMM completo	103
23.5. Carga asíncrona: iniciar no significa terminar	103
23.6. El epílogo también tiene arquitectura	104
23.7. CuTe, CUTLASS y cuTile no son tres nombres de lo mismo	104
23.8. Una prueba oral que detecta comprensión	104
24. AMD: portar el significado antes de perseguir instrucciones	105
24.1. Una traducción que parece demasiado fácil	105
24.2. Tres capas del port	105
24.3. Un ejemplo de reducción dependiente del grupo	105
24.4. LDS y memoria compartida: parecido no significa idéntico coste	106
24.5. Dónde mirar en el ecosistema	106
24.6. Laboratorio de port con presupuesto acotado	107
25. Huawei Ascend y DeepSeek: separar cálculo, comunicación y compilación	108
25.1. La palabra kernel estaba escondiendo dos cosas	108
25.2. Una fábrica con estaciones especializadas	108
25.3. Un GEMM con epílogo como primer port	108
25.4. Qué publica cada repositorio de DeepSeek	109
25.5. La lectura del README también es parte del laboratorio	109
25.6. Cuantización por bloques sin perder la escala	110
25.7. Planificar una tubería con estados explícitos	110
25.8. Qué conservar de esta comparación	111
26. Linux y los controladores: el suelo sobre el que corre el compilador	112
26.1. Una aplicación no posee físicamente toda la GPU	112
26.2. Dirección virtual, residencia y transferencia	112
26.3. Fences: una promesa de finalización	112
26.4. Fallo de compilación, fallo de ejecución y reinicio del dispositivo	113
26.5. Observación segura para principiantes	113

26.6.	El límite útil del semestre	114
27.	Aprender significa cambiar números: la derivada desde cero	115
27.1.	Un error que podemos calcular a mano	115
27.2.	Derivar no es dividir dos números escritos	115
27.3.	La regla de la cadena como transmisión de cambios	116
27.4.	Dos caminos obligan a sumar	116
27.5.	Qué necesitamos guardar del avance	116
27.6.	Una derivada numérica para comprobar, no para entrenar todo	117
27.7.	Primera práctica con Lumbre	117
27.8.	Dónde acaba esta intuición	117
28.	Autodiferenciación inversa sobre un grafo de UOps	119
28.1.	El retroceso empieza con una pregunta	119
28.2.	Qué significa un adjunto	119
28.3.	Reglas locales que debemos poder justificar	120
28.4.	Broadcasting hacia delante, suma hacia atrás	120
28.5.	Reducir hacia delante, expandir hacia atrás	120
28.6.	El producto de matrices explicado con una celda	121
28.7.	Movimientos: deshacer direcciones, no derivar etiquetas	121
28.8.	Comparaciones, selección y stop-gradient	121
28.9.	Autodiff también se compila	122
28.10.	Pruebas por capas	122
29.	Optimizar parámetros sin corromper el estado	123
29.1.	Descenso de gradiente con un ejemplo completo	123
29.2.	Por qué la actualización no debe pisar sus entradas	123
29.3.	Momentum: recordar una dirección reciente	124
29.4.	AdamW en cuatro piezas	124
29.5.	Recortar el gradiente sin cambiar cada componente a ojo	124
29.6.	Gradientes acumulados y lotes efectivos	125
29.7.	Checkpoint: mucho más que pesos	125
30.	De letras a predicciones: construir un modelo de lenguaje	126
30.1.	Texto, token e identificador	126
30.2.	La tabla de embeddings	126
30.3.	La etiqueta está desplazada una posición	127
30.4.	Logits y probabilidades	127
30.5.	Una derivada especialmente útil	127
30.6.	Causalidad: no mirar la solución	128
30.7.	Qué significa «entrenar un LLM» en este curso	128
31.	Atención: consultar recuerdos con pesos aprendidos	129
31.1.	Tres papeles para una misma secuencia	129
31.2.	Una consulta y dos recuerdos	129
31.3.	Todas las consultas juntas	129
31.4.	Por qué aparece la raíz de d	130
31.5.	Múltiples cabezas: varias consultas en paralelo	130

31.6.	GQA: compartir claves y valores	130
31.7.	Qué memoria crece más deprisa	131
31.8.	El retroceso de atención por etapas	131
32.	RMSNorm, RoPE y SwiGLU: entender cada pieza del decoder	132
32.1.	Normalizar sin borrar la identidad de la fila	132
32.2.	Derivar RMSNorm para comprobar el grafo	132
32.3.	RoPE: rotar parejas según la posición	132
32.4.	Posición absoluta y relación relativa	133
32.5.	SwiGLU: una puerta que modula contenido	133
32.6.	Conexiones residuales: sumar una corrección	134
32.7.	Embedding compartido con la salida	134
33.	Ensamblar el Transformer y comprobarlo antes de entrenar	135
33.1.	El contrato del constructor	135
33.2.	Contar parámetros para detectar duplicaciones	135
33.3.	Seguir una capa con shapes	135
33.4.	Prueba uno: el avance tiene números finitos	136
33.5.	Prueba dos: cambiar el futuro no cambia el pasado	136
33.6.	Prueba tres: reducir a una instancia diminuta	136
33.7.	Prueba cuatro: sobreajustar un batch deliberadamente	137
33.8.	Prueba cinco: el checkpoint conserva el significado	137
34.	Tokenización por bytes y BPE: una extensión completa	138
34.1.	El carácter desconocido revela una decisión pendiente	138
34.2.	Una caja de piezas que aprende a pegar	138
34.3.	Parejas solapadas no pueden sustituirse dos veces	138
34.4.	Un orden reproducible para los empates	139
34.5.	Codificar y decodificar	139
34.6.	Ajustar el tokenizador solo con entrenamiento	140
34.7.	Usarlo con el compilador	140
34.8.	Qué demuestra la prueba incluida	140
35.	Ejecutar el entrenamiento completo y leer sus resultados	141
35.1.	El primer comando tiene un objetivo pequeño	141
35.2.	Lo que ocurre en una iteración	141
35.3.	Una configuración ejecutada durante la preparación	141
35.4.	La muestra no autoriza una afirmación de calidad	142
35.5.	Una ejecución anterior también enseña algo	142
35.6.	Reanudar significa añadir pasos	143
35.7.	Entrenar con texto propio	143
35.8.	La ruta GPU del mismo modelo	143
35.9.	Qué medir al aumentar el tamaño	144
36.	Evaluar sin engañarse: datos, métricas y experimentos	145
36.1.	Entrenamiento, validación y prueba responden preguntas distintas	145
36.2.	Pérdida y perplejidad	145
36.3.	Una pérdida finita puede estar mal normalizada	145

36.4.	Contaminación y duplicados	146
36.5.	Comparación de implementaciones con los mismos pesos	146
36.6.	Ablaciones: cambiar una cosa con una razón	146
36.7.	Semillas y variabilidad	147
36.8.	Informe final de un experimento	147
37.	Atención online: resolver una fila sin guardar todos sus pesos	148
37.1.	La receta densa guarda más de lo necesario	148
37.2.	Restar un máximo que todavía no conocemos	148
37.3.	Una ejecución entera con tres puntuaciones	149
37.4.	Inicialización y filas vacías	149
37.5.	De un elemento a un bloque	149
37.6.	La operación nueva en Lumbre	150
37.7.	Qué memoria ahorra exactamente	150
38.	Del algoritmo online a FlashAttention y sus derivados	151
38.1.	Qué idea investigamos en los artículos	151
38.2.	Reutilizar Q mientras recorremos K y V	151
38.3.	El coste no aritmético puede dominar	151
38.4.	Guardar log-sum-exp para el retroceso	152
38.5.	Una derivación que reduce un término del backward	152
38.6.	La propiedad de los gradientes importa	152
38.7.	Dropout y reproducibilidad del retroceso	153
38.8.	Atención de entrenamiento y de generación	153
38.9.	Reproducción de un artículo frente a inspiración	153
39.	Kernels persistentes, planificación y límites del autotuning	154
39.1.	Cuando los tiles no tienen el mismo trabajo	154
39.2.	Un scheduler de tareas como modelo independiente	154
39.3.	El orden de tareas también afecta a la caché	154
39.4.	Especializar sin compilar infinitas variantes	155
39.5.	Autotuning con un presupuesto explícito	155
39.6.	El mejor candidato de una muestra puede sobreajustar	155
40.	Escalar un modelo: primero contar memoria y trabajo	156
40.1.	Un presupuesto por categorías	156
40.2.	Una cuenta que descarta un plan antes de ejecutarlo	156
40.3.	Una arquitectura intermedia concreta	157
40.4.	Activaciones: el contexto puede dominar	157
40.5.	Checkpointing de activaciones no es guardar el entrenamiento en disco	158
40.6.	Cuánto trabajo aproxima un entrenamiento denso	158
40.7.	Datos y parámetros compiten por el presupuesto	158
40.8.	Puertas de aceptación antes de un entrenamiento largo	159
41.	Precisión mixta y cuantización: ahorrar sin cambiar a ciegas el cálculo	160
41.1.	Almacenar, multiplicar y acumular pueden usar tipos distintos	160
41.2.	Por qué el backward puede necesitar otro rango	160
41.3.	Escalado de la pérdida	160

41.4.	Cuantización simétrica con una cuenta pequeña	161
41.5.	La calibración no debe mirar el examen	161
41.6.	Formatos muy compactos y escalas por bloques	161
41.7.	Una política de error por operación	161
42.	Entrenamiento distribuido: repartir estado y reunir contribuciones . .	163
42.1.	Dos estudiantes calculan mitades de un lote	163
42.2.	All-reduce, reduce-scatter y all-gather	163
42.3.	Particionar estados para no repetirlos	163
42.4.	Tensor parallelism con una multiplicación pequeña	164
42.5.	No reunir un tensor solo para volver a partirlo	164
42.6.	Pipeline parallelism y burbujas	164
42.7.	Solapar comunicación y cómputo	165
42.8.	Fallos y orden colectivo	165
43.	Mezclas de expertos y modelos de frontera en 2026	166
43.1.	Más especialistas, no todos trabajando a la vez	166
43.2.	Un ejemplo de dispatch completo	166
43.3.	Expertos vacíos y tamaños irregulares	166
43.4.	Comunicación y cómputo son bibliotecas distintas por una razón	167
43.5.	Un mapa de diferencias respecto al decoder del curso	167
43.6.	Qué exigiría una reproducción de calidad competitiva	167
44.	Generación eficiente: la caché KV es parte del sistema	168
44.1.	Recalcular todo es correcto, pero caro	168
44.2.	Una cuenta de memoria	168
44.3.	Posiciones y desplazamientos	168
44.4.	Páginas para longitudes variables	169
44.5.	Prefill y decode necesitan métricas separadas	169
44.6.	Caché y entrenamiento no son la misma ruta	169
45.	Leer tinygrad sin perderse: del tensor al programa	170
45.1.	George Hotz y la idea de estudiar una pila completa	170
45.2.	Qué versión se está leyendo	170
45.3.	Cuatro traducciones de una misma intención	171
45.4.	Cómo seguir una reducción agrupada	171
45.5.	Qué copiar como idea y qué no copiar a ciegas	172
46.	Tiny Corp más allá del compilador: sistema, firmware y colas	173
46.1.	Un mapa sin mezclar responsabilidades	173
46.2.	Una cola explicada con pedidos de cocina	173
46.3.	Tres errores que parecen uno	174
46.4.	Estado, comandos y eventos no son intercambiables	174
46.5.	Un proyecto de firmware puede estudiarse sin instalarlo	175
47.	LLVM y MLIR: comprender SSA antes de aprender sus siglas	176
47.1.	Dar un nombre nuevo a cada resultado	176
47.2.	Una bifurcación necesita reunir resultados	176
47.3.	Un bucle es una recurrencia, no un círculo arbitrario	177

47.4.	Dialectos: conservar información mientras sea útil	177
47.5.	Alias: dos nombres pueden señalar la misma caja	178
47.6.	Diseñar diagnósticos para alguien que empieza	178
48.	Triton, CuTe, TileLang y bibliotecas: elegir una abstracción	179
48.1.	La pregunta no es quién tiene la sintaxis más corta	179
48.2.	Un contrato común para comparar soluciones	179
48.3.	Triton: un programa procesa un bloque de datos	180
48.4.	CuTe y CUTLASS: el layout también es un programa	180
48.5.	TileLang: separar la intención del tile y su planificación	180
48.6.	Bibliotecas de kernels frente a compiladores completos	181
48.7.	Decisión de proyecto: tres rutas legítimas	181
49.	Leer artículos de 2026 como investigador, no como espectador	182
49.1.	Una afirmación necesita su dominio	182
49.2.	Tres preguntas complementarias de la literatura reciente	182
49.3.	Diseñar una tabla que no oculte fracasos	183
49.4.	Separar prueba formal, comprobación diferencial y muestreo	183
49.5.	Elaborar una réplica de alcance honesto	184
50.	Laboratorio 1. Tu primera cadena completa: de cuatro números a C	185
50.1.	Objetivo y preparación	185
50.2.	Predicción antes del programa	185
50.3.	Construcción y ejecución	186
50.4.	Comparar dos planificaciones sin cambiar la fórmula	186
50.5.	Ejercicio de reescritura con límites explícitos	187
50.6.	Entrega y condición de avance	187
51.	Laboratorio 2. Índices simbólicos, intervalos y máscaras	188
51.1.	El problema que vamos a resolver	188
51.2.	Calcular una tabla completa del borde	188
51.3.	Construir la expresión en el módulo simbólico	188
51.4.	Probar una identidad y sus hipótesis	189
51.5.	Una máscara debe proteger el acceso, no solo el resultado	189
51.6.	Intervalos conservadores y precisión del análisis	190
51.7.	Entrega resuelta y variación	190
52.	Laboratorio 3. Vistas sin copia y reducciones sin confusión	191
52.1.	Del dibujo al almacenamiento	191
52.2.	Por qué aplanar no siempre es una vista contigua	191
52.3.	Broadcasting como stride cero	192
52.4.	Reducir una dimensión y conservar las otras	192
52.5.	Composición que obliga a seguir índices	192
52.6.	Condición de éxito	193
53.	Laboratorio 4. Planificar kernels y reciclar memoria sin romper datos	194
53.1.	Un grafo con una dependencia compartida	194
53.2.	Calcular vidas a mano	194
53.3.	Un ejemplo donde sí se puede reutilizar	195

53.4.	Comparación de las cuatro combinaciones	195
53.5.	Una escritura de parámetros es una transacción	195
53.6.	Entrega	196
54.	Laboratorio 5. GEMM: mejorar sin esconder el coste	197
54.1.	Una pregunta experimental concreta	197
54.2.	Cuenta de operaciones y bytes	197
54.3.	Comprobar bordes antes de medir cuadrados bonitos	198
54.4.	Entender una medición local real	198
54.5.	Experimento de una sola variable	198
54.6.	Condición de entrega	199
55.	Laboratorio 6. Convolución desde índices hasta gradientes	200
55.1.	Una imagen diminuta y un filtro que puedes calcular	200
55.2.	Resolver el gradiente del filtro a mano	200
55.3.	Dos implementaciones con propósitos distintos	201
55.4.	Padding y dilatación en coordenadas	201
55.5.	Prueba numérica de un gradiente	202
55.6.	Entrega	202
56.	Laboratorio 7. Llevar el mismo grafo a CUDA o HIP	203
56.1.	Preparar un inventario, no una instalación a ciegas	203
56.2.	Primera prueba: 257 elementos	203
56.3.	Leer el kernel generado	204
56.4.	Segunda prueba: GEMM con memoria compartida	204
56.5.	Tercera prueba: WMMA separado del compilador	205
56.6.	Depuración en orden	205
57.	Laboratorio 8. Autodiferenciación con una cuenta que puedes verificar	206
57.1.	Una regresión de dos parámetros	206
57.2.	Construir el paso como otro grafo	206
57.3.	Por qué una contribución no puede sobrescribir otra	207
57.4.	Broadcasting en el backward	207
57.5.	Diferencias finitas: escoger una perturbación razonable	207
57.6.	Entrega y ampliación	208
58.	Laboratorio 9. Un decoder completo y un checkpoint que realmente reanuda	209
58.1.	Qué vamos a demostrar	209
58.2.	Primera ejecución pequeña	209
58.3.	Reproducir la configuración principal	209
58.4.	Revisar un token de supervisión	210
58.5.	La prueba fuerte de reanudación	210
58.6.	Interpretar fallos con orden	211
59.	Laboratorio 10. Atención online: misma función, otra memoria	212
59.1.	La recurrencia que queremos probar	212
59.2.	Un programa de comparación	212
59.3.	Comprobar backward sin prometer una memoria que no existe	213

59.4.	Separar efecto algorítmico y efecto de implementación	213
59.5.	Pruebas que no debes omitir	213
60.	Laboratorio 11. Tokenización, particiones y pérdidas comparables . . .	214
60.1.	Por qué los datos también forman parte del experimento	214
60.2.	Una prueba de ida y vuelta	214
60.3.	Fusiones solapadas: un caso pequeño pero decisivo	214
60.4.	Dividir antes de aprender el vocabulario	215
60.5.	Entrenar con tokens de bytes	215
60.6.	De nats por token a una unidad interpretable	215
60.7.	Entrega	216
61.	Laboratorio 12. Una entrega reproducible y una defensa técnica	217
61.1.	El objetivo final no es una carpeta que solo funciona en tu sesión	217
61.2.	Qué pertenece al manifiesto	217
61.3.	Separar tres rutas de ejecución	218
61.4.	Preparar una defensa con un caso que no conozcas	218
61.5.	Rúbrica orientativa para el cierre del curso	218
61.6.	Entrega final del estudiante	219
62.	Proyecto final A. Atención con memoria acotada: una réplica semántica completa	220
62.1.	Alcance y contribución del proyecto de referencia	220
62.2.	Definir la función exacta	220
62.3.	Derivación del estado que basta conservar	221
62.4.	Integración en el compilador	221
62.5.	Plan de pruebas que acompaña al resultado	222
62.6.	Resultados locales del forward	222
62.7.	Qué experimento realizar después	222
62.8.	Entregables y defensa del proyecto	223
63.	Proyecto final B. Pasaporte de índices para un acelerador desconocido	224
63.1.	La idea original	224
63.2.	Definición del mapa	224
63.3.	Resolver el contraejemplo completo	225
63.4.	Del ensayo finito a un argumento general	225
63.5.	Por qué sirve para hardware extraño	225
63.6.	TDD del proyecto	226
63.7.	Extensión a layouts de fragmentos	226
63.8.	Entrega del proyecto	226
64.	Proyecto final C. Competitividad CPU y GPU con un contrato medible	227
64.1.	Convertir «competitivo» en una pregunta que pueda fallar	227
64.2.	Baseline de tres niveles	227
64.3.	Una escalera de optimizaciones CPU	227
64.4.	Repartir trabajo sin false sharing innecesario	228
64.5.	Escalera GPU y puertas de aceptación	228
64.6.	Función de puntuación sin premiar errores	229

64.7.	Calendario de cuatro semanas	229
64.8.	Qué resultado haría original el trabajo	229
65.	Proyecto final D. Un agente propone kernels; el contrato decide	230
65.1.	Motivación científica y límite de la automatización	230
65.2.	Elegir un espacio de búsqueda seguro y pequeño	230
65.3.	Un evaluador en fases	230
65.4.	No enseñar todas las pruebas al proponente	231
65.5.	Un presupuesto que no desaparece del artículo	231
65.6.	Detectar soluciones tramposas o frágiles	231
65.7.	Entrega y conexión con los LLM del curso	232
66.	Proyecto final E. Del decoder pequeño al entrenamiento de un LLM mayor	233
66.1.	Qué significa ampliar sin cambiar de problema	233
66.2.	Tres tamaños con objetivos distintos	233
66.3.	Un presupuesto de memoria antes de reservar	233
66.4.	Orden de las extensiones	234
66.5.	Datos y evaluación que no se puedan confundir con la demo	234
66.6.	Distribuido: un paso debe conservar su significado	234
66.7.	Puertas de aceptación del proyecto LLM	235
66.8.	Informe final y posible contribución	235
67.	Boletín de integración: resolver antes de ejecutar	236
67.1.	Problema 1. Un paso completo de aprendizaje con matrices pequeñas . . .	236
67.2.	Problema 2. Fusión que duplica trabajo	237
67.3.	Problema 3. Una vista invertida y una dirección válida	237
67.4.	Problema 4. Un techo de rendimiento no es una medida	237
67.5.	Problema 5. Dos barreras y un hilo sin salida	238
67.6.	Problema 6. Dos lotes y una media mal calculada	238
67.7.	Problema 7. Un caché aparentemente válido	238
67.8.	Problema 8. Qué significa un máximo con empate	239
67.9.	Problema 9. Compresión de pesos y memoria de entrenamiento	239
67.10.	Problema 10. Interpretar una tabla de un artículo	239
68.	Glosario razonado y conexiones que conviene conservar	241
68.1.	De la intención al programa	241
68.2.	De las coordenadas a la máquina	241
68.3.	Del programa al aprendizaje	242
68.4.	De la observación a la conclusión	243
69.	CPU y CUDA en la misma máquina: de la prueba a la medida	244
69.1.	El entorno que realmente ejecutó el código	244
69.2.	La escalera de aceptación	245
69.3.	Comparar valores y gradientes antes de comparar segundos	245
69.4.	Medir con una frontera explícita	246
69.5.	Entrenar la misma configuración en ambos destinos	247
69.6.	WMMA: una comprobación con alcance propio	248

69.7.	Qué entregar al terminar la comparación	248
70.	Kernels portables y estado persistente: dos lecturas de DeepSeek . . .	250
70.1.	DeepGEMM-Ascend: el contrato público y la representación física	250
70.2.	Leer un kernel industrial sin empezar por todos sus detalles	250
70.3.	Presupuestar KV antes de hablar de compresión	251
70.4.	Qué añade la lectura de DeepSeek-V4.1-Flash	252
70.5.	Un experimento de reconstrucción exacta y aproximada	252
70.6.	Un proyecto que sí cabe en esta entrega	253
71.	Contrato de UOps: de la especificación a una IR verificable	254
71.1.	Tres representaciones con responsabilidades distintas	254
71.2.	Hash-consing con tipos, ámbitos y efectos	254
71.3.	Semana 1: operaciones puras y sus dominios	255
71.4.	División hacia menos infinito y C	256
71.5.	Reescribir de abajo arriba y saber cuándo detenerse	256
71.6.	Semana 2: tipo de valor, forma y dirección	256
71.7.	Una función C sin bucles y una dependencia real	257
71.8.	Semana 3: rangos, cierre y acumulación	258
71.9.	El fragmento de intercambio proporcionado	258
71.10.	Un intercambio que se pueda comprobar	259
72.	De rangeify al entrenamiento: preservar el contrato	260
72.1.	Movimientos como mapas de coordenadas	260
72.2.	Reducción por número de ejes	260
72.3.	Derivar GEMM con un ejemplo rectangular	260
72.4.	Derivar convolución sin un operador privilegiado	261
72.5.	Qué elimina rangeify y qué debe conservar	261
72.6.	STAGE: pagar memoria para evitar trabajo repetido	262
72.7.	Autodiff antes de introducir efectos	262
72.8.	El primer entrenamiento: MNIST como entrega	263
72.9.	Semana 6: LOOP, UPCAST, THREAD y GROUP	263
72.10.	Contar operaciones y localizar el límite	264
72.11.	Atención online y el objetivo FlashAttention	264
72.12.	GPU: la dependencia lógica se convierte en ejecución	264
72.13.	Formas simbólicas y el paso de entrenamiento	265
A.	Código completo de Lumbre y sus pruebas	266
A.1.	examples/bench_kernels.py	266
A.2.	examples/train_decoder.py	267
A.3.	examples/validate_book.py	270
A.4.	examples/verify_launch.py	272
A.5.	kernels/conv.c	272
A.6.	kernels/wmma_demo.cu	273
A.7.	lumbre/__init__.py	274
A.8.	lumbre/compiler.py	274
A.9.	lumbre/ir.py	282
A.10.	lumbre/layout.py	288

A.11.	lumbre/nn.py	289
A.12.	lumbre/nnops.py	291
A.13.	lumbre/passport.py	292
A.14.	lumbre/symbolic.py	294
A.15.	lumbre/tokenizer.py	295
A.16.	tests/conftest.py	297
A.17.	tests/test_conv.py	297
A.18.	tests/test_conv_ir.py	298
A.19.	tests/test_lumbre.py	298
A.20.	tests/test_passport.py	302
A.21.	tests/test_tokenizer.py	303
B.	Registro de validación y alcance de la entrega	304
B.1.	Entorno observado	304
B.2.	Batería y ejemplos numéricos	304
B.3.	Entrenamiento principal	305
B.4.	Campaña secuencial CPU/CUDA	306
B.5.	Cómo distinguir reportes históricos	306
C.	Reproducir, modificar y reconstruir el material	308
C.1.	Primera instalación con una ruta única	308
C.2.	Reproducción fría y caliente	308
C.3.	Modificar el compilador por capas	309
C.4.	Diagnóstico de problemas frecuentes	309
C.5.	Compilar el libro	309
D.	Mapa de cobertura y puertas del semestre	311
D.1.	Correspondencia con el temario solicitado	311
D.2.	Qué tiene que saber explicar el estudiante	312
D.3.	Frontera entre libro y proyecto de producción	312
	Bibliografía y recursos verificables	313